

Android Từ Cơ Bản Đến Nâng Cao

Dành cho người đã biết Java OOP cơ bản

Mục lục

Phần 0 — Chuẩn bị môi trường

- Chương 1. Giới thiệu Android: kiến trúc hệ điều hành, vòng đời phiên bản, hệ sinh thái
- Chương 2. Cài đặt JDK, Android Studio, SDK/SDK Manager
- Chương 3. Tạo và cấu hình máy ảo (AVD), kết nối thiết bị thật (USB debugging, ADB)
- Chương 4. Tạo project đầu tiên: cấu trúc project, Gradle, AndroidManifest.xml
- Chương 5. Build & deploy ứng dụng "Hello World" đầu tiên lên emulator/thiết bị thật

Phần 1 — Nền tảng

- Chương 6. Vòng đời Activity & Application
- Chương 7. Layout & Views (XML layout, ConstraintLayout, ViewGroup)
- Chương 8. Sự kiện, Intent (explicit/implicit), điều hướng giữa các màn hình
- Chương 9. Fragment & Navigation Component
- Chương 10. RecyclerView & Adapter (danh sách, hiệu năng)
- Chương 11. Resource: string/dimens/style, đa ngôn ngữ, đa kích thước màn hình

Phần 2 — Lưu trữ dữ liệu

- Chương 12. SharedPreferences (cấu hình, trạng thái đơn giản)
- Chương 13. SQLite & Room (cơ sở dữ liệu quan hệ)
- Chương 14. File storage: internal vs external, scoped storage
- Chương 15. Jetpack DataStore (thay thế SharedPreferences hiện đại)

Phần 3 — Chạy ngầm & bất đồng bộ

- Chương 16. Service & foreground service
- Chương 17. BroadcastReceiver, hệ thống sự kiện của Android
- Chương 18. WorkManager (tác vụ nền có lịch/điều kiện)
- Chương 19. Xử lý bất đồng bộ: Thread/Executor (Java) và Coroutine (mở rộng Kotlin)
- Chương 20. Notification

Phần 4 — Mạng & API

- Chương 21. Gọi HTTP với OkHttp/Retrofit
- Chương 22. Parse JSON (Gson/Moshi), mapping dữ liệu
- Chương 23. Xử lý lỗi mạng, retry, cache offline-first cơ bản

Phần 5 — Kiểm thử, Sandbox & Bảo mật

- Chương 24. Unit test với JUnit

Chương 25. UI test với Espresso

Chương 26. Sandbox của Android: mô hình quyền (permission), runtime permission

Chương 27. Bảo mật cơ bản: ProGuard/R8, keystore, ký ứng dụng

Phần 6 — Kiến trúc nâng cao

Chương 28. MVVM, ViewModel & LiveData

Chương 29. Dependency Injection với Hilt

Chương 30. Giới thiệu Jetpack Compose

Chương 31. Clean Architecture cơ bản cho ứng dụng Android

Phần 7 — Giao tiếp liên ứng dụng & phần cứng nâng cao

Chương 32. Content Provider: chia sẻ dữ liệu có kiểm soát giữa các ứng dụng

Chương 33. Giao tiếp liên ứng dụng nâng cao: App Links/Deep Link, Share sheet, PendingIntent

Chương 34. AIDL & Bound Service: giao tiếp liên tiến trình (IPC) giữa các app

Chương 35. Bluetooth: Classic & BLE — quét thiết bị, kết nối, truyền/nhận dữ liệu

Chương 36. NFC: đọc/ghi tag, Host Card Emulation (HCE), giao tiếp giữa 2 thiết bị

Chương 37. Quyền & sandbox cho phần cứng: location cho BLE/NFC, background location, runtime permission theo API level

Phần 8 — Xuất bản ứng dụng

Chương 38. Chuẩn bị release build, ký ứng dụng, tối ưu kích thước (App Bundle)

Chương 39. Đưa ứng dụng lên Google Play Console

Chương 40. Theo dõi sau phát hành: Crashlytics, Analytics cơ bản

Phụ lục

Phụ lục A — Bảng tổng hợp lỗi thường gặp & cách xử lý

Phụ lục B — Tài liệu tham khảo & nguồn học thêm

Phần 0 — Chuẩn bị môi trường

Chương 1: Giới thiệu Android

Mục tiêu học

Sau chương này, bạn sẽ:

- Giải thích được Android là gì và vị trí của nó trong hệ sinh thái thiết bị di động.
- Mô tả được các tầng kiến trúc của hệ điều hành Android và vai trò từng tầng.
- Hiểu khái niệm **API level** và vì sao nó quan trọng khi viết ứng dụng.
- Có cái nhìn tổng quan (chưa đi sâu) về vòng đời một ứng dụng Android, để làm nền cho Chương 6.

Bạn đã biết Java/OOP nhưng chưa cần biết gì về Android — chương này không có code, chỉ có khái niệm và sơ đồ. Từ Chương 4 trở đi bạn sẽ bắt đầu gõ code thật.

1.1 Android là gì?

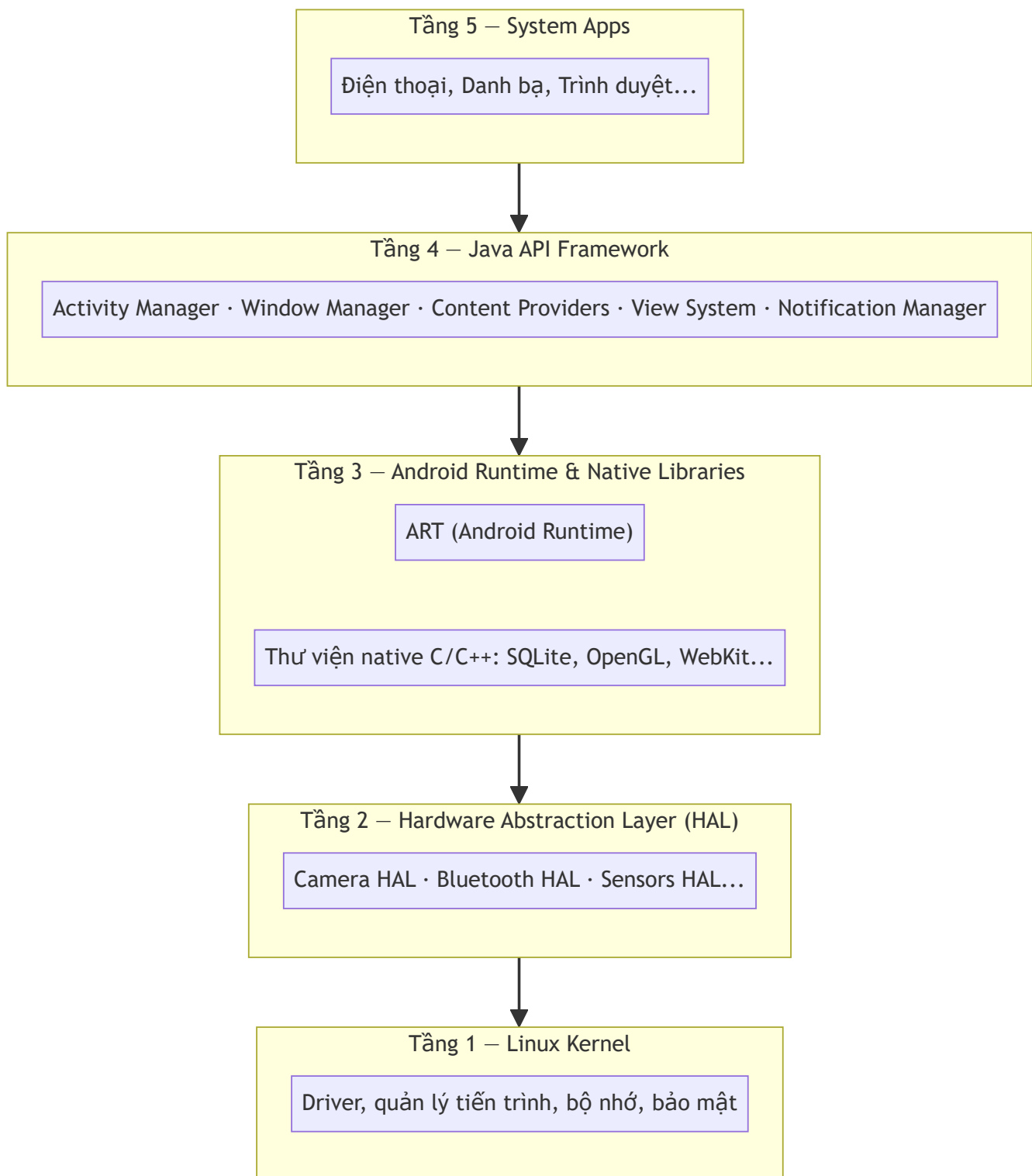
Android là hệ điều hành mã nguồn mở (dựa trên nhân Linux) do Google phát triển, dùng chủ yếu cho điện thoại, máy tính bảng, TV (Android TV), đồng hồ (Wear OS), và cả một số thiết bị nhúng (Android Things/Android Automotive).

Điểm khác biệt lớn nhất so với lập trình Java "thông thường" mà bạn đã quen:

- Ứng dụng **không có hàm** `main()` chạy tuyến tính. Hệ điều hành gọi vào các **thành phần** (Activity, Service, BroadcastReceiver, ContentProvider) theo vòng đời riêng của từng loại.
 - Mỗi ứng dụng chạy trong **sandbox** riêng (tiến trình Linux riêng, user ID riêng) — chi tiết ở Chương 26.
 - Tài nguyên (chuỗi, hình ảnh, layout...) tách khỏi code, quản lý qua hệ thống **resource** — chi tiết ở Chương 11.
-

1.2 Kiến trúc hệ điều hành Android

Android được xếp thành các tầng, tầng trên dùng dịch vụ của tầng dưới:

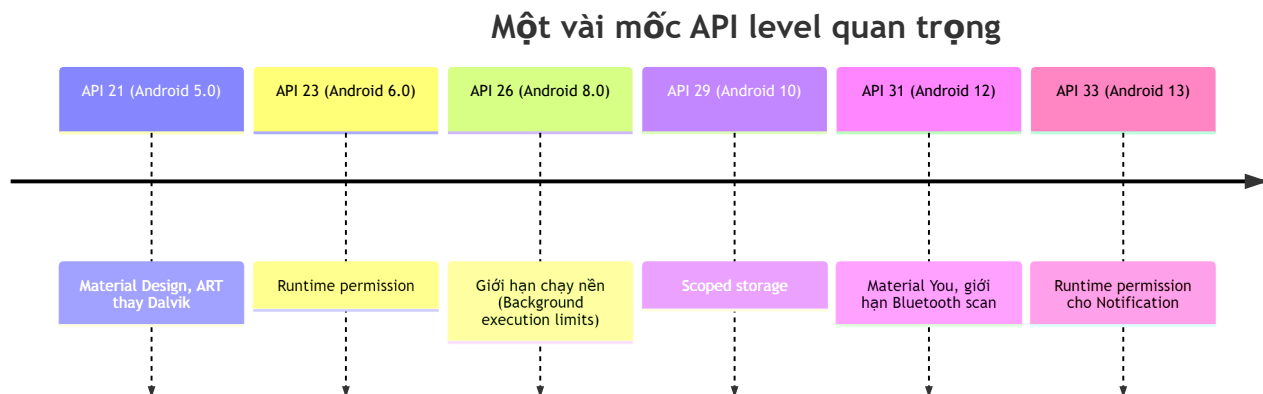


Ứng dụng bạn viết chạy trên tầng **Java API Framework**, được ART biên dịch và thực thi. Bạn hầu như không cần đụng tới HAL hay Linux Kernel trực tiếp — nhưng biết chúng tồn tại giúp bạn hiểu tại sao có khái niệm **permission** (Chương 26) hay vì sao truy cập phần cứng như Bluetooth/NFC (Chương 35–36) phải đi qua framework API thay vì gọi thẳng driver.

1.3 API level và vòng đời phiên bản

Mỗi phiên bản Android có một **API level** (số nguyên tăng dần). Khi viết ứng dụng, bạn khai báo:

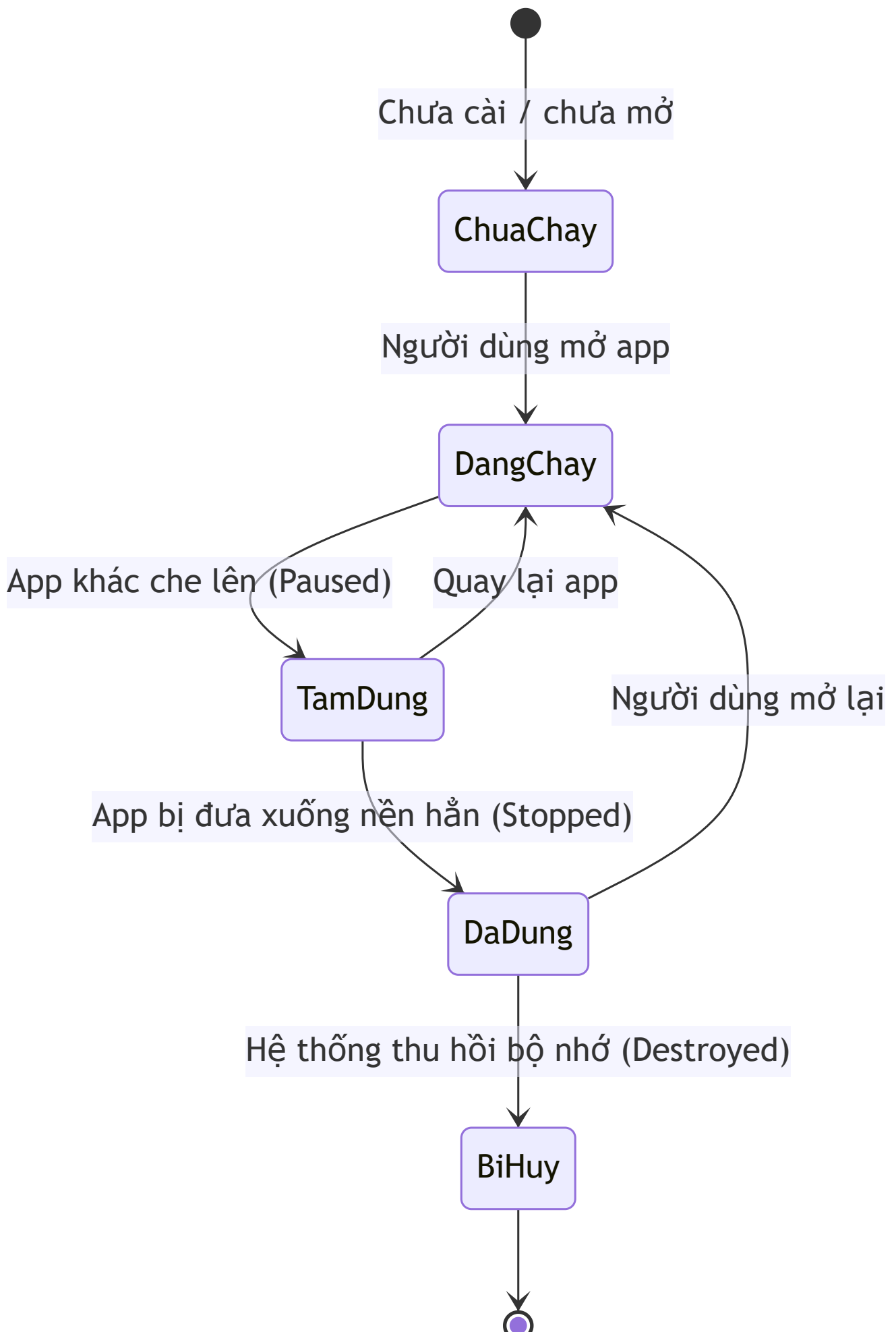
- `minSdkVersion` : phiên bản thấp nhất máy người dùng phải có để cài được app.
- `targetSdkVersion` : phiên bản bạn đã test và tối ưu hành vi cho nó.



Đây là lý do vì sao nhiều chương sau (lưu trữ file, chạy nền, Bluetooth...) đều phải nói rõ "hành vi này khác nhau theo API level" — Android không "đứng yên", và sách sẽ luôn ghi chú mốc phiên bản liên quan.

1.4 Nhìn trước: vòng đời một ứng dụng

Bạn chưa cần nhớ chi tiết sơ đồ dưới đây — đây chỉ là bức tranh toàn cảnh để bạn thấy trước khi vào Chương 6:



Khác với ứng dụng desktop, **hệ điều hành có quyền tự ý huỷ tiến trình ứng dụng của bạn** khi thiết bị thiếu bộ nhớ — kể cả khi người dùng không chủ động thoát app. Đây là lý do Android bắt buộc bạn thiết kế app theo tư duy "trạng thái có thể mất bất cứ lúc nào", chứ không phải chi tiết vật vãnh.

1.5 Ví dụ minh họa: một AndroidManifest.xml thực tế

Mọi ứng dụng Android đều có một file khai báo trung tâm là `AndroidManifest.xml`. Đây là ví dụ rút gọn (bạn sẽ tự tạo file này ở Chương 4, giờ chỉ cần đọc hiểu):

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="vn.example.helloandroid">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:label="Hello Android"
        android:icon="@mipmap/ic_launcher">

        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Ba điều đáng chú ý ngay từ bây giờ:

1. `package` là định danh duy nhất của app trên máy (và trên Google Play).
2. `<uses-permission>` là nơi bạn **khai báo trước** những gì app cần quyền truy cập — hệ quả trực tiếp của mô hình sandbox (Chương 26).
3. `<intent-filter>` với `MAIN / LAUNCHER` là cách hệ thống biết "đây là màn hình khởi động khi người dùng bấm icon app" — một ví dụ cụ thể của cơ chế Intent (Chương 8).

Bài tập

1. Vẽ lại sơ đồ kiến trúc 5 tầng ở mục 1.2 theo cách hiểu của riêng bạn, thêm chú thích ví dụ thực tế cho mỗi tầng (không cần đúng thuật ngữ, miễn giải thích được vai trò).
2. Tra cứu (tìm trên tài liệu chính thức của Android) xem thiết bị bạn đang dùng chạy phiên bản Android nào, tương ứng API level bao nhiêu.

3. Từ sơ đồ vòng đời 1.4, dự đoán: nếu bạn đang nhập một đoạn văn bản dài trong app, rồi có cuộc gọi đến làm app bị che khuất, dữ liệu bạn gõ có khả năng bị mất không? Ghi lại dự đoán — Chương 6 sẽ giải thích chính xác.
-

Lỗi thường gặp

- **Nhầm "phiên bản Android" với "API level":** Android 13 không phải lúc nào cũng là API 33 trên mọi thiết bị (một số thiết bị OEM có thể chậm cập nhật). Khi viết code, luôn dựa vào **API level**, không dựa vào tên phiên bản.
 - **Cho rằng app của mình sẽ luôn chạy nền vô hạn định:** từ API 26 trở đi, hệ thống giới hạn mạnh việc chạy nền (xem Chương 16–18). Nhiều lỗi "tại sao thông báo của tôi không đến" bắt nguồn từ đây.
-

Tóm tắt & tiếp theo

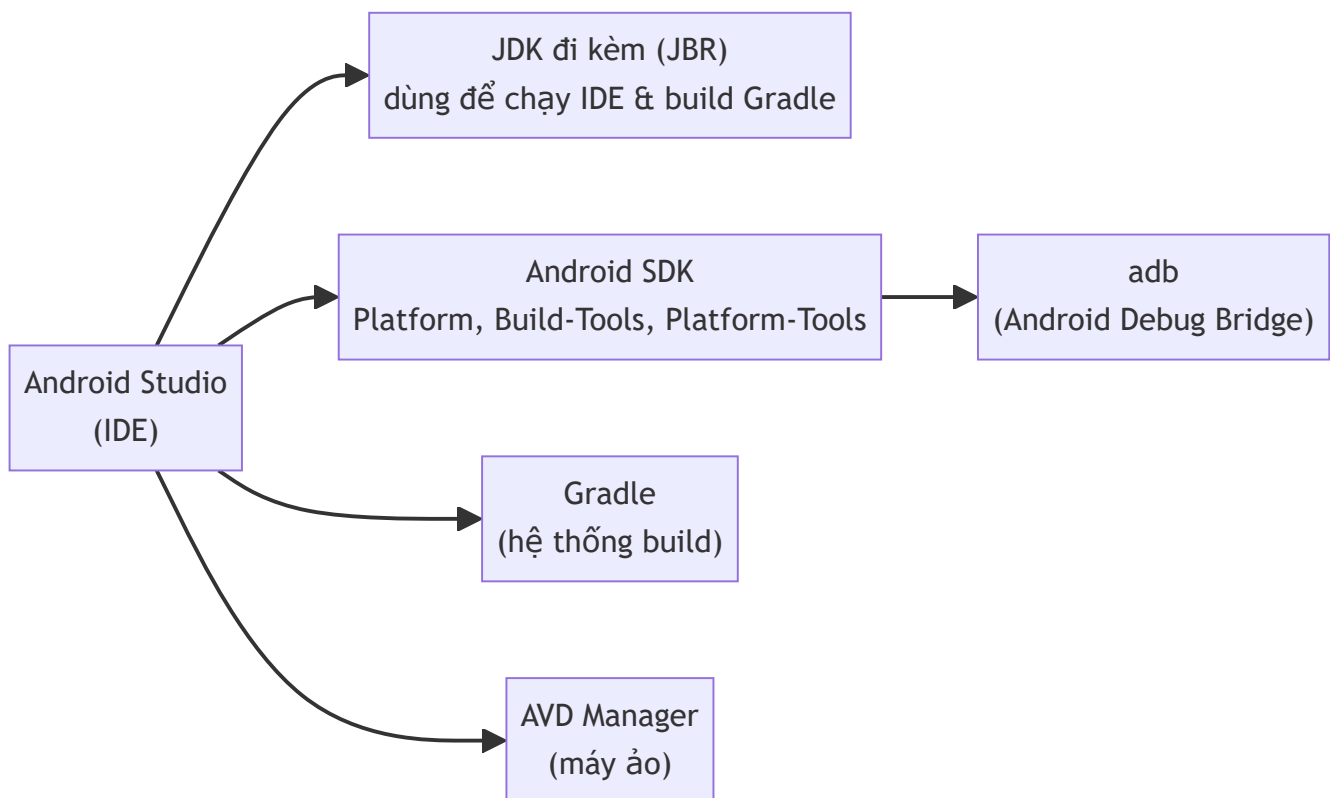
Bạn đã có bức tranh tổng quan về Android: kiến trúc hệ thống, khái niệm API level, và một cái nhìn sơ bộ về vòng đời ứng dụng. Chương 2 sẽ bắt đầu phần thực hành: cài đặt JDK, Android Studio và SDK để chuẩn bị viết dòng code Android đầu tiên.

Chương 2: Cài đặt JDK, Android Studio, SDK/SDK Manager

Mục tiêu học

- Cài đặt được Android Studio trên Windows và hoàn tất setup wizard lần đầu.
- Hiểu vai trò của JDK, Android SDK, SDK Manager, Gradle trong bộ công cụ — cái nào Android Studio đã tự lo, cái nào bạn cần biết để tự xử lý khi có lỗi.
- Cấu hình biến môi trường để dùng được `adb` từ dòng lệnh (ngoài Android Studio).
- Xác nhận cài đặt thành công bằng vài lệnh kiểm tra cụ thể.

2.1 Bộ công cụ gồm những gì?



Điểm nhiều tài liệu cũ còn nhầm: **từ các bản Android Studio gần đây, bạn không cần tự cài JDK riêng** — Android Studio đi kèm sẵn một bản JDK (JetBrains Runtime, dựa trên OpenJDK) và tự dùng nó để chạy IDE lẫn build Gradle. Bạn chỉ cần cài JDK riêng nếu:

- Build project bằng dòng lệnh (`gradlew`) mà không mở Android Studio, trên máy chưa từng cài Android Studio.
- Cần một phiên bản JDK cụ thể khác với bản đi kèm vì lý do riêng của dự án.

Ở sách này, mặc định dùng JDK đi kèm Android Studio — đủ cho toàn bộ nội dung.

2.2 Cài đặt Android Studio (Windows)

1. Tải bộ cài từ trang chính thức của Android Studio (bản Windows).
2. Chạy installer, giữ nguyên các lựa chọn mặc định: cài **Android SDK**, **Android Virtual Device**, và **performance (Intel HAXM)** — nếu máy dùng CPU Intel; máy AMD/ARM sẽ có lựa chọn tương ứng cho tăng tốc ảo hoá.
3. Chọn thư mục cài đặt Android SDK (nhớ đường dẫn này — dùng ở bước 2.4). Mặc định thường là `C:\Users\<tên-bạn>\AppData\Local\Android\Sdk`.
4. Sau khi cài xong, Android Studio mở **Setup Wizard** lần đầu: chọn kiểu cài "Standard" — wizard sẽ tự tải về:
 - SDK Platform mới nhất ổn định (API level cao nhất hiện tại).
 - SDK Build-Tools tương ứng.
 - Một bản system image để tạo máy ảo (chi tiết ở Chương 3).
 - Android Emulator, Platform-Tools (chứa `adb`).

Bước này cần mạng ổn định — SDK component khá nặng (vài GB nếu tải nhiều API level).

2.3 SDK Manager — nơi quản lý mọi phiên bản Android

Mở qua **Tools** → **SDK Manager** trong Android Studio. Ba tab quan trọng:

- **SDK Platforms:** từng phiên bản Android (API level) bạn muốn build/test. Cài ít nhất phiên bản `targetSdkVersion` bạn dự định dùng (sách này khuyến nghị `compileSdk/targetSdk 34` — Android 14).
- **SDK Tools:** `Android SDK Build-Tools`, `Android SDK Platform-Tools` (chứa `adb`), `Android Emulator`, `Android SDK Command-line Tools`.
- **SDK Update Sites:** danh sách nguồn cập nhật, thường không cần đổi.

Mỗi API level cài thêm tốn khoảng 500MB–1.5GB — không cần cài hết tất cả, chỉ cài phiên bản bạn thật sự target và test.

2.4 Cấu hình biến môi trường (để dùng `adb` ngoài Android Studio)

Android Studio tự biết đường dẫn SDK, nhưng terminal ngoài IDE (PowerShell, Command Prompt) thì không, trừ khi bạn khai báo:

1. Mở **System Properties** → **Environment Variables** (gõ "environment variables" vào Windows Search).
2. Thêm biến hệ thống mới: `ANDROID_HOME` = đường dẫn SDK (vd `C:\Users\<tên-bạn>\AppData\Local\Android\Sdk`).
3. Thêm vào biến `Path` hai mục:
 - `%ANDROID_HOME%\platform-tools`

- `%ANDROID_HOME%\emulator`

4. Mở lại terminal (PowerShell) để biến môi trường có hiệu lực.

2.5 Ví dụ thực hành: xác nhận cài đặt

Sau khi cài xong, mở PowerShell và chạy lần lượt:

```
adb --version
```

Kết quả mong đợi: in ra phiên bản Android Debug Bridge, ví dụ `Android Debug Bridge version 1.0.41`.

```
sdkmanager --list-installed
```

(Nếu lệnh `sdkmanager` chưa nhận, dùng đường dẫn đầy đủ: `%ANDROID_HOME%\cmdline-tools\latest\bin\sdkmanager.bat --list-installed`.) Kết quả liệt kê các package SDK đã cài — dùng để đối chiếu khi gặp lỗi thiếu component.

Bài tập

1. Sau khi cài đặt, ghi lại: đường dẫn `ANDROID_HOME` trên máy bạn, API level bạn đã cài qua SDK Manager.
2. Chạy `adb --version` — nếu báo lỗi "not recognized", tự tra lại bước 2.4 và sửa cho đến khi chạy được (đây là bài tập bắt buộc — Chương 3 giả định `adb` đã dùng được).
3. Mở SDK Manager, thử cài thêm một API level cũ hơn (ví dụ API 26) — quan sát dung lượng tải về.

Lỗi thường gặp

- `'adb' is not recognized as an internal or external command`: biến môi trường `Path` chưa trở đúng tới `platform-tools`, hoặc terminal đang mở từ trước khi bạn cập nhật biến môi trường — đóng và mở lại terminal.
- **Setup Wizard tải rất chậm hoặc treo**: thường do mạng chặn domain tải SDK — thử đổi mạng, hoặc cấu hình proxy trong `Settings → Appearance & Behavior → System Settings → HTTP Proxy`.
- **Không thấy tùy chọn Intel HAXM khi cài**: máy dùng CPU AMD hoặc đã bật Hyper-V/WSL2 (Windows) — không sao, Chương 3 sẽ nói rõ ảnh hưởng của việc này tới tốc độ máy ảo và cách xử lý.
- **Cài nhiều API level "cho chắc"**: mỗi API level tốn dung lượng đáng kể — chỉ cài các phiên bản bạn thực sự cần test.

Tóm tắt & tiếp theo

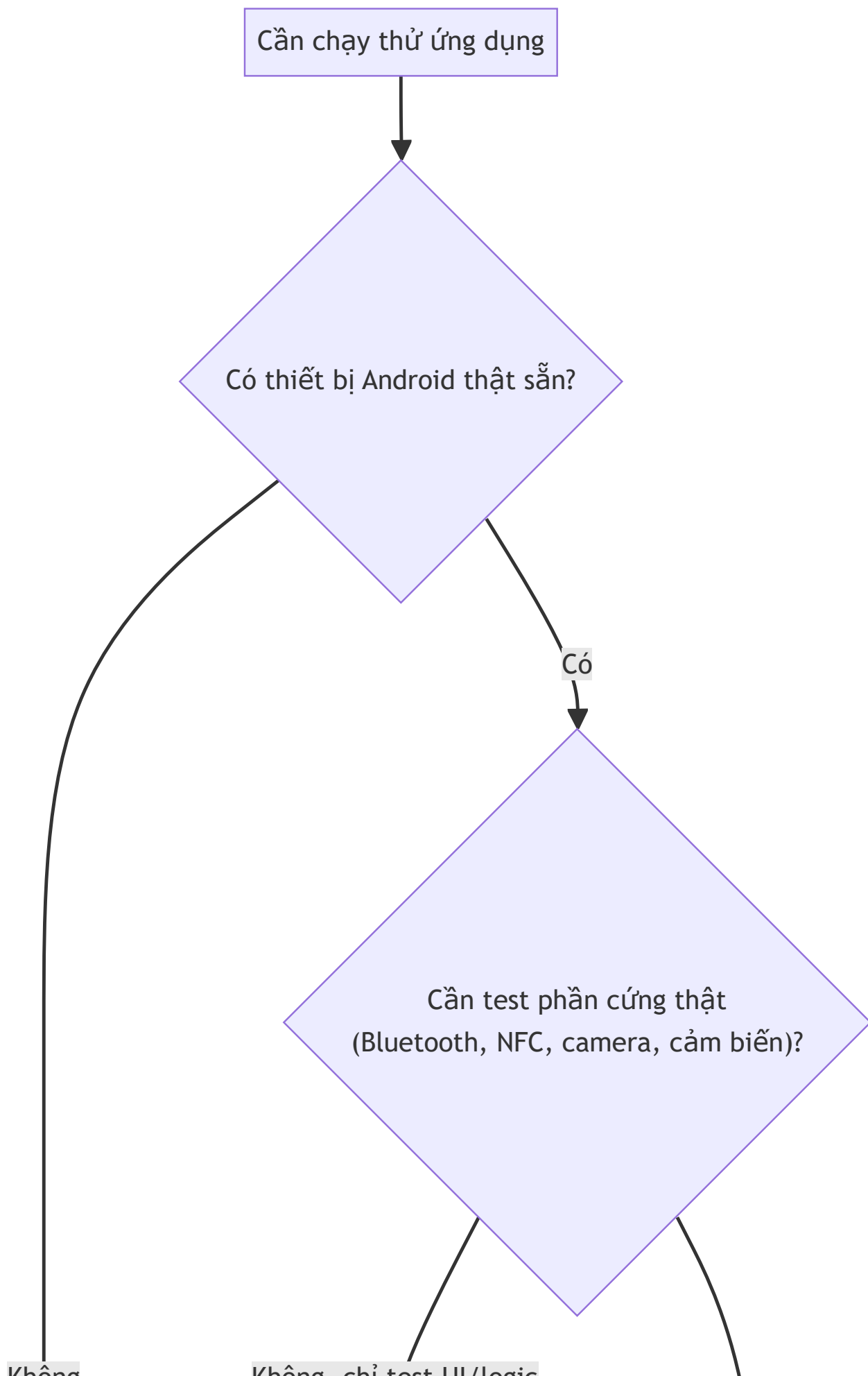
Bạn đã có Android Studio, SDK, và `adb` chạy được từ dòng lệnh. Chương 3 sẽ dùng đúng những thứ này để tạo máy ảo (AVD) và/hoặc kết nối thiết bị Android thật — chuẩn bị nơi để chạy ứng dụng đầu tiên ở Chương 4–5.

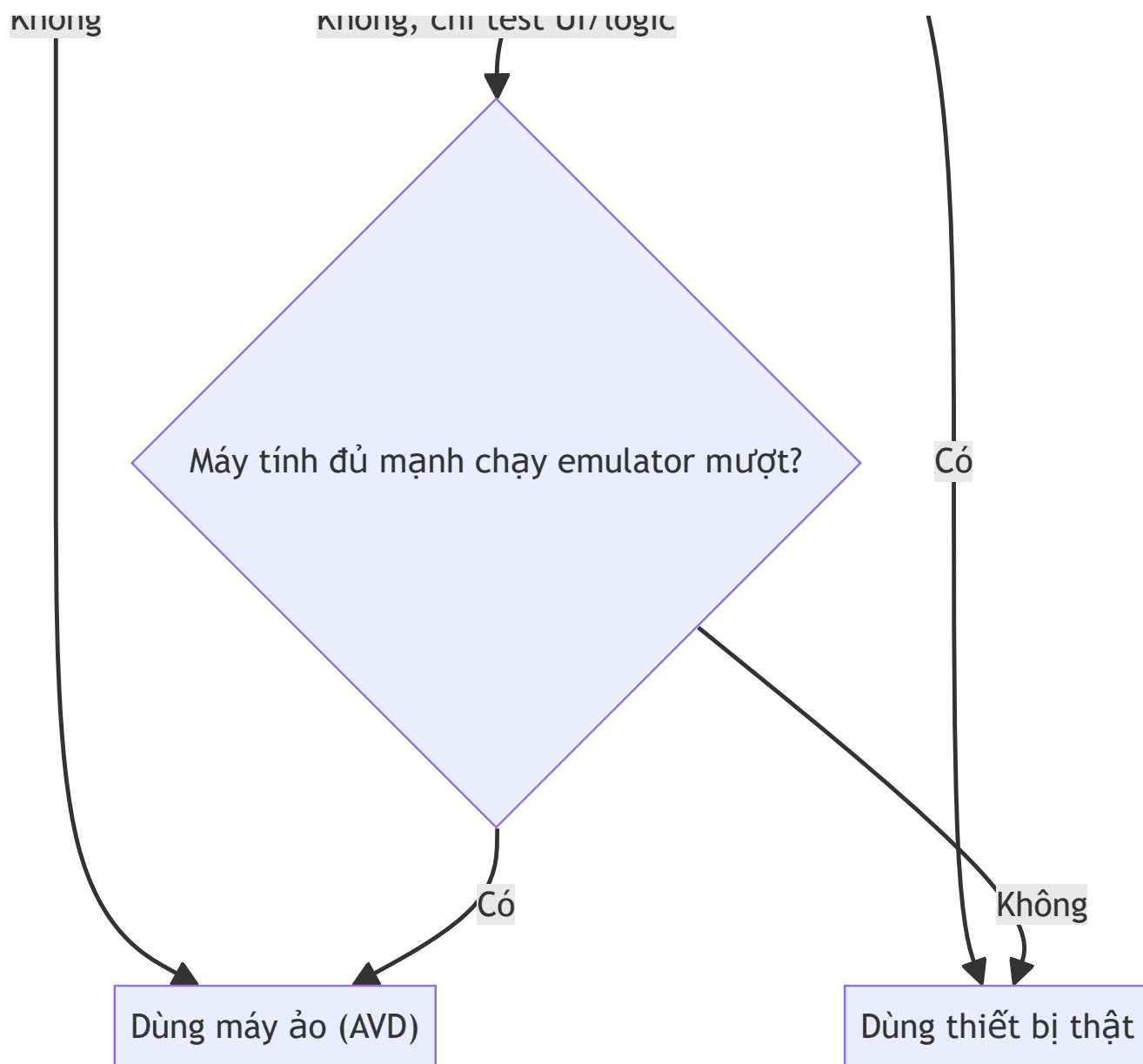
Chương 3: Tạo và cấu hình máy ảo (AVD), kết nối thiết bị thật

Mục tiêu học

- Tạo được một máy ảo Android (AVD) chạy được trong Android Studio.
- Bật Developer Options và USB debugging để kết nối thiết bị Android thật.
- Dùng được các lệnh `adb` cơ bản để kiểm tra thiết bị/máy ảo đang kết nối.
- Biết chọn máy ảo hay thiết bị thật tùy tình huống.

3.1 Máy ảo hay thiết bị thật?





Máy ảo tiện cho việc test nhanh, nhiều cấu hình màn hình/API level khác nhau mà không cần nhiều máy thật. Nhưng một số phần cứng (Bluetooth thật, NFC, cảm biến chuyển động thật) **bắt buộc phải test trên thiết bị thật** — nội dung này sẽ quan trọng trở lại ở Chương 35–36.

3.2 Tạo máy ảo (AVD)

1. Mở **Tools** → **Device Manager** trong Android Studio.
2. Chọn **Create Device**.
3. Chọn một **Device definition** (ví dụ Pixel 6) — quyết định kích thước màn hình, mật độ điểm ảnh.
4. Chọn **System Image** — đây là phiên bản Android sẽ chạy trong máy ảo:
 - Ưu tiên bản có nhãn **x86_64** (chạy nhanh hơn nhiều so với ARM trên máy Windows/Intel thông qua ảo hoá phần cứng).
 - Chọn API level trùng với `targetSdkVersion` bạn định dùng (Chương 2 đã cài).
5. Đặt tên AVD, có thể chỉnh thêm RAM/dung lượng lưu trữ nếu cần test app nặng.

6. Bấm **Finish** — lần đầu chạy máy ảo sẽ hơi chậm (khởi tạo), các lần sau nhanh hơn nhiều nhờ snapshot.

3.3 Thao tác cơ bản trên máy ảo

Khi máy ảo đã chạy, thanh công cụ bên cạnh cho phép:

- Xoay màn hình (rotate).
- Giả lập vị trí GPS (Extended controls → Location) — hữu ích khi học các chương liên quan tới vị trí (Chương 37).
- Giả lập cuộc gọi/SMS đến.
- Chụp ảnh màn hình, quay video màn hình.
- Giả lập mức pin yếu, mất mạng — để test app trong điều kiện bất lợi.

3.4 Kết nối thiết bị Android thật

1. Trên điện thoại: vào **Settings** → **About phone**, bấm liên tục 7 lần vào **Build number** để mở khóa **Developer options**.
2. Vào **Settings** → **System** → **Developer options**, bật **USB debugging**.
3. Cắm điện thoại vào máy tính bằng cáp USB (ưu tiên cáp hỗ trợ truyền dữ liệu, không phải cáp chỉ sạc).
4. Trên điện thoại sẽ hiện hộp thoại "Allow USB debugging?" kèm vân tay RSA của máy tính — chọn **Allow** (có thể tick "Always allow from this computer" để khỏi hỏi lại).
5. Xác nhận từ máy tính:

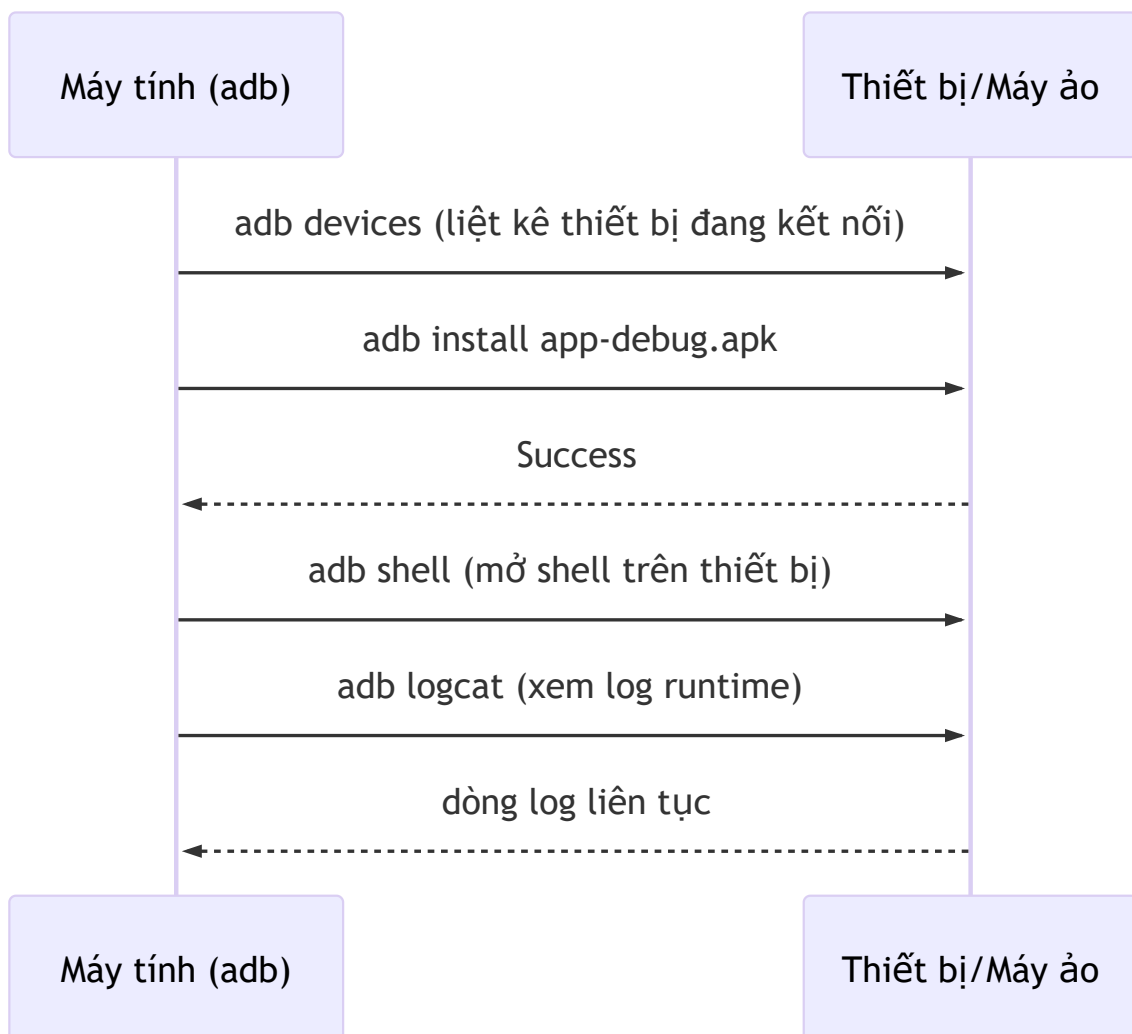
```
adb devices
```

Kết quả mong đợi:

```
List of devices attached
R58N60XXXXX    device
```

Nếu cột thứ hai hiện `unauthorized` thay vì `device`, quay lại điện thoại và xác nhận lại hộp thoại cho phép ở bước 4.

3.5 ADB cơ bản — bạn sẽ dùng lại nhiều lần trong sách



Vài lệnh dùng thường xuyên:

```
adb devices          # liệt kê thiết bị/máy ảo đang kết nối
adb install app.apk  # cài 1 file APK vào thiết bị đang chọn
adb uninstall vn.example.helloandroid # gỡ theo package name
adb logcat           # xem log runtime (Ctrl+C để dừng)
adb shell            # mở shell Linux trên thiết bị (gõ exit để thoát)
```

Nếu có nhiều thiết bị/máy ảo cùng kết nối, thêm `-s <device-id>` (lấy từ `adb devices`) vào trước lệnh để chỉ định đích.

Bài tập

1. Tạo một AVD Pixel 6, API level trùng với bản bạn đã cài ở Chương 2, khởi động thành công.
 2. Nếu có điện thoại Android thật: bật Developer options + USB debugging, chạy `adb devices` cho tới khi thấy trạng thái `device` (không phải `unauthorized` hay trống).
 3. Thử `adb shell`, gõ `exit` để thoát — xác nhận bạn vào được shell của máy ảo/thiết bị.
-

Lỗi thường gặp

- **Máy ảo chạy cực chậm / bị treo khi khởi động:** thường do chưa bật ảo hoá phần cứng (Intel VT-x/AMD-V) trong BIOS, hoặc xung đột với Hyper-V/WSL2 đang bật sẵn trên Windows. Cần nhắc dùng thiết bị thật nếu máy không hỗ trợ tốt.
 - `adb devices` **không thấy thiết bị thật:** thiếu driver USB (một số hãng máy Android cần driver riêng trên Windows), hoặc cáp chỉ hỗ trợ sạc không truyền dữ liệu.
 - **Thiết bị hiện `unauthorized`:** chưa xác nhận hộp thoại "Allow USB debugging" trên điện thoại, hoặc vân tay RSA cũ bị đổi — vào lại Developer options, chọn **Revoke USB debugging authorizations**, rút và cắm lại cáp.
 - **Máy ảo mất kết nối mạng bên trong:** thường do phần mềm VPN/firewall trên máy tính chặn — tắt thử VPN rồi khởi động lại máy ảo.
-

Tóm tắt & tiếp theo

Bạn đã có nơi để chạy ứng dụng: một máy ảo và/hoặc một thiết bị thật đã kết nối qua `adb`. Chương 4 sẽ tạo project Android đầu tiên và giải thích từng phần trong cấu trúc project đó.

Chương 4: Tạo project đầu tiên — cấu trúc project, Gradle, AndroidManifest.xml

Mục tiêu học

- Tạo được một project Android mới trong Android Studio.
- Đọc hiểu cấu trúc thư mục mặc định của project — biết file nào để làm gì.
- Hiểu vai trò của `build.gradle` (cấp project và cấp module) trong việc build ứng dụng.
- Đọc hiểu sâu hơn `AndroidManifest.xml` so với cái nhìn sơ bộ ở Chương 1.

Code mẫu đầy đủ của chương này (và Chương 5) nằm ở `code/ch04-05-hello-world/` trong repo — mở bằng Android Studio để chạy trực tiếp thay vì gõ lại từng file.

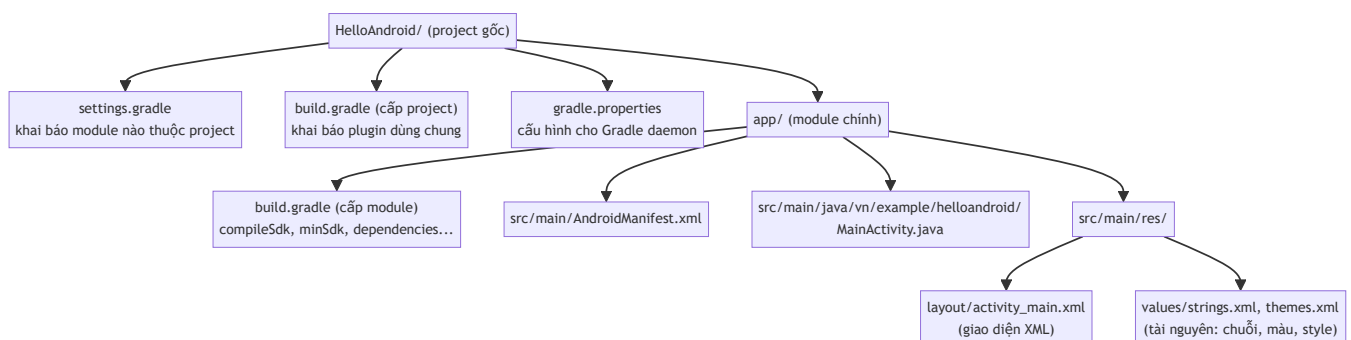
4.1 Tạo project mới

Trong Android Studio: **File** → **New** → **New Project** → **Empty Views Activity**, sau đó điền:

- **Name:** `HelloAndroid`
- **Package name:** `vn.example.helloandroid` (định danh duy nhất, xem lại 1.5)
- **Language:** Java
- **Minimum SDK:** API 24 trở lên (đủ để dùng gần hết API hiện đại mà vẫn tương thích máy cũ)

Android Studio sinh ra project theo đúng cấu trúc chuẩn dưới đây.

4.2 Cấu trúc thư mục project



Ba điều quan trọng cần phân biệt ngay:

1. **build.gradle cấp project** (nằm ở thư mục gốc) khai báo plugin dùng chung cho toàn bộ project — bạn hiếm khi sửa file này.

2. **build.gradle** **cấp module** (`app/build.gradle`) mới là nơi bạn thường sửa: thêm dependency (thư viện), đổi `minSdk` / `targetSdk`, cấu hình build type (debug/release).
3. **settings.gradle** khai báo project gồm những module nào (`include ':app'`) — một app lớn có thể tách nhiều module, sách này dùng 1 module `app` xuyên suốt cho đơn giản.

4.3 Đọc `app/build.gradle`

```
android {  
    namespace 'vn.example.helloandroid'  
    compileSdk 34 // phiên bản SDK dùng để BIÊN DỊCH code  
  
    defaultConfig {  
        applicationId "vn.example.helloandroid" // ID duy nhất khi phát hành (Chương 39)  
        minSdk 24 // phiên bản THẤP NHẤT máy người dùng cần  
        có  
        targetSdk 34 // phiên bản bạn đã test hành vi  
        versionCode 1 // số nguyên tăng dần mỗi lần phát hành  
        versionName "1.0" // chuỗi hiển thị cho người dùng  
    }  
}  
  
dependencies {  
    implementation 'androidx.appcompat:appcompat:1.7.0'  
    implementation 'com.google.android.material:material:1.12.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
}
```

`compileSdk` khác `minSdk` / `targetSdk` — đây là điểm hay gây nhầm cho người mới:

- `compileSdk`: bộ API bạn được PHÉP DÙNG khi viết code (luôn nên là bản mới nhất bạn có).
- `minSdk` / `targetSdk`: hành vi RUNTIME trên máy người dùng (đã nói ở 1.3).

Phần `dependencies` khai báo thư viện ngoài — cú pháp `implementation 'group:artifact:version'`. Các chương sau (Room, Retrofit, Hilt...) đều thêm dòng vào đúng khối này.

4.4 Đọc lại `AndroidManifest.xml` — sâu hơn Chương 1

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@android:drawable/sym_def_app_icon"
        android:label="@string/app_name"
        android:theme="@style/Theme.HelloAndroid">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Chú ý `android:exported="true"` trên `<activity>` — **bắt buộc phải khai báo rõ từ Android 12 (API 31)** cho bất kỳ Activity/Service/Receiver nào có `<intent-filter>`, nếu không app sẽ không build được. Đây là ví dụ cụ thể của việc "API level ảnh hưởng tới cách bạn viết manifest" đã nói ở Chương 1.

`android:label` và các chuỗi hiển thị không viết cứng trong manifest mà tham chiếu tới `@string/...` — định nghĩa trong `res/values/strings.xml`. Đây là quy ước bắt buộc cho đa ngôn ngữ (chi tiết Chương 11), nên làm quen từ bây giờ thay vì viết chuỗi cứng.

Bài tập

1. Mở code mẫu `code/ch04-05-hello-world/` bằng Android Studio, để Gradle sync xong (chưa cần Run).
2. Trong `app/build.gradle`, thử đổi `minSdk` xuống 21 rồi sync lại — quan sát Android Studio có cảnh báo API nào không dùng được ở API 21 không (thử nếu có dùng API mới trong code, ở đây có thể chưa thấy vì code còn đơn giản).
3. Mở `AndroidManifest.xml`, thử xóa `android:exported="true"` rồi build — đọc thông báo lỗi Gradle đưa ra, đối chiếu với giải thích ở mục 4.4.

Lỗi thường gặp

- **"SDK location not found"**: thiếu file `local.properties` (Android Studio tự tạo khi mở project lần đầu, không nên xoá hay commit file này lên git vì nó chứa đường dẫn máy cá nhân).
- **Gradle sync failed do version plugin không khớp**: phiên bản Android Gradle Plugin (khai báo ở `build.gradle` cấp project) cần tương thích với phiên bản Android Studio đang dùng — Android Studio thường tự đề xuất bản phù hợp khi mở project cũ.
- **Quên `android:exported`**: build lỗi ngay với thông báo rõ ràng tên Activity thiếu khai báo — đọc kỹ thông báo lỗi Gradle thay vì đoán, phần lớn lỗi manifest đều chỉ đúng dòng.

Tóm tắt & tiếp theo

Bạn đã hiểu cấu trúc project, vai trò từng file Gradle, và đọc sâu hơn `AndroidManifest.xml`. Chương 5 sẽ dùng chính project này: sửa giao diện, chạy lên máy ảo/thiết bị thật, và build APK bằng dòng lệnh.

Chương 5: Build & deploy ứng dụng "Hello World" đầu tiên

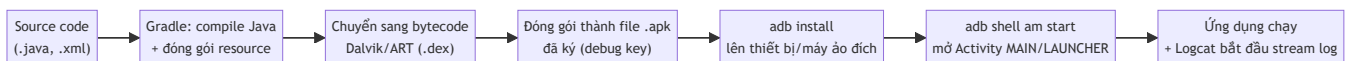
Mục tiêu học

- Chạy được ứng dụng từ Chương 4 lên máy ảo hoặc thiết bị thật qua Android Studio.
- Hiểu được pipeline build & deploy diễn ra bên dưới nút **Run**.
- Build được file APK bằng dòng lệnh (`gradlew`) và cài thử công bằng `adb`.
- Đọc được Logcat để quan sát ứng dụng đang chạy.

5.1 Chạy ứng dụng qua Android Studio

1. Mở lại `code/ch04-05-hello-world/` (hoặc project bạn tự tạo ở Chương 4).
2. Ở thanh công cụ, chọn thiết bị đích: máy ảo đã tạo (Chương 3) hoặc thiết bị thật đã kết nối.
3. Bấm **Run** ► (hoặc Shift+F10).
4. Chờ build xong — ứng dụng tự mở trên thiết bị đích, hiển thị dòng chữ "Xin chào, Android!" (định nghĩa ở `strings.xml`, hiển thị qua `activity_main.xml`).

5.2 Điều gì xảy ra khi bạn bấm Run?



Nút **Run** trong Android Studio thực chất chỉ là giao diện gói gọn lại đúng các bước trên — mọi bước đều có thể tự làm bằng dòng lệnh, như mục 5.3 dưới đây. Hiểu rõ pipeline này giúp bạn tự debug khi Android Studio báo lỗi mơ hồ, vì bạn biết lỗi đang xảy ra ở bước nào (compile? đóng gói? cài đặt? khởi chạy?).

5.3 Build & cài bằng dòng lệnh

Từ thư mục project (PowerShell):

```
.\gradlew.bat assembleDebug
```

Lệnh này chạy đúng 3 bước đầu của sơ đồ trên (compile → dex → đóng gói APK), không cần Android Studio. Kết quả nằm ở:

```
app\build\outputs\apk\debug\app-debug.apk
```

Cài thủ công lên thiết bị/máy ảo đang kết nối (Chương 3):

```
adb install app\build\outputs\apk\debug\app-debug.apk
```

Mở ứng dụng thủ công bằng `adb` (không cần chạm vào màn hình):

```
adb shell am start -n vn.example.helloandroid/.MainActivity
```

5.4 Đọc Logcat

Logcat là dòng log runtime của toàn hệ thống Android, lọc theo ứng dụng của bạn để dễ theo dõi:

```
adb logcat --pid=$(adb shell pidof -s vn.example.helloandroid)
```

Trong Android Studio, tab **Logcat** ở dưới màn hình làm việc này tự động, có ô lọc theo package name hoặc theo mức log (Verbose/Debug/Info/Warn/Error). Thói quen nên có: khi app "không có phản ứng gì", luôn mở Logcat trước khi đoán nguyên nhân.

Bài tập

1. Sửa `strings.xml` trong code mẫu, đổi `hello_message` thành một câu khác, Run lại — quan sát Android Studio chỉ build lại phần cần thiết (nhanh hơn lần đầu).
2. Build APK bằng `gradlew assembleDebug`, cài bằng `adb install`, mở app bằng lệnh `adb shell am start` ở mục 5.3 — không chạm tay vào thiết bị.
3. Cố tình gây lỗi (ví dụ sửa sai tên resource trong `activity_main.xml`), Run lại, đọc thông báo lỗi Gradle và tự sửa.

Lỗi thường gặp

- **INSTALL_FAILED_UPDATE_INCOMPATIBLE**: máy đã cài một bản app cùng `applicationId` nhưng ký bằng key khác (thường do trước đó cài bản build từ máy/IDE khác) — gỡ bản cũ (`adb uninstall vn.example.helloandroid`) rồi cài lại.
- **App cài xong nhưng bấm icon không mở được / crash ngay khi mở**: mở Logcat, tìm dòng `FATAL EXCEPTION` — stack trace luôn chỉ đúng dòng code gây lỗi.

- `gradlew.bat` báo **"not recognized"** hoặc **thiếu file**: thư mục project chưa có Gradle wrapper — mở project bằng Android Studio một lần để IDE tự sinh wrapper, hoặc chạy `gradle wrapper` nếu đã cài Gradle riêng.
 - **Build rất chậm ở lần chạy đầu tiên**: bình thường — Gradle tải và cache dependency lần đầu; các lần sau nhanh hơn đáng kể nhờ cache.
-

Tóm tắt & tiếp theo

Bạn đã hoàn thành cột mốc đầu tiên: viết, build, và chạy thành công một ứng dụng Android thật, cả qua Android Studio lẫn dòng lệnh thuần. Đây cũng là điểm kết thúc **Phần 0 — Chuẩn bị môi trường**. Chương 6 bắt đầu **Phần 1 — Nền tảng**, đi sâu vào vòng đời Activity mà Chương 1 mới chỉ giới thiệu sơ bộ.

Phần 1 — Nền tảng

Chương 6: Vòng đời Activity & Application

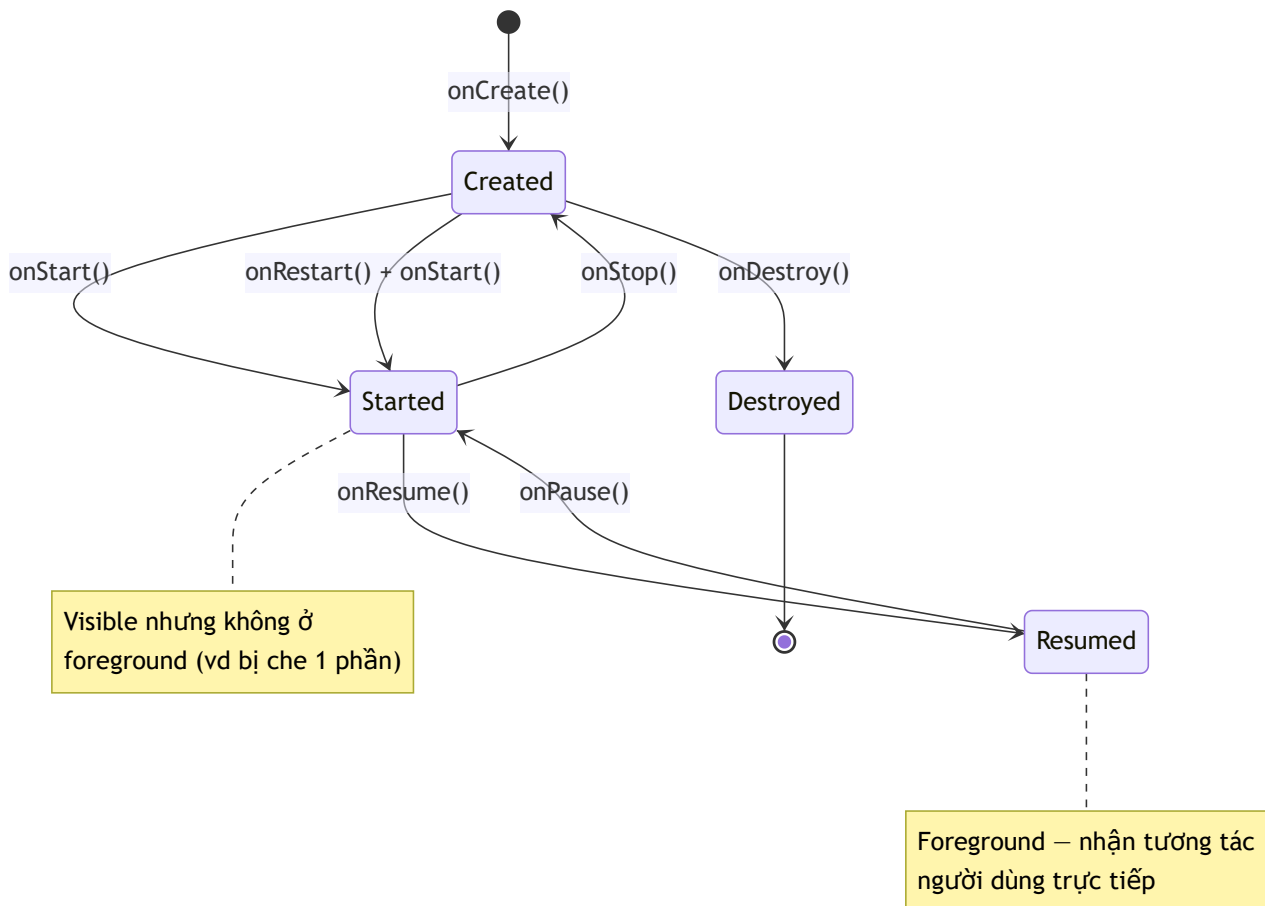
Mục tiêu học

- Gọi tên chính xác và đúng thứ tự các callback vòng đời của Activity.
- Phân biệt "activity bị che khuất tạm thời" với "activity bị huỷ hẳn" — và biết dùng đúng callback cho từng tình huống.
- Hiểu vì sao xoay màn hình lại tạo lại (recreate) Activity, và cách giữ dữ liệu qua `onSaveInstanceState`.
- Biết `Application` class dùng để làm gì và khi nào nên dùng.

Code mẫu: `code/ch06-lifecycle-demo/` — mở bằng Android Studio, chạy và đọc Logcat song song với chương này sẽ dễ hiểu hơn nhiều so với chỉ đọc.

6.1 Sơ đồ đầy đủ

Chương 1 đã cho xem sơ đồ rút gọn. Đây là bản đầy đủ với tên callback thật:



Vài quy tắc cần nhớ, không cần học vẹt tên hàm:

- `onPause()` **luôn được gọi trước khi activity mất hoàn toàn tương tác** — kể cả khi chỉ bị che một phần (ví dụ có dialog nổi lên). Code trong `onPause()` phải cực nhanh, vì hệ thống có thể đang chờ nó xong để cho activity khác chạy tiếp.
- `onStop()` **nghĩa là activity không còn hiển thị chút nào**, nhưng vẫn còn tồn tại trong bộ nhớ — bấm nút Home rồi quay lại app sẽ thấy `onRestart()` → `onStart()` → `onResume()`, KHÔNG chạy lại `onCreate()`.
- `onDestroy()` **không phải lúc nào cũng được gọi** trước khi hệ thống giết tiến trình (Android có quyền kill tiến trình thẳng tay khi thiếu bộ nhớ, không đảm bảo gọi `onDestroy()`). Vì vậy đừng đặt logic "phải chạy" (như lưu dữ liệu quan trọng) vào `onDestroy()` — dùng `onPause()` / `onStop()` cho việc đó.

6.2 Configuration change: vì sao xoay màn hình lại "chạy lại app"?

Khi xoay màn hình, đổi ngôn ngữ hệ thống, hoặc bật/tắt chế độ tối, mặc định Android **huỷ và tạo lại Activity** — vì layout/resource phù hợp có thể khác (ví dụ layout riêng cho màn ngang, string riêng cho ngôn ngữ mới). Thứ tự:

```
onPause() → onStop() → onSaveInstanceState() → onDestroy() → onCreate() → onStart() → onResume()
```

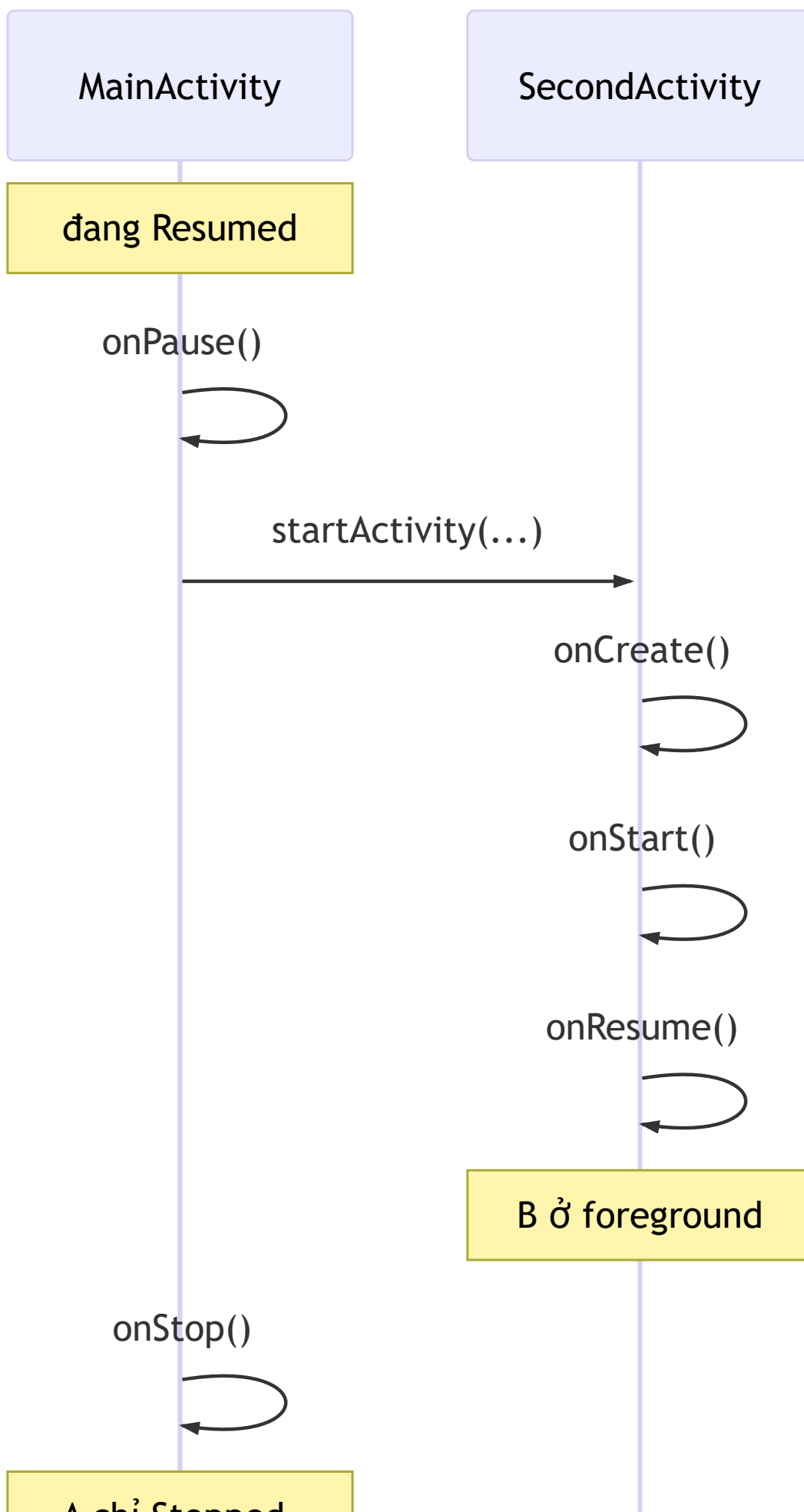
`onSaveInstanceState(Bundle outState)` là nơi bạn lưu tạm dữ liệu UI (ví dụ nội dung đang gõ dở, vị trí cuộn) vào `Bundle`, để đọc lại trong `onCreate(Bundle savedInstanceState)` ngay sau đó. Đây **không phải chỗ lưu dữ liệu lâu dài** — chỉ sống sót qua configuration change, mất hẳn nếu người dùng thoát app hẳn (xem Chương 12–13 cho lưu trữ thật sự).

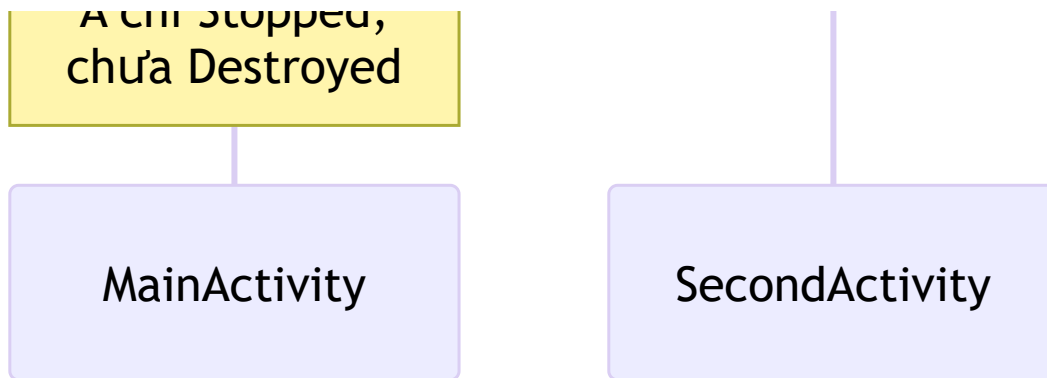
Code mẫu minh họa đúng cặp này:

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("counter", ++counter);
}

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    if (savedInstanceState != null) {
        counter = savedInstanceState.getInt("counter", 0);
    }
    ...
}
```

6.3 Hai Activity cùng lúc: ai onPause trước, ai onResume sau?





Thứ tự quan trọng cần nhớ: `A.onPause()` **chạy xong TRƯỚC KHI** `B.onResume()` **được gọi**, nhưng `A.onStop()` chỉ chạy SAU KHI `B` đã lên foreground. Đây là lý do việc chuyển màn hình mượt — activity cũ chưa "biến mất" ngay, nó chỉ dừng hẳn sau khi activity mới đã sẵn sàng.

6.4 Application class — chạy trước cả Activity đầu tiên

```
public class DemoApplication extends Application {
    @Override
    public void onCreate() {
        super.onCreate();
        // Khởi tạo thư viện dùng chung toàn app ở đây
    }
}
```

Khai báo trong manifest bằng `android:name=".DemoApplication"` trên thẻ `<application>`. `Application.onCreate()` chạy đúng **một lần** khi tiến trình app khởi tạo — trước cả `MainActivity.onCreate()`. Dùng để khởi tạo thư viện toàn app (ví dụ crash reporting ở Chương 40), **không dùng để làm việc nặng** vì nó chặn toàn bộ quá trình khởi động app.

Bài tập

1. Chạy code mẫu, mở Logcat lọc tag `Lifecycle`. Bấm nút mở màn hình 2, quan sát đúng thứ tự ở sơ đồ 6.3.
2. Bấm Home rồi mở lại app từ danh sách app gần đây — xác nhận `onCreate()` KHÔNG chạy lại, chỉ có `onRestart()/onStart()/onResume()`.
3. Nếu máy ảo bật được auto-rotate: xoay ngang rồi xoay dọc lại — quan sát toàn bộ chuỗi callback ở mục 6.2, và giá trị `counter` được giữ nguyên nhờ `onSaveInstanceState`.

Lỗi thường gặp

- **Giữ tham chiếu Activity trong biến `static`**: gây rò rỉ bộ nhớ (memory leak) vì Activity không bao giờ được giải phóng dù đã `onDestroy()`. Nếu cần truyền dữ liệu sống lâu hơn Activity, dùng `Application` hoặc kiến trúc `ViewModel` (Chương 28).
- **Làm việc nặng (đọc file lớn, gọi mạng đồng bộ) trong `onCreate()`**: làm chậm hoặc treo màn hình khởi động — chuyển sang bất đồng bộ (Chương 19).
- **Tương `onDestroy()` luôn chạy để "dọn dẹp"**: sai — hệ thống có thể kill tiến trình mà không gọi `onDestroy()`. Dọn tài nguyên quan trọng (đóng kết nối, huỷ listener) nên đặt ở `onPause()` / `onStop()`.
- **Quên gọi `super.onXxx()`**: hầu hết callback vòng đời bắt buộc gọi `super` trước, quên sẽ gây `SuperNotCalledException` khi build debug.

Tóm tắt & tiếp theo

Vòng đời Activity là nền tảng chi phối gần như mọi chương sau — từ chỗ lưu dữ liệu, chạy tác vụ nền, đến khi nào an toàn để cập nhật UI. Chương 7 tạm rời vòng đời để tập trung vào layout và các loại View — thứ bạn nhìn thấy trực tiếp trên màn hình.

Chương 7: Layout & Views (XML layout, ConstraintLayout, ViewGroup)

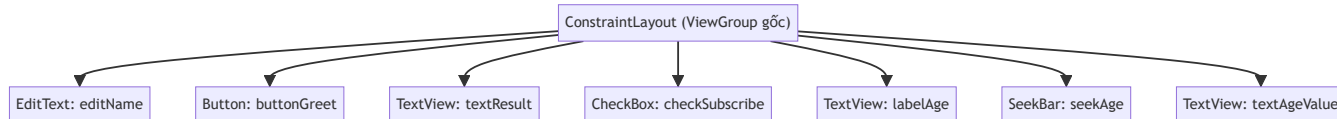
Mục tiêu học

- Phân biệt `View` và `ViewGroup`, hiểu cây phân cấp (view hierarchy) của một layout.
- Dùng thành thạo `ConstraintLayout` — layout mặc định và khuyến nghị hiện nay.
- Đọc hiểu vì sao `findViewById` dễ gây lỗi runtime, và chuyển sang dùng `ViewBinding`.
- Biết một số View cơ bản: `TextView`, `EditText`, `Button`, `CheckBox`, `SeekBar`.

Code mẫu: `code/ch07-layout-views/`.

7.1 View và ViewGroup

Mọi thứ hiển thị trên màn hình Android đều là một `View`. `ViewGroup` là một `View` đặc biệt có thể chứa các `View` khác bên trong (kể cả `ViewGroup` khác) — tạo thành cây phân cấp:



Cây càng sâu (ViewGroup lồng nhiều tầng), hệ thống càng tốn công đo đạc/vẽ lại (`measure` / `layout` pass) mỗi khi màn hình cập nhật — đây là lý do `ConstraintLayout` ra đời: cho phép tạo layout **phẳng** (ít lồng tầng) mà vẫn định vị được các View phức tạp tương đối với nhau.

7.2 ConstraintLayout — định vị bằng ràng buộc

Thay vì lồng nhiều `LinearLayout`, `ConstraintLayout` định vị mỗi View bằng các **ràng buộc** (**constraint**) tới cạnh của View khác hoặc của layout cha:

```
<Button
    android:id="@+id/buttonGreet"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toBottomOf="@id/editName"
    app:layout_constraintStart_toStartOf="parent" />
```

Đọc thành lời: "cạnh trên của `buttonGreet` nằm dưới cạnh dưới của `editName`; cạnh trái của `buttonGreet` thẳng với cạnh trái của layout cha". Bốn hướng ràng buộc cơ bản: `Top`, `Bottom`, `Start`, `End` (dùng `Start` / `End` thay vì `Left` / `Right` để tự động đổi chiều cho ngôn ngữ viết phải-sang-trái).

Quy tắc thực dụng: mỗi View cần **ít nhất một ràng buộc theo chiều ngang và một theo chiều dọc** để có vị trí xác định — thiếu sẽ khiến View "nhảy" về góc trên-trái (0,0) lúc chạy thật dù Preview trong Android Studio có thể vẫn hiển thị đúng.

7.3 `findViewById` và vì sao nên chuyển sang ViewBinding

Cách truyền thống:

```
TextView textResult = findViewById(R.id.textResult);
textResult.setText(" ... ");
```

Vấn đề: `findViewById` trả kiểu `View` (phải tự ép kiểu), và nếu bạn gõ sai `R.id.textResultt`, lỗi chỉ lộ ra **lúc chạy** (`NullPointerException`), không phải lúc biên dịch.

ViewBinding giải quyết cả hai: bật trong `app/build.gradle`:

```
android {
    buildFeatures {
        viewBinding true
    }
}
```

Gradle tự sinh ra class `ActivityMainBinding` (tên suy từ `activity_main.xml`) với field cho mỗi View có `android:id`, đúng kiểu sẵn:

```
private ActivityMainBinding binding;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    binding.buttonGreet.setOnClickListener(v → { ... }); // gõ sai sẽ báo lỗi compile
}
```

Từ chương này trở đi, sách dùng ViewBinding mặc định cho mọi ví dụ có giao diện.

7.4 Một vài View cơ bản dùng trong code mẫu

View	Vai trò	Sự kiện thường dùng
<code>TextView</code>	Hiển thị chữ (không cho sửa)	—
<code>EditText</code>	Ô nhập liệu	đọc qua <code>.getText().toString()</code>
<code>Button</code>	Nút bấm	<code>setOnClickListener</code>
<code>CheckBox</code>	Chọn có/không	<code>setOnCheckedChangeListener</code>
<code>SeekBar</code>	Thanh trượt chọn giá trị số	<code>setOnSeekBarChangeListener</code>

Ví dụ đọc dữ liệu người dùng nhập và phản hồi lại — mẫu lặp lại rất nhiều lần trong các chương sau:

```
binding.buttonGreet.setOnClickListener(v → {
    String name = binding.editName.getText().toString().trim();
    binding.textResult.setText(name.isEmpty()
        ? getString(R.string.result_empty_name)
        : getString(R.string.result_greeting, name));
});
```

`getString(R.string.result_greeting, name)` minh họa chuỗi có tham số (`%1$s` trong `strings.xml`) — cách chuẩn để ghép chuỗi có dữ liệu động mà vẫn giữ được khả năng dịch đa ngôn ngữ (Chương 11), thay vì nối chuỗi bằng `+`.

Bài tập

1. Chạy code mẫu, thử nhập tên rỗng rồi bấm Chào — xác nhận đúng thông báo lỗi hiển thị.
2. Thêm một `TextView` mới hiển thị trạng thái của `CheckBox` ("Đã đăng ký" / "Chưa đăng ký"), cập nhật qua `setOnCheckedChangeListener`.
3. Thử xóa ràng buộc `app:layout_constraintStart_toStartOf="parent"` của `buttonGreet`, build lại và quan sát vị trí nút bị lệch — đối chiếu với cảnh báo ở mục 7.2.

Lỗi thường gặp

- **Unresolved reference khi dùng ViewBinding:** quên bật `viewBinding true` trong `build.gradle`, hoặc build cache cũ — thử **Build** → **Clean Project** rồi build lại.
- **View "biến mất" hoặc nằm sai vị trí dù XML trông đúng:** thiếu ràng buộc theo một chiều (ngang hoặc dọc) — xem lại mục 7.2.
- **Lồng quá nhiều LinearLayout bên trong nhau:** build được nhưng vẽ chậm hơn — cân nhắc chuyển sang `ConstraintLayout` phẳng nếu layout phức tạp.

- Gọi `.getText().toString()` khi `EditText` là `null`: xảy ra khi gọi trước `setContentView / binding.inflate` — luôn đảm bảo binding đã khởi tạo trước khi truy cập.
-

Tóm tắt & tiếp theo

Bạn đã biết dựng giao diện bằng `ConstraintLayout`, thao tác với View qua `ViewBinding` an toàn hơn `findViewById`. Chương 8 sẽ dùng chính những kỹ năng này để xây 2 màn hình và học cách điều hướng, truyền dữ liệu giữa chúng bằng `Intent`.

Chương 8: Sự kiện, Intent (explicit/implicit), điều hướng giữa các màn hình

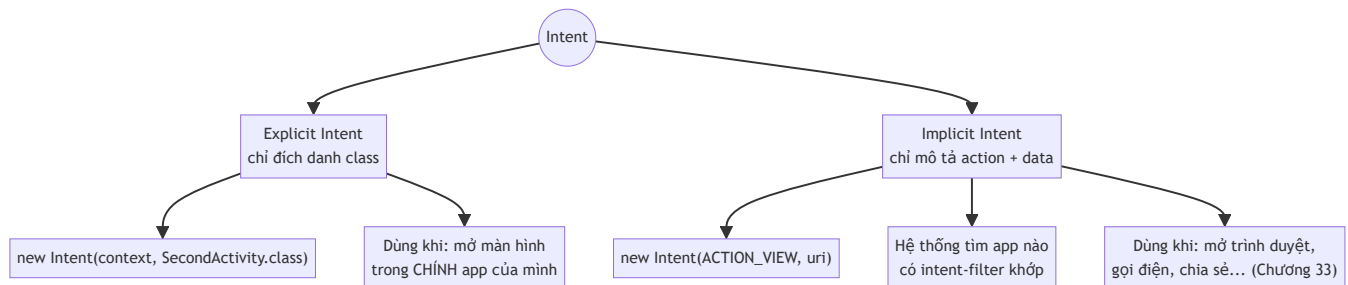
Mục tiêu học

- Phân biệt explicit Intent (mở màn hình cụ thể trong app mình) và implicit Intent (nhờ hệ thống tìm app phù hợp xử lý).
- Truyền dữ liệu giữa hai Activity, và nhận lại kết quả bằng Activity Result API.
- Hiểu vì sao `startActivityForResult()` cũ đã lỗi thời và không nên dùng trong code mới.

Code mẫu: `code/ch08-intent-navigation/`.

8.1 Intent là gì?

`Intent` là một "thông điệp" mô tả một hành động cần thực hiện — có thể là mở một Activity, khởi động một Service (Chương 16), hoặc gửi broadcast (Chương 17). Có hai kiểu:



8.2 Explicit Intent — truyền dữ liệu bằng `putExtra`

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra(EXTRA_MESSAGE, message);
startActivity(intent);
```

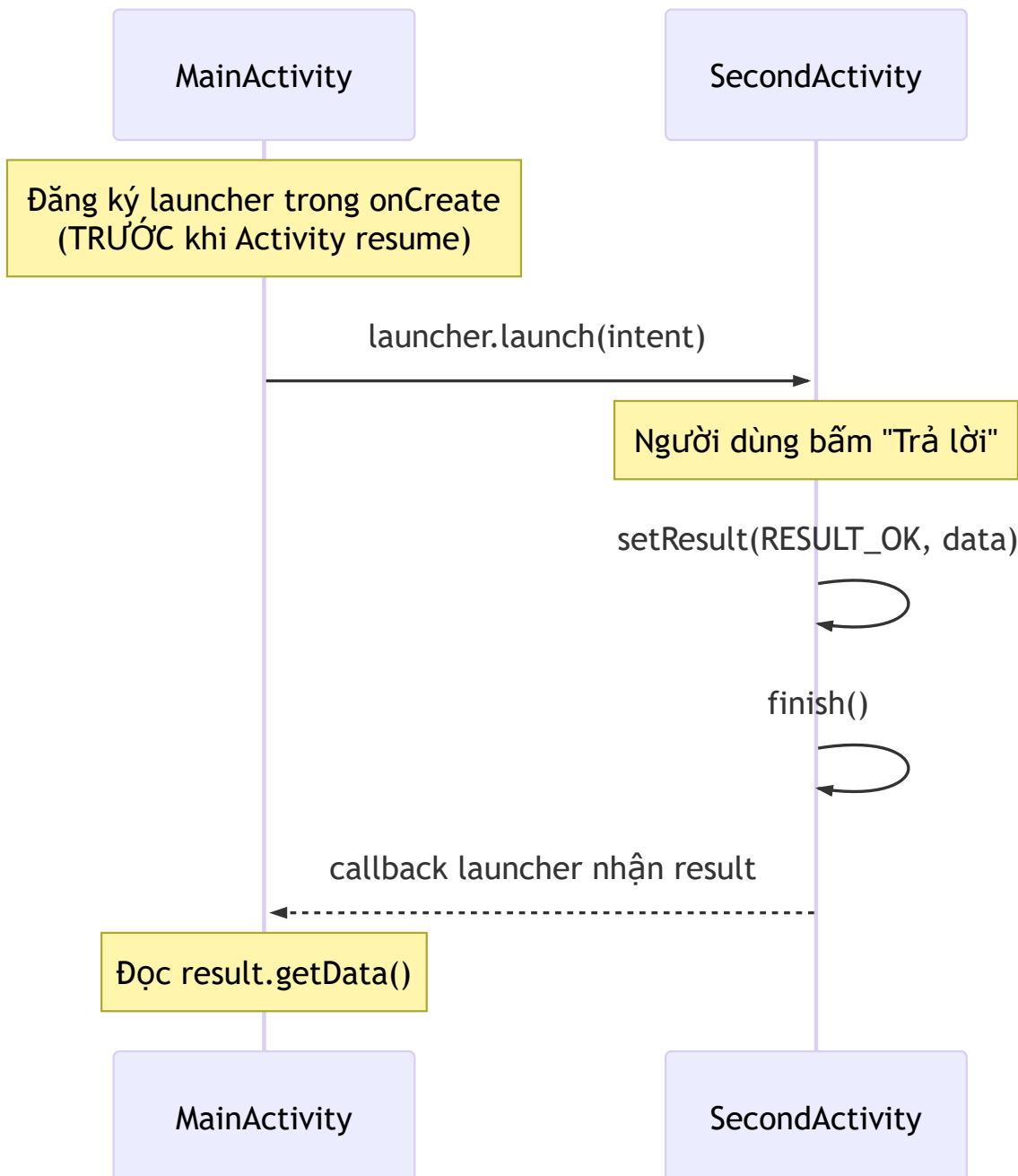
Phía nhận đọc lại bằng `getIntent()`:

```
String message = getIntent().getStringExtra(EXTRA_MESSAGE);
```

Lưu ý dùng **hằng số** `public static final String` cho key của extra (như `EXTRA_MESSAGE` trong code mẫu) thay vì viết chuỗi `"message"` trực tiếp ở cả hai nơi — tránh gõ sai key giữa Activity gửi và nhận, một lỗi runtime rất khó phát hiện vì không có cảnh báo lúc build.

8.3 Nhận kết quả trả về — Activity Result API

Tài liệu cũ hay dạy `startActivityForResult()` + override `onActivityResult()` — cách này **đã lỗi thời**, Google khuyến nghị thay bằng Activity Result API:



```
private final ActivityResultLauncher<Intent> secondActivityLauncher =
    registerForActivityResult(new ActivityResultContracts.StartActivityForResult(),
result → {
    if (result.getResultCode() == RESULT_OK && result.getData() ≠ null) {
        String reply = result.getData().getStringExtra(EXTRA_REPLY);
        // cập nhật UI với reply
    }
});
```

`registerForActivityResult( ... )` **phải gọi ở cấp field hoặc trong `onCreate` trước khi Activity resume** — gọi trễ hơn (ví dụ trong `onClickListener`) sẽ ném lỗi runtime. Đây là điểm khác biệt quan trọng nhất so với cách cũ, và cũng là lỗi người mới hay gặp nhất khi mới chuyển qua API này.

Phía `SecondActivity` trả kết quả:

```
Intent result = new Intent();
result.putExtra(EXTRA_REPLY, "Đã nhận, cảm ơn!");
setResult(RESULT_OK, result);
finish();
```

8.4 Implicit Intent — nhờ hệ thống tìm app xử lý

```
Intent intent = new Intent(Intent.ACTION_VIEW,
Uri.parse("https://developer.android.com"));
startActivity(intent);
```

Bạn không cần biết (và không nên quan tâm) máy người dùng cài trình duyệt nào — hệ thống tự tìm app có khai báo `<intent-filter>` khớp với `ACTION_VIEW` + scheme `https`. Nếu nhiều app cùng khớp, hệ thống hiện hộp thoại "Open with" cho người dùng chọn. Cơ chế này sẽ quay lại chi tiết hơn ở Chương 33 khi bàn về App Links và chia sẻ dữ liệu giữa các app.

Bài tập

1. Chạy code mẫu, thử luồng gửi tin nhắn → nhận phản hồi đầy đủ như mô tả ở README.
2. Thêm một extra thứ hai (ví dụ giờ gửi tin, dùng `putExtra` với kiểu `long`), hiển thị nó ở `SecondActivity`.
3. Đổi implicit intent ở mục 8.4 sang `Intent.ACTION_DIAL` với `Uri.parse("tel:0123456789")` — quan sát hệ thống mở app quay số thay vì trình duyệt.

Lỗi thường gặp

- **Gọi `registerForActivityResult` bên trong `onClickListener`**: ném `IllegalStateException`, phải đăng ký ở cấp `field/ onCreate` như mục 8.3.
- **Quên `android:exported="false"` cho Activity không cần app khác gọi tới** (như `SecondActivity` trong code mẫu) — từ Android 12 phải khai báo rõ giá trị này, không có mặc định ngầm.
- **Đọc `getStringExtra()` mà không kiểm tra `null`**: nếu Activity được mở theo cách khác (không qua đúng luồng bạn nghĩ), extra có thể không tồn tại — luôn có nhánh xử lý `null`.
- **Implicit intent không tìm được app xử lý**: gây crash `ActivityNotFoundException` nếu máy không cài app nào khớp — nên bọc trong `try/catch` hoặc kiểm tra bằng `intent.resolveActivity()` trước khi gọi `startActivity`.

Tóm tắt & tiếp theo

Bạn đã biết điều hướng giữa các màn hình bằng Intent theo cả hai chiều (gửi dữ liệu đi, nhận kết quả về), và cách nhờ hệ thống mở app khác. Chương 9 giới thiệu `Fragment` — đơn vị giao diện có thể tái sử dụng bên trong một Activity, cùng Navigation Component để quản lý điều hướng giữa nhiều Fragment gọn gàng hơn so với việc tạo nhiều Activity.

Chương 9: Fragment & Navigation Component

Mục tiêu học

- Hiểu `Fragment` là gì, khác `Activity` ở điểm nào, và vì sao vòng đời của nó phức tạp hơn.
- Dựng được mô hình "single-activity" — một Activity chứa nhiều Fragment.
- Dùng Navigation Component để điều hướng và truyền dữ liệu an toàn kiểu (Safe Args) giữa các Fragment.

Code mẫu: `code/ch09--fragment-navigation/`.

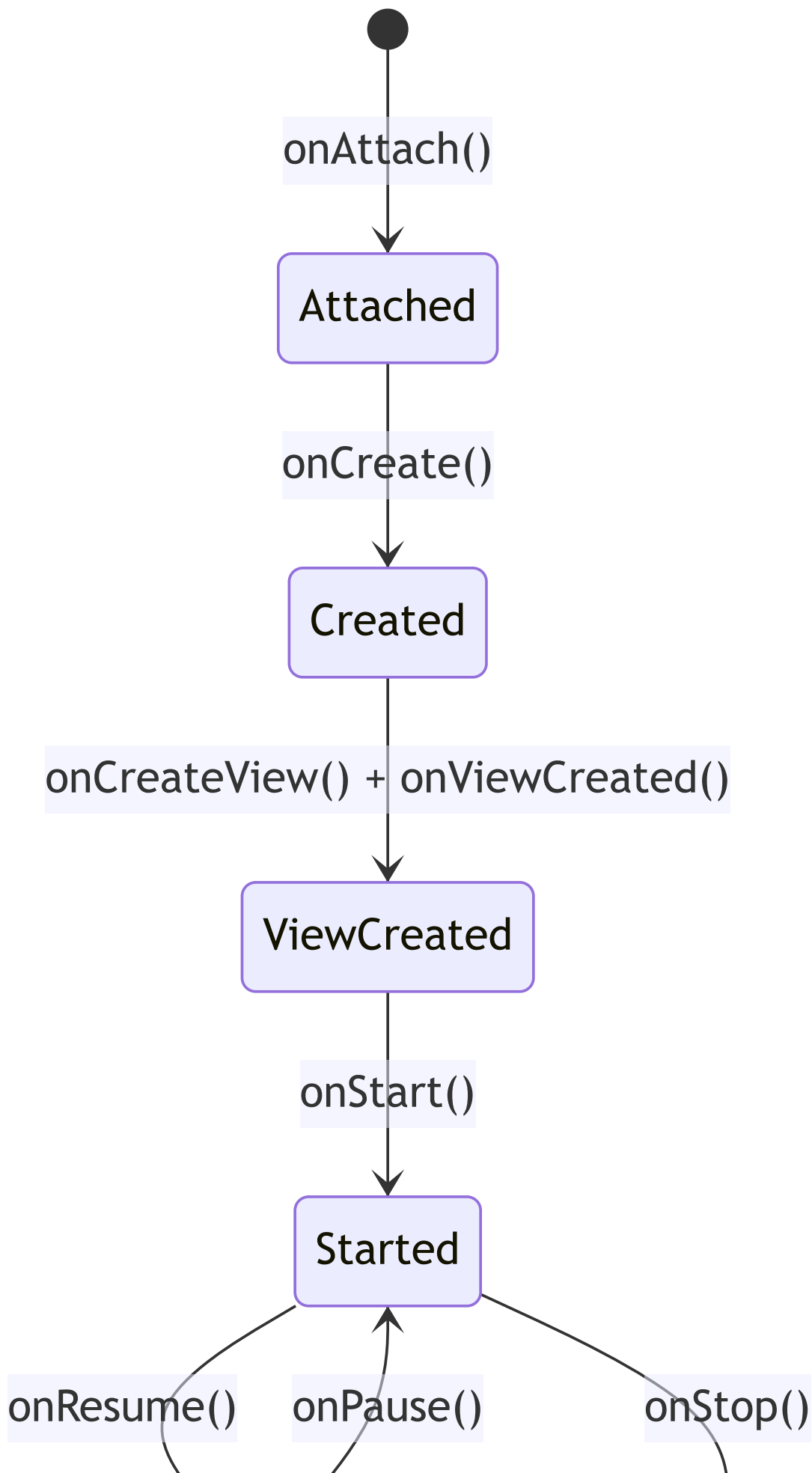
9.1 Vì sao cần Fragment nếu đã có Activity?

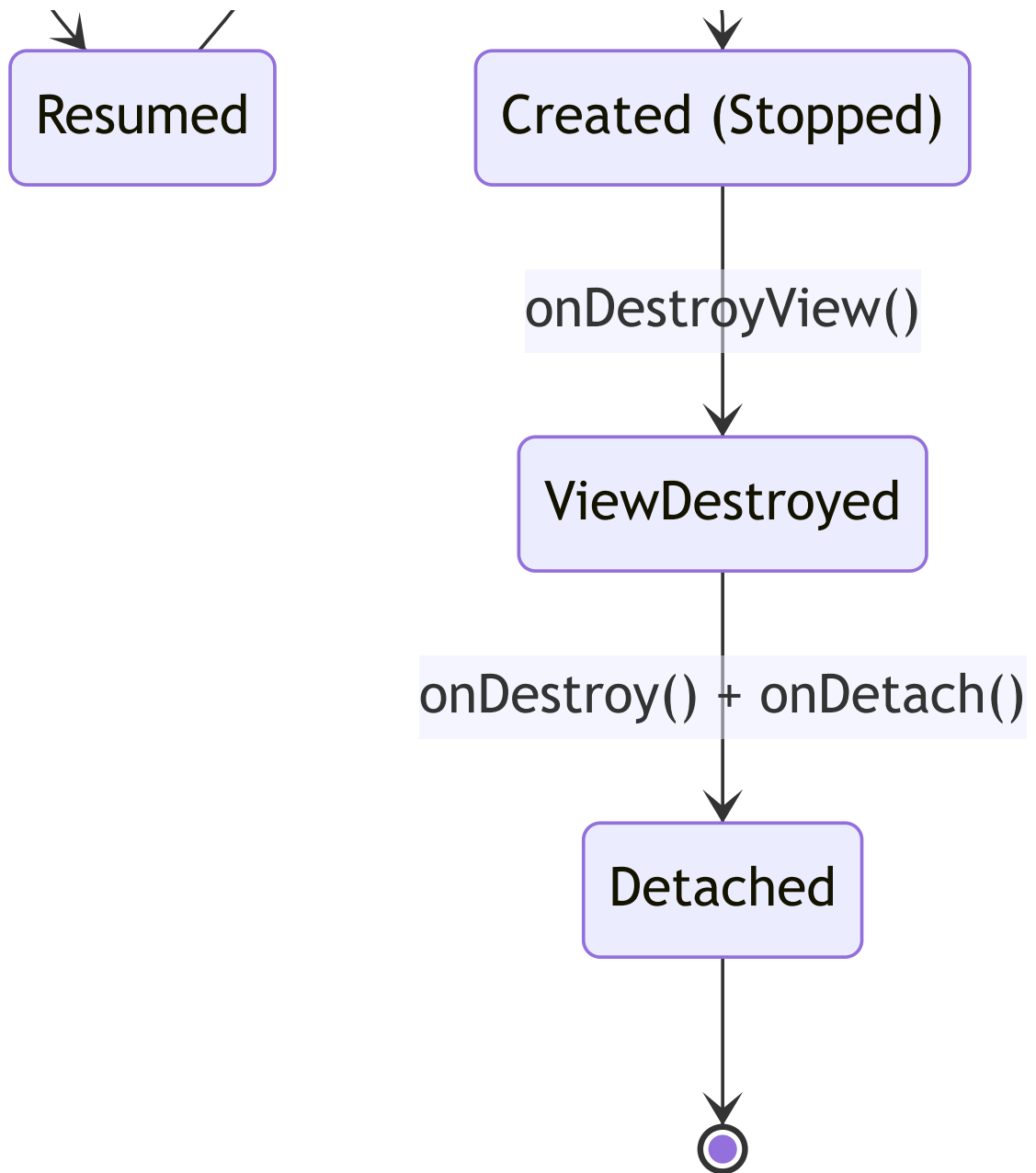
Ở Chương 8, mỗi màn hình là một Activity riêng. Cách này có hai giới hạn:

- Muốn hiển thị 2 màn hình cạnh nhau (ví dụ layout máy tính bảng: danh sách bên trái, chi tiết bên phải) — không làm được nếu mỗi màn hình là một Activity độc lập.
- Chuyển màn hình bằng Activity luôn kèm chi phí tạo mới cửa sổ hệ thống, nặng hơn cần thiết cho những màn hình đơn giản chỉ là "một phần nội dung thay đổi".

`Fragment` giải quyết việc này: là một **mảnh giao diện có vòng đời riêng, nhưng sống bên trong một Activity**. Nhiều app hiện đại dùng đúng **một Activity duy nhất**, mọi màn hình còn lại đều là Fragment — mô hình dùng trong code mẫu chương này.

9.2 Vòng đời Fragment — phức tạp hơn Activity một bậc





Điểm khác biệt quan trọng nhất so với Activity: **Fragment có** `onCreateView()` / `onDestroyView()` **tách riêng khỏi** `onCreate()` / `onDestroy()`. Lý do: View của Fragment có thể bị huỷ và tạo lại (ví dụ khi Fragment tạm thời không hiển thị trong `ViewPager`) trong khi bản thân đối tượng Fragment vẫn còn sống. Đây là lý do code mẫu luôn **null hoá** `binding` **trong** `onDestroyView()`:

```
@Override
public void onDestroyView() {
    super.onDestroyView();
    binding = null; // View đã chết - giữ tham chiếu là rò rỉ bộ nhớ
}
```

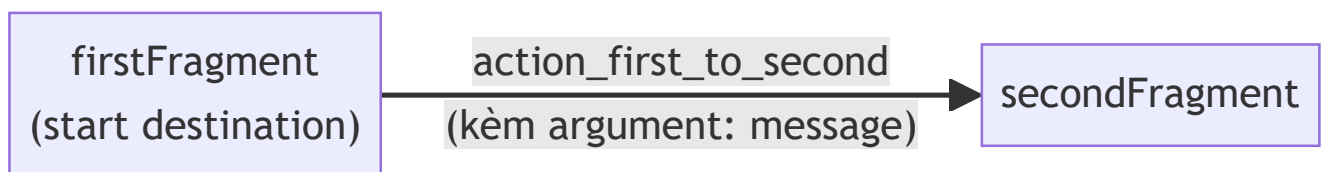
9.3 NavHostFragment — nơi chứa các Fragment điều hướng qua lại

```

<!-- activity_main.xml -->
<fragment
    android:id="@+id/navHostFragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/nav_graph" />

```

`app:navGraph` trỏ tới file định nghĩa sơ đồ điều hướng — mở bằng Android Studio sẽ thấy giao diện kéo-thả trực quan (Navigation Editor):



`app:defaultNavHost="true"` để `NavHostFragment` tự xử lý nút Back của hệ thống — bấm Back sẽ tự quay lại Fragment trước đó trong back stack, không cần code tay.

9.4 Safe Args — truyền dữ liệu có kiểm tra kiểu

Ở Chương 8, `putExtra` / `getStringExtra` không có gì đảm bảo hai bên dùng đúng cùng một key và đúng kiểu — sai chỉ lộ ra lúc chạy. Navigation Component giải quyết bằng **Safe Args**: khai báo tham số ngay trong `nav_graph.xml`:

```
<fragment android:id="@+id/secondFragment" ... >
    <argument
        android:name="message"
        app:argType="string" />
</fragment>
```

Plugin Safe Args (id 'androidx.navigation.safeargs' , khai báo ở build.gradle) sinh ra class FirstFragmentDirections và SecondFragmentArgs lúc build:

```
// Gửi đi - FirstFragment
NavDirections action = FirstFragmentDirections.actionFirstToSecond(message);
NavHostFragment.findNavController(this).navigate(action);

// Nhận - SecondFragment
SecondFragmentArgs args = SecondFragmentArgs.fromBundle(requireArguments());
String message = args.getMessage();
```

Nếu bạn đổi app:argType="string" thành "integer" mà quên sửa code Java tương ứng, **lỗi lộ ra ngay lúc biên dịch** — không phải khi chạy thử mới phát hiện, khác hẳn cách làm ở Chương 8.

Bài tập

1. Chạy code mẫu, xác nhận luồng nhập tin nhắn → chuyển Fragment → hiển thị đúng dữ liệu → bấm Back quay lại.
2. Mở nav_graph.xml bằng Navigation Editor trong Android Studio, thử kéo thêm một Fragment thứ ba, nối action từ secondFragment tới nó.
3. Thêm một argument kiểu integer (ví dụ độ ưu tiên tin nhắn), truyền và hiển thị nó ở SecondFragment.

Lỗi thường gặp

- **Quên null hoá binding trong onDestroyView()**: rò rỉ bộ nhớ, đặc biệt nghiêm trọng nếu Fragment nằm trong danh sách vượt qua lại nhiều lần.
- **Gọi thao tác trên binding sau khi View đã bị huỷ** (ví dụ trong callback bất đồng bộ hoàn thành trễ) → NullPointerException. Luôn kiểm tra Fragment còn "alive"/View còn tồn tại trước khi cập nhật UI.
- **Không thấy class FirstFragmentDirections dù đã khai báo action**: quên áp dụng plugin Safe Args trong app/build.gradle, hoặc chưa build lại project sau khi sửa nav_graph.xml.

- `IllegalArgumentException: Required argument "message" is missing`: điều hướng tới `secondFragment` mà không qua action đã khai báo argument (ví dụ gọi thẳng `navigate(R.id.secondFragment)`), khiến Safe Args không có dữ liệu để truyền.

Tóm tắt & tiếp theo

Bạn đã biết tách một Activity thành nhiều Fragment, điều hướng và truyền dữ liệu giữa chúng an toàn kiểu. Chương 10 sẽ dùng Fragment/Activity để hiển thị **danh sách** dữ liệu hiệu quả bằng `RecyclerView` — thứ gần như mọi ứng dụng thực tế đều cần.

Chương 10: RecyclerView & Adapter (danh sách, hiệu năng)

Mục tiêu học

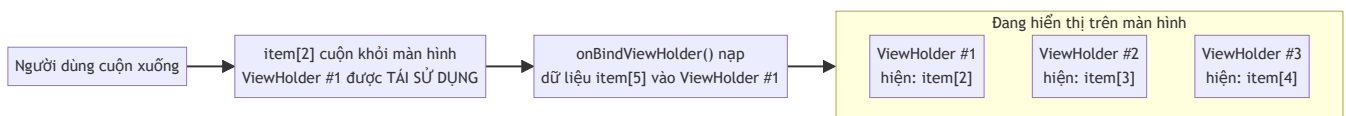
- Hiểu vì sao `RecyclerView` "tái sử dụng" View thay vì tạo mới cho mỗi item — và tại sao điều đó quan trọng với hiệu năng.
- Viết được `Adapter` + `ViewHolder` theo đúng mẫu chuẩn.
- Dùng `ListAdapter` + `DiffUtil` để cập nhật danh sách hiệu quả, có animation, thay vì `notifyDataSetChanged()`.

Code mẫu: `code/ch10-recyclerview-adapter/`.

10.1 Vì sao gọi là "Recycler"?

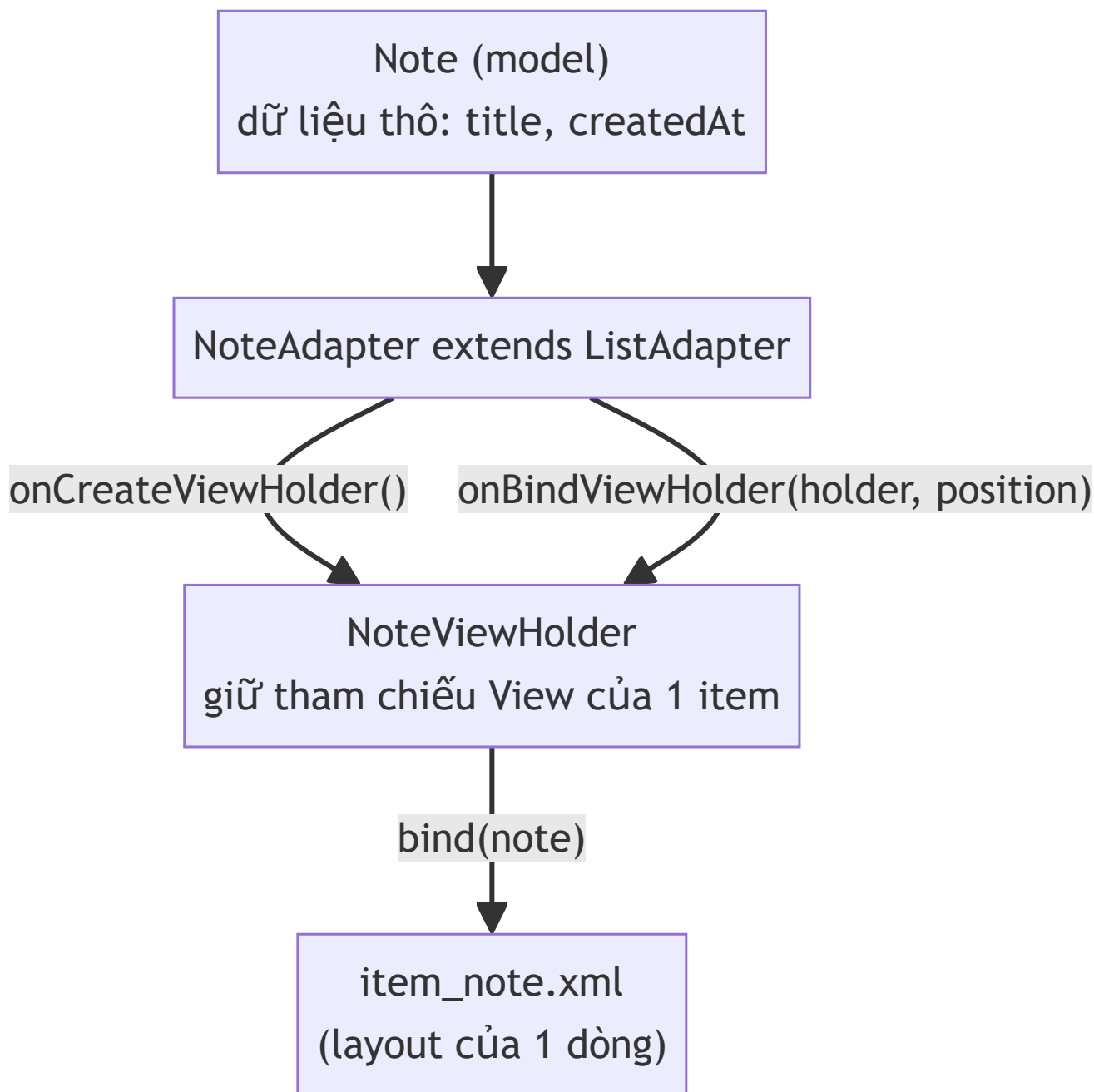
Tưởng tượng một danh sách 10.000 ghi chú nhưng màn hình chỉ hiển thị được 8 item cùng lúc. Nếu tạo mới 10.000 View, gần như toàn bộ sẽ không bao giờ hiển thị — lãng phí bộ nhớ và thời gian.

`RecyclerView` chỉ tạo ra vừa đủ View để lấp đầy màn hình (cộng thêm vài cái đệm), rồi **tái chế (recycle)** — khi một item cuộn ra khỏi màn hình, View của nó không bị huỷ mà được nạp lại (bind) dữ liệu của item mới sắp cuộn vào.



Đây cũng là lý do `ViewHolder` **luôn phải gán lại toàn bộ dữ liệu cần thiết** trong `onBindViewHolder()` — kể cả những trường hợp "trông giống mặc định" (ví dụ ẩn/hiện một icon) — vì ViewHolder có thể đang mang trạng thái sót lại từ item trước đó nó từng hiển thị.

10.2 Ba mảnh ghép: Model — ViewHolder — Adapter



```

static class NoteViewHolder extends RecyclerView.ViewHolder {
    private final ItemNoteBinding binding;

    NoteViewHolder(ItemNoteBinding binding) {
        super(binding.getRoot());
        this.binding = binding;
    }

    void bind(Note note, OnNoteClickListener listener) {
        binding.textTitle.setText(note.getTitle());
        binding.textTimestamp.setText(note.getCreatedAt());
        binding.getRoot().setOnClickListener(v → listener.onNoteClick(note));
    }
}

```

Ba phương thức bắt buộc override trong `Adapter` truyền thống: `onCreateViewHolder()` (tạo ViewHolder mới, chỉ gọi khi CHƯA đủ View để tái sử dụng), `onBindViewHolder()` (gán dữ liệu, gọi RẤT NHIỀU LẦN khi cuộn), và `getItemCount()`. Code mẫu dùng `ListAdapter` (mục 10.3) nên `getItemCount()` đã có sẵn.

10.3 `ListAdapter` + `DiffUtil` — cập nhật danh sách đúng cách

Cách cũ hay thấy: `notifyDataSetChanged()` — báo cho RecyclerView "toàn bộ danh sách có thể đã đổi, vẽ lại hết đi". Cách này **luôn đúng nhưng luôn kém hiệu quả** và mất animation (thêm/xoá/di chuyển item mượt mà).

`ListAdapter` (kế thừa thay vì `RecyclerView.Adapter`) cùng `DiffUtil.ItemCallback` tự so sánh danh sách cũ và mới, chỉ báo đúng phần thay đổi:

```

private static final DiffUtil.ItemCallback<Note> DIFF_CALLBACK = new
DiffUtil.ItemCallback<Note>() {
    @Override
    public boolean areItemsTheSame(@NonNull Note oldItem, @NonNull Note newItem) {
        return oldItem.getId() == newItem.getId(); // có phải "cùng một" item
không?
    }

    @Override
    public boolean areContentsTheSame(@NonNull Note oldItem, @NonNull Note newItem) {
        return oldItem.getTitle().equals(newItem.getTitle())
            && oldItem.getCreatedAt().equals(newItem.getCreatedAt()); // nội dung
có đổi không?
    }
};

```

Khi có danh sách mới, chỉ cần gọi:

```
adapter.submitList(new ArrayList<>(notes));
```

`ListAdapter` tự chạy `DiffUtil` trên một thread nền, tính toán chính xác item nào thêm/xoá/đổi vị trí, rồi gọi đúng các hàm `notifyItemInserted()` / `notifyItemRemoved()` /... tương ứng — có animation mượt, hiệu năng tốt hơn hẳn `notifyDataSetChanged()` với danh sách lớn.

Lưu ý luôn **truyền một List MỚI** vào `submitList()` (code mẫu bọc lại bằng `new ArrayList<>(notes)`) — nếu truyền cùng một tham chiếu List đã sửa đổi tại chỗ, `DiffUtil` sẽ không phát hiện được gì khác biệt vì nó so sánh dựa trên tham chiếu/nội dung tại 2 thời điểm.

Bài tập

1. Chạy code mẫu, bấm nhiều lần **Thêm ghi chú ngẫu nhiên**, quan sát animation item mới chèn vào đầu danh sách.
2. Thêm chức năng xoá: vuốt một item để xoá khỏi `notes`, gọi lại `submitList()` — quan sát animation xoá tự động từ `DiffUtil` (gợi ý: dùng `ItemTouchHelper`, có thể tự tìm hiểu thêm ngoài phạm vi chương này).
3. Thử đổi tạm `areContentsTheSame` để luôn trả `true` — quan sát item không cập nhật giao diện dù dữ liệu đã đổi, để thấy rõ vai trò của hàm này.

Lỗi thường gặp

- **Không override `getItemCount()` khi dùng `RecyclerView.Adapter` thuần** (không áp dụng nếu dùng `ListAdapter` như code mẫu): danh sách hiển thị trống dù có dữ liệu.
- **Sửa dữ liệu trực tiếp trong List đang hiển thị rồi gọi `notifyDataSetChanged()` tuý tiện**: hoạt động nhưng đánh mất toàn bộ lợi ích hiệu năng của `DiffUtil` — nên chuyển hẳn sang `submitList()`.
- **Giữ state UI (như trạng thái checkbox) trong `ViewHolder` mà không đồng bộ với model**: vì `ViewHolder` bị tái sử dụng, checkbox "được tích" có thể tự nhảy sang item khác khi cuộn — luôn lưu state vào model (`Note`), không lưu trong View.
- **`onNoteClick` bị gọi với dữ liệu item cũ sau khi danh sách đã cập nhật**: đảm bảo listener luôn lấy dữ liệu qua `getItem(position)` tại thời điểm bind, không cache riêng.

Tóm tắt & tiếp theo

Bạn đã biết hiển thị danh sách hiệu quả — kỹ năng dùng lại ở hầu hết mọi màn hình danh sách trong các chương sau (từ SQLite/Room ở Chương 13 đến kết quả API ở Chương 21). Chương 11 khép lại Phần 1 với chủ đề resource: string/dimens/style, và cách làm app hỗ trợ đa ngôn ngữ, đa kích thước màn hình.

Chương 11: Resource: string/dimens/style, đa ngôn ngữ, đa kích thước màn hình

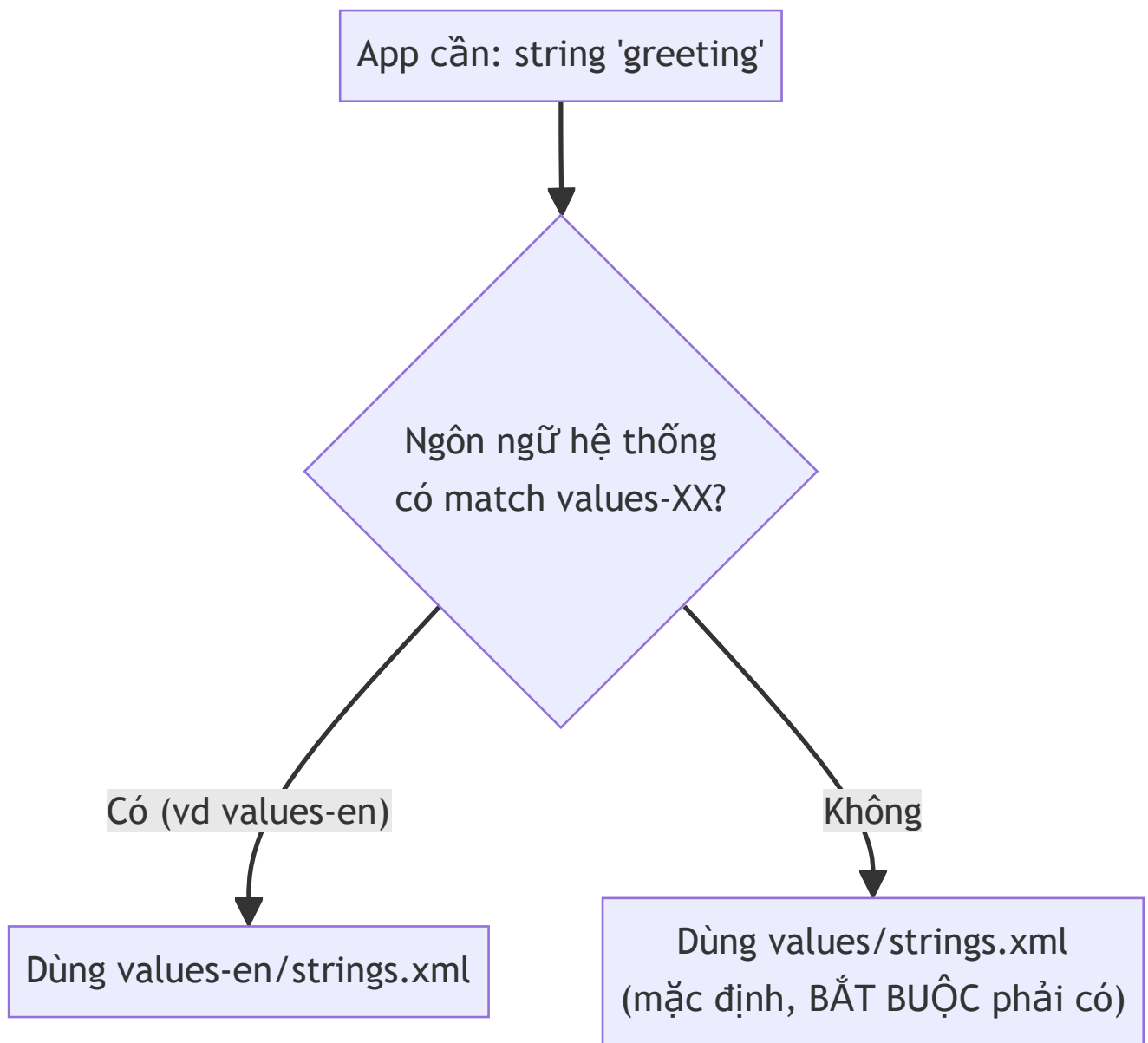
Mục tiêu học

- Hiểu cơ chế **resource qualifier** — cách Android tự chọn đúng file resource theo ngôn ngữ, chế độ sáng/tối, kích thước màn hình.
- Tổ chức được `strings.xml`, `dimens.xml`, `colors.xml` hỗ trợ đa ngôn ngữ và đa kích thước màn hình mà không cần rẽ nhánh code Java.
- Biết vì sao **luôn phải có resource mặc định** (không có hậu tố qualifier) làm phương án dự phòng.

Code mẫu: `code/ch11-resources-i18n/`.

11.1 Resource qualifier là gì?

Tên thư mục con trong `res/` có thể mang thêm hậu tố (qualifier) để chỉ báo "chỉ dùng file này khi điều kiện X đúng":



Một vài qualifier thường dùng:

Qualifier	Ý nghĩa	Ví dụ thư mục
Ngôn ngữ	Mã ngôn ngữ theo chuẩn ISO	values-en , values-ja
Chế độ tối	Dark theme đang bật	values-night
Chiều rộng màn hình tối thiểu	dp, không phụ thuộc mật độ điểm ảnh	values-sw600dp
Hướng màn hình	Ngang/dọc	layout-land
Mật độ điểm ảnh	ldpi/mdpi/hdpi/xhdpi...	drawable-xhdpi

Android tự chọn thư mục khớp nhất với thiết bị hiện tại tại **thời điểm chạy** (không phải lúc build) — cùng một file APK, cài trên máy tiếng Việt và máy tiếng Anh sẽ tự hiển thị đúng ngôn ngữ tương ứng.

11.2 `values/` mặc định — bắt buộc phải đầy đủ

Đây là lỗi người mới hay gặp nhất: chỉ định nghĩa string mới trong `values-en/strings.xml` mà quên định nghĩa nó trong `values/strings.xml` (mặc định). Kết quả: app chạy bình thường trên máy tiếng Anh, nhưng **crash** trên máy có ngôn ngữ khác (không phải Anh, không khớp bất kỳ qualifier nào bạn có) vì không tìm thấy resource nào cả.

Quy tắc: `values/` (không qualifier) phải luôn chứa **đầy đủ mọi string/dimen/color** app cần — coi nó là "sự thật cuối cùng", các thư mục có qualifier khác chỉ **ghi đè một phần**.

11.3 Ví dụ áp dụng trong code mẫu

```
<!-- res/values/strings.xml (mặc định - tiếng Việt) -->
<string name="greeting">Xin chào!</string>

<!-- res/values-en/strings.xml (ghi đè khi hệ thống English) -->
<string name="greeting">Hello!</string>
```

```
<!-- res/values/dimens.xml (mặc định - điện thoại) -->
<dimen name="screen_padding">24dp</dimen>

<!-- res/values-sw600dp/dimens.xml (ghi đè khi màn hình ≥ 600dp - tablet) -->
<dimen name="screen_padding">64dp</dimen>
```

```
<!-- res/values/colors.xml (mặc định - nền sáng) -->
<color name="screen_background">#FFFFFF</color>

<!-- res/values-night/colors.xml (ghi đè khi Dark theme bật) -->
<color name="screen_background">#14171C</color>
```

Layout chỉ tham chiếu resource bằng tên, không quan tâm giá trị cụ thể đến từ thư mục nào:

```
<TextView
    android:text="@string/greeting"
    android:textSize="@dimen/greeting_text_size" />
```

11.4 Vì sao không nối chuỗi bằng `+`?

```
// SAI - không dịch được, và sai ngữ pháp với nhiều ngôn ngữ
String msg = "Xin chào, " + name + "!";
```

```
<!-- ĐÚNG - placeholder %1$s, dịch tự nhiên theo từng ngôn ngữ -->
<string name="greeting_name">Xin chào, %1$s!</string>
```

```
String msg = getString(R.string.greeting_name, name);
```

Đã dùng cách này từ Chương 7 (`result_greeting`) — lý do sâu xa chính là để bản dịch tiếng Anh/ngôn ngữ khác có thể sắp xếp lại vị trí tham số nếu ngữ pháp yêu cầu (`%1$s` cho phép đổi thứ tự bằng cách đổi vị trí `%1$s` trong chuỗi dịch, code Java không cần đổi gì).

Bài tập

1. Chạy code mẫu, đổi ngôn ngữ hệ thống sang English trong Settings, mở lại app — xác nhận nội dung đổi theo `values-en/`.
2. Bật Dark theme của hệ thống, xác nhận màu nền/chữ đổi theo `values-night/`.
3. Thêm một string mới CHỈ trong `values-en/strings.xml` mà không thêm vào `values/strings.xml` — build và chạy thử trên thiết bị/máy ảo có ngôn ngữ khác English lẫn tiếng Việt hệ mặc định (ví dụ đổi máy ảo sang tiếng Nhật) để tự chứng kiến lỗi ở mục 11.2.

Lỗi thường gặp

- **Thiếu resource trong `values/` mặc định:** app crash trên ngôn ngữ không được hỗ trợ tường minh — luôn kiểm tra `values/` có đủ mọi key trước khi thêm bản dịch khác.
- **Viết cứng chuỗi/kích thước trong code Java hoặc layout** (`android:padding="24dp"` thay vì `@dimen/ ...`): mất khả năng điều chỉnh tập trung, tablet và điện thoại buộc dùng chung một giá trị.
- **Nối chuỗi bằng `+` thay vì placeholder:** khó dịch đúng ngữ pháp, một số ngôn ngữ cần đổi thứ tự từ.
- **Quên rằng qualifier áp dụng theo TÊN THƯ MỤC, không phải theo tên file:** `values-en/string.xml` (thiếu "s") sẽ không được Android nhận diện đúng cách nếu đặt sai tên thư mục, dù nội dung file đúng.

Tóm tắt & tiếp theo

Bạn đã hoàn thành **Phần 1 — Nền tảng**: vòng đời, layout, Intent, Fragment, RecyclerView, và hệ thống resource. Đây là bộ kỹ năng đủ để xây một ứng dụng có giao diện hoàn chỉnh. Phần 2 bắt đầu từ Chương 12, chuyển sang chủ đề lưu trữ dữ liệu — bắt đầu với `SharedPreferences`.

Phần 2 — Lưu trữ dữ liệu

Chương 12: SharedPreferences (cấu hình, trạng thái đơn giản)

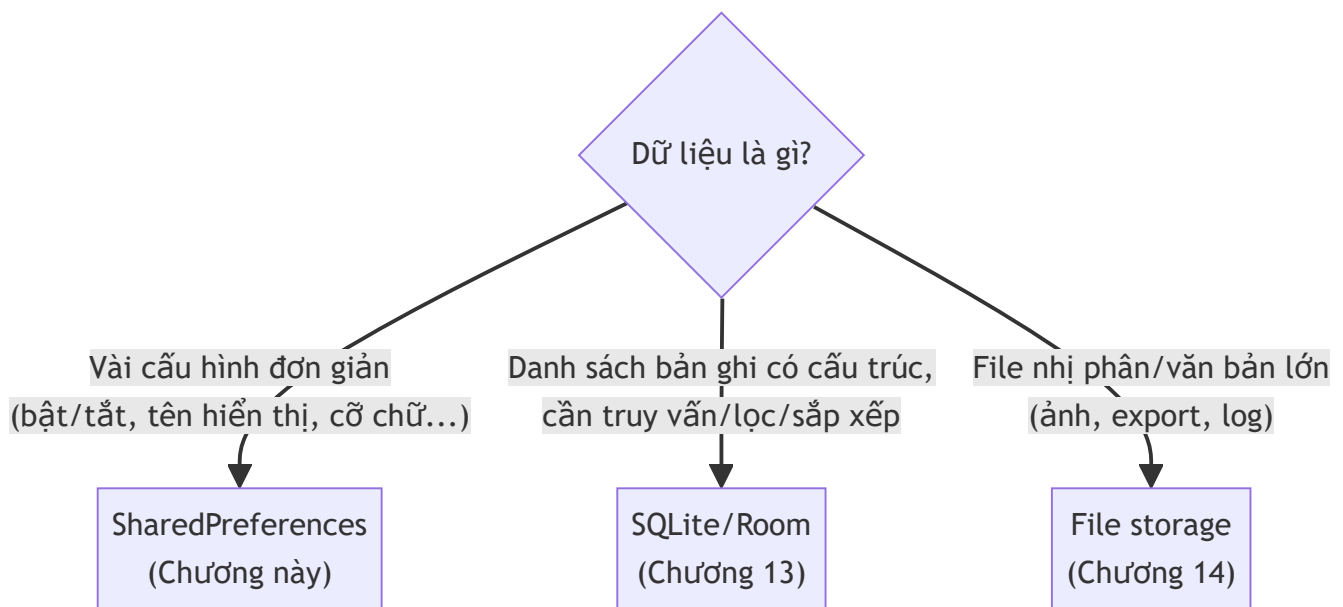
Mục tiêu học

- Hiểu `SharedPreferences` phù hợp cho loại dữ liệu nào, không phù hợp cho loại nào.
- Đọc/ghi được các kiểu dữ liệu cơ bản, phân biệt `apply()` và `commit()`.
- Biết gom logic đọc/ghi vào một class quản lý riêng thay vì rải key khắp nơi.
- Lắng nghe thay đổi bằng `OnSharedPreferenceChangeListener` và biết khi nào phải huỷ đăng ký.

Code mẫu: `code/ch12-shared-preferences/`.

12.1 SharedPreferences là gì, dùng khi nào?

`SharedPreferences` lưu dữ liệu dạng **key-value** đơn giản (chuỗi, số, boolean, tập hợp chuỗi) xuống một file XML riêng của app, tồn tại **qua mọi lần khởi động lại tiến trình** — khác hẳn `onSaveInstanceState` ở Chương 6 (chỉ sống qua configuration change, mất khi app bị đóng hẳn).



Dùng đúng chỗ: cờ "đã xem hướng dẫn lần đầu chưa", tên hiển thị người dùng, cỡ chữ, trạng thái bật/tắt tính năng. **Không dùng** cho danh sách lớn hay dữ liệu cần truy vấn phức tạp — đó là việc của Room (Chương 13).

12.2 Đọc và ghi cơ bản

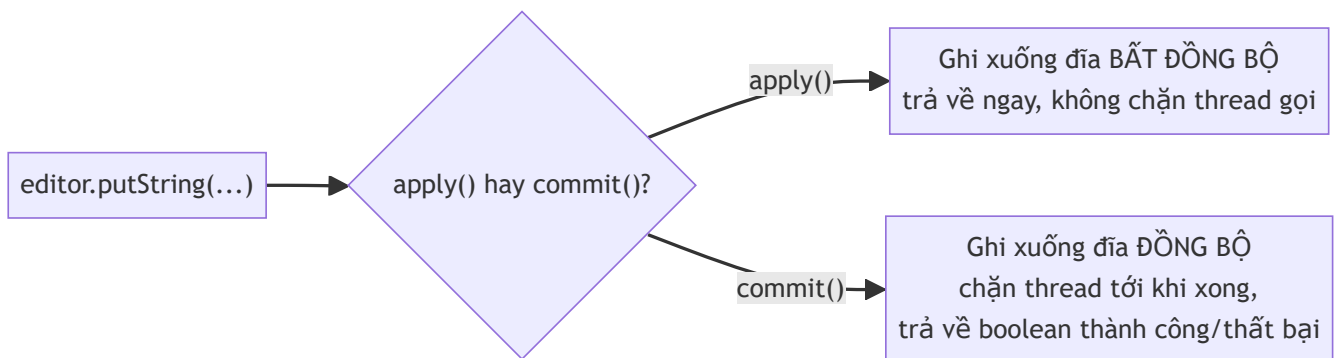
```
SharedPreferences prefs = context.getSharedPreferences("app_settings",
Context.MODE_PRIVATE);

// Đọc - luôn kèm giá trị mặc định cho lần đầu chưa từng lưu
String name = prefs.getString("display_name", "");
boolean notificationsEnabled = prefs.getBoolean("notifications_enabled", true);
int fontSize = prefs.getInt("font_size", 16);

// Ghi - phải qua Editor
SharedPreferences.Editor editor = prefs.edit();
editor.putString("display_name", "Chi");
editor.putBoolean("notifications_enabled", true);
editor.apply();
```

`Context.MODE_PRIVATE` là chế độ **duy nhất nên dùng** — chỉ app của mình đọc/ghi được file này (các mode khác cho phép app khác đọc/ghi đã bị Android loại bỏ từ lâu vì lý do bảo mật).

12.3 `apply()` và `commit()` — khác nhau ở đâu?



Mặc định luôn dùng `apply()` — code mẫu chương này dùng `apply()` cho mọi trường hợp. Chỉ cần nhắc `commit()` khi bạn thật sự cần biết ngay lập tức việc ghi đã thành công hay chưa trước khi làm bước tiếp theo (hiếm gặp, và gọi trên main thread có thể gây giật UI vì bị chặn).

12.4 Gom logic vào một class quản lý riêng

Rải chuỗi key (`"display_name"`, `"font_size"` ...) khắp các Activity là nguồn lỗi gõ sai (typo) rất khó phát hiện — sai một ký tự, `getString()` âm thầm trả về giá trị mặc định thay vì báo lỗi. Cách làm tốt hơn — class `PrefsManager` bọc toàn bộ:


```

public class PrefsManager {
    private static final String PREFS_NAME = "app_settings";
    private static final String KEY_DISPLAY_NAME = "display_name";
    private final SharedPreferences prefs;

    public PrefsManager(Context context) {
        prefs = context.getApplicationContext().getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE);
    }

    public String getDisplayName(String defaultValue) {
        return prefs.getString(KEY_DISPLAY_NAME, defaultValue);
    }
    // ...
}

```

Lưu ý dùng `context.getApplicationContext()` khi lấy `SharedPreferences` bên trong một class không phải Activity — tránh giữ tham chiếu tới `Context` của Activity (rò rỉ bộ nhớ nếu `PrefsManager` sống lâu hơn Activity tạo ra nó).

12.5 Lắng nghe thay đổi

```

private final SharedPreferences.OnSharedPreferenceChangeListener changeListener =
    (sharedPreferences, key) → Log.d(TAG, "Preference đã đổi: " + key);

@Override
protected void onStart() {
    super.onStart();
    prefsManager.registerOnChangeListener(changeListener);
}

@Override
protected void onStop() {
    super.onStop();
    prefsManager.unregisterOnChangeListener(changeListener); // bắt buộc, xem 12.6
}

```

Hữu ích khi nhiều màn hình cùng cần biết một cấu hình vừa đổi (ví dụ đổi ngôn ngữ ở màn hình cài đặt, màn hình khác cần refresh lại UI ngay).

Bài tập

1. Chạy code mẫu, lưu vài giá trị, đóng hẳn app rồi mở lại — xác nhận dữ liệu còn nguyên.

2. Thêm một `CheckBox` mới (ví dụ "Tự động đồng bộ"), lưu và đọc lại giá trị boolean tương ứng qua `PrefsManager`.
3. Thử comment dòng `unregisterOnChangeListener` trong `onStop()`, xoay màn hình nhiều lần (tạo lại Activity nhiều lần) — dùng Android Studio Profiler (Memory) quan sát số lượng Activity instance không được giải phóng tăng dần.

Lỗi thường gặp

- **Quên `unregisterOnChangeListener`**: mỗi Activity mới đăng ký thêm một listener nhưng listener cũ (trở tới Activity đã destroy) không bao giờ được gỡ — rò rỉ bộ nhớ tích lũy dần.
- **Dùng `SharedPreferences` cho danh sách lớn** (ví dụ tự chuyển một `List<Note>` thành JSON rồi nhét vào một key string): hoạt động được ở quy mô nhỏ nhưng không truy vấn/loc được, không hiệu quả khi dữ liệu lớn dần — nên chuyển sang Room (Chương 13) ngay khi có dấu hiệu này.
- **Rải key trực tiếp trong nhiều Activity thay vì qua một class quản lý**: chỉ cần gõ sai `"display_name"` thành `"displayname"` ở một chỗ, dữ liệu coi như "biến mất" (thực ra vẫn nằm ở key cũ) mà không có lỗi nào báo.
- **Đọc `SharedPreferences` trên main thread trong vòng lặp lớn**: bản thân việc đọc/ghi thường nhanh, nhưng thao tác dồn dập không cần thiết trên main thread vẫn nên tránh — cân nhắc thực hiện trên thread nền nếu logic phức tạp hơn ví dụ trong chương này.

Tóm tắt & tiếp theo

Bạn đã biết lưu cấu hình đơn giản bền vững qua các lần mở app. Chương 13 chuyển sang bài toán lớn hơn: lưu trữ danh sách bản ghi có cấu trúc, truy vấn được — bằng SQLite thông qua thư viện Room.

Chương 13: SQLite & Room (cơ sở dữ liệu quan hệ)

Mục tiêu học

- Hiểu vì sao Room ra đời để thay thế việc dùng SQLite trực tiếp bằng tay.
- Định nghĩa được bảng dữ liệu (`@Entity`), câu truy vấn (`@Dao`), và điểm truy cập database (`RoomDatabase`).
- Hiểu vì sao Room bắt buộc thao tác off main thread, và cách xử lý bằng `Executor`.
- Dùng `LiveData` để UI tự cập nhật mỗi khi dữ liệu trong database thay đổi.

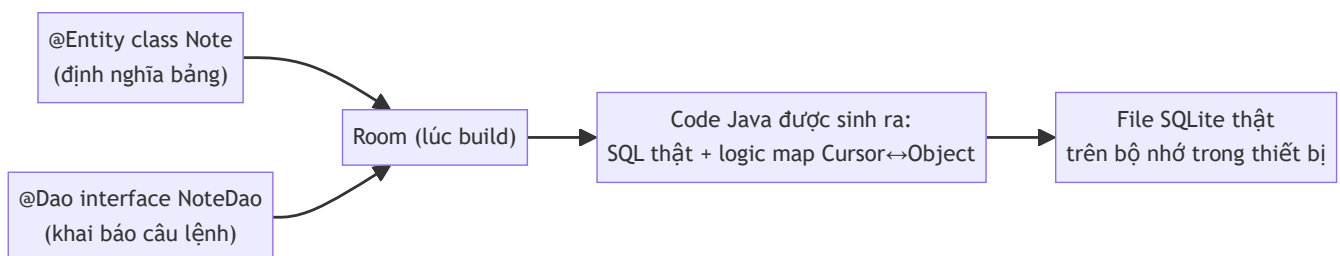
Code mẫu: `code/ch13-sqlite-room/`.

13.1 Vì sao không viết SQL tay như trước đây?

Android có sẵn `SQLiteOpenHelper` để làm việc trực tiếp với SQLite — cách làm truyền thống, nhưng có ba vấn đề:

- Phải tự viết chuỗi SQL (`"CREATE TABLE notes (id INTEGER PRIMARY KEY, title TEXT)"`) — gõ sai chỉ phát hiện lúc chạy.
- Phải tự map thủ công giữa `Cursor` (kết quả truy vấn) và object Java — code lặp lại, dễ quên field.
- Phải tự quản lý version migration khi đổi cấu trúc bảng.

Room (thư viện chính thức của Google, thuộc Jetpack) giải quyết cả ba: bạn khai báo bảng bằng annotation trên class Java, Room tự sinh SQL và code map dữ liệu lúc build, **kiểm tra câu truy vấn ngay lúc biên dịch** (gõ sai tên cột sẽ báo lỗi compile, không phải runtime).



13.2 @Entity — định nghĩa một bảng

```
@Entity(tableName = "notes")
public class Note {

    @PrimaryKey(autoGenerate = true)
    public long id;

    @NonNull
    @ColumnInfo(name = "title")
    public String title;

    @ColumnInfo(name = "created_at")
    public long createdAt;

    public Note(@NonNull String title, long createdAt) {
        this.title = title;
        this.createdAt = createdAt;
    }
}
```

`@PrimaryKey(autoGenerate = true)` tương đương `INTEGER PRIMARY KEY AUTOINCREMENT` trong SQL thuần — Room tự gán `id` khi insert, bạn không cần tự sinh giá trị.

13.3 @Dao — khai báo câu lệnh, không viết logic

```
@Dao
public interface NoteDao {

    @Query("SELECT * FROM notes ORDER BY id DESC")
    LiveData<List<Note>> getAll();

    @Insert
    void insert(Note note);

    @Delete
    void delete(Note note);
}
```

Đây là điểm khác biệt lớn nhất so với JDBC/SQLiteOpenHelper truyền thống: bạn **chỉ khai báo interface**, không viết thân hàm. Room đọc annotation (`@Query`, `@Insert`, `@Delete`) lúc build, tự sinh class triển khai thật (`NoteDao_Impl`) — kiểm tra được ngay lúc biên dịch nếu câu SQL trong `@Query` tham chiếu sai tên cột/bảng.

13.4 RoomDatabase — điểm truy cập duy nhất

```
@Database(entities = {Note.class}, version = 1, exportSchema = false)
public abstract class AppDatabase extends RoomDatabase {

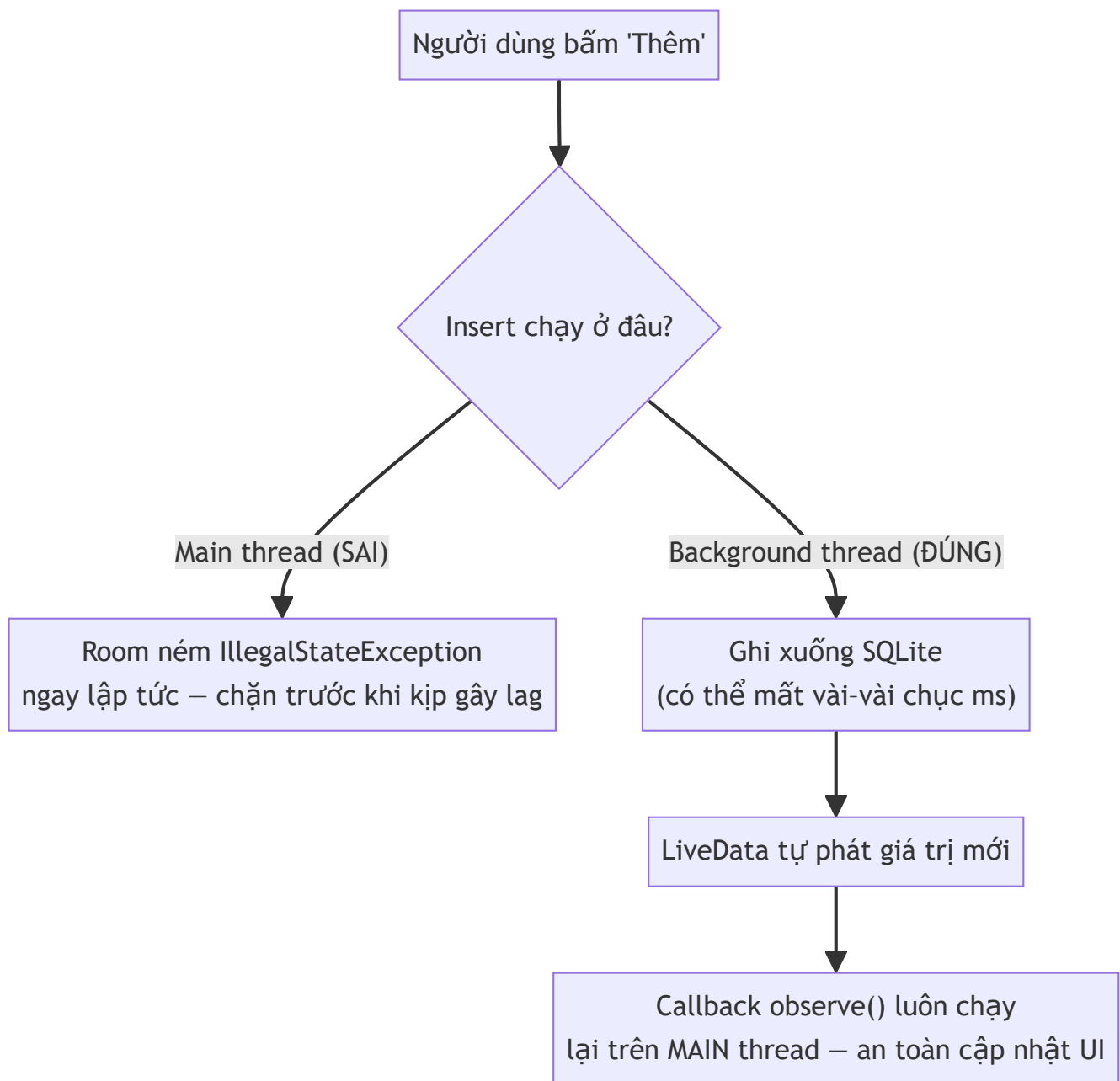
    public abstract NoteDao noteDao();

    private static volatile AppDatabase instance;

    public static AppDatabase getInstance(Context context) {
        if (instance == null) {
            synchronized (AppDatabase.class) {
                if (instance == null) {
                    instance = Room.databaseBuilder(
                        context.getApplicationContext(), AppDatabase.class,
                        "notes.db")
                        .build();
                }
            }
        }
        return instance;
    }
}
```

Luôn dùng một instance RoomDatabase duy nhất cho cả app (mẫu singleton như trên) — tạo nhiều instance trở tới cùng file `.db` gây tốn tài nguyên và có thể xung đột khoá file.

13.5 Vì sao Room cấm chạy trên main thread?



I/O đĩa (đọc/ghi file `.db`) có thể mất từ vài đến vài chục mili-giây — đủ để làm khung hình bị giật nếu chạy trên main thread, nặng hơn có thể gây ANR (Application Not Responding). Room **chủ động chặn** việc này bằng cách ném lỗi ngay lúc gọi nhầm, thay vì để bạn tự phát hiện qua hiện tượng giật lag mờ mờ. Code mẫu dùng `ExecutorService` riêng cho việc này:

```
private final ExecutorService dbExecutor = Executors.newSingleThreadExecutor();

binding.buttonAdd.setOnClickListener(v → {
    Note note = new Note(title, System.currentTimeMillis());
    dbExecutor.execute(() → database.noteDao().insert(note));
});
```

Chương 19 sẽ bàn kỹ hơn về các lựa chọn xử lý bất đồng bộ trong Android — `Executor` ở đây là cách đơn giản nhất phù hợp với những gì đã học tới thời điểm này.

13.6 `LiveData` — UI tự cập nhật khi dữ liệu đổi

```
database.noteDao().getAll().observe(this, adapter::submitList);
```

Khi `NoteDao.getAll()` trả về `LiveData<List<Note>>`, Room tự động theo dõi bảng `notes` — **bất kỳ lúc nào** có insert/update/delete (kể cả từ một đoạn code khác trong cùng app), `observe()` callback sẽ tự chạy lại với danh sách mới nhất, luôn đảm bảo chạy trên main thread nên gọi thẳng `adapter::submitList` (từ Chương 10) an toàn, không cần tự `runOnUiThread`.

Bài tập

1. Chạy code mẫu, thêm vài ghi chú, xóa thử một ghi chú, đóng hẳn app và mở lại — xác nhận dữ liệu còn nguyên trong SQLite thật (khác `SharedPreferences` ở chỗ đây là dữ liệu có cấu trúc, truy vấn được).
2. Thêm một cột mới vào `Note` (ví dụ `boolean isPinned`), thêm một `@Query` mới trong `NoteDao` để lấy riêng các ghi chú đã ghim (`WHERE is_pinned = 1`).
3. Mở **Database Inspector** trong Android Studio khi app đang chạy, tự viết một câu `SELECT` để kiểm tra dữ liệu trực tiếp trong bảng `notes`.

Lỗi thường gặp

- **`IllegalStateException: Cannot access database on the main thread`**: gọi trực tiếp `dao.insert(...)` trong `onClickListener` mà không bọc qua `Executor`/thread nền — đúng như thiết kế bảo vệ của Room ở mục 13.5.
- **Đổi cấu trúc `@Entity` (thêm/xóa cột) mà không tăng `version` trong `@Database`**: app cũ cài trên máy thật sẽ crash khi mở vì Room phát hiện schema không khớp — cần tăng `version` và cung cấp `Migration` (nằm ngoài phạm vi chương này, nhưng bắt buộc phải biết tồn tại trước khi phát hành bản cập nhật đổi schema).

- **Tạo nhiều instance** `RoomDatabase` **thay vì dùng singleton**: lãng phí tài nguyên, có thể gây lỗi khoá file khó debug.
 - **Quên** `@NonNull` **trên field kiểu tham chiếu bắt buộc có giá trị** (như `title`): Room cho phép cột đó nhận `NULL` trong SQL dù ý định của bạn là bắt buộc — khai báo rõ ràng giúp Room sinh đúng ràng buộc `NOT NULL`.
-

Tóm tắt & tiếp theo

Bạn đã có một database SQLite thật hoạt động qua Room — đủ để lưu trữ danh sách dữ liệu có cấu trúc, truy vấn được, tự động cập nhật UI. Chương 14 chuyển sang một dạng lưu trữ khác: làm việc trực tiếp với file (văn bản, nhị phân) trên bộ nhớ trong và bộ nhớ ngoài của thiết bị.

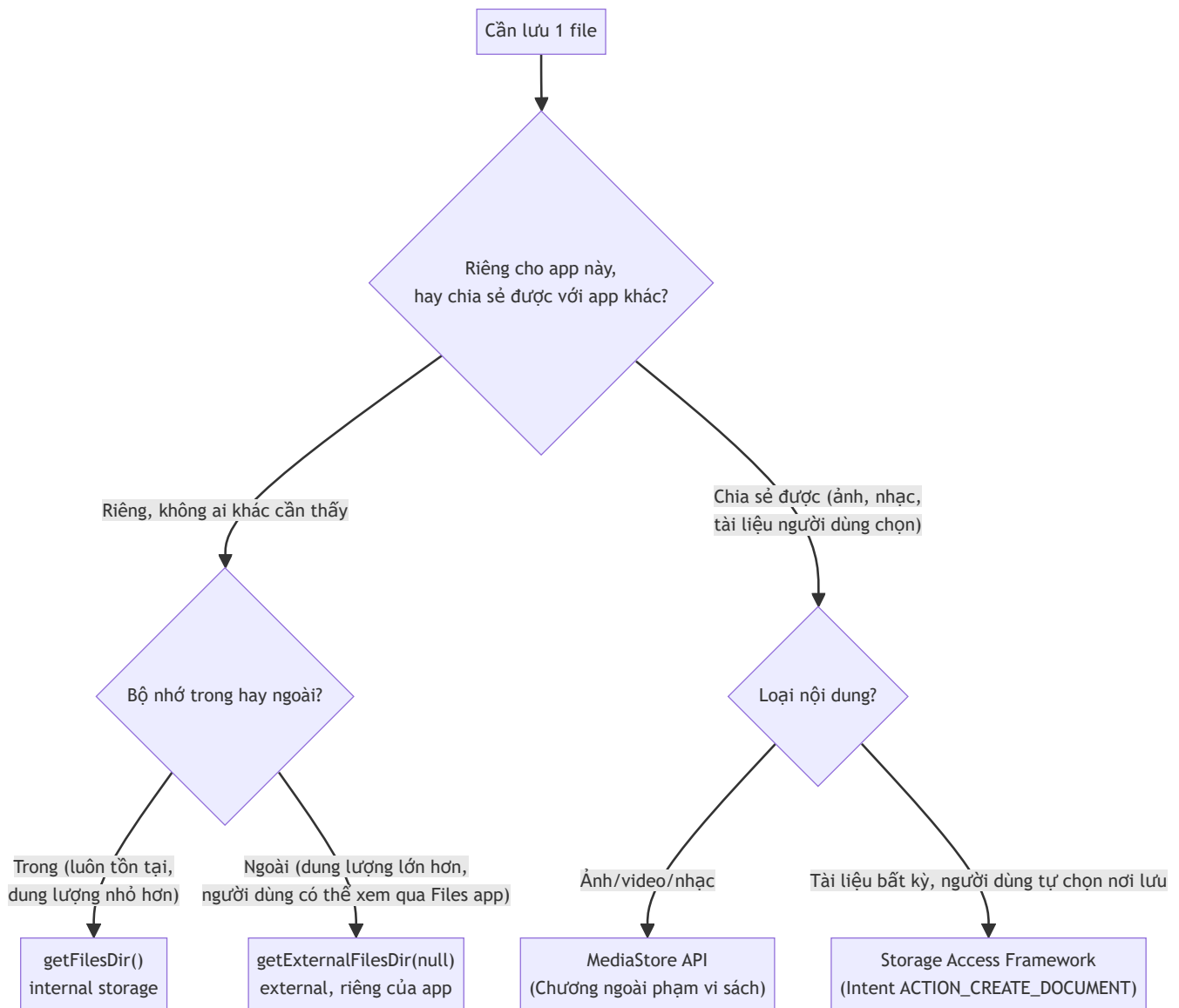
Chương 14: File storage — internal vs external, scoped storage

Mục tiêu học

- Phân biệt bộ nhớ trong (internal) và bộ nhớ ngoài (external), và thư mục "riêng của app" so với thư mục "dùng chung".
- Đọc/ghi file bằng API chuẩn (`FileInputStream` / `FileOutputStream`), chạy đúng cách trên background thread.
- Hiểu khái niệm **scoped storage** (từ Android 10/API 29) và vì sao nó ra đời.
- Biết khi nào cần dùng `MediaStore` /Storage Access Framework thay vì `File` trực tiếp (giới thiệu, không đi sâu).

Code mẫu: `code/ch14-file-storage/`.

14.1 Bốn "nơi" lưu file khác nhau



Chương này tập trung vào hai ô **Internal** và **External riêng của app** — không cần xin bất kỳ quyền runtime nào (khác các chương liên quan tới Bluetooth/NFC/vị trí ở Phần 7). Hai ô còn lại (MediaStore, Storage Access Framework) chỉ giới thiệu khái niệm để bạn biết tồn tại khi cần tra cứu thêm.

14.2 Bộ nhớ trong (internal) — luôn riêng tư, luôn tồn tại

```
File file = new File(getFilesDir(), "note.txt");
try (FileOutputStream out = new FileOutputStream(file)) {
    out.write(content.getBytes(StandardCharsets.UTF_8));
}
```

`getFilesDir()` trả về thư mục **chỉ app của bạn truy cập được** (nhờ sandbox — Chương 26 sẽ giải thích sâu hơn cơ chế này), không cần khai báo quyền gì trong manifest, và **tự động bị xoá khi người dùng gỡ cài đặt app**. Phù hợp cho: cache, dữ liệu tạm, file cấu hình nội bộ.

14.3 Bộ nhớ ngoài, thư mục riêng của app

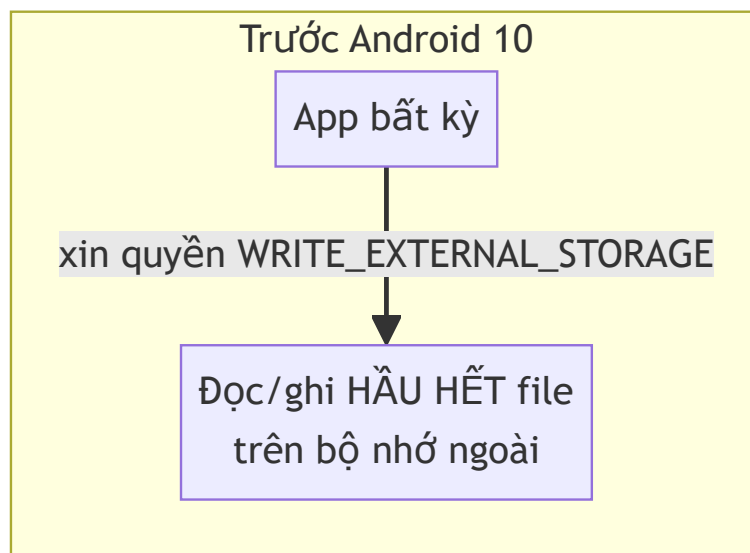
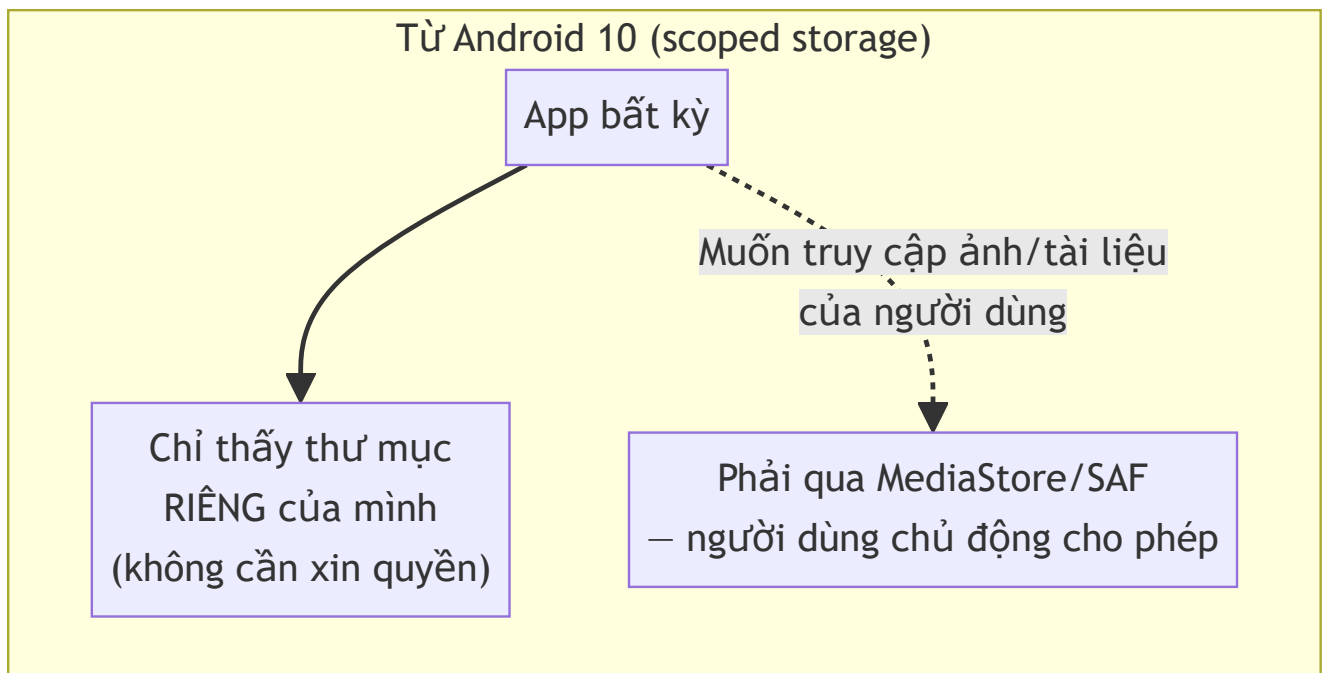
```
File dir = getExternalFilesDir(null); // null = thư mục gốc riêng của app
File file = new File(dir, "note.txt");
```

Khác bộ nhớ trong ở chỗ: dung lượng thường lớn hơn, và **người dùng có thể tự xem file này qua ứng dụng Quản lý file** (đường dẫn dạng `Android/data/<package>/files/`) — nhưng vẫn **không app nào khác** đọc/ghi được (từ Android 10 trở đi, đây chính là quy tắc "scoped storage" ở mục 14.4). Cũng tự động bị xoá khi gỡ app, và **không cần khai báo quyền** `WRITE_EXTERNAL_STORAGE` — điểm hay bị hiểu nhầm vì tài liệu cũ (trước Android 10) yêu cầu quyền này cho MỌI thao tác ghi ra bộ nhớ ngoài.

14.4 Scoped storage là gì, vì sao Android 10 đổi luật chơi?

Trước Android 10 (API 29), một app xin quyền `WRITE_EXTERNAL_STORAGE` có thể đọc/ghi **gần như mọi file** trên bộ nhớ ngoài — kể cả file do app khác tạo ra. Đây là rủi ro bảo mật/riêng tư lớn (một app "đèn pin" xin quyền này có thể âm thầm đọc ảnh riêng tư của bạn).

Scoped storage giới hạn lại: mỗi app mặc định chỉ thấy được thư mục riêng của mình (`getExternalFilesDir()`) mà **không cần xin quyền**. Muốn truy cập file người dùng thật sự sở hữu (ảnh trong thư viện, tài liệu bất kỳ), phải đi qua các API tôn trọng lựa chọn của người dùng: `MediaStore` (cho ảnh/video/nhạc) hoặc Storage Access Framework (cho tài liệu bất kỳ, người dùng tự chọn qua hộp thoại hệ thống) — cả hai đều nằm ngoài phạm vi sách này nhưng nên biết tên để tra cứu khi cần.



14.5 Vì sao đọc/ghi file trên background thread?

Tương tự Room ở Chương 13, thao tác đĩa có thể mất thời gian đáng kể — code mẫu dùng `ExecutorService` riêng cho I/O:

```
private final ExecutorService ioExecutor = Executors.newSingleThreadExecutor();

private void saveTo(File file, String label) {
    String content = binding.editContent.getText().toString();
    ioExecutor.execute(() -> {
        try (FileOutputStream out = new FileOutputStream(file)) {
            out.write(content.getBytes(StandardCharsets.UTF_8));
            runOnUiThread(() -> Toast.makeText(this, "...", Toast.LENGTH_LONG).show());
        } catch (IOException e) {
            runOnUiThread(() -> Toast.makeText(this, "Lưu thất bại.",
                Toast.LENGTH_SHORT).show());
        }
    });
}
```

Khác với Room, API `File` **không tự ép buộc** bạn tránh main thread (không ném lỗi nếu gọi sai chỗ) — kỷ luật tự giác quan trọng hơn ở đây. `runOnUiThread()` đưa phần cập nhật UI (Toast, TextView) quay lại đúng main thread sau khi việc I/O hoàn tất trên thread nền.

Bài tập

1. Chạy code mẫu, lưu nội dung vào cả hai nơi, đọc lại và so sánh đường dẫn hiển thị trong Toast.
2. Dùng `adb shell run-as <package>` (Chương 3) để tự tay xem file `note.txt` nằm ở đâu trên thiết bị thật/máy ảo.
3. Thử xóa dòng `try (FileOutputStream out = ...)` sang gọi trực tiếp trên main thread (không qua `ioExecutor`) — dùng Android Studio Profiler quan sát main thread bị chiếm dụng, dù ngắn, trong lúc ghi file.

Lỗi thường gặp

- **Tương** `getExternalFilesDir()` **cần quyền** `WRITE_EXTERNAL_STORAGE`: sai với thư mục riêng của app — quyền này chỉ từng cần cho việc ghi vào thư mục DÙNG CHUNG trước Android 10, và hiện gần như không còn tác dụng do scoped storage.
- **Không kiểm tra** `getExternalFilesDir()` **trả về** `null`: có thể xảy ra nếu thẻ nhớ ngoài không sẵn sàng (hiếm nhưng có thật) — code mẫu có kiểm tra và dùng bộ nhớ trong làm phương án dự phòng.
- **Đọc file lớn bằng cách đọc hết vào một mảng byte** (như code mẫu làm để đơn giản): ổn với file nhỏ như ghi chú văn bản, nhưng với file lớn (vài chục MB trở lên) cần đọc theo luồng (stream) từng phần thay vì nạp hết vào bộ nhớ.
- **Nhầm lẫn "bộ nhớ ngoài" (external storage) với "thẻ nhớ SD vật lý"**: trên phần lớn thiết bị hiện đại, "external storage" chỉ là một phân vùng logic trong bộ nhớ trong máy, không nhất thiết là thẻ SD rời.

Tóm tắt & tiếp theo

Bạn đã biết ba cách lưu trữ dữ liệu: cấu hình đơn giản (SharedPreferences), dữ liệu có cấu trúc (Room), và file thô (chương này) — cùng khái niệm scoped storage chi phối toàn bộ hệ sinh thái Android hiện đại. Chương 15 giới thiệu `DataStore` — giải pháp hiện đại của Jetpack, dần thay thế `SharedPreferences` cho các trường hợp Chương 12 đã đề cập.

Chương 15: Jetpack DataStore (thay thế SharedPreferences hiện đại)

Mục tiêu học

- Hiểu những giới hạn của `SharedPreferences` mà `DataStore` được thiết kế để khắc phục.
- Đọc/ghi được dữ liệu qua Preferences DataStore trong project Java (dùng cầu nối RxJava3 chính thức).
- Hiểu khái niệm cập nhật có tính giao dịch (transactional update) qua `updateDataAsync`.

Code mẫu: `code/ch15-datastore/` — cùng chức năng với Chương 12 để dễ so sánh trực tiếp.

15.1 SharedPreferences có vấn đề gì?

`SharedPreferences` (Chương 12) hoạt động tốt cho phần lớn trường hợp đơn giản, nhưng có vài giới hạn Google chỉ ra làm lý do ra đời `DataStore`:

- `getString()` / `getBoolean()` ... chạy **đồng bộ trên thread gọi nó** — nếu file preferences lớn hoặc bị chặn I/O, có thể làm giạt main thread mà không có cảnh báo nào (khác Room ở Chương 13, vốn chủ động chặn hành vi sai).
- `commit()` đồng bộ có thể **âm thầm thất bại** mà code gọi không kiểm tra giá trị trả về vẫn tiếp tục chạy như không có gì.
- Không có cách "quan sát" một giá trị cụ thể đổi theo kiểu luồng dữ liệu (stream) hiện đại — phải tự đăng ký/hủy đăng ký listener thủ công như Chương 12 đã làm.

`DataStore` (Jetpack) giải quyết bằng: **mọi thao tác đều bất đồng bộ**, cập nhật có tính giao dịch (đọc-sửa-ghi trọn vẹn, không bị xen ngang), và trả dữ liệu dưới dạng luồng (Kotlin `Flow`, hoặc `Flowable` / `Observable` nếu dùng cầu nối RxJava như code mẫu Java trong sách).

DataStore (chương này)

`data()` — LUÔN bất đồng bộ,
trả về luồng (Flow/Flowable)

`updateDataAsync()` — đọc-sửa-ghi
TRỌN VẸN trong 1 giao dịch

SharedPreferences (Chương 12)

`getString()` — ĐỒNG BỘ,
có thể chặn thread gọi

`commit()` ĐỒNG BỘ / `apply()` bất đồng bộ
— không có cơ chế giao dịch

15.2 Vì sao code mẫu dùng RxJava thay vì Kotlin Flow?

Thư viện lỗi `androidx.datastore:datastore-preferences` được viết dựa trên Kotlin Coroutines/ `Flow` — không gọi trực tiếp thuận tiện từ Java thuần (ngôn ngữ chính của sách này). Google cung cấp cầu nối chính thức `datastore-preferences-rxjava3`, expose cùng chức năng qua kiểu `RxDataStore<Preferences>` với `Flowable` / `Single` — quen thuộc hơn với lập trình viên Java, dùng trong toàn bộ chương này.

```
implementation 'androidx.datastore:datastore-preferences-rxjava3:1.1.1'
implementation 'io.reactivex.rxjava3:rxjava:3.1.8'
implementation 'io.reactivex.rxjava3:rxandroid:3.0.2'
```

15.3 Đọc dữ liệu — luôn là một luồng (stream)

```
RxDataStore<Preferences> datastore = SettingsDataStore.getInstance(this);

datastore.data()
    .observeOn(AndroidSchedulers.mainThread())
    .subscribe(prefs -> {
        String name = prefs.get(SettingsKeys.DISPLAY_NAME);
        binding.editDisplayName.setText(name != null ? name : "");
    });
```


`dataStore.data()` trả về `Flowable<Preferences>`: phát ngay giá trị hiện tại lúc `subscribe()`, rồi tiếp tục phát mỗi khi dữ liệu đổi — không cần tự đăng ký/hủy đăng ký listener thủ công như `OnSharedPreferenceChangeListener` ở Chương 12 (dù bản chất vẫn phải `dispose()` subscription đúng lúc, xem mục 15.5).

15.4 Ghi dữ liệu có tính giao dịch

```
Single<Preferences> update = datastore.updateDataAsync(prefsIn → {
    MutablePreferences mutablePrefs = prefsIn.toMutablePreferences();
    mutablePrefs.set(SettingsKeys.DISPLAY_NAME, name);
    mutablePrefs.set(SettingsKeys.NOTIFICATIONS_ENABLED, notificationsEnabled);
    return Single.just(mutablePrefs);
});

update.subscribe(
    prefs → { /* thành công */ },
    throwable → { /* thất bại - DataStore CÓ báo lỗi rõ ràng, khác commit() im lặng */ });
```

`updateDataAsync` nhận vào giá trị **hiện tại** (`prefsIn`), bạn sửa nó và trả về giá trị **mới** — toàn bộ diễn ra như một giao dịch duy nhất. Nếu hai nơi trong app cùng gọi cập nhật gần như đồng thời, `DataStore` đảm bảo chúng không ghi đè mất dữ liệu của nhau (khác nguy cơ tiềm ẩn khi tự đọc rồi tự ghi hai bước riêng với `SharedPreferences.Editor`).

Lưu ý quan trọng: `updateDataAsync` trả về một `Single` — theo đúng bản chất "lazy" của RxJava, **nếu không gọi `.subscribe()`, việc ghi sẽ không bao giờ thực sự xảy ra**. Đây là lỗi rất dễ mắc khi mới làm quen RxJava.

15.5 Quản lý subscription bằng `CompositeDisposable`

```
private final CompositeDisposable disposables = new CompositeDisposable();

@Override
protected void onStart() {
    super.onStart();
    disposables.add(dataStore.data().subscribe(this::applyToUi));
}

@Override
protected void onStop() {
    super.onStop();
    disposables.clear(); // huỷ TẤT CẢ subscription đã add, cùng lúc
}
```

Nguyên tắc giống hệt lý do phải `unregisterOnChangeListener` ở Chương 12: một `subscribe()` không được huỷ đúng lúc sẽ tiếp tục giữ tham chiếu và chạy callback vào một Activity đã không còn hiển thị (thậm chí đã bị huỷ) — rò rỉ bộ nhớ. `CompositeDisposable` chỉ là công cụ gom nhiều subscription lại để huỷ một lượt cho gọn.

Bài tập

1. Chạy code mẫu, lưu vài giá trị, đóng hẳn app và mở lại — xác nhận dữ liệu còn nguyên, tương tự Chương 12.
2. So sánh trực tiếp file `MainActivity.java` của `ch12-shared-preferences` và `ch15-datastore` — liệt kê ra 3 điểm khác nhau rõ nhất trong cách viết code.
3. Thử bỏ dòng `.subscribe( ... )` sau `dataStore.updateDataAsync( ... )`, build và chạy lại — bấm Lưu và xác nhận dữ liệu **không** thực sự được ghi (minh chứng cho tính "lazy" nói ở mục 15.4).

Lỗi thường gặp

- Gọi `updateDataAsync()` mà quên `.subscribe()`: không có lỗi nào hiện ra, chỉ đơn giản là việc ghi không xảy ra — rất khó phát hiện nếu không biết trước (xem bài tập 3).
- Không `dispose()` subscription từ `dataStore.data()`: rò rỉ bộ nhớ tương tự quên `unregisterOnChangeListener` ở Chương 12.
- Trộn lẫn cả `SharedPreferences` lẫn `DataStore` cho cùng một loại dữ liệu trong cùng một app: gây khó hiểu, chọn một trong hai cho mỗi loại cấu hình và nhất quán trong toàn bộ codebase.

- **Cập nhật UI trực tiếp trong callback của `subscribe()` mà quên `observeOn(AndroidSchedulers.mainThread())`**: một số thao tác I/O của DataStore chạy trên thread nền — callback mặc định có thể không nằm trên main thread, gây crash khi đụng tới View.
-

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 2 — Lưu trữ dữ liệu**: từ cấu hình đơn giản (SharedPreferences/DataStore), dữ liệu có cấu trúc (Room), tới file thô (internal/external storage). Phần 3 chuyển hướng sang một chủ đề hoàn toàn khác: cách Android xử lý các tác vụ chạy ngầm, bắt đầu bằng `Service` ở Chương 16.

Phần 3 — Chạy ngầm & bất đồng bộ

Chương 16: Service & foreground service

Mục tiêu học

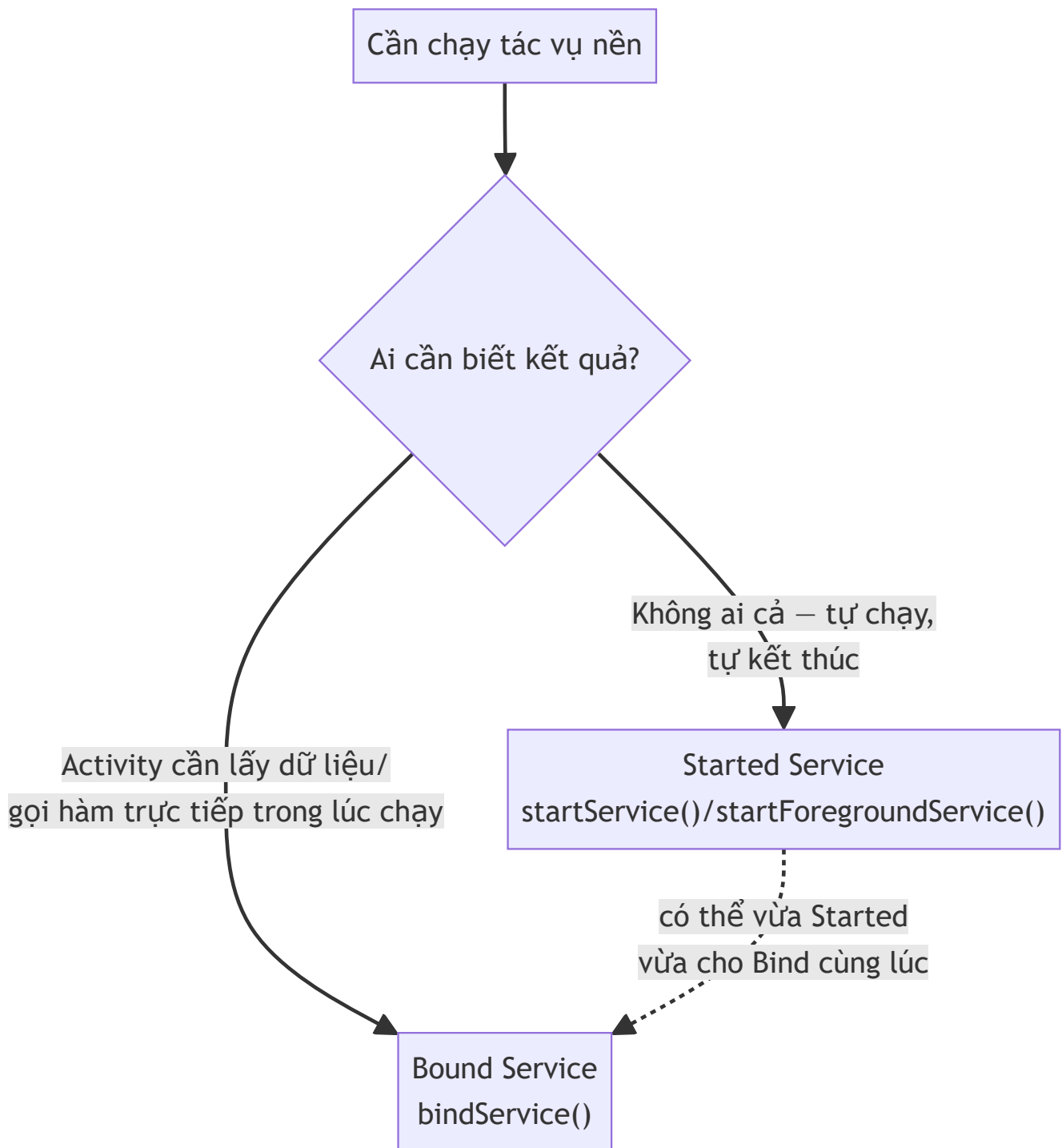
- Hiểu `Service` là gì — và điều nó KHÔNG phải: không tự động chạy trên thread riêng, không phải "mini Activity không giao diện".
- Phân biệt Service dạng **started** và dạng **bound**, biết khi nào dùng loại nào.
- Hiểu vì sao **foreground service** ra đời và bắt buộc phải có notification liên tục.
- Biết `startForegroundService()` khác `startService()` ở đâu, và giới hạn chạy nền từ Android 8+.

Code mẫu: `code/ch16-service/`.

16.1 Ngộ nhận đầu tiên: Service KHÔNG tự chạy trên thread riêng

Đây là hiểu lầm phổ biến nhất với người mới: `Service` chỉ là một **thành phần có vòng đời riêng, được hệ thống ưu tiên giữ sống lâu hơn Activity thông thường** — mọi callback của nó (`onCreate`, `onStartCommand` ...) mặc định vẫn chạy trên **main thread**, giống hệt Activity. Muốn làm việc nặng, bạn vẫn phải tự tạo thread/Executor bên trong Service, đúng như đã làm với Room (Chương 13) hay file I/O (Chương 14).

16.2 Hai kiểu Service



Code mẫu chương này minh họa **cả hai cùng lúc** trên một `CounterService`: khởi động bằng `startForegroundService()` (chạy độc lập, đếm số giây), đồng thời cho phép `MainActivity` `bindService()` vào để đọc số đếm hiện tại trực tiếp — không cần disk, không cần Intent, chỉ cần gọi hàm Java bình thường qua `Binder`.

16.3 Started Service — tự chạy, không cần ai bind

```
Intent intent = new Intent(this, CounterService.class);
ContextCompat.startForegroundService(this, intent);
```

Sau khi gọi, `CounterService.onStartCommand()` chạy — Service **tiếp tục tồn tại** cho tới khi tự gọi `stopSelf()` hoặc bị `stopService()` từ bên ngoài, **kể cả khi Activity đã bị đóng**. Giá trị trả về của `onStartCommand()` quyết định hành vi khi hệ thống buộc phải kill tiến trình để giải phóng bộ nhớ:

Giá trị trả về	Hành vi sau khi bị kill
<code>START_STICKY</code>	Hệ thống tự khởi động lại Service (intent = <code>null</code>) — dùng cho tác vụ nên tiếp tục (như code mẫu)
<code>START_NOT_STICKY</code>	Không tự khởi động lại — dùng cho tác vụ chỉ cần chạy một lần
<code>START_REDELIVER_INTENT</code>	Tự khởi động lại VÀ gửi lại đúng Intent ban đầu — dùng khi cần biết lại tham số gốc

16.4 Bound Service — Activity gọi hàm trực tiếp

```
public class LocalBinder extends Binder {
    public CounterService getService() {
        return CounterService.this;
    }
}

@Nullable
@Override
public IBinder onBind(Intent intent) {
    return binder;
}
```

Phía Activity:

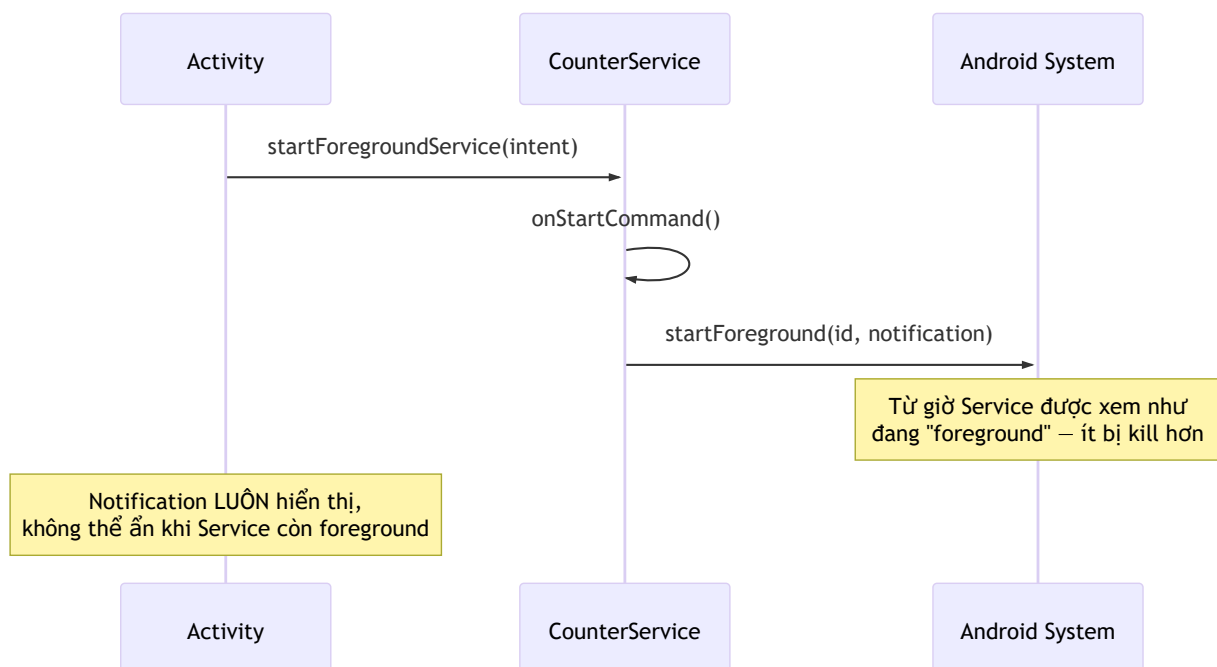
```
private final ServiceConnection connection = new ServiceConnection() {
    @Override
    public void onServiceConnected(ComponentName name, IBinder service) {
        CounterService.LocalBinder localBinder = (CounterService.LocalBinder) service;
        boundService = localBinder.getService();
        boundService.setListener(MainActivity.this);
    }
    @Override
    public void onServiceDisconnected(ComponentName name) { ... }
};

bindService(intent, connection, Context.BIND_AUTO_CREATE);
```

`LocalBinder` chỉ hoạt động vì Activity và Service **chạy trong cùng một tiến trình** (cùng app) — gọi thẳng phương thức Java, không qua serialization nào. Giao tiếp giữa hai tiến trình khác nhau (hai app riêng biệt) cần cơ chế khác hẳn: AIDL, sẽ học ở Chương 34.

Điểm dễ nhầm nhất: `unbindService()` **không** làm Service dừng nếu nó đã được khởi động bằng `startForegroundService()` — Service (và notification) tiếp tục chạy độc lập. Chỉ `stopSelf()` / `stopService()` mới thực sự dừng nó.

16.5 Foreground Service — vì sao bắt buộc có notification?



Trước Android 8, một app có thể âm thầm chạy Service nền vô thời hạn — nguồn gốc của nhiều app "ăn pin" mà người dùng không biết lý do. Từ Android 8 (API 26), Google **giới hạn mạnh** việc chạy Service nền thông thường, và bắt buộc: muốn chạy tác vụ dài hơi mà người dùng có thể nhận biết, phải dùng

foreground service — đổi lại được ưu tiên không bị hệ thống kill, nhưng **bắt buộc hiển thị notification liên tục** để người dùng luôn biết app đang làm gì:

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    // Bắt buộc gọi trong vài giây đầu – trễ hơn sẽ bị hệ thống coi là lỗi (ANR)
    startForeground(NOTIFICATION_ID, buildNotification());
    ...
}
```

Từ Android 14 (API 34), còn phải khai báo rõ **loại** foreground service trong manifest (`android:foregroundServiceType`) — hệ thống áp dụng ràng buộc khác nhau tùy loại (ví dụ loại `location` cho phép chạy lâu hơn loại thông thường, nhưng đòi hỏi quyền vị trí tương ứng — liên quan tới Chương 37).

```
<service
    android:name=".CounterService"
    android:exported="false"
    android:foregroundServiceType="dataSync" />
```

Bài tập

1. Chạy code mẫu, bấm Bắt đầu, rời khỏi app (Home) — quan sát notification tiếp tục cập nhật số đếm mỗi giây dù không có màn hình nào của app hiển thị.
2. Mở lại app, quan sát `textCount` đồng bộ ngay với số hiện tại của Service (nhờ `bindService()` đọc trực tiếp `getCount()`).
3. Thử đổi `START_STICKY` thành `START_NOT_STICKY`, mô tả tình huống hành vi sẽ khác nhau (không cần code thêm, chỉ cần giải thích bằng lời dựa vào bảng ở mục 16.3).

Lỗi thường gặp

- Gọi `startService()` thay vì `startForegroundService()` khi target Android 8+, rồi gọi `startForeground()` trễ: hệ thống ném `ForegroundServiceDidNotStartInTimeException`, app bị crash.
- Quên khai báo `foregroundServiceType` trên Android 14+: crash ngay khi gọi `startForeground()` với thông báo lỗi rõ ràng về thiếu type.
- Tưởng `bindService()` một mình đủ để Service chạy nền bền vững: sai — Service chỉ sống bằng đúng thời gian còn ít nhất một client bind, trừ khi cũng được start bằng `startService()` / `startForegroundService()` (chính là lý do code mẫu dùng cả hai).

- **Không huỷ đăng ký listener (`setListener(null)`) khi Activity dừng bind:** giữ tham chiếu Activity trong Service lâu hơn cần thiết — rò rỉ bộ nhớ, cùng nguyên tắc với Chương 12/15.
-

Tóm tắt & tiếp theo

Bạn đã biết chạy tác vụ nền dài hơi, đúng luật của Android hiện đại (foreground service + notification bắt buộc). Chương 17 giới thiệu `BroadcastReceiver` — cách lắng nghe sự kiện hệ thống (như thay đổi pin, kết nối mạng) và giao tiếp giữa các thành phần bằng broadcast.

Chương 17: BroadcastReceiver, hệ thống sự kiện của Android

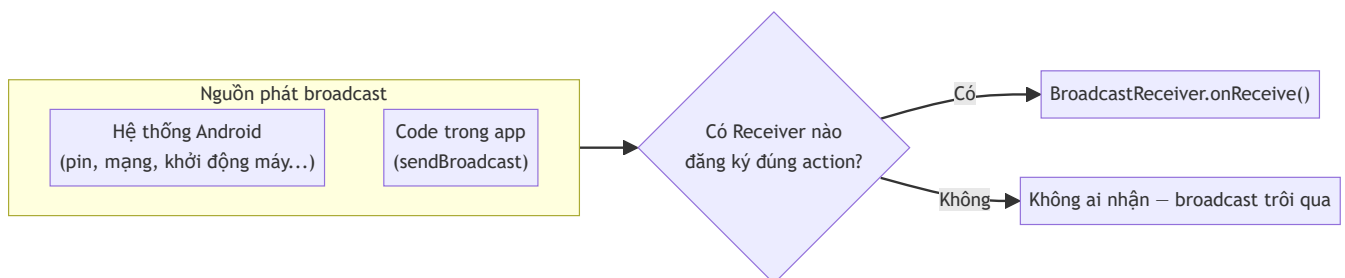
Mục tiêu học

- Hiểu `BroadcastReceiver` là gì, và Android dùng nó để thông báo sự kiện hệ thống ra sao.
- Phân biệt đăng ký receiver qua manifest và đăng ký động (runtime) — biết vì sao cách đầu gần như không còn dùng được.
- Gửi và nhận được broadcast tự định nghĩa trong phạm vi app của mình.
- Hiểu khái niệm "sticky broadcast" qua ví dụ pin thiết bị.

Code mẫu: `code/ch17-broadcast-receiver/`.

17.1 BroadcastReceiver là gì?

`BroadcastReceiver` là một thành phần lắng nghe **thông điệp phát rộng (broadcast)** — có thể do hệ thống Android phát ra (pin thay đổi, kết nối mạng đổi, thiết bị khởi động xong...), hoặc do chính app tự phát ra để các phần khác trong app phản ứng lại.



17.2 Đăng ký động (runtime) — cách chuẩn hiện nay

```
private final BroadcastReceiver batteryReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        int level = intent.getIntExtra(BatteryManager.EXTRA_LEVEL, -1);
        int scale = intent.getIntExtra(BatteryManager.EXTRA_SCALE, -1);
        int percent = Math.round(level * 100f / scale);
        // cập nhật UI
    }
};

@Override
protected void onStart() {
    super.onStart();
    ContextCompat.registerReceiver(this, batteryReceiver,
        new IntentFilter(Intent.ACTION_BATTERY_CHANGED),
        ContextCompat.RECEIVER_NOT_EXPORTED);
}

@Override
protected void onStop() {
    super.onStop();
    unregisterReceiver(batteryReceiver);
}
```

Đăng ký trong `onStart()` /huỷ trong `onStop()` — cùng nguyên tắc với listener ở Chương 12: receiver chỉ nên "sống" khi Activity thật sự đang hiển thị, tránh nhận sự kiện và giữ tham chiếu Activity không cần thiết khi app đã ẩn.

Tham số cuối `RECEIVER_NOT_EXPORTED` (bắt buộc khai báo rõ từ Android 13/API 33) nghĩa là **chỉ app của chính mình mới gửi broadcast tới được receiver này** — dùng `RECEIVER_EXPORTED` nếu bạn thật sự cần app khác gửi broadcast tới (hiếm gặp, và cần cân nhắc kỹ về bảo mật).

17.3 Vì sao không đăng ký qua `AndroidManifest.xml` nữa?

Tài liệu cũ hay dạy khai báo receiver tĩnh trong manifest:

```
←!— CÁCH CŨ – không còn hoạt động với hầu hết broadcast hệ thống từ Android 8+ →  
<receiver android:name=".MyReceiver">  
  <intent-filter>  
    <action android:name="android.intent.action.BATTERY_CHANGED" />  
  </intent-filter>  
</receiver>
```

Từ Android 8 (API 26), Google giới hạn mạnh: **hầu hết implicit broadcast hệ thống không còn đánh thức được receiver khai báo tĩnh trong manifest** — lý do tương tự Chương 16 (chống app âm thầm chạy nền tốn pin). `ACTION_BATTERY_CHANGED` thậm chí **chưa bao giờ** hỗ trợ đăng ký qua manifest, kể cả trước Android 8 — bắt buộc phải đăng ký động như mục 17.2. Quy tắc thực dụng: **luôn đăng ký động trong code**, chỉ tra cứu ngoại lệ hiếm hoi (như `BOOT_COMPLETED`) khi thật sự cần nhận sự kiện lúc app chưa chạy.

17.4 Gửi broadcast tự định nghĩa trong app

```
Intent intent = new Intent(PingReceiver.ACTION_PING);  
intent.putExtra(PingReceiver.EXTRA_SEQUENCE, ++pingSequence);  
intent.setPackage(getPackageName()); // giới hạn trong phạm vi app của mình  
sendBroadcast(intent);
```

`setPackage(getPackageName())` là bước quan trọng: từ Android 8+, một broadcast implicit (không set package) với action tự định nghĩa **có thể không tới được receiver nào cả** nếu hệ thống coi nó là broadcast "công khai" bị giới hạn. Giới hạn rõ ràng vào package của chính mình vừa an toàn hơn (không app lạ nào nhận được), vừa đảm bảo hoạt động nhất quán.

Lưu ý: nếu chỉ cần giao tiếp **giữa các thành phần trong cùng một app** (ví dụ Activity với Activity, hay với Service), các chương sau sẽ giới thiệu những cách trực tiếp và gọn hơn broadcast rất nhiều — `LiveData` /callback đơn giản (đã dùng ở Chương 16), hay `ViewModel` chia sẻ (Chương 28). Broadcast phù hợp nhất khi cần lắng nghe sự kiện **hệ thống**, hoặc giao tiếp **giữa các app khác nhau** (liên quan Chương 32–33).

17.5 Sticky broadcast — vì sao pin hiện đúng ngay cả khi không đổi?

`ACTION_BATTERY_CHANGED` là một **sticky broadcast**: hệ thống luôn giữ sẵn Intent chứa giá trị mới nhất, nên **đăng ký receiver vào bất kỳ lúc nào** cũng nhận được giá trị hiện tại ngay lập tức — không cần đợi tới lần pin thay đổi tiếp theo. Đây là lý do code mẫu hiển thị đúng % pin ngay khi mở app, dù không có sự kiện pin nào vừa xảy ra.

Bài tập

1. Chạy code mẫu, quan sát % pin hiển thị ngay khi mở app.
2. Bấm nút gửi tín hiệu vài lần liên tiếp, xác nhận bộ đếm và số thứ tự đúng.
3. Thử xóa dòng `intent.setPackage(getPackageName())`, build lại, kiểm tra xem tín hiệu còn được nhận trên máy ảo của bạn không (kết quả có thể khác nhau tùy phiên bản Android, đó chính là lý do nên luôn set package tường minh thay vì phụ thuộc hành vi ngầm định).

Lỗi thường gặp

- **Quên `unregisterReceiver()`**: rò rỉ Activity, và tệ hơn — receiver vẫn tiếp tục chạy xử lý sự kiện dù màn hình đã ẩn từ lâu.
- **Gọi `unregisterReceiver()` hai lần (hoặc khi chưa từng đăng ký)**: ném `IllegalArgumentException: Receiver not registered` — đảm bảo cặp đăng ký/hủy đăng ký luôn khớp nhau (`onStart` / `onStop`, không lệch sang `onCreate` / `onDestroy` rồi quên).
- **Kỳ vọng receiver khai báo tĩnh trong manifest vẫn nhận được broadcast hệ thống**: hầu hết không còn hoạt động từ Android 8+, xem mục 17.3.
- **Gửi broadcast implicit không giới hạn package cho giao tiếp nội bộ app**: hành vi không nhất quán giữa các phiên bản Android, dễ tạo lỗ hổng để app khác giả mạo gửi broadcast trùng action vào app của bạn.

Tóm tắt & tiếp theo

Bạn đã biết lắng nghe sự kiện hệ thống và tự định nghĩa broadcast riêng khi cần. Chương 18 giới thiệu `WorkManager` — công cụ hiện đại để lên lịch tác vụ nền có điều kiện (chỉ chạy khi có mạng, khi đang sạc...), đảm bảo chạy được kể cả khi app đã đóng hoặc thiết bị khởi động lại.

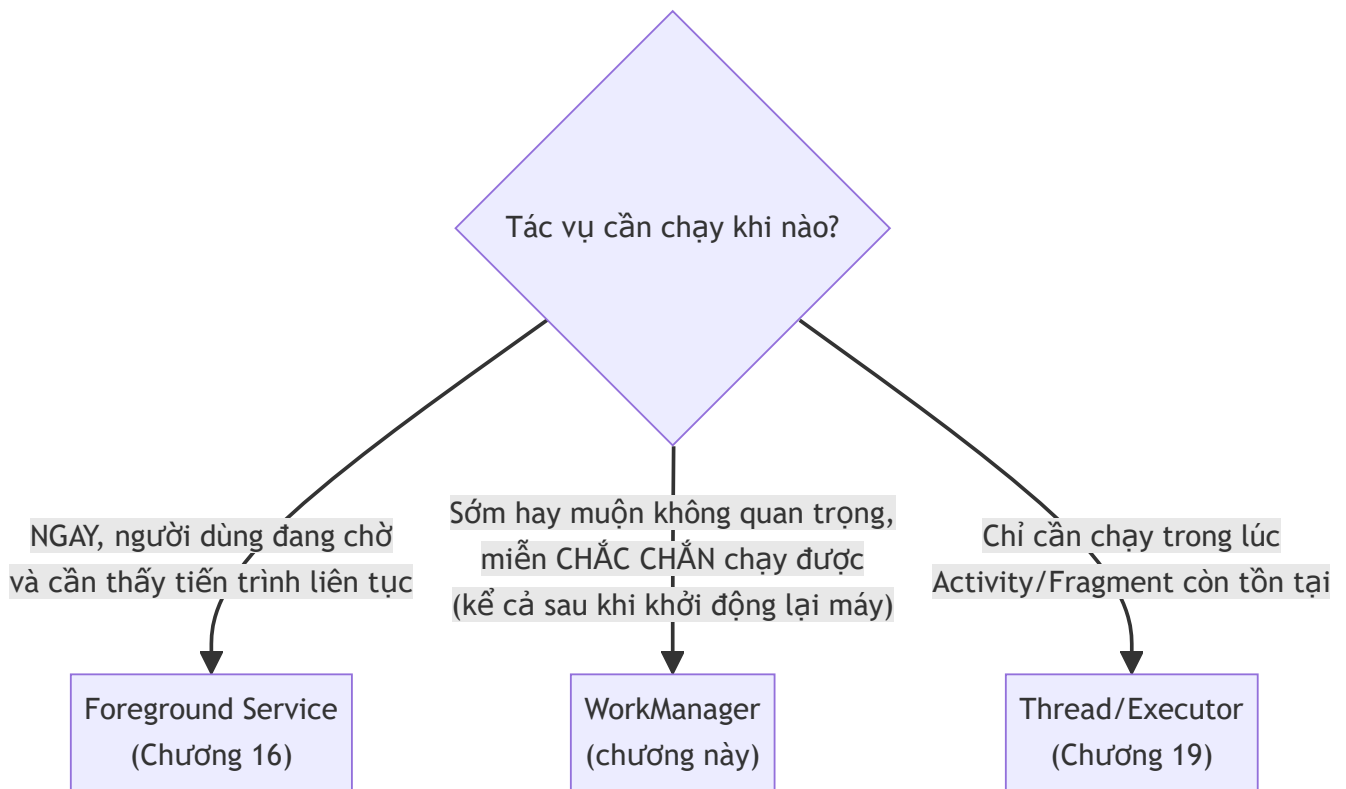
Chương 18: WorkManager (tác vụ nền có lịch/điều kiện)

Mục tiêu học

- Hiểu WorkManager giải quyết bài toán gì mà Service (Chương 16) không phù hợp.
- Viết được một `Worker`, chạy một lần (`OneTimeWorkRequest`) với ràng buộc điều kiện (`Constraints`).
- Theo dõi trạng thái công việc qua `WorkInfo` và `LiveData`.
- Biết cú pháp `PeriodicWorkRequest` và giới hạn chu kỳ tối thiểu.

Code mẫu: `code/ch18-workmanager/`.

18.1 WorkManager khác Service ở đâu?



WorkManager phù hợp cho tác vụ như: đồng bộ dữ liệu định kỳ, upload log, nén ảnh trước khi gửi — **không cần chạy ngay lập tức**, nhưng **phải đảm bảo chạy được**, kể cả khi app đã bị đóng hoàn toàn hoặc thiết bị vừa khởi động lại. WorkManager tự chọn cơ chế phù hợp nhất bên dưới tùy phiên bản Android (`JobScheduler`, `AlarmManager`...) — bạn chỉ cần lập trình với một API duy nhất, không cần tự xử lý khác biệt giữa các phiên bản.

18.2 Viết một `Worker`

```
public class SyncWorker extends Worker {

    public SyncWorker(@NonNull Context context, @NonNull WorkerParameters params) {
        super(context, params);
    }

    @NonNull
    @Override
    public Result doWork() {
        // Chạy trên thread nền do WorkManager tự quản lý – an toàn Thread.sleep(),
        // gọi mạng trực tiếp, không cần tự tạo Executor như Chương 13/14.
        Thread.sleep(3000);

        Data output = new Data.Builder()
            .putString("synced_at", "12:00:00")
            .build();
        return Result.success(output);
    }
}
```

Ba giá trị `Result` có thể trả về: `Result.success(data)` (xong, có thể kèm dữ liệu output), `Result.failure()` (thất bại hẳn, không thử lại), `Result.retry()` (thất bại tạm thời, WorkManager tự lên lịch thử lại sau).

18.3 Ràng buộc điều kiện (`Constraints`)

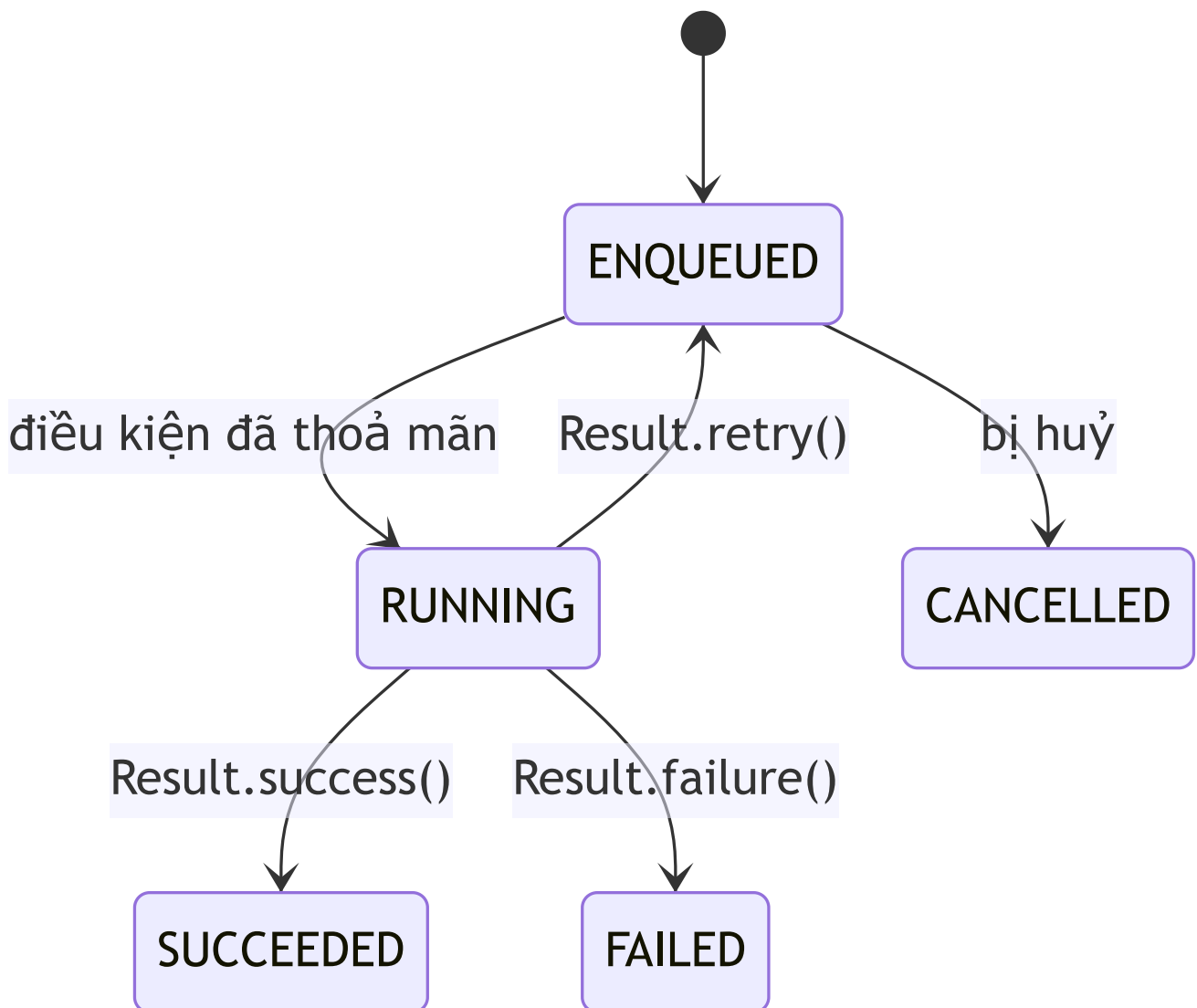
```
Constraints constraints = new Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build();

OneTimeWorkRequest request = new OneTimeWorkRequest.Builder(SyncWorker.class)
    .setConstraints(constraints)
    .build();

WorkManager.getInstance(context).enqueue(request);
```

WorkManager tự chờ tới khi điều kiện thoả mãn mới thật sự chạy `doWork()` — ví dụ `NetworkType.CONNECTED` khiến công việc bị giữ ở trạng thái `ENQUEUED` cho tới khi thiết bị có mạng, hoàn toàn tự động, không cần bạn tự kiểm tra kết nối mạng thủ công. Các ràng buộc khác thường dùng: `setRequiresCharging(true)`, `setRequiresBatteryNotLow(true)`, `setRequiresDeviceIdle(true)`.

18.4 Theo dõi trạng thái bằng `WorkInfo`



```
workManager.getWorkInfoByIdLiveData(request.getId()).observe(this, workInfo -> {  
    if (workInfo.getState() == WorkInfo.State.SUCCEEDED) {  
        String syncedAt = workInfo.getOutputData().getString("synced_at");  
        // cập nhật UI  
    }  
});
```

Trạng thái công việc được `WorkManager` **lưu xuống database nội bộ riêng** — vẫn tồn tại kể cả khi Activity đang quan sát nó bị huỷ và tạo lại (xoay màn hình, Chương 6), hoặc kể cả khi cả app bị đóng hoàn toàn rồi mở lại.

18.5 `PeriodicWorkRequest` — lặp lại định kỳ

```
PeriodicWorkRequest request = new PeriodicWorkRequest.Builder(SyncWorker.class, 15,
    TimeUnit.MINUTES)
    .setConstraints(constraints)
    .build();

workManager.enqueueUniquePeriodicWork(
    "periodic_sync", ExistingPeriodicWorkPolicy.KEEP, request);
```

Hai điểm quan trọng cần nhớ:

- **Chu kỳ tối thiểu là 15 phút** — đặt số nhỏ hơn, WorkManager tự động nâng lên 15 phút mà không báo lỗi. Đây không phải giới hạn kỹ thuật ngẫu nhiên mà là chính sách có chủ đích của Android để hạn chế app đánh thức máy quá thường xuyên, tốn pin.
- `enqueueUniquePeriodicWork` với `ExistingPeriodicWorkPolicy.KEEP` đảm bảo dù gọi hàm này nhiều lần (ví dụ mỗi lần mở app), **chỉ một lịch chạy định kỳ duy nhất tồn tại** — tránh tạo hàng chục bản lặp trùng nếu người dùng mở app nhiều lần.

Bài tập

1. Chạy code mẫu, bấm **Đồng bộ ngay**, quan sát đúng trình tự trạng thái ENQUEUED → RUNNING → SUCCEEDED trong khoảng 3 giây.
2. Bật chế độ máy bay trước khi bấm nút — xác nhận trạng thái dừng lại ở ENQUEUED cho tới khi tắt chế độ máy bay.
3. Sửa `SyncWorker.doWork()` để thỉnh thoảng trả về `Result.retry()` (ví dụ dựa vào `getRunAttemptCount()` để giả lập thất bại ở lần thử đầu), quan sát WorkManager tự động chạy lại.

Lỗi thường gặp

- **Kỳ vọng** `OneTimeWorkRequest` **chạy NGAY LẬP TỨC**: WorkManager có thể trì hoãn một chút để tối ưu pin/hệ thống, không đảm bảo chạy tức thời như gọi hàm bình thường — không phù hợp cho việc cần phản hồi ngay (dùng Executor, Chương 19, cho trường hợp đó).
- **Không dùng** `enqueueUniquePeriodicWork`: gọi `enqueuePeriodicWork` thông thường nhiều lần tạo ra nhiều lịch chạy song song, gây tác vụ chạy trùng lặp không kiểm soát.
- **Đặt chu kỳ** `PeriodicWorkRequest` **nhỏ hơn 15 phút rồi thắc mắc sao chạy chậm hơn khai báo**: đây là hành vi đã định, không phải lỗi — xem lại mục 18.5.

- **Làm việc nặng trực tiếp trên `doWork()` mà không xử lý `InterruptedException`** : WorkManager có thể huỷ Worker giữa chừng (ví dụ hệ thống cần tài nguyên gấp) — nên kiểm tra khả năng dừng sớm với tác vụ dài, tương tự nguyên tắc huỷ Thread ở Chương 19.
-

Tóm tắt & tiếp theo

Bạn đã có công cụ đúng đắn cho tác vụ nền cần đảm bảo chạy được, có điều kiện, có thể định kỳ. Chương 19 lùi lại một bước, hệ thống hoá các lựa chọn xử lý bất đồng bộ trong Android (`Thread` , `Handler` , `Executor`) mà sách đã dùng rải rác từ Chương 13 tới giờ.

Chương 19: Xử lý bất đồng bộ — Thread/Executor (Java) và Coroutine (mở rộng Kotlin)

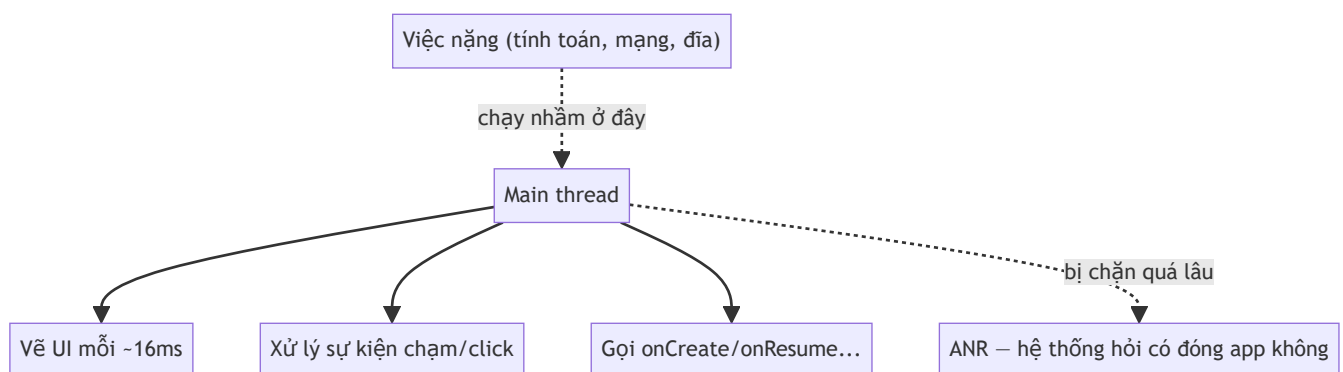
Mục tiêu học

- Hệ thống hoá lại một nguyên tắc đã lặp lại từ Chương 13: **không làm việc nặng trên main thread**.
- Hiểu vì sao dùng `ExecutorService` thay vì tự tạo `Thread` trực tiếp trong hầu hết trường hợp.
- Biết cách huỷ một tác vụ nền đúng cách (cooperative cancellation), không phải "giết" thread thô bạo.
- Biết `AsyncTask` đã lỗi thời, và Kotlin Coroutine tồn tại như một hướng thay thế hiện đại (ngoài phạm vi code Java của sách).

Code mẫu: `code/ch19-async-thread-executor/`.

19.1 Vì sao main thread quan trọng đến vậy?

Android vẽ lại giao diện khoảng 60 lần/giây (mỗi khung hình ~16ms) — toàn bộ việc này chạy trên **một thread duy nhất gọi là main thread** (cũng là thread chạy mọi callback vòng đời Activity/Fragment, mọi `onClickListener`). Bất kỳ đoạn code nào chạy quá lâu trên thread này đều làm khung hình bị bỏ lỡ (giật, lag) — và nếu chặn quá ~5 giây, hệ thống hiện hộp thoại "Ứng dụng không phản hồi" (ANR) rồi có thể tự đóng app.



Đây chính là lý do xuyên suốt sách luôn nhấn mạnh: Room (Chương 13) tự chặn bạn làm sai, file I/O (Chương 14) và giờ là mọi tính toán nặng đều cần chuyển ra khỏi main thread.

19.2 Thread thô — hoạt động nhưng thiếu kiểm soát

```
new Thread(() → {  
    int result = doHeavyWork();  
    runOnUiThread(() → textResult.setText(String.valueOf(result)));  
}).start();
```

Cách này **đúng về nguyên tắc** (việc nặng chạy ngoài main thread, cập nhật UI quay lại qua `runOnUiThread()`), nhưng có vấn đề khi dùng ở quy mô lớn hơn: mỗi `new Thread()` tạo ra một thread hệ điều hành thật (tốn tài nguyên để tạo/hủy), không có cơ chế quản lý số lượng thread đang chạy cùng lúc, và không dễ hủy giữa chừng một cách có tổ chức.

19.3 ExecutorService — quản lý thread có tổ chức

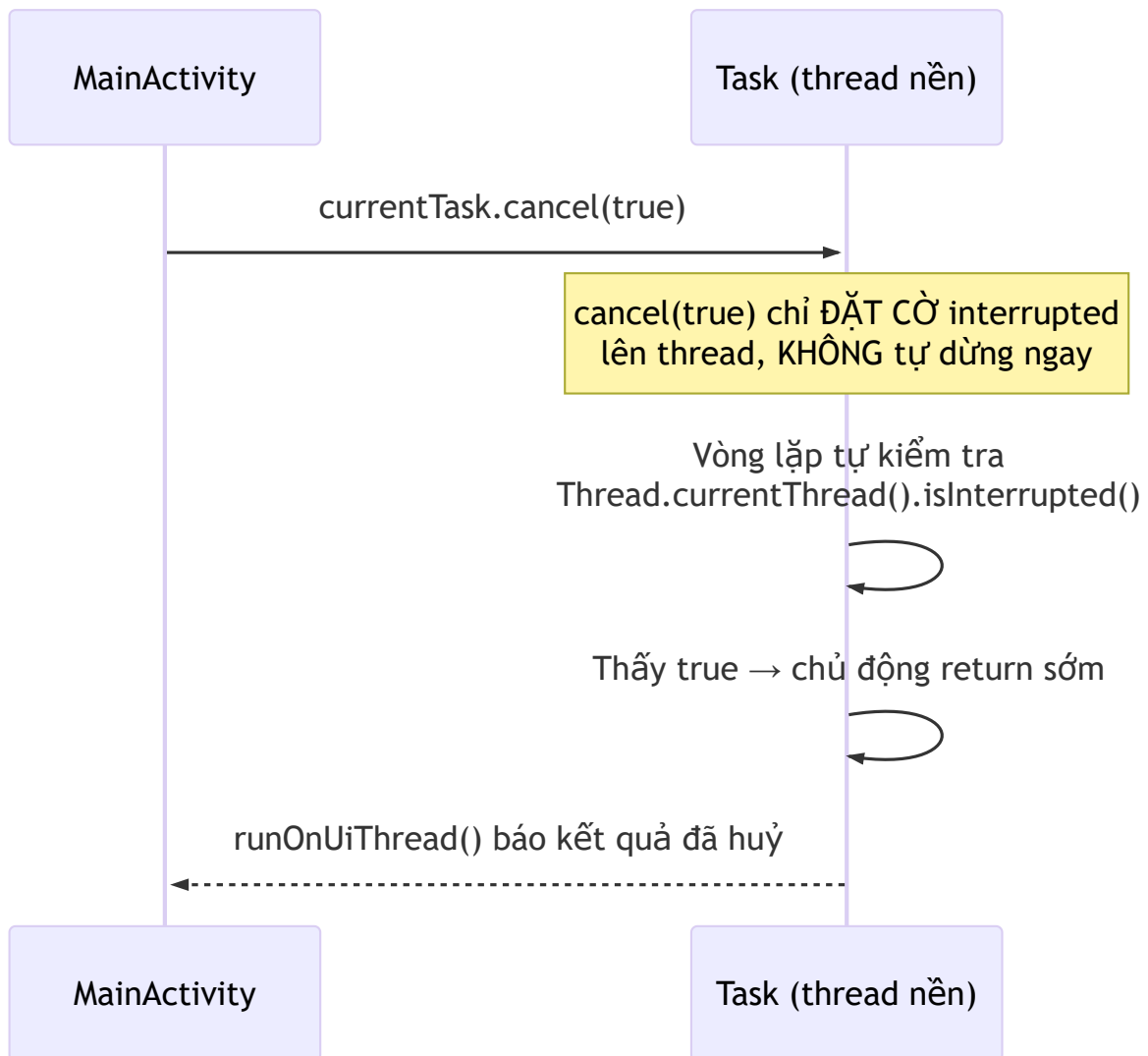
```
private final ExecutorService executor = Executors.newSingleThreadExecutor();  
  
currentTask = executor.submit(() → {  
    int result = PrimeCounter.countPrimes(limit, percent →  
        runOnUiThread(() → binding.progressBar.setProgress(percent)));  
    runOnUiThread(() → binding.textStatus.setText("Xong: " + result));  
});
```

`Executors.newSingleThreadExecutor()` tạo ra **một thread nền duy nhất, tái sử dụng** cho mọi task gửi vào qua `submit()` — task xếp hàng chạy tuần tự, không tạo/hủy thread liên tục như cách 19.2. Executor cũng đã được dùng ở Chương 13 (Room) và Chương 14 (file I/O) — chương này chính thức hệ thống hoá lại mẫu đó.

Vài loại Executor thường gặp:

Loại	Khi nào dùng
<code>Executors.newSingleThreadExecutor()</code>	Các task cần chạy TUẦN TỰ, không chồng chéo (như code mẫu)
<code>Executors.newFixedThreadPool(n)</code>	Nhiều task độc lập, muốn chạy song song có giới hạn
<code>Executors.newCachedThreadPool()</code>	Nhiều task ngắn, số lượng biến động — cần thận có thể tạo quá nhiều thread nếu dùng sai

19.4 Huỷ tác vụ đúng cách — "hợp tác", không "ép buộc"



```
public static int countPrimes(int limit, ProgressCallback callback) {
    int count = 0;
    for (int n = 2; n ≤ limit; n++) {
        if (Thread.currentThread().isInterrupted()) {
            return count; // dừng SỚM một cách chủ động
        }
        if (isPrime(n)) count++;
    }
    return count;
}
```

Điểm quan trọng nhất chương này: `Future.cancel(true)` **không có phép màu nào tự dừng vòng lặp đang chạy** — nó chỉ gọi `Thread.interrupt()`, đặt một cờ nội bộ lên. Code bên trong task phải **tự nguyện kiểm tra** cờ đó (`isInterrupted()`) tại những điểm hợp lý (ở đây là đầu mỗi vòng lặp) và tự

quyết định dừng. Bỏ qua bước kiểm tra này, `cancel(true)` gọi xong nhưng vòng lặp vẫn chạy tới hết — hiện tượng "bấm Huỷ mà không huỷ" rất khó hiểu nếu không biết cơ chế này.

19.5 `AsyncTask` đã lỗi thời — vì sao không dùng?

Nhiều tài liệu cũ dạy `AsyncTask` — lớp này đã bị Google **đánh dấu deprecated (API 30) và gỡ bỏ hoàn toàn khỏi Android 13 SDK**. Lý do: hành vi chạy tuần tự/song song của nó thay đổi khó lường giữa các phiên bản Android, và dễ gây rò rỉ Activity nếu dùng sai. Nếu gặp code cũ dùng `AsyncTask`, nên chuyển sang `ExecutorService` (như chương này) hoặc `WorkManager` (Chương 18) tùy tình huống.

19.6 Kotlin Coroutine — hướng đi hiện đại (ngoài phạm vi code Java)

Cộng đồng Android hiện đại phần lớn đã chuyển sang **Kotlin Coroutines** cho xử lý bất đồng bộ — cú pháp gần giống code đồng bộ thông thường (`suspend fun`) nhưng chạy không chặn thread, tích hợp sẵn cơ chế huỷ (`cancel()`) hợp tác tương tự mục 19.4 nhưng được ngôn ngữ hỗ trợ trực tiếp) và xử lý lỗi gọn hơn nhiều so với callback lồng nhau. Sách này viết bằng Java nên không đi sâu vào cú pháp Coroutine, nhưng **nhên biết tên và khái niệm này tồn tại** — nếu sau này chuyển một phần project sang Kotlin (hoàn toàn có thể làm xen kẽ trong cùng một project Android), đây là hướng đáng tìm hiểu tiếp theo.

Bài tập

1. Chạy code mẫu, bấm Bắt đầu rồi bấm Huỷ giữa chừng — xác nhận số lượng nguyên tố hiển thị đúng bằng số đã đếm được TỚI THỜI ĐIỂM huỷ, không phải 0.
 2. Thử xoá dòng kiểm tra `Thread.currentThread().isInterrupted()` trong `PrimeCounter.countPrimes()`, build lại — bấm Huỷ và quan sát tác vụ vẫn chạy tới hết (minh chứng trực tiếp cho mục 19.4).
 3. Đổi `Executors.newSingleThreadExecutor()` thành `Executors.newFixedThreadPool(2)`, thử bấm Bắt đầu hai lần liên tiếp với hai giá trị giới hạn khác nhau — quan sát cả hai cùng chạy song song thay vì xếp hàng.
-

Lỗi thường gặp

- **Cập nhật View trực tiếp từ trong task chạy trên Executor** (quên `runOnUiThread()`): ném `CalledFromWrongThreadException` ngay lập tức — Android chủ động chặn hành vi này, tương tự tinh thần bảo vệ của Room ở Chương 13.
- **Tương `cancel(true)` dừng tác vụ ngay lập tức**: sai, xem mục 19.4 — tác vụ chỉ dừng nếu chủ động kiểm tra cờ interrupted.

- **Không gọi** `executor.shutdown()` / `shutdownNow()` **khi Activity bị huỷ**: executor (và thread nó tạo ra) tiếp tục tồn tại dù không còn Activity nào cần tới, rò rỉ tài nguyên.
 - **Dùng** `AsyncTask` **trong code mới**: đã bị gỡ khỏi SDK ở các phiên bản Android mới — code sẽ không biên dịch được nếu compileSdk đủ mới.
-

Tóm tắt & tiếp theo

Bạn đã có bộ công cụ đầy đủ cho xử lý bất đồng bộ trong Java: `Executor` để chạy nền có tổ chức, và cơ chế huỷ hợp tác đúng cách. Chương 20 khép lại Phần 3 với `Notification` — cách giao tiếp với người dùng ngay cả khi app không ở foreground, đã dùng sơ bộ ở Chương 16, giờ sẽ đi sâu đầy đủ.

Chương 20: Notification

Mục tiêu học

- Tạo `NotificationChannel` đúng cách, hiểu vì sao không thể đổi importance bằng code sau khi tạo.
- Xây một notification đầy đủ: tiêu đề, nội dung dài (`BigTextStyle`), tap để mở app, nút hành động xử lý ngay trên thông báo.
- Hiểu `PendingIntent` là gì và vì sao nó khác `Intent` thường.
- Xin đúng quyền `POST_NOTIFICATIONS` (Android 13+).

Code mẫu: `code/ch20-notification/` — dùng lại và mở rộng notification đã xuất hiện sơ bộ ở Chương 16.

20.1 `NotificationChannel` — bắt buộc từ Android 8

```
NotificationChannel channel = new NotificationChannel(
    CHANNEL_ID,
    "Nhắc nhở",
    NotificationManager.IMPORTANCE_DEFAULT);
NotificationManager manager = getSystemService(NotificationManager.class);
manager.createNotificationChannel(channel);
```

Từ Android 8 (API 26), **mọi notification phải thuộc về một channel** — người dùng kiểm soát riêng từng channel (tắt tiếng, đổi mức độ quan trọng) qua Settings, thay vì chỉ bật/tắt toàn bộ thông báo của app như trước. Code mẫu tạo channel trong `Application.onCreate()` (Chương 6 đã giới thiệu `Application` — đây là một ví dụ thực tế cho việc "khởi tạo một lần khi app chạy" mà chương đó nói tới).

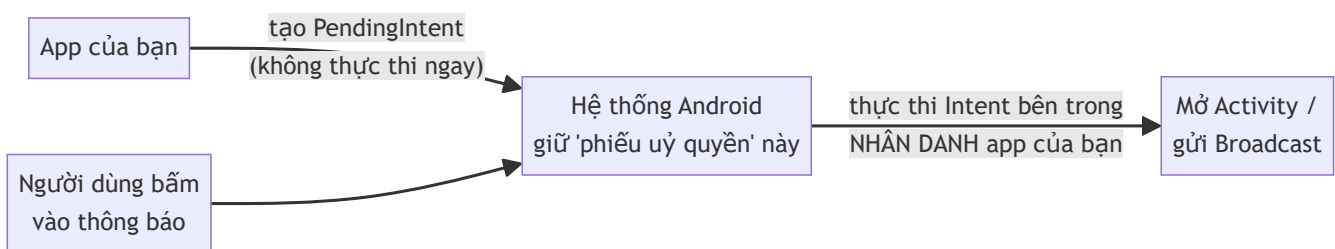
Lưu ý quan trọng: gọi `createNotificationChannel()` nhiều lần với cùng ID là an toàn (không tạo trùng), nhưng **một khi channel đã tồn tại, code không thể tự đổi `importance` của nó nữa** — chỉ người dùng tự đổi được trong Settings. Đặt đúng mức quan trọng ngay từ đầu, và tạo channel MỚI (ID khác) nếu cần một mức khác hẳn về sau.

20.2 Xây notification đầy đủ

```
NotificationCompat.Builder builder = new NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(android.R.drawable.ic_dialog_info)
    .setContentTitle("Đừng quên ôn bài!")
    .setContentText(longText)
    .setStyle(new NotificationCompat.BigTextStyle().bigText(longText))
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .setAutoCancel(true);
```

`setStyle(BigTextStyle ...)` cho phép người dùng kéo giãn thông báo để đọc đầy đủ nội dung dài — thiếu dòng này, nội dung dài sẽ bị cắt ngắn còn một dòng. `setAutoCancel(true)` khiến thông báo tự biến mất khi người dùng bấm vào nó (không cần tự gọi `cancel()`).

20.3 `PendingIntent` — "phiếu uỷ quyền" cho hệ thống



`Intent` thường thực thi ngay khi bạn gọi `startActivity()` / `sendBroadcast()`. `PendingIntent` thì khác: nó là một **"phiếu uỷ quyền"** — bạn tạo ra và trao cho hệ thống (qua notification), hệ thống giữ nó và chỉ thực thi khi có sự kiện tương ứng xảy ra (người dùng bấm vào thông báo/nút), **thực thi nhân danh app của bạn** dù lúc đó app có đang chạy hay không.

```
Intent contentIntent = new Intent(context, MainActivity.class);
PendingIntent pendingIntent = PendingIntent.getActivity(
    context, notificationId, contentIntent,
    PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);
```

`FLAG_IMMUTABLE` (bắt buộc khai báo rõ từ Android 12) nghĩa là hệ thống **không được sửa đổi** nội dung Intent bên trong trước khi thực thi — chỉ dùng `FLAG_MUTABLE` khi có lý do kỹ thuật cụ thể cần hệ thống chỉnh sửa (hiếm gặp, ngoài phạm vi sách này).

20.4 Nút hành động ngay trên thông báo — không cần mở app

```
Intent markReadIntent = new Intent(context, MarkReadReceiver.class);
markReadIntent.setAction(MarkReadReceiver.ACTION_MARK_READ);
PendingIntent markReadPendingIntent = PendingIntent.getBroadcast(
    context, notificationId, markReadIntent,
    PendingIntent.FLAG_UPDATE_CURRENT | PendingIntent.FLAG_IMMUTABLE);

builder.addAction(icon, "Đánh dấu đã đọc", markReadPendingIntent);
```

Đây là lúc `BroadcastReceiver` (Chương 17) và `Notification` phối hợp: bấm nút hành động gửi một broadcast, `MarkReadReceiver.onReceive()` xử lý (ở đây là huỷ notification và hiện Toast) **mà không cần mở Activity nào cả**.

Ngoại lệ hợp lệ đối với nguyên tắc "luôn đăng ký động" ở Chương 17: `MarkReadReceiver` phải khai báo **tĩnh** trong manifest, vì `PendingIntent` có thể được kích hoạt bất kỳ lúc nào sau này — kể cả khi tiến trình app đã bị hệ thống kill hoàn toàn, lúc đó không có Activity/Service nào đang sống để tự đăng ký receiver động.

```
<receiver android:name=".MarkReadReceiver" android:exported="false" />
```

20.5 Xin quyền `POST_NOTIFICATIONS` (Android 13+)

```
if (Build.VERSION.SDK_INT ≥ Build.VERSION_CODES.TIRAMISU
    && ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS)
    ≠ PackageManager.PERMISSION_GRANTED) {
    requestPermission.launch(Manifest.permission.POST_NOTIFICATIONS);
} else {
    sendReminderNotification();
}
```

Từ Android 13 (API 33), hiển thị BẤT KỲ notification nào cũng cần quyền runtime — dùng đúng Activity Result API đã học ở Chương 8. Chương 26 sẽ giải thích đầy đủ mô hình permission runtime của Android; ở đây chỉ cần biết notification là một trong những trường hợp cần xin quyền.

Bài tập

1. Chạy code mẫu, thử cả hai luồng: bấm vào nội dung thông báo (mở app), và bấm nút hành động (không mở app) — quan sát khác biệt.
 2. Đổi `IMPORTANCE_DEFAULT` thành `IMPORTANCE_HIGH` trong `ReminderApplication`, gỡ cài đặt app rồi cài lại (để channel cũ không còn tồn tại), quan sát thông báo mới hiện dạng "heads-up" (nổi lên đầu màn hình) thay vì chỉ nằm trong thanh trạng thái.
 3. Thử bỏ `FLAG_IMMUTABLE` khỏi `PendingIntent`, build lại trên compileSdk 34 — quan sát Android Studio/Gradle có cảnh báo hoặc lỗi gì liên quan tới yêu cầu bắt buộc chỉ định rõ mutability.
-

Lỗi thường gặp

- **Quên tạo `NotificationChannel` trên Android 8+:** notification không hiện ra, không có lỗi rõ ràng nào cả — rất khó debug nếu không biết trước quy tắc này.
 - **Đổi `importance` của channel đã tồn tại bằng code, tưởng sẽ có tác dụng:** không có gì thay đổi — phải hướng dẫn người dùng tự vào Settings, hoặc tạo channel ID mới.
 - **Dùng `sendBroadcast()` / `startActivity()` trực tiếp thay vì `PendingIntent` khi cấu hình notification:** không biên dịch được — API notification yêu cầu đúng kiểu `PendingIntent`, đây là điểm khác biệt nền tảng đã giải thích ở mục 20.3.
 - **Quên xin quyền `POST_NOTIFICATIONS` trên Android 13+:** `notify()` gọi xong không ném lỗi, nhưng notification âm thầm không hiển thị — kiểm tra kỹ quyền trước khi kết luận code sai.
-

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 3 — Chạy ngầm & bất đồng bộ**: Service, BroadcastReceiver, WorkManager, xử lý đa luồng, và Notification — năm mảnh ghép phối hợp chặt chẽ với nhau trong phần lớn ứng dụng Android thực tế. Phần 4 chuyển hướng sang giao tiếp mạng, bắt đầu với việc gọi HTTP API ở Chương 21.

Phần 4 — Mạng & API

Chương 21: Gọi HTTP với OkHttp

Mục tiêu học

- Khai báo đúng quyền `INTERNET` và hiểu vì sao Android không tự cho phép truy cập mạng.
- Gửi được một request GET bằng OkHttp, xử lý response bất đồng bộ.
- Hiểu callback của OkHttp chạy trên thread nào — và vì sao phải `runOnUiThread()`.
- Phân biệt lỗi MẠNG (không kết nối được) với lỗi HTTP (server phản hồi nhưng báo lỗi).

Code mẫu: [code/ch21-http-okhttp/](#). Chương này cố tình dùng OkHttp thuần + parse JSON thủ công để bạn thấy rõ những gì Retrofit (Chương 22) sẽ tự động hoá giúp bạn.

21.1 Quyền `INTERNET`

```
<uses-permission android:name="android.permission.INTERNET" />
```

Đây là quyền **duy nhất trong sách không cần xin runtime** (khác `POST_NOTIFICATIONS` ở Chương 20) — chỉ cần khai báo trong manifest là đủ, vì Google xếp nó vào nhóm quyền "normal" (rủi ro thấp), sẽ giải thích đầy đủ mô hình phân loại quyền ở Chương 26. Thiếu dòng này, mọi request mạng sẽ thất bại với `SecurityException`, dù logic code hoàn toàn đúng — lỗi hay gặp nhất với người mới bắt đầu phần mạng.

21.2 Gửi một request GET

```
private final OkHttpClient client = new OkHttpClient.Builder()
    .connectTimeout(10, TimeUnit.SECONDS)
    .readTimeout(10, TimeUnit.SECONDS)
    .build();

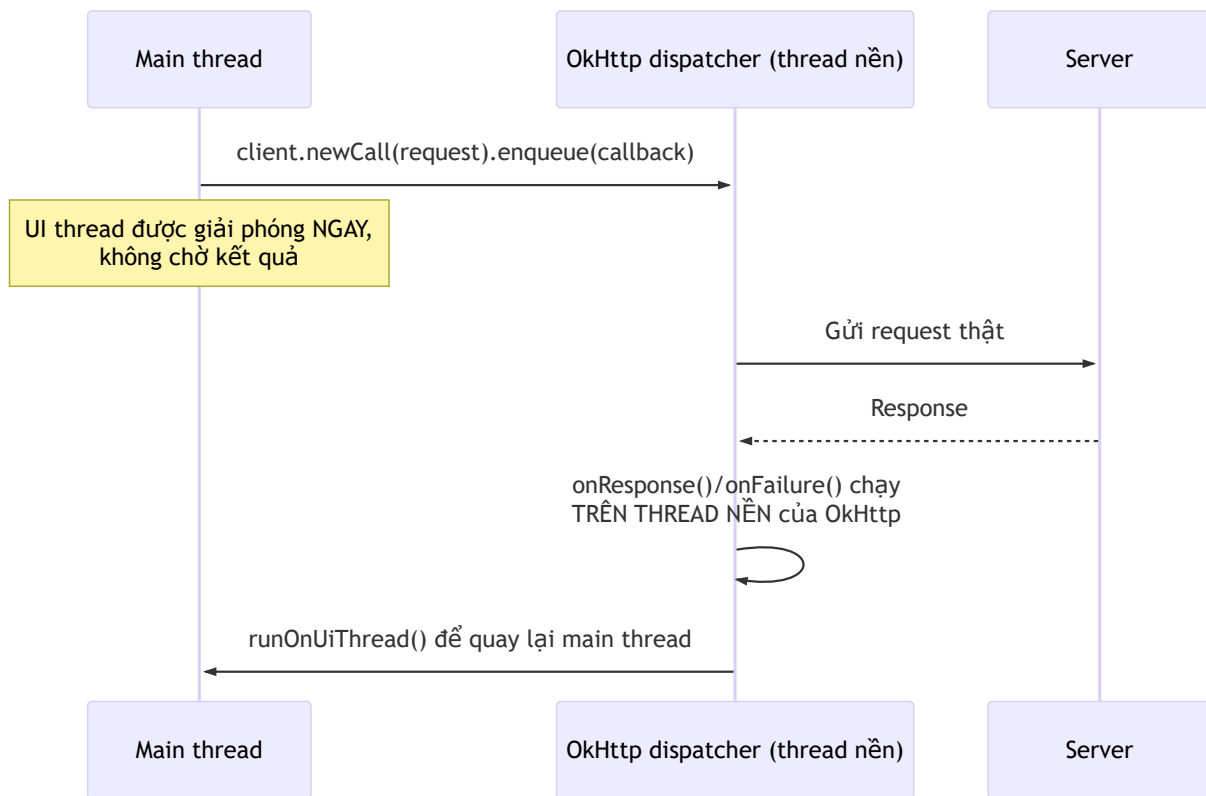
Request request = new Request.Builder()
    .url("https://jsonplaceholder.typicode.com/posts")
    .get()
    .build();

client.newCall(request).enqueue(new Callback() {
    @Override
    public void onFailure(Call call, IOException e) { ... }

    @Override
    public void onResponse(Call call, Response response) throws IOException { ... }
});
```

`OkHttpClient` nên được **tạo một lần, dùng lại cho cả app** — nó tự quản lý connection pool bên trong, tạo mới liên tục cho mỗi request sẽ lãng phí và chậm hơn hẳn. `enqueue()` gửi request bất đồng bộ (không chặn thread gọi nó) — có `execute()` đồng bộ nhưng **không bao giờ gọi trên main thread** (Android chủ động ném `NetworkOnMainThreadException` nếu bạn thử làm vậy, tương tự tinh thần bảo vệ của Room ở Chương 13).

21.3 Callback chạy trên thread nào?



Đây là điểm dễ nhầm nhất: callback `onResponse` / `onFailure` **không chạy trên main thread** — khác hẳn Retrofit ở Chương 22 (Retrofit tự động đưa callback về main thread khi chạy trên Android). Với OkHttp thuần, mọi cập nhật UI trong callback đều phải bọc qua `runOnUiThread()`, đúng nguyên tắc đã lặp lại xuyên suốt từ Chương 13.

```
@Override
public void onResponse(Call call, Response response) {
    String json = response.body().string();
    List<Post> posts = parsePosts(json);
    runOnUiThread(() -> adapter.submitList(posts)); // bắt buộc
}
```


21.4 Hai loại lỗi khác nhau — đừng gộp chung

```
@Override
public void onFailure(Call call, IOException e) {
    // Lỗi MẠNG: không kết nối được server (mất mạng, timeout, sai domain...)
    runOnUiThread() → showError("Lỗi mạng: " + e.getMessage());
}

@Override
public void onResponse(Call call, Response response) {
    if (!response.isSuccessful()) {
        // Lỗi HTTP: SERVER ĐÃ TRẢ LỖI, nhưng với mã lỗi (404, 500...)
        runOnUiThread() → showError("Server trả lỗi HTTP " + response.code());
        return;
    }
    // response.isSuccessful() true - xử lý dữ liệu bình thường
}
```

Hai tình huống cần thông báo khác nhau cho người dùng: `onFailure` nghĩa là **request chưa bao giờ tới được server** (kiểm tra lại kết nối mạng); `response.isSuccessful() == false` bên trong `onResponse` nghĩa là **server đã nhận và xử lý, nhưng trả về lỗi** (ví dụ dữ liệu không tồn tại, server đang gặp sự cố) — hai loại lỗi này cần hướng xử lý khác nhau, sẽ bàn kỹ hơn ở Chương 23.

21.5 Parse JSON thủ công — tại sao chương này cố tình làm "khó"?

```
private List<Post> parsePosts(String json) throws JSONException {
    List<Post> posts = new ArrayList<>();
    JSONArray array = new JSONArray(json);
    for (int i = 0; i < array.length(); i++) {
        JSONObject obj = array.getJSONObject(i);
        posts.add(new Post(
            obj.getInt("id"),
            obj.getInt("userId"),
            obj.getString("title"),
            obj.getString("body")));
    }
    return posts;
}
```

Với API có nhiều field, lồng nhau nhiều tầng, cách viết tay này nhanh chóng trở nên dài dòng và dễ gõ sai tên field (`"titel"` thay vì `"title"` chỉ phát hiện lúc chạy). Chương 22 giới thiệu Gson/Retrofit tự động hoá toàn bộ phần này — cố tình cho bạn thấy công việc thủ công trước để hiểu rõ **Retrofit thực chất đang làm gì bên dưới**, thay vì dùng nó như một "hộp đen".

Bài tập

1. Chạy code mẫu, tải danh sách bài viết thành công.
 2. Bật chế độ máy bay, bấm tải lại — xác nhận thấy đúng thông báo lỗi MẠNG (`onFailure`).
 3. Sửa tạm `POSTS_URL` thành `".../postss"` (sai đường dẫn, JSONPlaceholder trả 404) — xác nhận thấy đúng thông báo lỗi HTTP (khác thông báo ở bài tập 2).
-

Lỗi thường gặp

- **Quên khai báo** `<uses-permission android:name="android.permission.INTERNET" />`: mọi request thất bại với `SecurityException`, dễ nhầm tưởng lỗi code logic.
 - **Gọi `execute()` (đồng bộ) trên main thread thay vì `enqueue()`**: crash ngay với `NetworkOnMainThreadException`.
 - **Cập nhật View trực tiếp trong `onResponse` / `onFailure` mà quên `runOnUiThread()`**: `CalledFromWrongThreadException`, cùng nguyên nhân đã gặp ở Chương 19.
 - **Gộp chung xử lý lỗi mạng và lỗi HTTP vào một nhánh**: người dùng nhận thông báo mơ hồ ("Có lỗi xảy ra") thay vì gợi ý hữu ích ("Kiểm tra kết nối mạng" so với "Server đang bảo trì").
-

Tóm tắt & tiếp theo

Bạn đã gọi được API thật, hiểu rõ cơ chế bất đồng bộ và hai loại lỗi mạng/HTTP. Chương 22 giới thiệu Retrofit + Gson — xây lại đúng chức năng này với ít code hơn nhiều, tự động hoá phần parse JSON vừa làm thủ công ở đây.

Chương 22: Parse JSON (Gson/Moshi), mapping dữ liệu

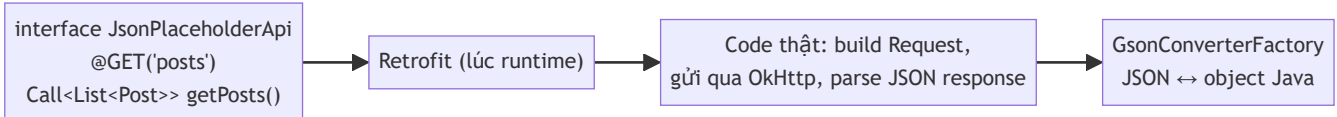
Mục tiêu học

- Hiểu Retrofit tự động hoá những gì bạn vừa làm thủ công ở Chương 21.
- Khai báo API bằng interface + annotation, để Retrofit tự sinh code gọi mạng.
- Dùng Gson map JSON sang object Java, xử lý trường hợp tên field khác nhau bằng `@SerializedName`.
- Biết Moshi tồn tại như một lựa chọn thay thế Gson, và khi nào nên cân nhắc.

Code mẫu: `code/ch22-json-parsing/` — cùng chức năng với Chương 21, viết lại bằng Retrofit + Gson để so sánh trực tiếp.

22.1 Retrofit là gì?

Retrofit biến việc gọi API thành gọi một **hàm Java bình thường**: bạn khai báo interface mô tả API, Retrofit tự sinh code thật (dựa trên OkHttp bên dưới) để gửi request và parse response — hoàn toàn tương tự cách `@Dao` của Room (Chương 13) chỉ cần khai báo, không viết thân hàm.



22.2 Khai báo API bằng interface

```
public interface JsonPlaceholderApi {  
    @GET("posts")  
    Call<List<Post>> getPosts();  
  
    @GET("posts/{id}")  
    Call<Post> getPost(@Path("id") int id);  
}
```

`@GET("posts")` ghép với `baseUrl` thành URL đầy đủ. `@Path("id")` thay thế `{id}` trong đường dẫn bằng giá trị tham số truyền vào — Retrofit hỗ trợ tương tự cho query parameter (`@Query`), request body (`@Body`), header (`@Header`)... đủ dùng cho hầu hết REST API mà không cần tự ghép chuỗi URL bằng tay như Chương 21.

22.3 Khởi tạo Retrofit — một singleton dùng chung

```
public final class ApiClient {
    private static final String BASE_URL = "https://jsonplaceholder.typicode.com/";
    private static volatile JsonPlaceholderApi api;

    public static JsonPlaceholderApi getApi() {
        if (api == null) {
            synchronized (ApiClient.class) {
                if (api == null) {
                    Retrofit retrofit = new Retrofit.Builder()
                        .baseUrl(BASE_URL)
                        .addConverterFactory(GsonConverterFactory.create())
                        .build();
                    api = retrofit.create(JsonPlaceholderApi.class);
                }
            }
        }
        return api;
    }
}
```

Mẫu singleton **giống hệt** `AppDatabase` ở Chương 13 — cùng lý do: chỉ nên có một instance Retrofit (và OkHttpClient bên dưới nó) cho toàn bộ app.

22.4 Gọi API — so sánh trực tiếp với Chương 21

```
ApiClient.getApi().getPosts().enqueue(new Callback<List<Post>>() {
    @Override
    public void onResponse(Call<List<Post>> call, Response<List<Post>> response) {
        if (response.isSuccessful() && response.body() != null) {
            adapter.submitList(response.body()); // đã là List<Post>, không cần tự
parse!
        }
    }

    @Override
    public void onFailure(Call<List<Post>> call, Throwable t) {
        showError(t.getMessage());
    }
});
```

Hai khác biệt quan trọng so với OkHttpClient thuần (Chương 21):

1. **Không còn bước parse JSON thủ công** — `response.body()` đã là `List<Post>` sẵn.
2. **Callback chạy thẳng trên main thread** khi dùng trên Android — không cần `runOnUiThread()` nữa, Retrofit tự phát hiện đang chạy trên Android và tự động đưa callback về đúng main thread.

22.5 `@SerializedName` — khi tên field Java khác tên key JSON

```
public class Post {
    public int id;

    @SerializedName("userId")    // JSON dùng "userId", ta muốn field Java tên khác
    public int authorId;

    public String title;
    public String body;
}
```

Gson mặc định map field JSON và field Java **cùng tên** — field nào không khớp tên sẽ bị bỏ qua (không lỗi, chỉ âm thầm giữ giá trị mặc định). `@SerializedName("tên-key-json")` cho phép đặt tên field Java theo ý muốn (rõ nghĩa hơn, đúng convention đặt tên của bạn) mà vẫn map đúng dữ liệu.

22.6 Moshi — lựa chọn thay thế Gson

Moshi (cũng của Square, cùng nhà với Retrofit/OkHttp) là một thư viện parse JSON khác, thường được nhắc tới như lựa chọn thay thế Gson. Cú pháp tương tự, dùng annotation `@Json(name = "...")` thay cho `@SerializedName`, và cần đổi converter factory:

```
implementation 'com.squareup.retrofit2:converter-moshi:2.11.0'
```

```
Retrofit retrofit = new Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(MoshiConverterFactory.create())
    .build();
```

Khác biệt thực dụng: Moshi kiểm tra chặt hơn (mặc định coi field thiếu là lỗi thay vì âm thầm bỏ qua như Gson), và hiệu năng nhỉnh hơn ở một số benchmark. Với quy mô project trong sách này, cả hai đều dùng tốt — Gson được chọn làm mặc định vì phổ biến hơn trong tài liệu tiếng Việt và dễ tiếp cận hơn cho người mới.

Bài tập

1. Chạy code mẫu, xác nhận `authorId` hiển thị đúng giá trị `userId` gốc từ JSON.
 2. Thêm một field mới vào `Post` khớp tên JSON có sẵn (ví dụ không có trong API này, thử thêm field `email` không tồn tại) — quan sát Gson gán giá trị `null` mặc định thay vì báo lỗi, khác hành vi bạn có thể ngờ.
 3. So sánh trực tiếp số dòng code giữa `MainActivity.java` của Chương 21 và Chương 22 — liệt kê những phần đã biến mất.
-

Lỗi thường gặp

- **Gõ sai giá trị trong `@SerializedName`**: field Java âm thầm nhận giá trị mặc định (`0`, `null`) mà không có lỗi báo — luôn double-check tên field JSON thực tế (xem response mẫu từ Postman/trình duyệt trước khi viết model).
 - **Quên `addConverterFactory(GsonConverterFactory.create())`**: Retrofit không biết cách chuyển JSON thành object, ném lỗi runtime ngay khi gọi API đầu tiên.
 - **Trộn lẫn kiểu dữ liệu** (khai báo field Java là `int` nhưng JSON trả về chuỗi `"123"`): Gson khá khoan dung và tự chuyển đổi được nhiều trường hợp, nhưng không phải luôn luôn — kiểm tra kỹ khi gặp `JsonSyntaxException`.
 - **Dùng `response.body()` mà không kiểm tra `null`**: một số trường hợp server trả `204 No Content` hoặc lỗi bất thường khiến `body()` là `null` dù `isSuccessful()` vẫn `true`.
-

Tóm tắt & tiếp theo

Bạn đã thấy Retrofit + Gson giảm tải đáng kể công việc gọi API so với làm thủ công. Chương 23 khép lại Phần 4 với chủ đề xử lý lỗi mạng bài bản hơn, chiến lược retry, và cache offline-first — kết hợp lại với Room (Chương 13) để app vẫn dùng được khi mất mạng.

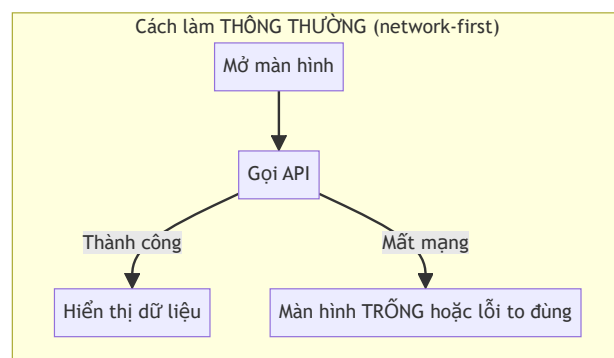
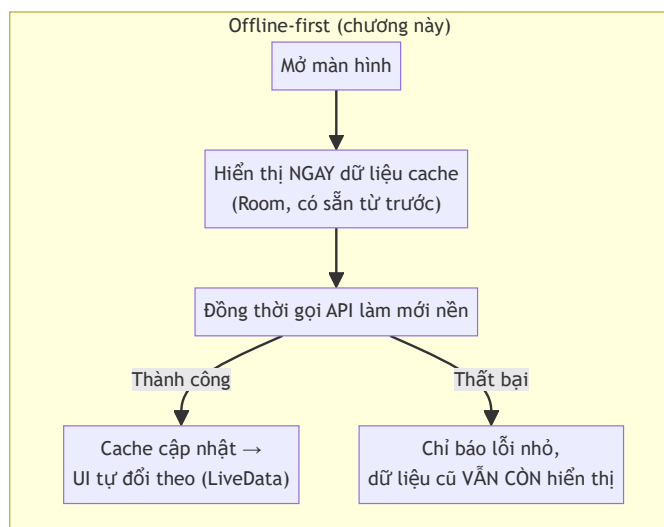
Chương 23: Xử lý lỗi mạng, retry, cache offline-first cơ bản

Mục tiêu học

- Thiết kế màn hình theo tư duy **offline-first**: luôn hiển thị dữ liệu cục bộ trước, mạng chỉ để làm mới.
- Kết hợp Room (Chương 13) và Retrofit (Chương 22) qua một `Repository` gọn gàng.
- Viết cơ chế retry đơn giản, biết phân biệt lỗi nên thử lại và lỗi không nên.
- Hiển thị lỗi mạng mà không phá hỏng trải nghiệm người dùng đang có.

Code mẫu: `code/ch23-network-cache/` — capstone của Phần 4, ghép lại toàn bộ kỹ năng từ Chương 13, 21, 22.

23.1 Offline-first là gì, vì sao quan trọng?



Phần lớn ứng dụng thực tế (mạng xã hội, ghi chú, email...) đều áp dụng tư duy này: **mất mạng không nên đồng nghĩa với "app không dùng được"**. Đây cũng là lý do Chương 13 dạy Room từ rất sớm — nó chính là nền tảng lưu cache cho chương này.

23.2 Repository — gộp hai nguồn dữ liệu thành một API

```
public class PostRepository {  
  
    public LiveData<List<Post>> getCachedPosts() {  
        return dao.getAll(); // luôn đọc từ Room  
    }  
  
    public void refresh(RefreshCallback callback) {  
        fetchWithRetry(1, callback); // gọi mạng, kết quả ghi ngược lại vào Room  
    }  
}
```

UI chỉ cần biết hai việc: **quan sát** `getCachedPosts()` để luôn có dữ liệu hiển thị, và **gọi** `refresh()` khi muốn làm mới — không cần tự phối hợp giữa Room và Retrofit. Đây là bước đệm nhẹ cho khái niệm "Repository pattern" sẽ chính thức xuất hiện trong kiến trúc MVVM ở Chương 28.

```
repository.getCachedPosts().observe(this, adapter::submitList);  
repository.refresh(callback);
```

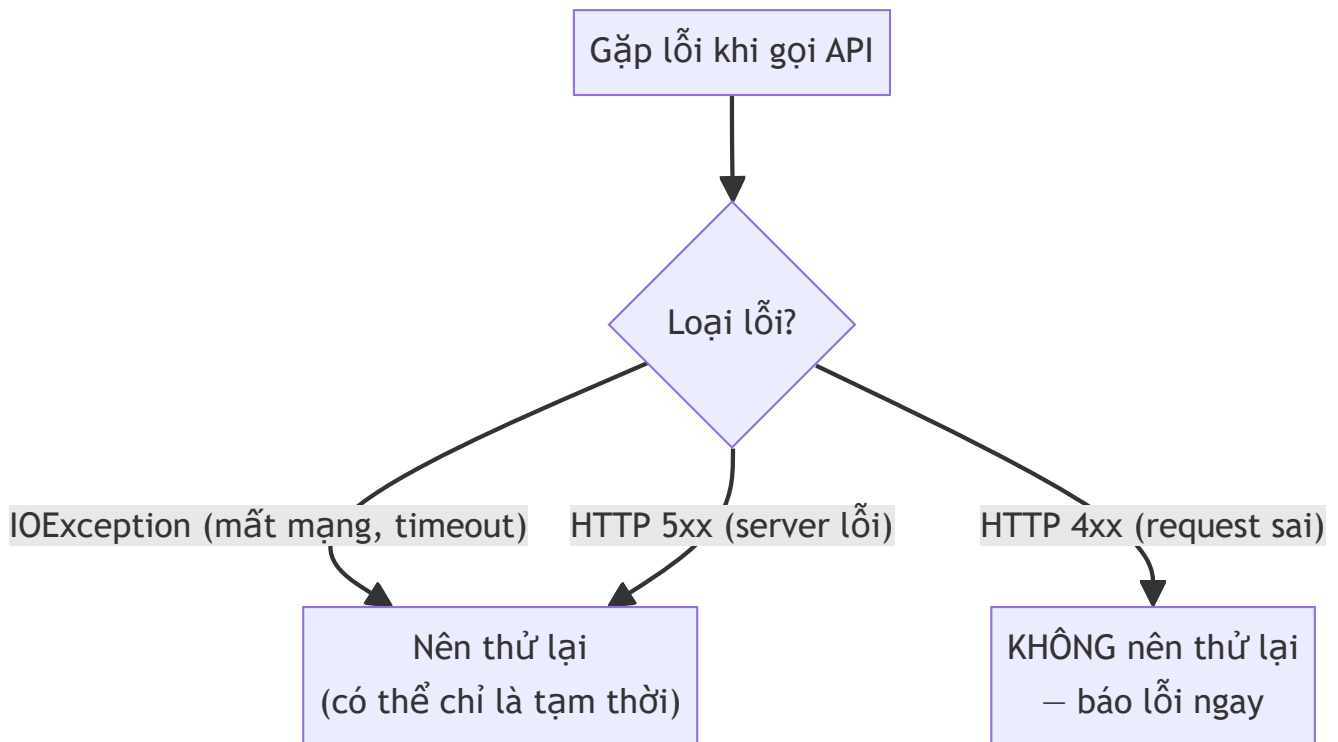
Vòng khép kín quan trọng nhất: khi `refresh()` thành công, nó **ghi dữ liệu mới vào Room**, và vì UI đang `observe()` trực tiếp Room (không phải kết quả API), UI **tự động cập nhật** — không cần code nào nối trực tiếp "API trả về" với "adapter.submitList()" như Chương 22 đã làm.

23.3 Retry — thử lại khi nào, và khi nào không nên

```
private void handleFailure(int attempt, String message, RefreshCallback callback) {  
    if (attempt < MAX_ATTEMPTS) {  
        mainHandler.postDelayed(() → fetchWithRetry(attempt + 1, callback),  
RETRY_DELAY_MS);  
    } else {  
        callback.onError(message);  
    }  
}
```

Nguyên tắc quan trọng hay bị bỏ qua: **không phải lỗi nào cũng nên thử lại**. Lỗi mạng thoáng qua (timeout, mất sóng tạm thời) hợp lý để thử lại. Nhưng lỗi HTTP 4xx (ví dụ 401 Unauthorized, 404 Not Found) là lỗi **do bản chất request sai** — thử lại y hệt sẽ luôn nhận cùng kết quả, chỉ tổ tốn pin/dữ liệu

di động. Code mẫu đơn giản hoá bằng cách thử lại mọi loại lỗi một lần duy nhất — đủ dùng cho quy mô sách, nhưng trong hệ thống thực tế nên kiểm tra `response.code()` để chỉ retry với lỗi 5xx/lỗi mạng, không retry với 4xx.



23.4 Hiện thị lỗi mà không phá dữ liệu đang có

```
@Override
public void onError(String message) {
    binding.progressBar.setVisibility(View.GONE);
    binding.textError.setVisibility(View.VISIBLE);
    binding.textError.setText(getString(R.string.error_refresh_kept_cache, message));
    // KHÔNG gọi adapter.submitList(emptyList()) hay bất kỳ thao tác nào xóa dữ liệu
}
```

Sai lầm thường gặp: khi `refresh()` thất bại, nhiều người có phản xạ xóa danh sách hoặc hiện toàn màn hình lỗi — xóa mất dữ liệu vẫn còn giá trị sử dụng được. Code mẫu chỉ hiện một dòng thông báo nhỏ, **giữ nguyên** danh sách cache đang hiển thị.

Bài tập

1. Chạy code mẫu với mạng bình thường, đóng app, bật chế độ máy bay, mở lại — xác nhận danh sách vẫn đầy đủ.

2. Tắt máy bay, bấm **Làm mới**, xác nhận thông báo lỗi biến mất và dữ liệu (nếu đổi ở server) được cập nhật.
3. Sửa `PostRepository` để phân biệt lỗi HTTP 4xx (không retry, báo lỗi ngay) với lỗi mạng/5xx (giữ nguyên cơ chế retry hiện tại) — dựa vào gợi ý ở mục 23.3.

Lỗi thường gặp

- **Chỉ hiển thị dữ liệu khi API thành công, bỏ qua Room hoàn toàn:** quay lại đúng vấn đề "network-first" ở mục 23.1 — mất mạng là màn hình trống.
- **Retry vô hạn lần không giới hạn:** có thể khiến app gọi API liên tục, tốn pin/dữ liệu di động nghiêm trọng nếu server đang gặp sự cố kéo dài — luôn giới hạn số lần thử (`MAX_ATTEMPTS` trong code mẫu).
- **Xoá sạch dữ liệu hiển thị mỗi khi có lỗi mạng:** trải nghiệm tệ hơn nhiều so với việc chỉ báo lỗi nhỏ và giữ dữ liệu cũ, như đã bàn ở mục 23.4.
- **Insert dữ liệu mới vào Room trên main thread** (quên bọc qua `dbExecutor` như Chương 13 đã dạy): dễ bỏ sót khi code đã "quen tay" gọi trực tiếp sau một chuỗi callback dài.

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 4 — Mạng & API**: từ gọi HTTP thô, tự động hoá bằng Retrofit/Gson, tới thiết kế offline-first hoàn chỉnh kết hợp với Room. Phần 5 chuyển hướng sang chủ đề kiểm thử và bảo mật — bắt đầu với Unit test bằng JUnit ở Chương 24.

Phần 5 — Kiểm thử, Sandbox & Bảo mật

Chương 24: Unit test với JUnit

Mục tiêu học

- Phân biệt **Unit test cục bộ** (chạy trên JVM máy tính) và **Instrumented test** (chạy trên thiết bị/máy ảo, Chương 25).
- Viết được test case với JUnit: trường hợp bình thường, trường hợp biên, trường hợp ném ngoại lệ.
- Hiểu vì sao tách logic ra khỏi Activity giúp việc test dễ dàng hơn nhiều.

Code mẫu: `code/ch24-unit-test-junit/`.

24.1 Hai loại test trong Android — chạy Ở ĐÂU là khác biệt cốt lõi

Instrumented test (app/src/androidTest/) — Chương 25				Unit test cục bộ (app/src/test/)			
Chạy TRÊN máy ảo/thiết bị thật	Có đầy đủ Android framework	Chậm hơn nhiều (giây)	Test được UI, Activity thật	Chạy TRÊN MÁY TÍNH, trong JVM thường	KHÔNG cần máy ảo/thiết bị	Rất nhanh (mili-giây)	KHÔNG gọi được API của android.* thật

Chương này chỉ dùng **Unit test cục bộ** — nằm trong thư mục `app/src/test/java/`, khác thư mục `app/src/main/java/` (code chạy thật) và `app/src/androidTest/java/` (Chương 25). Vì chạy trên JVM thường (không có Android thật), code được test **không được phép** gọi thẳng các API của `android.*` — đây chính là lý do mục 24.2 nhấn mạnh việc tách logic.

24.2 Tách logic ra khỏi Activity để dễ test

```
// PasswordStrengthChecker.java — KHÔNG import bất kỳ thứ gì từ android.*
public final class PasswordStrengthChecker {
    public enum Strength { WEAK, MEDIUM, STRONG }

    public static Strength check(String password) {
        if (password == null) {
            throw new IllegalArgumentException("password không được null");
        }
        // ... logic thuần Java
    }
}
```

`MainActivity` chỉ **gọi** hàm này khi người dùng gõ, không tự chứa logic rẽ nhánh nào cần test riêng:

```
binding.editPassword.addTextChangedListener(new TextWatcher() {
    @Override
    public void onTextChanged(CharSequence s, int start, int before, int count) {
        PasswordStrengthChecker.Strength strength =
        PasswordStrengthChecker.check(s.toString());
        binding.textStrength.setText(labelFor(strength));
    }
    // ...
});
```

Nguyên tắc thực dụng: **business logic (quy tắc tính toán, xác thực dữ liệu...)** nên là các hàm/class **Java thuần**, tách khỏi Activity/Fragment — không phải vì "đúng chuẩn kiến trúc" trừu tượng, mà vì lý do rất cụ thể: chỉ có code như vậy mới test được bằng Unit test nhanh, không cần khởi động máy ảo mỗi lần chạy thử.

24.3 Viết test case với JUnit

```
public class PasswordStrengthCheckerTest {

    @Test
    public void matKhauNgan_luonYeu() {
        assertEquals(PasswordStrengthChecker.Strength.WEAK,
            PasswordStrengthChecker.check("ab1"));
    }

    @Test
    public void nullNemNgoaiLe() {
        assertThrows(IllegalArgumentException.class,
            () → PasswordStrengthChecker.check(null));
    }
}
```

Mỗi phương thức đánh dấu `@Test` là một trường hợp kiểm thử độc lập. `assertEquals(expected, actual)` so sánh giá trị mong đợi với giá trị thực tế; `assertThrows(ExceptionClass.class, lambda)` xác nhận một đoạn code ném đúng loại ngoại lệ mong muốn.

24.4 Test trường hợp biên (boundary) — nơi lỗi hay ẩn náu nhất

```
@Test
public void bienChinhXacDoDaiToiThieuCuaStrong() {
    // Đủ 3 loại ký tự nhưng CHƯA đủ 10 ký tự → chỉ MEDIUM, không phải STRONG.
    assertEquals(PasswordStrengthChecker.Strength.MEDIUM,
        PasswordStrengthChecker.check("Abc12345"));
}
```

Lỗi lập trình thường không nằm ở trường hợp "bình thường" mà ở đúng **ranh giới** của điều kiện (ví dụ `≥ 10` hay `> 10`, một lỗi off-by-one kinh điển). Thói quen tốt: với mỗi điều kiện dạng so sánh số trong code, luôn viết ít nhất một test case đúng đúng ngay tại ranh giới đó.

24.5 Chạy test — không cần máy ảo, không cần chờ đợi

Trong Android Studio: bấm nút mũi tên xanh cạnh tên class hoặc từng `@Test`. Từ dòng lệnh:

```
.\gradlew.bat testDebugUnitTest
```

Toàn bộ file `PasswordStrengthCheckerTest.java` (6 test case) chạy xong trong tích tắc — đây chính là giá trị thực dụng lớn nhất của Unit test: **phản hồi gần như tức thời** mỗi khi bạn sửa code, so với việc phải build lại app, chờ cài lên máy ảo, rồi tự tay thao tác lại từng bước để kiểm tra thủ công.

Bài tập

1. Chạy toàn bộ test có sẵn, xác nhận cả 6 test case đều pass (màu xanh).
2. Sửa `PasswordStrengthChecker.check()`, đổi điều kiện độ dài tối thiểu của `STRONG` từ `10` xuống `8`, chạy lại test — xác nhận đúng test `bienChinhXacDoDaiToiThieuCuaStrong` chuyển sang fail (vì giờ nó mong đợi kết quả khác với hành vi mới).
3. Viết thêm một test case cho chuỗi rỗng (`""`) — dự đoán kết quả trước, sau đó chạy để xác nhận.

Lỗi thường gặp

- **Viết logic quan trọng trực tiếp trong `onClick` / `onTextChanged` thay vì tách hàm riêng**: không có gì để gọi từ test, buộc phải test thủ công bằng tay mỗi lần — chậm và dễ bỏ sót trường hợp.
- **Chỉ test "đường vui" (happy path), bỏ qua input null/rỗng/âm**: đây thường là nguồn crash phổ biến nhất trong thực tế, và là chỗ Unit test phát huy giá trị rõ nhất.

- **Test phụ thuộc vào thứ tự chạy của các test khác** (ví dụ test B giả định test A đã chạy trước và để lại trạng thái nào đó): mỗi `@Test` nên độc lập hoàn toàn, JUnit không đảm bảo thứ tự chạy cố định.
 - **Nhầm Unit test cục bộ với Instrumented test:** Cỡ `import android.widget.TextView` trong file ở `app/src/test/java/` sẽ báo lỗi hoặc test luôn fail — loại API đó chỉ tồn tại thật sự trên thiết bị, thuộc phạm vi Chương 25.
-

Tóm tắt & tiếp theo

Bạn đã biết viết Unit test nhanh cho business logic thuần Java. Chương 25 chuyển sang Instrumented test với Espresso — kiểm thử tự động ngay trên giao diện thật, bù đắp cho phần Unit test không chạm tới được.

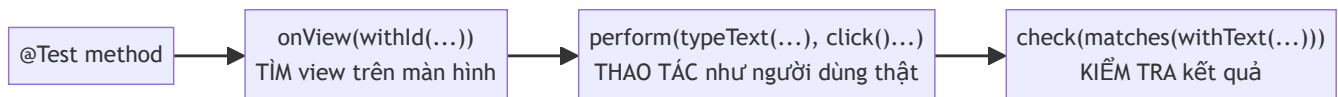
Chương 25: UI test với Espresso

Mục tiêu học

- Viết được Instrumented test — chạy trên máy ảo/thiết bị thật, thao tác lên UI như người dùng.
- Dùng ba bước cốt lõi của Espresso: tìm View (`onView`), thao tác (`perform`), kiểm tra (`check`).
- Biết `ActivityScenarioRule` tự động hoá việc mở/đóng Activity cho mỗi test.

Code mẫu: `code/ch25-ui-test-espresso/` — cùng app Chương 24, thêm test UI.

25.1 Espresso hoạt động thế nào?



Ba bước này lặp lại trong hầu hết mọi test Espresso: **tìm** một View bằng ID hoặc thuộc tính khác, **thao tác** lên nó (gõ chữ, bấm, cuộn...), rồi **kiểm tra** trạng thái sau đó đúng như mong đợi. Khác Unit test (Chương 24) gọi thẳng hàm Java, Espresso mô phỏng chính xác hành vi người dùng thật trên giao diện thật.

25.2 `ActivityScenarioRule` — tự mở/đóng Activity mỗi test

```
@RunWith(AndroidJUnit4.class)
public class MainActivityEspressoTest {

    @Rule
    public ActivityScenarioRule<MainActivity> activityRule =
        new ActivityScenarioRule<>(MainActivity.class);

    @Test
    public void goMatKhauYeu_hienThiNhanYeu() {
        onView(withId(R.id.editPassword))
            .perform(typeText("abc"), closeSoftKeyboard());

        onView(withId(R.id.textStrength))
            .check(matches(withText("Yếu")));
    }
}
```


`@Rule` là cơ chế của JUnit cho phép chạy code thiết lập/dọn dẹp quanh mỗi `@Test` — ở đây `ActivityScenarioRule` tự mở `MainActivity` **trước** mỗi test, tự đóng lại **sau** mỗi test, đảm bảo các test không ảnh hưởng lẫn nhau (một nguyên tắc đã nhắc ở Chương 24: mỗi test case nên độc lập).

25.3 Các thao tác (`ViewActions`) thường dùng

Hàm	Ý nghĩa
<code>typeText(" ... ")</code>	Gõ chữ vào ô nhập (như <code>EditText</code>)
<code>clearText()</code>	Xoá sạch nội dung đang có
<code>click()</code>	Bấm vào View (nút, item danh sách...)
<code>closeSoftKeyboard()</code>	Đóng bàn phím ảo — thường cần sau <code>typeText</code> để tránh bàn phím che View khác
<code>scrollTo()</code>	Cuộn tới một View đang nằm ngoài vùng nhìn thấy

```
onView(withId(R.id.editPassword))
    .perform(typeText("Abcdefgh123"), closeSoftKeyboard());
```

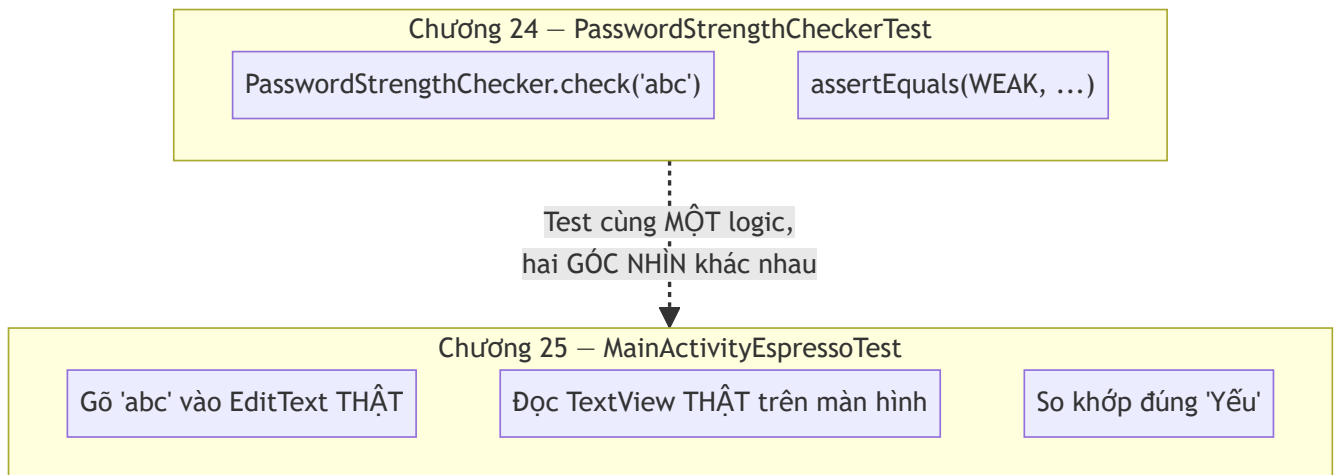
Nhiều thao tác có thể nối chuỗi trong cùng một `perform()` , thực hiện tuần tự theo đúng thứ tự liệt kê.

25.4 Các phép kiểm tra (`ViewAssertions` / `Matchers`) thường dùng

```
onView(withId(R.id.textStrength)).check(matches(withText("Mạnh")));
```

`matches(withText(" ... "))` kiểm tra View có đúng nội dung chữ. Các matcher khác thường gặp: `isDisplayed()` (View có đang hiển thị), `isChecked()` (với `CheckBox` / `Switch` , dùng lại được ngay cho code từ Chương 12), `hasChildCount(n)` (với `ViewGroup`).

25.5 So sánh trực tiếp với Unit test (Chương 24)



Cả hai chương đều đang kiểm tra cùng một quy tắc nghiệp vụ (`PasswordStrengthChecker`), nhưng từ hai góc độ bổ sung cho nhau: Unit test xác nhận **logic tính toán đúng**, Espresso xác nhận **logic đó thật sự được nối đúng dây với giao diện** (đúng ID, đúng sự kiện, hiển thị đúng chỗ). Một dự án lớn cần cả hai loại — Unit test nhiều vì nhanh, Espresso ít hơn nhưng bao quát đúng những luồng người dùng quan trọng nhất.

Bài tập

1. Chạy cả 3 test có sẵn trên máy ảo/thiết bị, xác nhận đều pass.
2. Thêm một test case mới kiểm tra trường hợp `Strength.MEDIUM` (gõ một chuỗi có đúng 2 loại ký tự, độ dài từ 6 trở lên).
3. Cố tình đổi `R.id.textStrength` thành một ID không tồn tại trong một test, chạy lại — đọc thông báo lỗi Espresso đưa ra (`NoMatchingViewException`) để làm quen với loại lỗi này khi gặp trong thực tế.

Lỗi thường gặp

- **Quên** `closeSoftKeyboard()` **sau** `typeText()` : bàn phím ảo có thể che mất View cần thao tác tiếp theo, gây lỗi "view not displayed" dù code test có vẻ đúng logic.
- **Đặt file test vào** `app/src/test/` **thay vì** `app/src/androidTest/` : sai vị trí khiến Gradle không nhận diện đây là Instrumented test, hoặc biên dịch lỗi vì thiếu Android framework thật.
- **Chạy Espresso test mà không có máy ảo/thiết bị nào đang kết nối**: Android Studio báo lỗi rõ ràng "no target device" — cần một máy ảo/thiết bị đã bật, khác Unit test ở Chương 24 không cần gì cả.
- **Viết test phụ thuộc animation/thời gian chờ không xác định**: Espresso mặc định tự đồng bộ với main thread khá tốt, nhưng animation dài hoặc network call thật trong lúc test có thể gây flaky test (lúc pass lúc fail) — nên tránh gọi mạng thật trong Espresso test cơ bản như ở chương

này.

Tóm tắt & tiếp theo

Bạn đã có cả hai công cụ kiểm thử tự động: Unit test cho logic, Espresso cho giao diện. Chương 26 chuyển hướng sang một chủ đề nền tảng khác của Android: mô hình sandbox và hệ thống quyền (permission) — đã được nhắc tới thoáng qua từ Chương 1, giờ sẽ giải thích đầy đủ.

Chương 26: Sandbox của Android — mô hình quyền (permission), runtime permission

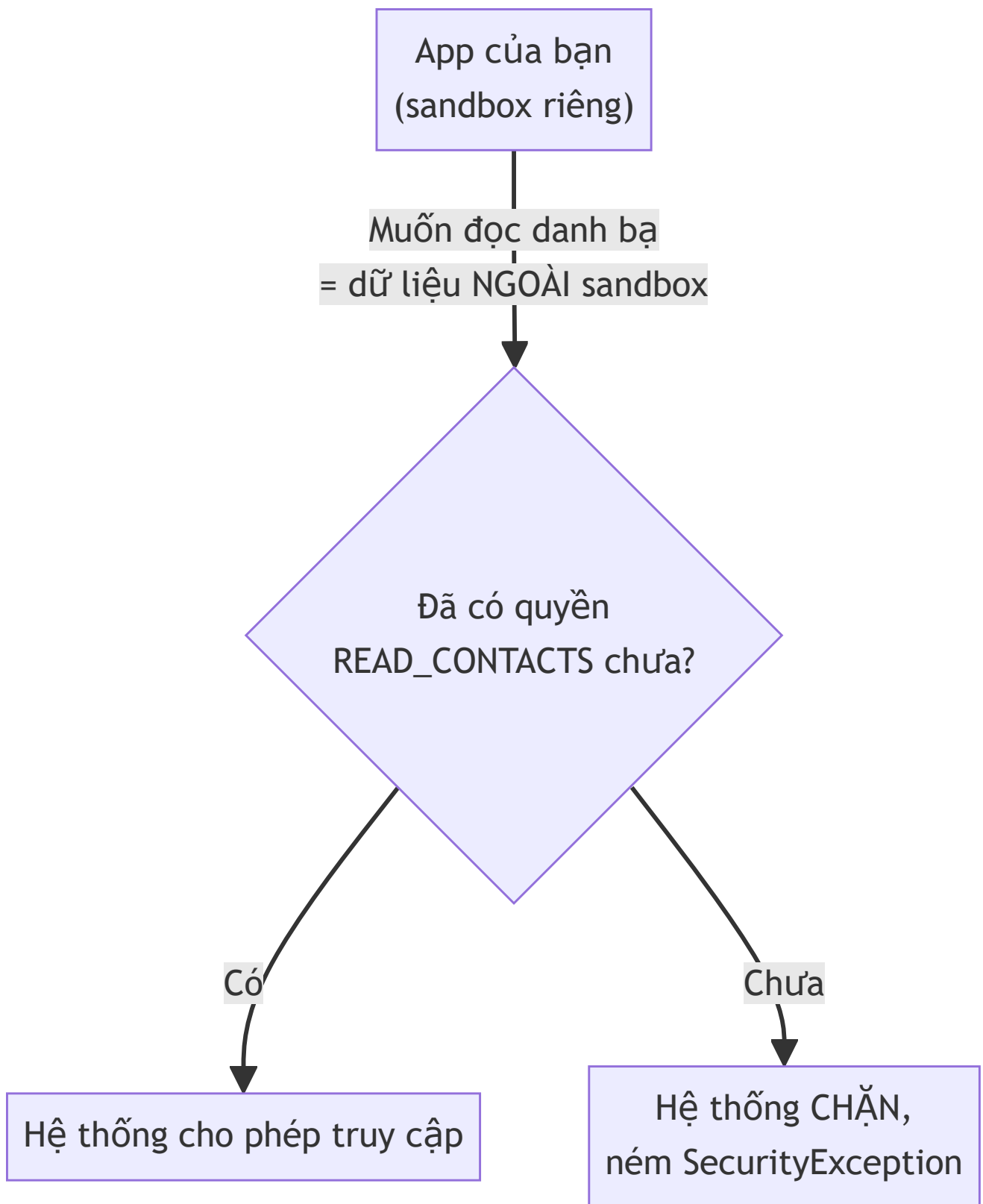
Mục tiêu học

- Hiểu cơ chế sandbox của Android ở tầng hệ điều hành — nhắc lại và đào sâu Chương 1.
- Phân biệt ba nhóm quyền: normal, dangerous, signature.
- Viết đúng luồng xin quyền runtime đầy đủ: kiểm tra, xin, xử lý từ chối, xử lý "không hỏi lại".
- Biết khi nào phải giải thích lý do (rationale) trước khi xin quyền.

Code mẫu: `code/ch26-permission/`. Chương này cũng đã dùng runtime permission ở Chương 16/20 (`POST_NOTIFICATIONS`) — giờ giải thích đầy đủ cơ chế đứng sau.

26.1 Nhắc lại: Sandbox là gì?

Chương 1 đã nhắc: mỗi app Android chạy trong **sandbox** riêng — tiến trình Linux riêng, user ID riêng (`UID`), thư mục dữ liệu riêng mà app khác không đọc/ghi được (đây chính là nền tảng cho `getFilesDir()` "luôn riêng tư" ở Chương 14). Permission là **cơ chế app xin phép vượt ra ngoài sandbox của chính mình** — đọc danh bạ (dữ liệu người dùng), dùng camera (phần cứng), truy cập mạng (Chương 21), v.v.



26.2 Ba nhóm quyền

Signature – nội bộ hệ thống/hãng

Chỉ cấp cho app ký cùng certificate với app khai báo quyền

Hiếm gặp trong app thông thường

Dangerous – ảnh hưởng riêng tư/an toàn

READ_CONTACTS, CAMERA, ACCESS_FINE_LOCATION...

PHẢI xin runtime, người dùng chủ động Allow/Deny

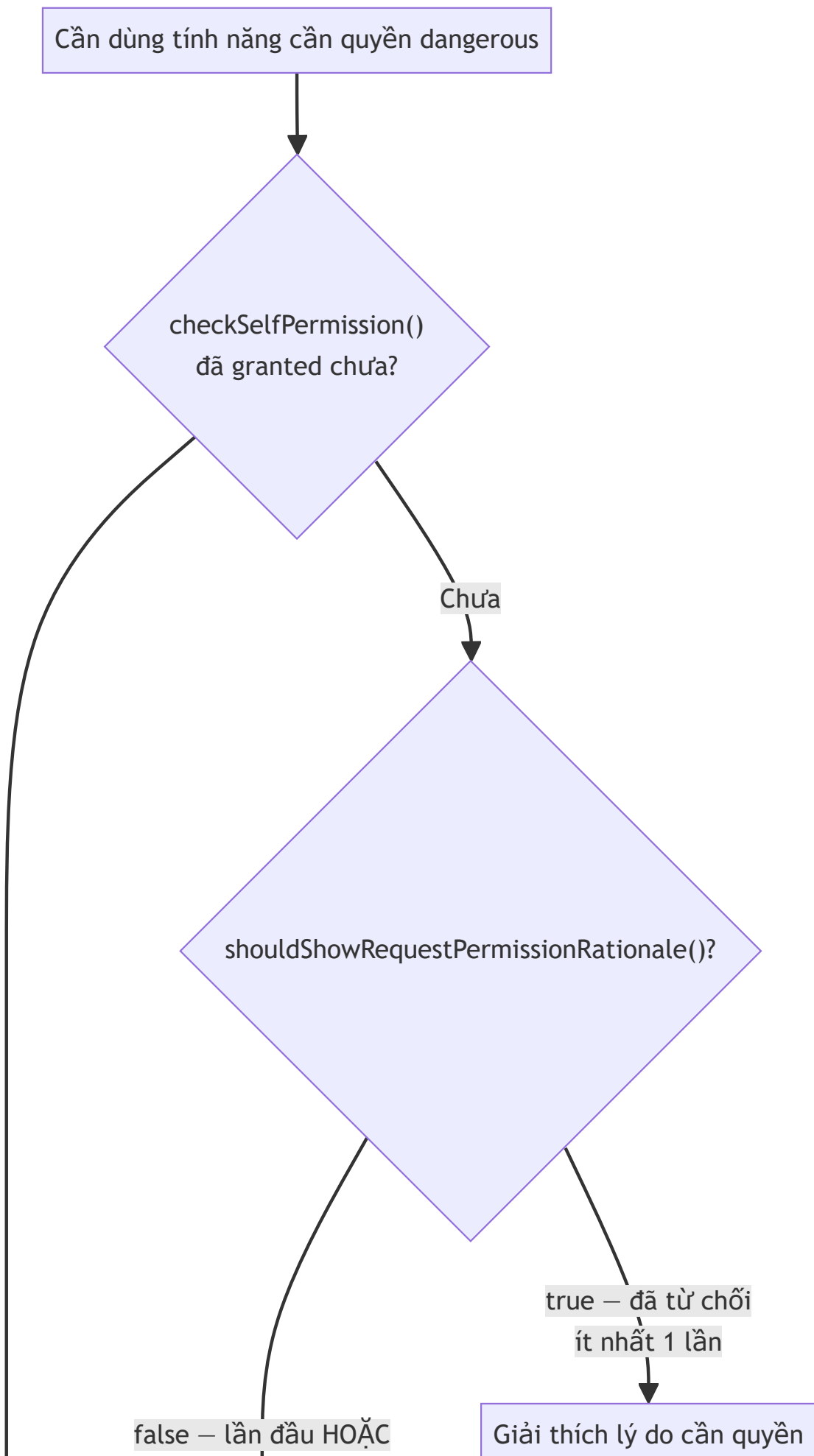
Normal – rủi ro thấp

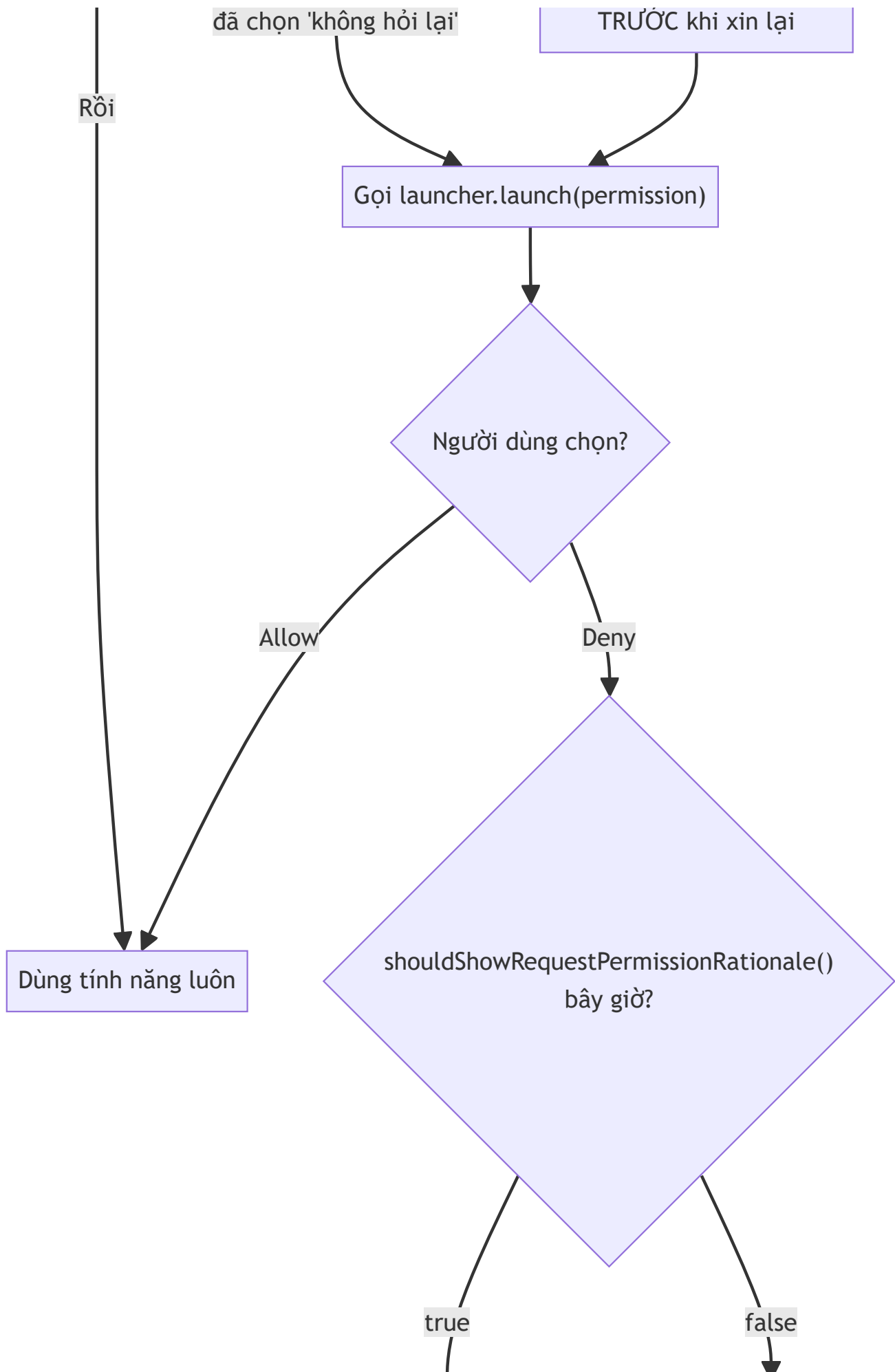
INTERNET (Chương 21)

Tự động cấp khi cài app, chỉ cần khai báo manifest

Sách đã gặp cả ba nhóm: `INTERNET` (Chương 21) và `FOREGROUND_SERVICE` (Chương 16) thuộc nhóm **normal** — chỉ cần khai báo trong manifest là đủ. `POST_NOTIFICATIONS` (Chương 20) và `READ_CONTACTS` (chương này) thuộc nhóm **dangerous** — khai báo manifest là điều kiện CẦN nhưng CHƯA ĐỦ, còn phải xin runtime lúc chạy. Nhóm **signature** hiếm gặp trong app thông thường nên không đi sâu.

26.3 Luồng xin quyền runtime đầy đủ





↓
Có thể xin lại sau

↑
Không thể tự xin lại —
hướng dẫn vào Settings

```
private void checkAndReadContacts() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS)
        == PackageManager.PERMISSION_GRANTED) {
        readContactsCount();
        return;
    }
    if (shouldShowRequestPermissionRationale(Manifest.permission.READ_CONTACTS)) {
        binding.textResult.setText(R.string.rationale_contacts);
    }
    requestPermissionLauncher.launch(Manifest.permission.READ_CONTACTS);
}
```

`launcher` ở đây dùng đúng Activity Result API đã học ở Chương 8/16/20 —
`ActivityResultContracts.RequestPermission()`.

26.4 "Không hỏi lại" — điểm dễ làm sai nhất

```
private void handleDenied() {
    if (!shouldShowRequestPermissionRationale(Manifest.permission.READ_CONTACTS)) {
        // false SAU KHI đã bị từ chối ⇒ người dùng chọn "Không hỏi lại"
        binding.buttonOpenSettings.setVisibility(View.VISIBLE);
    } else {
        binding.textResult.setText(R.string.permission_denied_once);
    }
}
```

`shouldShowRequestPermissionRationale()` trả về `true` / `false` mang hai ý nghĩa hoàn toàn khác nhau tùy ngữ cảnh gọi:

- Gọi **TRƯỚC KHI** xin quyền lần đầu: `false` (chưa từng hỏi, chưa có gì để "nên giải thích").
- Gọi **SAU KHI** bị từ chối: `true` nghĩa là "có thể xin lại, nên giải thích trước"; `false` nghĩa là **người dùng đã chọn "Không hỏi lại"** — từ giờ `launcher.launch()` sẽ tự động trả về `false` ngay lập tức, không hiện hộp thoại nào nữa. Cách duy nhất còn lại là mở màn hình Settings của app cho người dùng tự bật:

```
private void openAppSettings() {
    Intent intent = new Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
    intent.setData(Uri.fromParts("package", getPackageName(), null));
    startActivity(intent);
}
```

26.5 Dùng quyền sau khi được cấp — vẫn nên chạy nền

```
executor.execute(() → {
    int count = 0;
    try (Cursor cursor = getContentResolver().query(
        ContactsContract.Contacts.CONTENT_URI,
        new String[]{ContactsContract.Contacts._ID},
        null, null, null)) {
        if (cursor ≠ null) count = cursor.getCount();
    }
    runOnUiThread(() → binding.textResult.setText( ... ));
});
```

Được cấp quyền không có nghĩa thao tác truy cập dữ liệu trở nên "miễn phí" về hiệu năng —

`ContentResolver.query()` (đọc qua `ContentProvider`, sẽ học kỹ ở Chương 32) vẫn là một lời gọi liên tiến trình, nên vẫn tuân theo nguyên tắc đã xuyên suốt từ Chương 13: không chạy việc có thể chậm trên main thread.

Bài tập

1. Chạy code mẫu, cấp quyền, xác nhận số lượng liên hệ hiển thị đúng.
2. Gỡ cài đặt, cài lại, thử luồng từ chối rồi xin lại — xác nhận thấy đúng dòng giải thích rationale trước khi hộp thoại hiện lại lần hai.
3. Chọn "Không hỏi lại" ở lần từ chối tiếp theo, xác nhận nút "Mở Cài đặt ứng dụng" xuất hiện và dẫn đúng tới màn hình Settings của app.

Lỗi thường gặp

- Chỉ khai báo `<uses-permission>` trong manifest mà quên xin runtime cho quyền dangerous: crash ngay với `SecurityException` khi gọi API cần quyền đó — quên mất bước bắt buộc kể từ Android 6 (API 23).

- **Gọi `requestPermissionLauncher.launch()` liên tục dù người dùng đã chọn "Không hỏi lại":** không có tác dụng gì (không hiện hộp thoại), lãng phí và gây trải nghiệm khó hiểu — luôn kiểm tra `shouldShowRequestPermissionRationale()` để phát hiện đúng tình huống này.
- **Xin quyền ngay khi vừa mở app, trước khi người dùng hiểu vì sao cần nó:** tỷ lệ bị từ chối cao hơn hẳn so với xin đúng lúc người dùng vừa bấm vào tính năng cần quyền đó (nguyên tắc UX quan trọng, không chỉ là vấn đề kỹ thuật).
- **Quên rằng người dùng có thể thu hồi quyền bất kỳ lúc nào trong Settings,** kể cả sau khi đã cấp: luôn `checkSelfPermission()` lại mỗi lần thật sự cần dùng, không giả định quyền đã cấp sẽ mãi mãi còn hiệu lực.

Tóm tắt & tiếp theo

Bạn đã nắm vững mô hình quyền của Android — nền tảng chi phối mọi tính năng "nhạy cảm" từ đây tới hết sách (đặc biệt là Bluetooth/NFC/vị trí ở Phần 7). Chương 27 khép lại Phần 5 với bảo mật ở tầng build: ProGuard/R8 và ký ứng dụng bằng keystore.

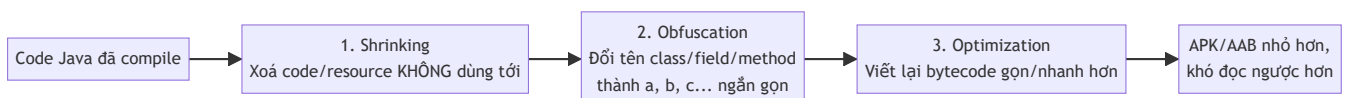
Chương 27: Bảo mật cơ bản — ProGuard/R8, keystore, ký ứng dụng

Mục tiêu học

- Hiểu R8 làm ba việc gì: shrinking, obfuscation, optimization.
- Viết đúng `proguard-rules.pro` cho code dùng reflection (Gson) — tránh lỗi "hoạt động ở debug, crash ở release".
- Hiểu khái niệm ký ứng dụng bằng keystore, và vì sao mất keystore là sự cố nghiêm trọng.
- Biết `mapping.txt` dùng để làm gì và vì sao phải lưu giữ nó.

Code mẫu: `code/ch27-proguard-keystore/` — bật R8 cho app Retrofit + Gson ở Chương 22.

27.1 R8 làm gì với code của bạn?



R8 (thay thế ProGuard cũ, nhưng vẫn dùng chung định dạng file cấu hình `proguard-rules.pro`) chạy tự động khi bật `minifyEnabled true` cho build type `release`. Hai lợi ích cụ thể: **APK/AAB nhỏ hơn** (ảnh hưởng tốc độ tải về, quan trọng ở Chương 39), và **khó dịch ngược hơn** — không phải bất khả xâm phạm, nhưng đủ để cản trở việc đọc hiểu logic app của người không có quyền truy cập source code.

27.2 Vì sao build release "vô cơ" crash mà debug thì không?

Đây là tình huống rất phổ biến và gây khó chịu cho người mới: app chạy hoàn hảo ở bản debug (`minifyEnabled false` mặc định), nhưng bản release đóng gói để test thử lại crash hoặc trả dữ liệu `null` không rõ lý do.

Thủ phạm gần như luôn là: **thư viện dùng reflection** (đọc/ghi field bằng tên chuỗi lúc chạy, không qua lời gọi hàm bình thường lúc biên dịch) — Gson (Chương 22) là ví dụ điển hình:

```
public class Post {
    public int id;
    public String title; // Gson gán giá trị vào field này bằng REFLECTION,
    public String body;  // dựa theo tên field khớp với key JSON "title", "body"
}
```

R8 không "nhìn thấy" được việc field `title` đang bị Gson truy cập theo tên chuỗi — nó chỉ thấy field này **không có lời gọi Java tường minh nào** nên coi là an toàn để đổi tên thành `a`, `b`. Sau khi đổi tên, Gson tìm field tên `"title"` để gán giá trị nhưng field đó giờ tên là `"a"` — **âm thầm không gán được gì cả**, không ném exception, chỉ để `null`.

27.3 Viết `proguard-rules.pro` đúng cách

```
# Giữ nguyên tên mọi field trong Post – Gson cần đúng tên gốc để map JSON.
-keepclassmembers class vn.example.ch27security.Post {
    <fields>;
}

# Giữ thông tin kiểu generic (Call<List<Post>>) – Retrofit/Gson cần lúc runtime
# để biết chính xác phải parse JSON thành kiểu dữ liệu nào.
-keepattributes Signature
-keepattributes *Annotation*
```

Quy tắc thực dụng: **bất kỳ class nào bị truy cập bằng reflection từ bên ngoài code Java thông thường** (model JSON, class dùng qua `Class.forName()`, đối tượng inject bằng Hilt ở Chương 29...) đều cần một dòng `-keep` tương ứng. Một cách khác, gọn hơn cho từng class riêng lẻ, là đánh dấu ngay trong code bằng annotation `@Keep` của AndroidX:

```
@Keep
public class Post { ... }
```

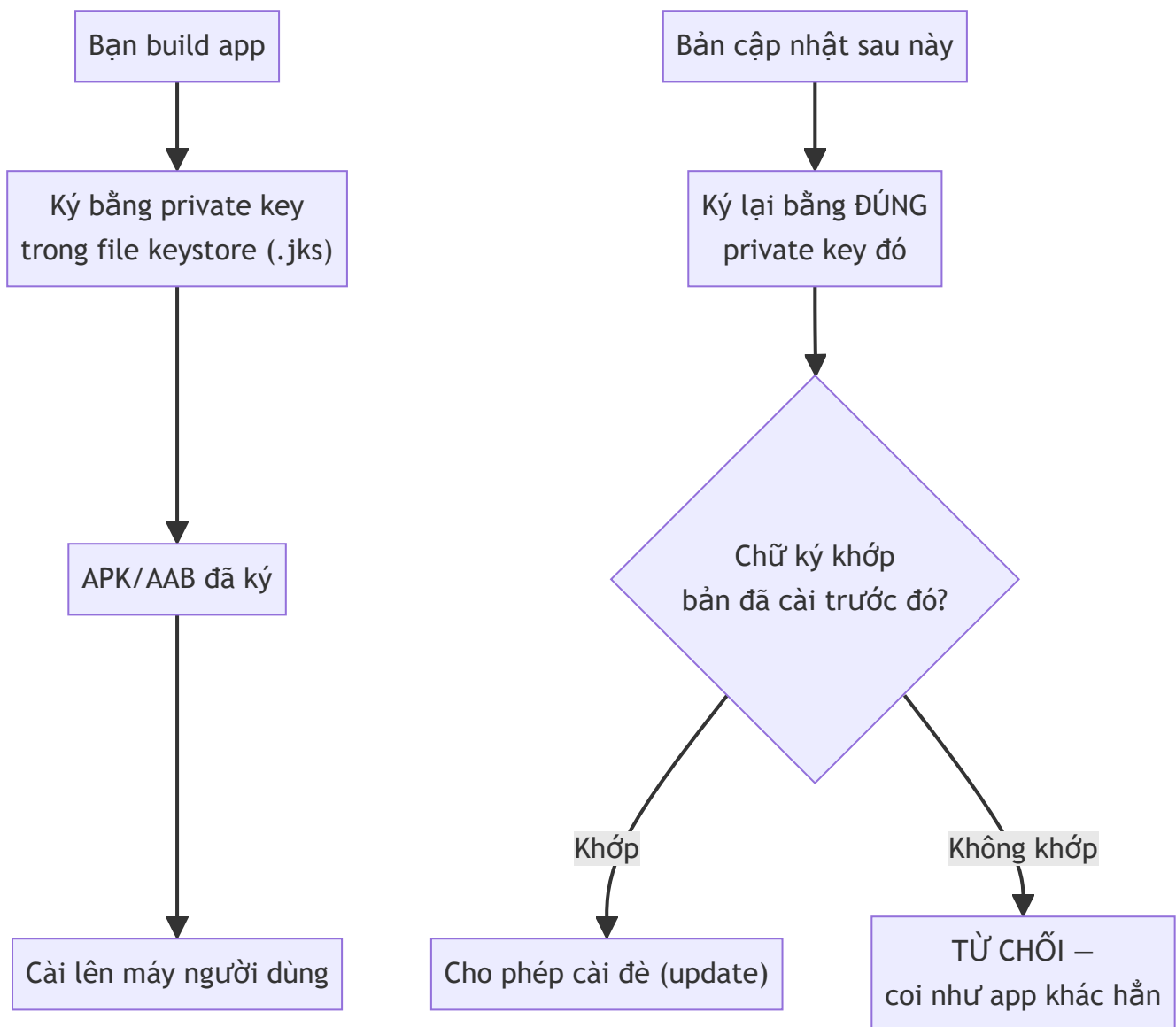
`@Keep` và viết tay trong `proguard-rules.pro` đạt cùng mục đích — chọn một cách nhất quán trong cả project để dễ theo dõi.

27.4 `mapping.txt` — chìa khoá đọc lại crash log

```
app/build/outputs/mapping/release/mapping.txt
```

File này ghi lại bảng đối chiếu tên gốc ↔ tên đã bị R8 đổi (`Post` → `a`, `getPosts` → `b`...). Khi app đã phát hành gặp crash thật ngoài thực tế, log lỗi thu về từ công cụ theo dõi (Chương 40 sẽ giới thiệu Crashlytics) sẽ chỉ hiện tên đã bị làm rối (`a.b.c: NullPointerException`) — **hoàn toàn vô nghĩa nếu không có đúng file `mapping.txt` của đúng phiên bản đó** để dịch ngược lại tên thật. Quy tắc bắt buộc: **lưu trữ `mapping.txt` cho mọi bản release đã phát hành**, thường bằng cách tải nó lên cùng công cụ theo dõi crash hoặc lưu trữ có đánh số phiên bản riêng.

27.5 Keystore — vì sao mọi APK đều phải được ký?



Android dùng chữ ký số (dựa trên cặp khoá bất đối xứng, lưu trong file **keystore**) để đảm bảo: bản cập nhật của một app **chắc chắn đến từ cùng một nhà phát triển** với bản đã cài trước đó — đây chính là nguyên nhân lỗi quen thuộc `INSTALL_FAILED_UPDATE_INCOMPATIBLE` đã gặp ở Chương 5, khi hai bản build được ký bằng hai key khác nhau.

Android Studio tự tạo sẵn một **debug keystore** (key mặc định, không bảo mật, dùng chung cho mọi máy phát triển) để bạn build và test nhanh — **không bao giờ dùng để phát hành**. Trước khi phát hành thật (Chương 38), bắt buộc tự tạo một **release keystore** riêng:

```
keytool -genkeypair -v -keystore release-key.jks -alias my-app-key -keyalg RSA -keysize 2048 -validity 10000
```

Cảnh báo quan trọng nhất chương này: làm mất file keystore này (và không dùng Google Play App Signing — Chương 38 sẽ giải thích cơ chế đó) đồng nghĩa với việc **không bao giờ phát hành được bản cập nhật nào nữa** cho app đã public dưới đúng định danh cũ — phải phát hành app mới hoàn toàn, mất hết lượt cài đặt/đánh giá cũ. Không commit file `.jks` hay mật khẩu của nó vào git.

Bài tập

1. Build thử bản release của code mẫu (`gradlew assembleRelease`), mở `mapping.txt` sinh ra, tìm dòng ánh xạ tên class `Post`.
 2. Thử xoá tạm rule `-keepclassmembers class ... Post` trong `proguard-rules.pro`, build lại, cài bản APK release lên máy ảo (dùng `adb install`, nhớ bản release ở dạng chưa ký cần cấu hình `signingConfig debug` tạm để cài thử được — hoặc đơn giản chỉ so sánh nội dung `mapping.txt` trước/sau mà không cần cài) — quan sát field `title` / `body` của `Post` cũng bị đổi tên trong mapping.
 3. Tự tạo một release keystore thử nghiệm bằng lệnh `keytool` ở mục 27.5 (dùng thông tin giả), sau đó tự xoá nó đi — chỉ để quen thao tác trước khi làm thật ở Chương 38.
-

Lỗi thường gặp

- **Bật `minifyEnabled true` mà không test kỹ bản release trước khi phát hành:** lỗi do thiếu `-keep` rule chỉ xuất hiện ở bản release, dễ lọt qua nếu chỉ test bản debug suốt quá trình phát triển.
 - **Đánh mất `mapping.txt` của một bản đã phát hành:** không thể đọc hiểu crash report từ bản đó nữa, dù bug vẫn còn nguyên trong code.
 - **Làm mất keystore release, hoặc để lộ nó công khai** (commit nhầm lên GitHub): mất keystore = không update được app cũ; lộ keystore = người khác có thể giả mạo phát hành bản cập nhật độc hại dưới danh nghĩa app của bạn.
 - **Tưởng debug keystore đủ an toàn để dùng cho bản phát hành thật:** debug keystore dùng chung mật khẩu mặc định trên mọi máy cài Android Studio — hoàn toàn không có giá trị bảo mật.
-

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 5 — Kiểm thử, Sandbox & Bảo mật**: JUnit, Espresso, mô hình quyền, và bảo mật ở tầng build/ký ứng dụng. Phần 6 chuyển sang kiến trúc nâng cao, bắt đầu với MVVM và `ViewModel` / `LiveData` ở Chương 28 — chính thức hệ thống hoá mẫu "Repository" đã dùng tạm ở Chương 23.

Phần 6 — Kiến trúc nâng cao

Chương 28: MVVM, ViewModel & LiveData

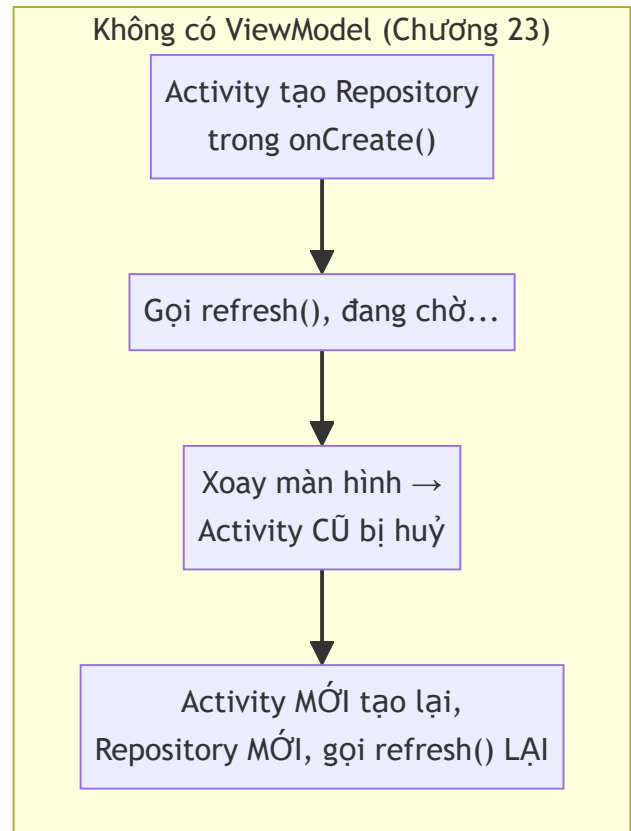
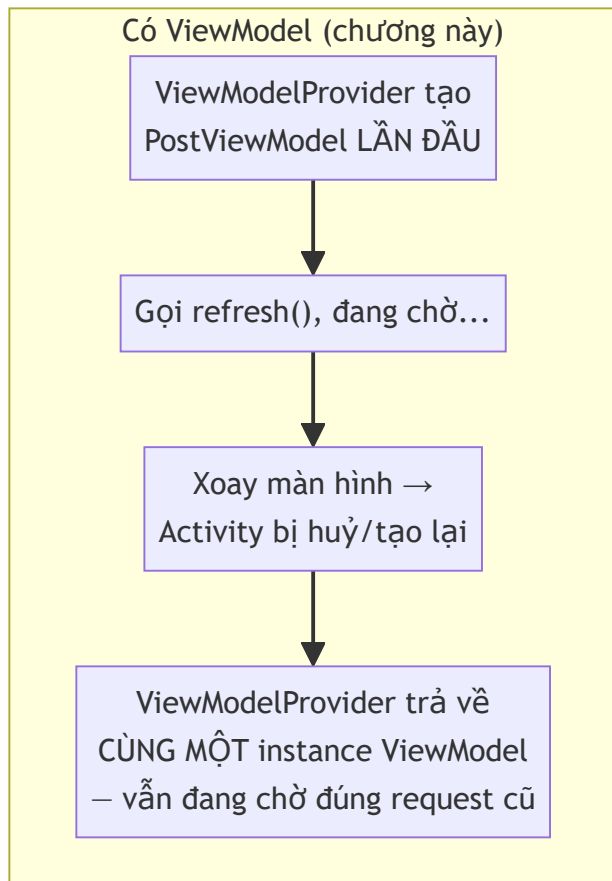
Mục tiêu học

- Hiểu vấn đề cụ thể `ViewModel` giải quyết: dữ liệu sống sót qua configuration change (Chương 6) mà không cần `onSaveInstanceState`.
- Tách Activity thành ba lớp trách nhiệm theo mô hình MVVM: View, ViewModel, Model (Repository — đã có từ Chương 23).
- Biết `ViewModelProvider` cấp phát instance đúng cách, và vòng đời `ViewModel` khác vòng đời Activity ra sao.

Code mẫu: `code/ch28-mvvm-livedata/` — chính thức hoá kiến trúc cho app đã xây từ Chương 13/22/23.

28.1 Vấn đề cụ thể: xoay màn hình làm mất gì?

Nhớ lại Chương 6: xoay màn hình huỷ và tạo lại Activity. Ở bản Chương 23, nếu `MainActivity` đang tự giữ một `PostRepository` và đang chờ kết quả `refresh()`, xoay màn hình giữa chừng sẽ **huỷ luôn cả Activity đang giữ callback đó** — kết quả trả về sau (nếu Activity cũ đã bị huỷ) rơi vào khoảng không, hoặc tệ hơn, Activity mới lại tự gọi `refresh()` từ đầu, gây gọi mạng trùng lặp lãng phí.



28.2 ViewModel sống lâu hơn Activity như thế nào?

```
public class PostViewModel extends AndroidViewModel {
    private final PostRepository repository;

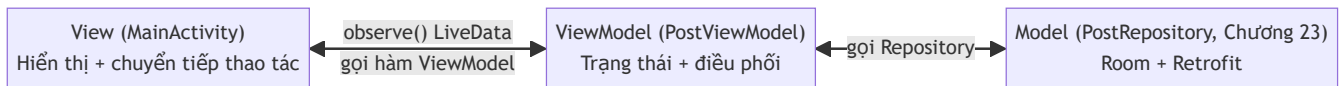
    public PostViewModel(@NonNull Application application) {
        super(application);
        repository = new PostRepository(application);
        refresh();
    }
    // ...
}
```

```
viewModel = new ViewModelProvider(this).get(PostViewModel.class);
```

`ViewModelProvider(this).get(...)` không phải lúc nào cũng tạo instance mới — nó được gắn với một phạm vi sống (ở đây là Activity, tính luôn qua các lần bị huỷ/tạo lại do configuration change) do hệ thống Android quản lý ở tầng thấp hơn cả Activity thường thấy. Kết quả: **cùng một instance `PostViewModel`** được trả về xuyên suốt vòng đời "logic" của màn hình, chỉ thật sự bị huỷ (`onCleared()`) khi Activity kết thúc hẳn (người dùng bấm Back, không phải xoay màn hình).

`AndroidViewModel` (thay vì `ViewModel` trần) có sẵn `Application` Context qua `getApplication()` — dùng khi `ViewModel` cần Context để khởi tạo thứ gì đó (ở đây là `PostRepository` cần Context cho Room, Chương 13). Lưu ý **Application Context là an toàn để giữ lâu dài**, khác hẳn Activity Context (nguồn rò rỉ bộ nhớ đã cảnh báo từ Chương 6) — đây là lý do `ViewModel` không bao giờ được phép giữ tham chiếu tới Activity/View.

28.3 Ba lớp trách nhiệm của MVVM



```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        viewModel = new ViewModelProvider(this).get(PostViewModel.class);

        viewModel.getPosts().observe(this, adapter::submitList);
        viewModel.getLoading().observe(this, isLoading -> ...);
        viewModel.getErrorMessage().observe(this, message -> ...);

        binding.buttonLoad.setOnClickListener(v -> viewModel.refresh());
    }
}
```

So với `MainActivity` ở Chương 23, Activity giờ **không còn logic nghiệp vụ nào** — không tự tạo `PostRepository`, không tự viết callback xử lý thành công/thất bại. Nó chỉ làm hai việc: **quan sát** (`observe`) dữ liệu từ `ViewModel` để cập nhật UI, và **chuyển tiếp** thao tác người dùng (`refresh()`) vào `ViewModel`. Toàn bộ trạng thái loading/lỗi giờ là `LiveData` do `ViewModel` quản lý:

```

private final MutableLiveData<Boolean> loading = new MutableLiveData<>{false};
private final MutableLiveData<String> errorMessage = new MutableLiveData<>();

public void refresh() {
    loading.setValue(true);
    repository.refresh(new PostRepository.RefreshCallback() {
        @Override
        public void onSuccess() {
            loading.setValue(false);
            errorMessage.setValue(null);
        }
        @Override
        public void onError(String message) {
            loading.setValue(false);
            errorMessage.setValue(message);
        }
    });
}
}

```

28.4 Vì sao Activity mỏng đi lại quan trọng?

Đây không phải "làm cho đẹp code" thuần túy — Activity càng ít logic, càng dễ:

- **Test được logic mà không cần UI thật:** `PostViewModel` không đụng gì tới `View` / `Activity`, có thể viết Unit test cho nó (Chương 24) mà không cần Espresso (Chương 25).
- **Không lo rò rỉ bộ nhớ do giữ sai Context:** quy tắc "ViewModel không giữ Activity/View" giúp tránh thẳng lỗi kinh điển đã cảnh báo từ Chương 6/12.
- **Activity/Fragment có thể thay đổi (thậm chí đổi từ Activity sang Fragment) mà không đụng tới logic nghiệp vụ** — vì logic đó nằm trọn trong ViewModel, độc lập với loại "View" đang hiển thị nó.

Bài tập

1. Chạy code mẫu, xoay màn hình (nếu máy ảo hỗ trợ) đúng lúc ProgressBar đang chạy — xác nhận trạng thái loading không bị "giật" về false rồi lại true.
2. Thêm một `LiveData<Integer>` mới trong `PostViewModel` đếm số lần `refresh()` đã được gọi, hiển thị nó trong `MainActivity`.
3. Thử viết một Unit test (Chương 24) đơn giản cho `PostViewModel` mà không cần Espresso — gợi ý: dùng `InstantTaskExecutorRule` của thư viện `androidx.arch.core:core-testing` để `LiveData` phát giá trị ngay lập tức trong môi trường test (tìm hiểu thêm ngoài phạm vi sách nếu muốn thử).

Lỗi thường gặp

- **Giữ tham chiếu** `Activity / View / Context` **(không phải Application)** bên trong `ViewModel`: rò rỉ bộ nhớ nghiêm trọng — `ViewModel` sống lâu hơn `Activity`, giữ `Activity` cũ mãi không giải phóng được.
- **Tạo** `ViewModel` **bằng** `new PostViewModel( ... )` **trực tiếp thay vì qua** `ViewModelProvider`: mất hoàn toàn lợi ích "sống sót qua configuration change" — về bản chất chỉ là một object Java bình thường, bị huỷ theo `Activity` như cũ.
- **Đặt** `LiveData` **trả về kiểu** `MutableLiveData` **công khai** (thay vì chỉ để `LiveData` không sửa được ra ngoài, giữ `MutableLiveData` `private`): View bên ngoài vô tình tự ý gọi `setValue()`, phá vỡ nguyên tắc "chỉ `ViewModel` được sửa trạng thái của chính nó".
- **Nhồi logic hiển thị thuần UI (định dạng ngày giờ theo layout cụ thể, kích thước màn hình...)** vào `ViewModel`: `ViewModel` không nên biết gì về `View / Resources` cụ thể của UI — ranh giới này dễ bị lạm dụng nếu không cẩn thận.

Tóm tắt & tiếp theo

Bạn đã chính thức hoá kiến trúc MVVM cho app xây dựng xuyên suốt từ Chương 13. Chương 29 giới thiệu Hilt — tự động hoá việc khởi tạo và truyền các đối tượng như `PostRepository`, `AppDatabase` (hiện đang tự tay viết singleton) thay vì làm thủ công.

Chương 29: Dependency Injection với Hilt

Mục tiêu học

- Hiểu Dependency Injection (DI) giải quyết vấn đề gì — nhìn lại các singleton tự viết tay từ Chương 13/22.
- Cấu hình được Hilt trong project Java: `@HiltAndroidApp`, `@AndroidEntryPoint`, `@Module`.
- Dùng `@Inject` để Hilt tự lắp ráp `Repository` / `ViewModel` thay vì tự tay `new` / singleton.

Code mẫu: `code/ch29-hilt-di/` — refactor lại app Chương 28 bằng Hilt.

29.1 Vấn đề: singleton tự viết tay đang ngày càng cồng kềnh

Nhìn lại các singleton đã tự viết xuyên suốt sách — `AppDatabase.getInstance()` (Chương 13), `ApiClient.getApi()` (Chương 22) — đều theo đúng một khuôn mẫu lặp lại: biến `static volatile`, kiểm tra `null`, khối `synchronized`. Khi số lượng đối tượng cần chia sẻ tăng lên (database, API client, các repository khác nhau...), code khởi tạo thủ công này phình to và dễ tạo phụ thuộc vòng (A cần B, B cần A) khó gỡ.

Dependency Injection đảo ngược cách tiếp cận: thay vì một class tự đi "tìm" các phụ thuộc nó cần (gọi `getInstance()`, `new` trực tiếp), phụ thuộc được **truyền vào từ bên ngoài** (thường qua constructor) — class chỉ khai báo "tôi cần một `PostDao`", không cần biết nó được tạo ra từ đâu.

Có Hilt (chương này)

PostRepository CHỈ khai báo:
`@Inject constructor(PostDao, JsonPlaceholderApi)`

Hilt tự tìm/tạo đúng
`PostDao`, `JsonPlaceholderApi`
rồi truyền vào

Không có DI (Chương 13/22/23/28)

PostRepository tự gọi
`AppDatabase.getInstance(context)`

PostRepository tự gọi
`ApiClient.getApi()`

29.2 Bốn mảnh ghép cơ bản của Hilt

```
@HiltAndroidApp
public class MyApplication extends Application { }
```

`@HiltAndroidApp` bắt buộc đặt trên class `Application` — sinh ra "container" gốc quản lý toàn bộ vòng đời DI của app, tương tự cách `Application.onCreate()` ở Chương 6 là nơi khởi tạo một lần cho toàn app.

```
@AndroidEntryPoint
public class MainActivity extends AppCompatActivity { ... }
```

`@AndroidEntryPoint` đánh dấu Activity/Fragment/Service... có thể nhận đối tượng do Hilt cung cấp — thiếu annotation này, Hilt không biết "chèn" phụ thuộc vào đâu.

```
@Module
@InstallIn(SingletonComponent.class)
public class AppModule {
    @Provides
    @Singleton
    public AppDatabase provideDatabase(@ApplicationContext Context context) {
        return Room.databaseBuilder(context, AppDatabase.class,
            "posts_cache.db").build();
    }
}
```

`@Module` là nơi viết "công thức" cho những kiểu dữ liệu Hilt **không thể tự đoán cách tạo** — Room/Retrofit cần gọi qua Builder tĩnh, không có constructor đơn giản để Hilt tự suy luận.

`@InstallIn(SingletonComponent.class)` nghĩa là mọi thứ khai báo ở đây sống theo vòng đời **toàn app** — đúng bằng đúng phạm vi các singleton tự viết tay trước đây.

```
@Singleton
public class PostRepository {
    @Inject
    public PostRepository(PostDao dao, JsonPlaceholderApi api) {
        this.dao = dao;
        this.api = api;
    }
}
```


Ngược lại, với class có **constructor đơn giản** như `PostRepository`, chỉ cần đánh dấu `@Inject` ngay trên constructor — không cần viết `@Module` riêng, Hilt tự biết cách tạo nó (miễn là biết cách tạo đủ các tham số của constructor đó, ở đây là `PostDao` và `JsonPlaceholderApi`, cả hai đã có công thức trong `AppModule`).

29.3 `@HiltViewModel` — nối Hilt với ViewModel (Chương 28)

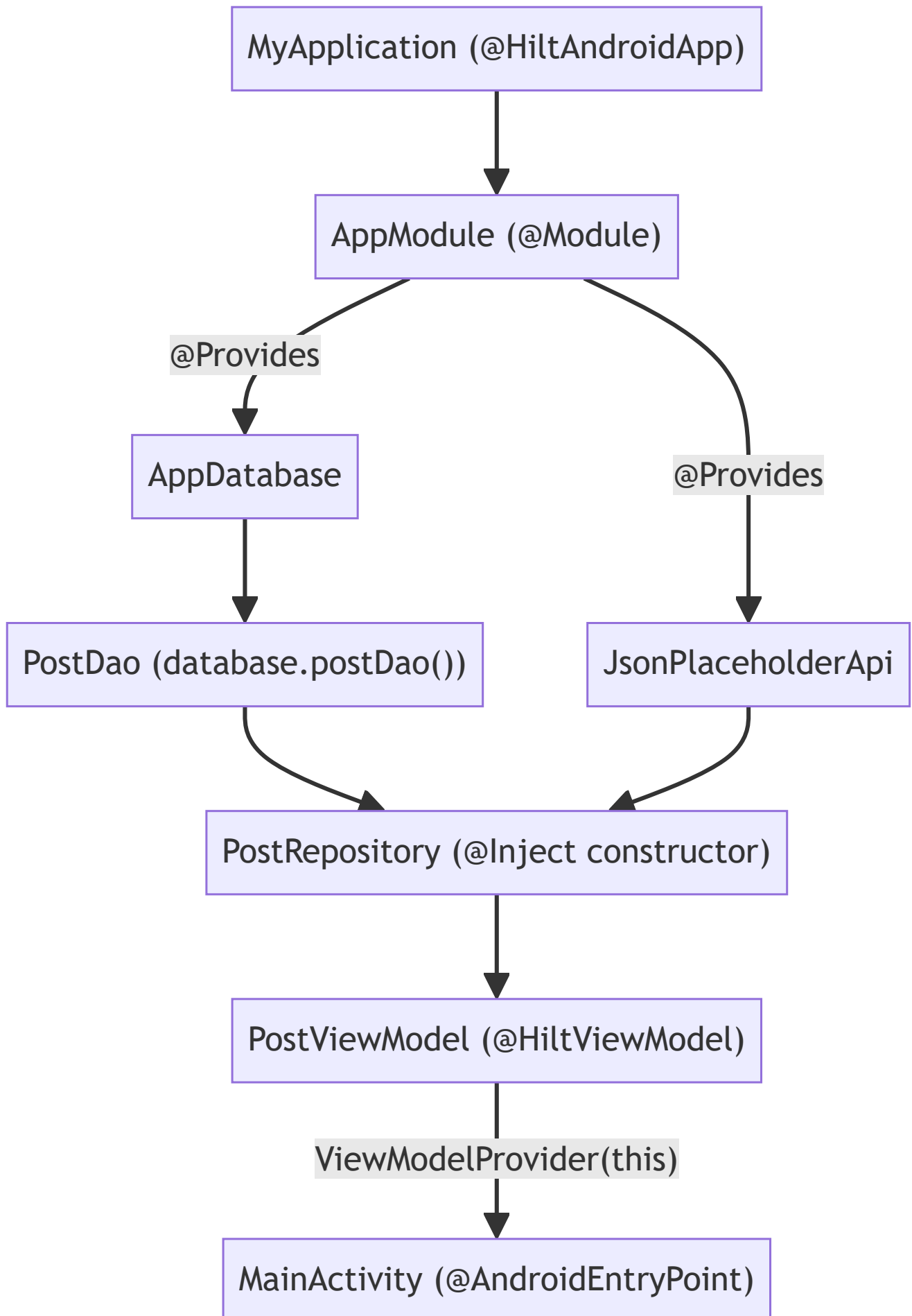
```
@HiltViewModel
public class PostViewModel extends ViewModel {
    @Inject
    public PostViewModel(PostRepository repository) {
        this.repository = repository;
        refresh();
    }
}
```

Phía `MainActivity`, cách gọi **không đổi gì** so với Chương 28:

```
viewModel = new ViewModelProvider(this).get(PostViewModel.class);
```

`@AndroidEntryPoint` trên Activity đã âm thầm thay thế factory mặc định của `ViewModelProvider` bằng một factory biết cách gọi đúng constructor có tham số của `PostViewModel` — đây là lý do `PostViewModel` ở Chương 28 phải kế thừa `AndroidViewModel` (để tự lấy `Application` rồi tự tạo `PostRepository`), còn ở chương này chỉ cần kế thừa `ViewModel` trần, nhận thẳng `PostRepository` đã lắp ráp sẵn.

29.4 Toàn cảnh: ai tạo ai?



Không một dòng code nào trong toàn bộ chuỗi trên gọi trực tiếp `new PostRepository( ... )` hay `AppDatabase.getInstance( ... )` — Hilt đọc toàn bộ khai báo `@Provides` / `@Inject` lúc build, tự sinh code lắp ráp thật (tương tự tinh thần "chỉ khai báo, không viết thân hàm" đã gặp ở Room `@Dao` từ Chương 13 và Retrofit interface từ Chương 22).

Bài tập

1. Chạy code mẫu, xác nhận hành vi giống hệt Chương 28.
 2. Thử tạo thêm một `@Provides` mới trong `AppModule` cho một giá trị cấu hình đơn giản (ví dụ base URL dạng `String`, đánh dấu `@Named("baseUrl")`), inject nó vào `provideApi()` thay vì viết cứng chuỗi URL — tìm hiểu thêm về `@Named` / `Qualifier` annotation nếu muốn mở rộng bài tập này.
 3. Xoá annotation `@Inject` khỏi constructor của `PostRepository`, build lại — đọc thông báo lỗi Hilt đưa ra lúc biên dịch (không phải lúc chạy) để thấy Hilt kiểm tra được sai sót ngay từ bước build.
-

Lỗi thường gặp

- **Quên `@AndroidEntryPoint` trên Activity/Fragment:** `ViewModelProvider` không tìm được factory phù hợp, ném lỗi runtime khi cố tạo `PostViewModel`.
 - **Quên khai báo `MyApplication` trong `AndroidManifest.xml` (`android:name=".MyApplication"`):** `@HiltAndroidApp` không có tác dụng gì nếu class đó không thật sự được dùng làm Application của app.
 - **Trộn lẫn `@InstallIn(SingletonComponent.class)` với vòng đời ngắn hơn cần thiết** (ví dụ đối tượng chỉ nên sống trong một Activity nhưng lại đặt ở `SingletonComponent`): giữ đối tượng tồn tại lâu hơn cần thiết — Hilt có các Component khác (`ActivityComponent` ...) cho từng phạm vi ngắn hơn, ngoài phạm vi giới thiệu của chương này.
 - **Tưởng Hilt là bắt buộc phải có để làm MVVM:** sai — Chương 28 đã làm MVVM đầy đủ mà không cần Hilt. DI là công cụ giúp việc lắp ráp phụ thuộc gọn hơn khi project lớn dần, không phải điều kiện tiên quyết của kiến trúc MVVM.
-

Tóm tắt & tiếp theo

Bạn đã thấy Hilt thay thế các singleton tự viết tay bằng khai báo tường minh, kiểm tra được lúc build. Chương 30 tạm rời xa các chủ đề kiến trúc thuần logic, giới thiệu Jetpack Compose — cách xây UI khai báo (declarative) hiện đại, khác hẳn XML layout đã dùng xuyên suốt từ Chương 7.

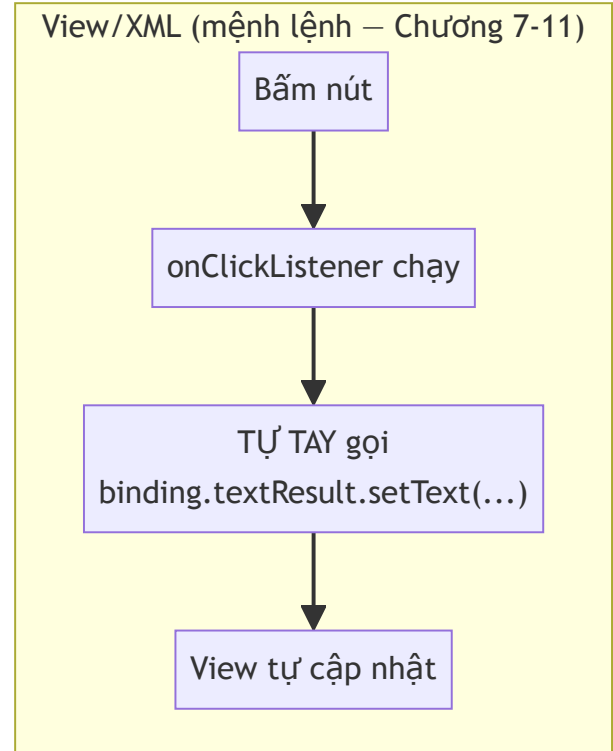
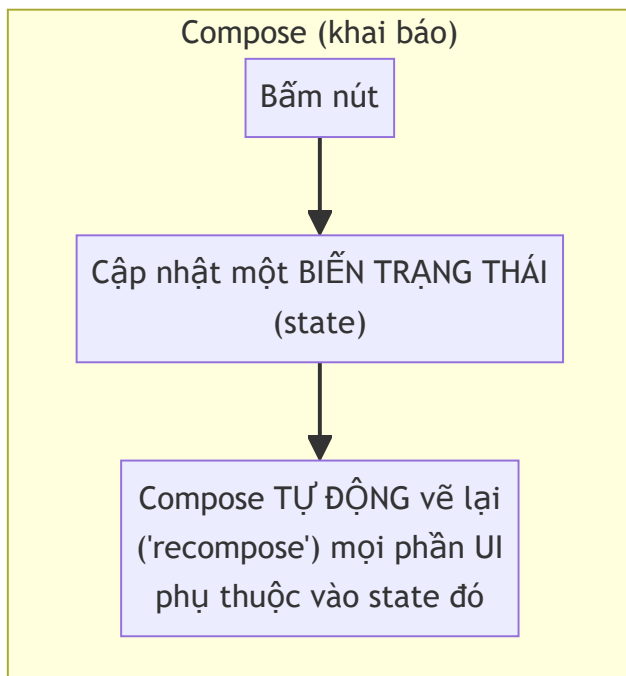
Chương 30: Giới thiệu Jetpack Compose

Lưu ý quan trọng trước khi đọc: toàn bộ sách dùng Java, nhưng **Jetpack Compose chỉ hoạt động với Kotlin** — không có API Compose nào cho Java. Code mẫu chương này (`code/ch30-jetpack-compose/`) là **project Kotlin duy nhất trong sách**, viết ra để bạn đọc hiểu và so sánh, không nhằm dạy Kotlin đầy đủ. Đây là chương **giới thiệu khái niệm** (đúng như tên chương), không phải chương thực hành sâu như các chương khác.

Mục tiêu học

- Hiểu Compose khác gì về bản chất so với XML layout (Chương 7) — lập trình khai báo (declarative) so với mệnh lệnh (imperative).
- Đọc hiểu được một đoạn code Compose đơn giản, dù không viết thạo Kotlin.
- Biết khi nào một project Android hiện đại chọn Compose, khi nào vẫn hợp lý dùng View/XML.

30.1 Hai triết lý xây UI



Với View/XML (Chương 7), bạn viết code **ra lệnh từng bước**: "tìm view này, đổi text của nó thành X". Với Compose, bạn chỉ mô tả **UI trông như thế nào ỨNG VỚI một trạng thái cho trước** — khi trạng thái đổi, Compose tự tính toán lại và vẽ lại phần cần thiết, bạn không tự tay gọi `setText()` / `setVisibility()` ở bất kỳ đâu.

30.2 So sánh trực tiếp: cùng một tính năng, hai cách viết

Bản XML + ViewBinding (Chương 7, Java):

```
binding.buttonGreet.setOnClickListener(v → {
    String name = binding.editName.getText().toString().trim();
    binding.textResult.setText(name.isEmpty()
        ? getString(R.string.result_empty_name)
        : getString(R.string.result_greeting, name));
});
```

Bản Compose (chương này, Kotlin):

```
@Composable
fun GreetingScreen() {
    var name by remember { mutableStateOf("") }
    var greeting by remember { mutableStateOf("") }

    Column(modifier = Modifier.padding(24.dp)) {
        TextField(value = name, onValueChange = { name = it }, ...)

        Button(onClick = {
            greeting = if (name.isBlank()) "Bạn chưa nhập tên." else "Xin chào, $name!"
        }) {
            Text("Chào")
        }

        Text(text = greeting)
    }
}
```

Vài điểm đọc hiểu quan trọng dù bạn chưa biết Kotlin:

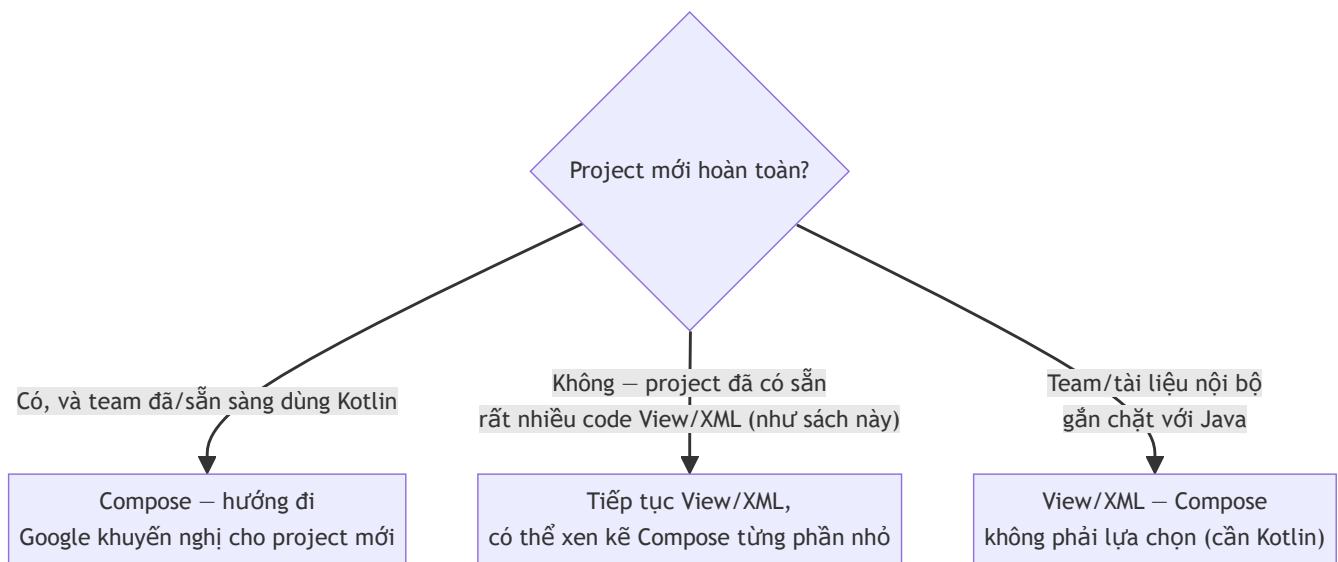
- `@Composable` đánh dấu một hàm "biết cách vẽ UI" — tương đương khái niệm một đoạn XML layout, nhưng là **code**, không phải file XML riêng.
- `remember { mutableStateOf( ... ) }` khai báo một **biến trạng thái** — Compose tự theo dõi, hề giá trị đổi (`name = it`, `greeting = ...`) thì tự vẽ lại đúng những `Text` / `Button` phụ thuộc vào nó.
- Không có `findViewById`, không có `binding.xxx.setText( ... )` ở bất kỳ đâu — toàn bộ giao diện là hệ quả trực tiếp của giá trị các biến trạng thái tại một thời điểm.

30.3 Không còn file XML layout riêng

Khác mọi chương trước (nơi UI luôn tách thành file `.xml` riêng trong `res/layout/`), Compose viết UI **ngay trong code Kotlin** — không có `activity_main.xml` nào cả. `setContent { ... }` trong `MainActivity.kt` thay thế hoàn toàn `setContentView(R.layout.activity_main)` đã dùng xuyên suốt từ Chương 4.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface {
                    GreetingScreen()
                }
            }
        }
    }
}
```

30.4 Khi nào chọn Compose, khi nào vẫn hợp lý dùng View/XML?



Compose và View/XML **có thể tồn tại xen kẽ trong cùng một project** (Google cung cấp cầu nối `ComposeView` để nhúng Compose vào layout XML và ngược lại) — không nhất thiết phải chọn một, bỏ hẳn cái kia ngay lập tức. Với một codebase Java lâu năm, hướng đi thực dụng thường là: giữ nguyên phần lớn View/XML, chỉ viết màn hình MỚI bằng Compose (kèm chuyển phần đó sang Kotlin) khi có lý do rõ ràng.

Bài tập

1. Chạy code mẫu, xác nhận hành vi giống hệt bản XML ở Chương 7.
 2. Không cần viết code — chỉ đọc lại `MainActivity.kt`, thử tự giải thích bằng lời từng dòng cho một người bạn tưởng tượng chưa biết Compose, dựa vào chú thích trong file.
 3. So sánh số dòng code và số file cần thiết giữa hai cách tiếp cận (Chương 7 cần `activity_main.xml` + `MainActivity.java`; chương này chỉ cần một file `.kt`) — tự rút ra nhận xét về đánh đổi giữa hai phong cách.
-

Lỗi thường gặp (khi tự tìm hiểu thêm ngoài sách)

- **Cố viết Compose bằng Java:** không thể — Compose Compiler chỉ xử lý code Kotlin, đây không phải giới hạn tạm thời mà là quyết định thiết kế cốt lõi của Compose.
 - **Quên `remember`, chỉ dùng `mutableStateOf(...)` trực tiếp:** giá trị bị khởi tạo lại mỗi lần hàm `@Composable` được gọi lại (recompose), mất trạng thái liên tục — lỗi rất phổ biến với người mới học Compose.
 - **Nhầm tưởng phải bỏ hết code View/XML đã có để "nâng cấp" lên Compose:** không cần thiết và thường không thực dụng cho codebase lớn đã ổn định.
-

Tóm tắt & tiếp theo

Bạn đã có cái nhìn khái quát về Compose — đủ để đọc hiểu, biết khi nào nên tìm hiểu sâu hơn, và tại sao nó đòi hỏi Kotlin. Chương 31 quay lại hoàn toàn với Java, khép lại Phần 6 bằng tư duy Clean Architecture — tổ chức code ở quy mô lớn hơn một app đơn giản.

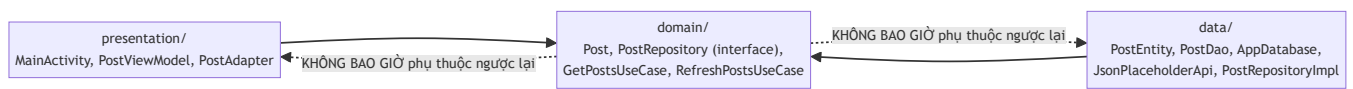
Chương 31: Clean Architecture cơ bản cho ứng dụng Android

Mục tiêu học

- Hiểu nguyên tắc cốt lõi của Clean Architecture: **phụ thuộc chỉ hướng vào trong**.
- Tổ chức code thành ba tầng: domain (nghiệp vụ thuần), data (chi tiết kỹ thuật), presentation (UI).
- Viết Use Case đơn giản, hiểu khi nào nó thật sự có giá trị so với gọi thẳng Repository.
- Biết đây là kỹ thuật cho project **lớn dần theo thời gian**, không phải yêu cầu bắt buộc cho mọi app.

Code mẫu: `code/ch31-clean-architecture/` — tổ chức lại đúng app đã xây từ Chương 13/22/23/28/29.

31.1 Nguyên tắc cốt lõi: phụ thuộc chỉ hướng vào trong



Cả `presentation` và `data` đều phụ thuộc vào `domain` — nhưng `domain` **không biết gì** về Android framework, Room, hay Retrofit. Đây là lý do `domain/Post.java` không có bất kỳ annotation nào (so sánh với `data/PostEntity.java` đầy `@Entity` / `@SerializedName`), và `domain/PostRepository.java` chỉ là một `interface` trống nghĩa vụ triển khai.

31.2 Ba tầng, ba trách nhiệm

Tầng	Biết gì	Không biết gì
domain	Quy tắc nghiệp vụ thuần (<code>Post</code> là gì, "lấy danh sách bài viết" nghĩa là gì)	Room, Retrofit, Android UI
data	Cách lấy dữ liệu thật (SQLite qua Room, HTTP qua Retrofit)	<code>MainActivity</code> , <code>PostViewModel</code> tồn tại
presentation	Cách hiển thị dữ liệu, phản ứng thao tác người dùng	<code>PostEntity</code> , <code>JsonPlaceholderApi</code> tồn tại

```
// domain/PostRepository.java - CHỈ khai báo, không có Room/Retrofit
public interface PostRepository {
    LiveData<List<Post>> getCachedPosts();
    void refresh(RefreshCallback callback);
}
```

```
// data/PostRepositoryImpl.java - cài đặt THẬT, biết Room và Retrofit
@Singleton
public class PostRepositoryImpl implements PostRepository {
    @Inject
    public PostRepositoryImpl(PostDao dao, JsonPlaceholderApi api) { ... }

    @Override
    public LiveData<List<Post>> getCachedPosts() {
        return Transformations.map(dao.getAll(), PostMapper::toDomainList);
    }
}
```

`Transformations.map( ... )` (dòng cuối) chính là **ranh giới vật lý** giữa hai tầng: `dao.getAll()` trả `LiveData<List<PostEntity>>` (kiểu của tầng data), được chuyển thành `LiveData<List<Post>>` (kiểu của tầng domain) ngay tại đây — `PostEntity` không bao giờ "rò rỉ" ra khỏi package `data`.

31.3 Nối hai tầng bằng Hilt `@Binds`

```
@Module
@InstallIn(SingletonComponent.class)
public abstract class RepositoryModule {

    @Binds
    public abstract PostRepository bindPostRepository(PostRepositoryImpl impl);
}
```

`@Binds` là cú pháp gọn hơn `@Provides` (Chương 29) dành riêng cho đúng một tình huống: "khi cần một `PostRepository`, hãy đưa một `PostRepositoryImpl`". Nhờ khai báo này, `PostViewModel` (tầng presentation) chỉ cần khai báo phụ thuộc vào **interface** `PostRepository` — Hilt tự biết phải truyền vào instance `PostRepositoryImpl` thật.

31.4 Use Case — khi nào thật sự có giá trị?

```
public class GetPostsUseCase {
    private final PostRepository repository;

    @Inject
    public GetPostsUseCase(PostRepository repository) {
        this.repository = repository;
    }

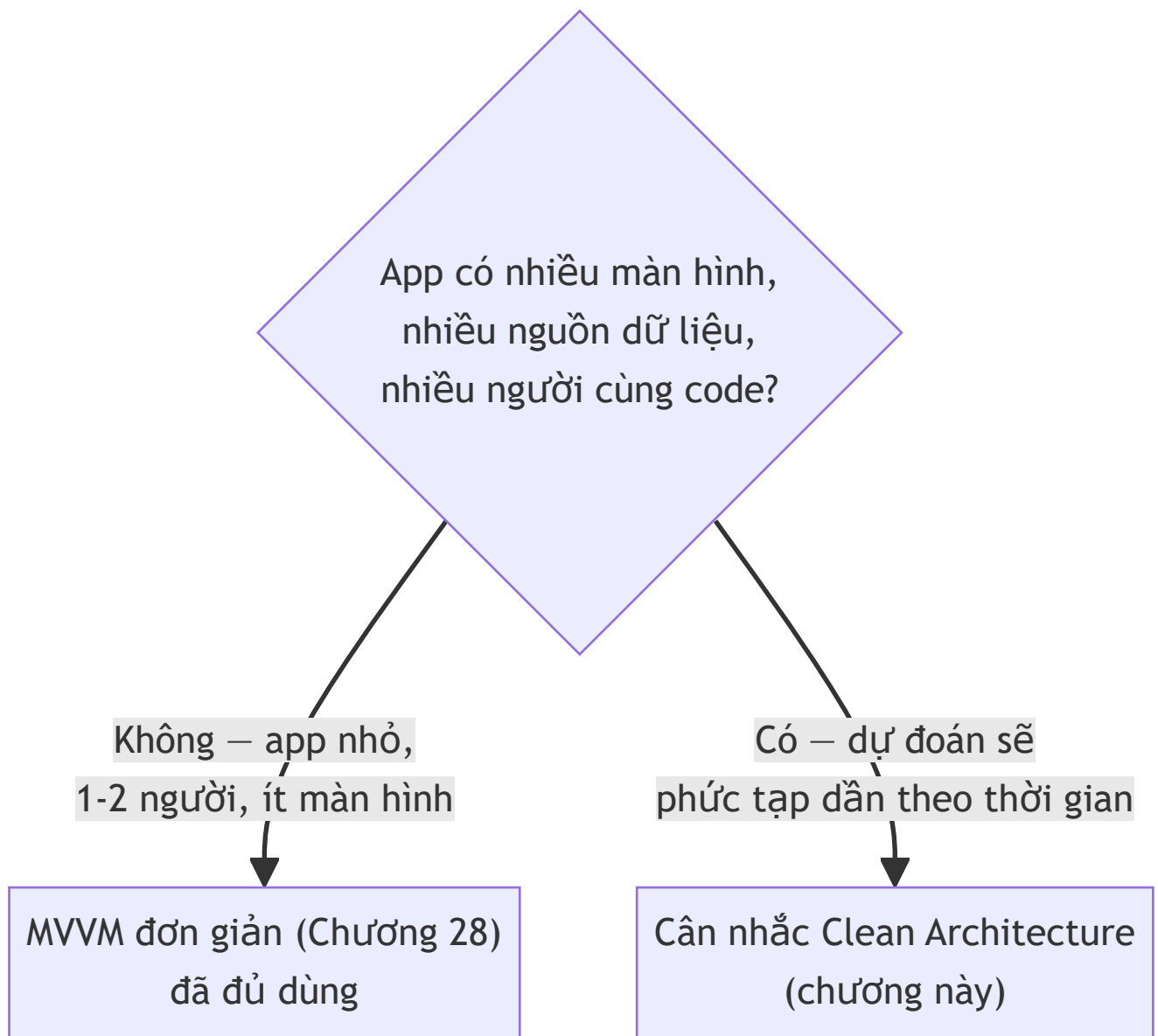
    public LiveData<List<Post>> execute() {
        return repository.getCachedPosts();
    }
}
```

Thành thật mà nói: với logic đơn giản như "lấy danh sách bài viết", `GetPostsUseCase` ở đây gần như chỉ **chuyển tiếp** lời gọi tới `repository.getCachedPosts()` — không thêm giá trị xử lý nào. Đây là điều nên thừa nhận thẳng thắn thay vì che giấu: Use Case phát huy giá trị thật sự khi nghiệp vụ **phức tạp hơn một lệnh gọi đơn giản** — ví dụ: lọc bài viết theo quyền hạn người dùng, ghép dữ liệu từ hai Repository khác nhau, áp dụng một quy tắc validate trước khi trả kết quả. Với app nhỏ, đơn giản, việc `ViewModel` gọi thẳng `Repository` (như Chương 28/29 đã làm) hoàn toàn hợp lý — không nhất thiết phải có tầng Use Case.

31.5 Đây có phải "must-have" cho mọi app không?

Không. Đây là điểm quan trọng nhất chương cần nhấn mạnh: Clean Architecture (và tầng Use Case nói riêng) là công cụ giải quyết vấn đề **quy mô và độ phức tạp tăng dần theo thời gian** — nhiều màn hình, nhiều nguồn dữ liệu, nhiều người cùng code, cần thay đổi chi tiết kỹ thuật (đổi từ Room sang một giải pháp lưu trữ khác, ví dụ) mà không muốn động vào logic hiển thị.

Với một app nhỏ, một người viết, ít màn hình — kiến trúc MVVM đơn giản ở Chương 28 (thậm chí không cần Hilt) đã là đủ, và việc áp Clean Architecture đầy đủ có thể chỉ tạo thêm nhiều file, nhiều tầng gián tiếp không cần thiết. Áp dụng đúng lúc, đúng quy mô — không áp dụng "cho có" chỉ vì nghe có vẻ chuyên nghiệp.



Bài tập

1. Chạy code mẫu, xác nhận hành vi giống hệt Chương 28/29.
2. Mở `domain/Post.java` và `data/PostEntity.java` cạnh nhau — liệt kê chính xác những annotation nào có ở bên này mà không có ở bên kia, và giải thích tại sao.
3. Thử tưởng tượng (không cần code thật): nếu muốn thêm một nguồn dữ liệu thứ hai — ví dụ đọc thêm bài viết yêu thích lưu trong `SharedPreferences` (Chương 12) — bạn sẽ sửa ở những file nào trong ba tầng? Xác nhận rằng `presentation/` không cần đổi gì nếu interface `PostRepository` vẫn giữ nguyên chữ ký.

Lỗi thường gặp

- Để `PostEntity` (hoặc bất kỳ model tầng data nào) rò rỉ vào `presentation/`: phá vỡ hoàn toàn mục đích tách tầng — kiểm tra bằng cách tự hỏi "package `presentation` có `import` gì từ package `data` không?", câu trả lời đúng luôn phải là "không".
- **Viết Use Case rỗng cho MỌI thao tác dù chẳng thêm logic gì** (như ví dụ thẳng thắn ở mục 31.4): tạo thêm tầng gián tiếp không cần thiết, làm code khó theo dõi hơn mà không đổi lại lợi ích gì.
- **Áp dụng Clean Architecture đầy đủ ngay từ app "Hello World" đầu tiên**: over-engineering — chi phí tổ chức nhiều tầng chỉ xứng đáng khi độ phức tạp thực tế đạt tới mức cần nó.
- **Quên `Transformations.map()` (hoặc mapper tương đương) khi nối data → domain**: dễ dẫn tới tình huống dở dang — vẫn còn `PostEntity` xuất hiện ở tầng domain vì "tiện" tái sử dụng model, phá vỡ ranh giới đã cố công dựng lên.

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 6 — Kiến trúc nâng cao**: MVVM, Dependency Injection với Hilt, giới thiệu Compose, và Clean Architecture — một bộ công cụ đầy đủ để tổ chức app Android ở quy mô lớn dần. Phần 7 chuyển hướng hoàn toàn sang một nhóm chủ đề khác: giao tiếp giữa các ứng dụng và điều khiển phần cứng tầng sâu — bắt đầu với Content Provider ở Chương 32.

Phần 7 — Giao tiếp liên ứng dụng & phần cứng nâng cao

Chương 32: Content Provider — chia sẻ dữ liệu có kiểm soát giữa các ứng dụng

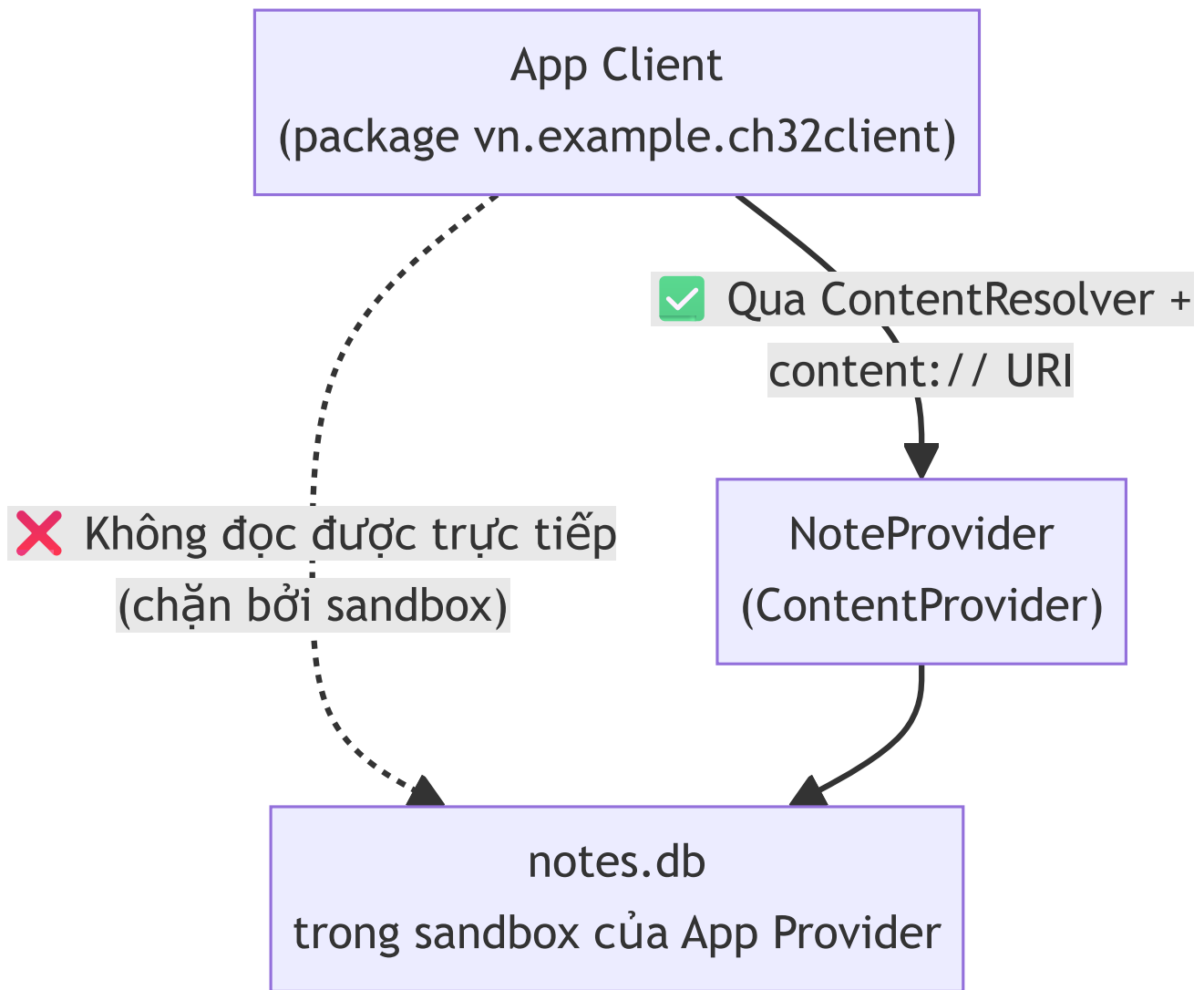
Mục tiêu học

- Hiểu vì sao sandbox (Chương 1/26) khiến app khác không đọc được trực tiếp file SQLite của app mình.
- Viết một `ContentProvider` công khai dữ liệu Room qua giao diện chuẩn (query/insert/delete).
- Truy cập dữ liệu app khác từ một app hoàn toàn riêng biệt bằng `ContentResolver`.
- Dùng `ContentObserver` để tự động nhận biết khi dữ liệu ở app kia thay đổi.

Code mẫu: `code/ch32-content-provider/` — **hai app thật sự riêng biệt** (`:app` và `:client`, hai APK khác nhau) để minh họa đúng bản chất chia sẻ liên-app, không phải mô phỏng trong cùng một app.

32.1 Vấn đề: sandbox chặn truy cập trực tiếp

Chương 1 và 26 đã nói: mỗi app có `UID` riêng, thư mục dữ liệu riêng (`getFilesDir()`, Chương 14) mà **app khác không đọc/ghi được**. File `notes.db` (SQLite của app Provider ở Chương 13) nằm trong sandbox đó — một app khác không thể mở file này trực tiếp, dù biết chính xác đường dẫn.



`ContentProvider` là **cửa duy nhất** được phép mở xuyên qua ranh giới sandbox này — nó tự quyết định app khác được đọc/ghi những gì, theo cách nào, không phải mở toang toàn bộ file.

32.2 Viết `ContentProvider` — bốn thao tác chuẩn

```
public class NoteProvider extends ContentProvider {

    @Override
    public boolean onCreate() {
        return true;    // chạy RẤT SỚM, kể cả trước Application.onCreate() - không làm
        // việc nặng ở đây
    }

    @Override
    public Cursor query(Uri uri, String[] projection, String selection,
                       String[] selectionArgs, String sortOrder) {
        Cursor cursor = writableDb().query(new SimpleSQLiteQuery(
            "SELECT id, title, created_at FROM notes ORDER BY id DESC"));
        cursor.setNotificationUri(requireContext().getContentResolver(), uri);
        return cursor;
    }

    @Override
    public Uri insert(Uri uri, ContentValues values) {
        long id = writableDb().insert("notes", SQLiteDatabase.CONFLICT_REPLACE, values);
        Uri result = ContentUris.withAppendedId(NoteContract.CONTENT_URI, id);
        requireContext().getContentResolver().notifyChange(result, null);
        return result;
    }
}
```

Bốn phương thức bắt buộc cài đặt: `query`, `insert`, `update`, `delete` (code mẫu bỏ qua `update` để gọn, một Provider thật cần đủ cả bốn). `writableDb()` lấy `SupportSQLiteDatabase` ngay từ `AppDatabase` (Room) đã xây ở Chương 13 — Provider chỉ là một **lớp giao diện mỏng** đặt trước database đã có sẵn, không cần viết lại logic lưu trữ.

32.3 `UriMatcher` và "hợp đồng" URI công khai

```
public final class NoteContract {
    public static final String AUTHORITY = "vn.example.ch32provider.provider";
    public static final Uri CONTENT_URI = Uri.parse("content://" + AUTHORITY +
"/notes");
    public static final String COLUMN_ID = "id";
    public static final String COLUMN_TITLE = "title";
    public static final String COLUMN_CREATED_AT = "created_at";
}
```

`AUTHORITY` là định danh duy nhất của Provider trên toàn hệ thống (tương tự `applicationId` cho app) — khai báo khớp trong manifest:

```
<provider
    android:name=".NoteProvider"
    android:authorities="vn.example.ch32provider.provider"
    android:exported="true" />
```

`android:exported="true"` **bắt buộc** để app khác truy cập được — ngược với mặc định "false" đã quen dùng cho Activity/Service nội bộ (Chương 8, 16). Đây chính là điểm cần cân nhắc bảo mật: Provider `exported=true` không kèm `readPermission` / `writePermission` nghĩa là **MỌI app khác trên máy** đều đọc/ghi được, không chỉ app Client trong ví dụ. Với dữ liệu nhạy cảm, luôn thêm quyền tùy chỉnh (`android:readPermission="com.example.app.permission.READ_NOTES"`) để giới hạn — nằm ngoài phạm vi trình bày chi tiết ở chương này nhưng bắt buộc biết để tự tra cứu khi làm dự án thật.

32.4 Phía app khách: `ContentResolver`

```
private static final Uri NOTES_URI =
    Uri.parse("content://vn.example.ch32provider.provider/notes");

try (Cursor cursor = getContentResolver().query(NOTES_URI, null, null, null, null)) {
    int idxTitle = cursor.getColumnIndexOrThrow("title");
    while (cursor.moveToNext()) {
        String title = cursor.getString(idxTitle);
        // ...
    }
}
```

Đây chính là API đã dùng để đọc danh bạ ở Chương 26

(`getContentResolver().query(ContactsContract ...)`) — danh bạ máy thực chất cũng chỉ là một `ContentProvider` do hệ thống cung cấp sẵn. App Client trong code mẫu **không hề import bất kỳ class nào của app Provider** — chỉ cần đúng chuỗi URI và tên cột, y hệt cách một app thật kết nối tới Provider của app khác mà không cần mã nguồn của nhau.

32.5 `ContentObserver` — tự động biết khi dữ liệu đổi

```
private final ContentObserver observer = new ContentObserver(mainHandler) {
    @Override
    public void onChange(boolean selfChange) {
        queryNotes(); // tự tải lại, không cần người dùng bấm nút
    }
};

getContentResolver().registerContentObserver(NOTES_URI, true, observer);
```

Hoạt động được là nhờ `cursor.setNotificationUri(...)` trong `query()` và `notifyChange(...)` trong `insert()` / `delete()` phía Provider — đúng nguyên tắc "đăng ký lúc `onStart`, huỷ đăng ký lúc `onStop`" đã lặp lại từ Chương 12/17: `unregisterContentObserver()` trong `onStop()` để tránh rò rỉ.

Bài tập

1. Cài cả hai app (`:app` và `:client`) lên cùng một máy ảo, làm theo README để xác nhận luồng đọc/ghi hai chiều hoạt động.
2. Gỡ cài đặt app `:app`, mở app `:client`, bấm đọc ghi chú — đọc thông báo lỗi hiển thị, xác nhận app không crash im lặng.
3. Thử thêm `android:readPermission` tùy chỉnh vào `NoteProvider` trong manifest của `:app`, không khai báo quyền tương ứng ở `:client` — build lại cả hai, quan sát lỗi `SecurityException` khi `:client` cố gọi `query()`.

Lỗi thường gặp

- **Quên** `android:exported="true"`: Provider tồn tại nhưng không app nào khác gọi tới được — từ Android 12 (API 31), giá trị này bắt buộc khai báo tường minh, không có mặc định ngầm.
- **Provider** `exported="true"` **không giới hạn quyền cho dữ liệu nhạy cảm**: bất kỳ app nào cài trên máy (không chỉ app bạn tự viết) đều đọc được — cân nhắc `readPermission` / `writePermission` ngay từ đầu với dữ liệu thật.
- **Quên gọi** `notifyChange()` **sau** `insert` / `delete`: `ContentObserver` phía client không bao giờ được kích hoạt, giao diện không tự cập nhật dù dữ liệu đã đổi thật sự.
- **Làm việc nặng trong** `onCreate()` **của Provider**: Provider khởi tạo cực sớm trong vòng đời tiến trình, việc nặng ở đây có thể làm chậm toàn bộ quá trình khởi động app, kể cả trước khi `Application.onCreate()` (Chương 6) chạy.

Tóm tắt & tiếp theo

Bạn đã biết cách chia sẻ dữ liệu có kiểm soát giữa hai app hoàn toàn tách biệt — cơ chế đứng sau danh bạ, lịch, và nhiều API hệ thống khác đã dùng ở Chương 26. Chương 33 mở rộng sang các hình thức giao tiếp liên ứng dụng khác: App Links/Deep Link, chia sẻ nội dung qua Share sheet, và `PendingIntent` (đã gặp ở Chương 20) nhìn từ góc độ liên ứng dụng.

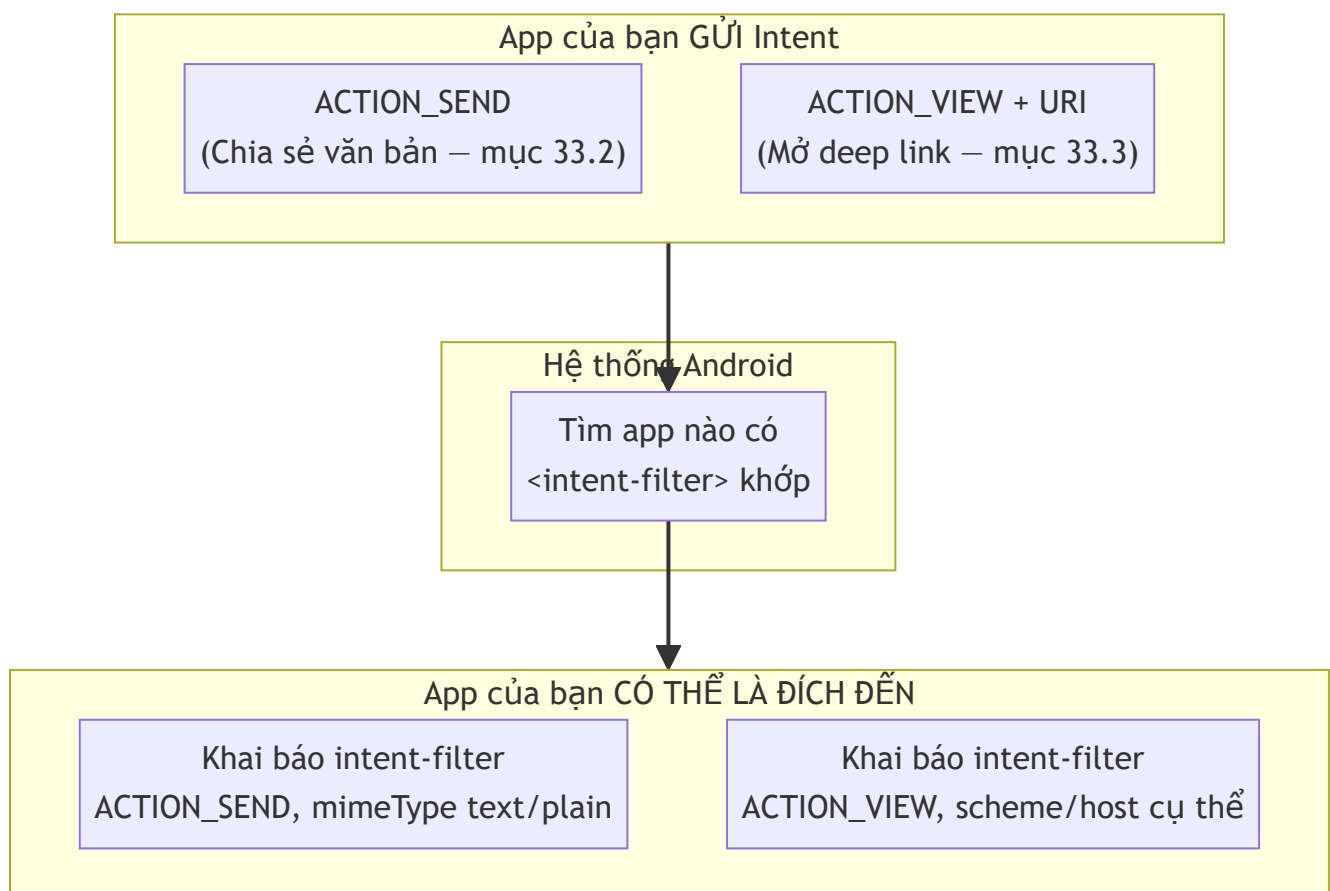
Chương 33: Giao tiếp liên ứng dụng nâng cao — App Links/Deep Link, Share sheet, PendingIntent

Mục tiêu học

- Làm app của mình **xuất hiện** trong Share sheet của app khác (nhận chia sẻ), không chỉ gửi đi.
- Phân biệt custom scheme deep link (luôn hoạt động) và App Link http/https có `autoVerify` (cần xác minh domain).
- Ôn lại `PendingIntent` (Chương 20) từ góc nhìn liên ứng dụng: ai thực thi nó, và tại sao đó lại an toàn.

Code mẫu: `code/ch33-app-links-share/`. Chương 8 đã dùng implicit Intent để **gửi đi** (mở trình duyệt); chương này mở rộng theo cả hai chiều gửi/nhận, và thêm cơ chế deep link.

33.1 Ba cơ chế, một nền tảng chung: Intent + `<intent-filter>`



Tất cả đều dựa trên implicit Intent đã học ở Chương 8 — điểm mới ở chương này là **app của bạn cũng có thể là bên nhận**, không chỉ bên gửi.

33.2 Share sheet — vừa gửi, vừa nhận

Gửi (mở hộp thoại "Chia sẻ qua"):

```
Intent sendIntent = new Intent(Intent.ACTION_SEND);
sendIntent.setType("text/plain");
sendIntent.putExtra(Intent.EXTRA_TEXT, message);
startActivity(Intent.createChooser(sendIntent, "Chia sẻ qua"));
```

`Intent.createChooser( ... )` **luôn** hiện hộp thoại chọn app, kể cả khi chỉ có một app khớp — khác gọi thẳng `startActivity(sendIntent)` (có thể tự mở app duy nhất khớp mà không hỏi, gây bất ngờ nếu người dùng không chú ý). Luôn dùng `createChooser` cho hành động chia sẻ.

Nhận — khai báo `<intent-filter>` để app xuất hiện trong Share sheet của **app khác**:

```
<activity android:name=".MainActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.SEND" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="text/plain" />
  </intent-filter>
</activity>
```

Đọc dữ liệu được chia sẻ tới:

```
if (Intent.ACTION_SEND.equals(intent.getAction()) &&
    intent.getType().startsWith("text/")) {
    String sharedText = intent.getStringExtra(Intent.EXTRA_TEXT);
    // hiển thị sharedText
}
```

33.3 Deep Link bằng custom scheme — luôn hoạt động, dễ test

```
<activity android:name=".NoteDetailActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="ch33share" android:host="note" />
  </intent-filter>
</activity>
```

`ch33share://note/42` khớp `scheme="ch33share"` + `host="note"` — phần `/42` còn lại đọc được qua `Uri.getLastPathSegment()`:

```
Uri uri = getIntent().getData();
String noteId = uri.getLastPathSegment(); // "42"
```

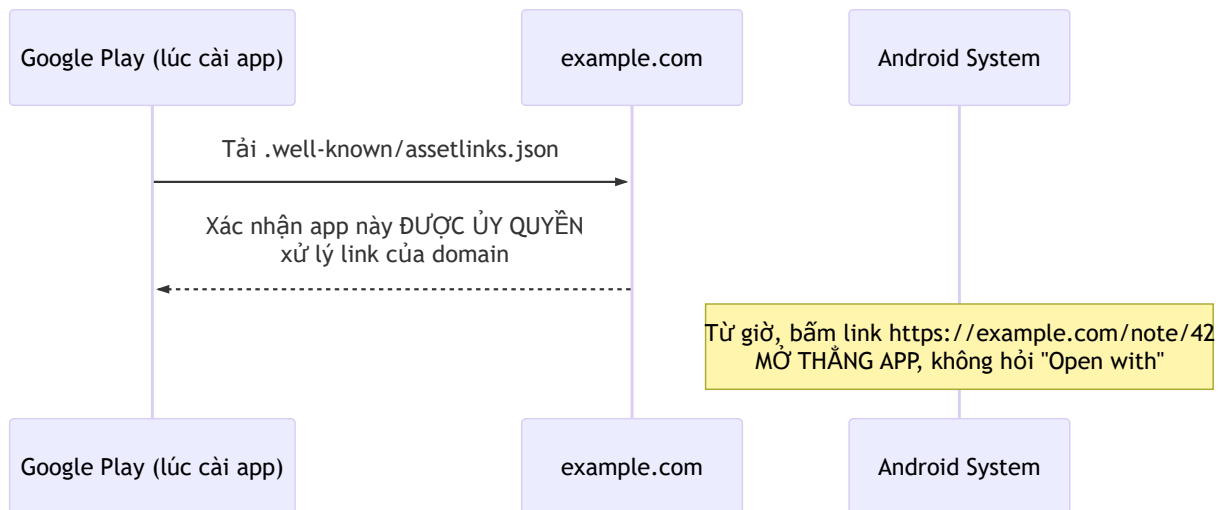
`category.BROWSABLE` cho phép trình duyệt/app khác mở được link này (không chỉ code trong chính app gọi `startActivity`) — test trực tiếp từ dòng lệnh, mô phỏng một nguồn bên ngoài bất kỳ:

```
adb shell am start -a android.intent.action.VIEW -d "ch33share://note/99"
```

Ưu điểm: hoạt động **ngay lập tức**, không cần cấu hình gì thêm. Nhược điểm: nếu chưa cài app, hệ thống không biết phải làm gì với link này (không tự động rơi về một trang web dự phòng như App Link thật).

33.4 App Link http/https — cần xác minh domain (`autoVerify`)

```
<intent-filter android:autoVerify="true">
  <action android:name="android.intent.action.VIEW" />
  <category android:name="android.intent.category.DEFAULT" />
  <category android:name="android.intent.category.BROWSABLE" />
  <data android:scheme="https" android:host="example.com" android:pathPrefix="/note" />
</intent-filter>
```



Khác custom scheme, App Link thật đòi hỏi bạn **sở hữu domain** và host file `/.well-known/assetlinks.json` chứng minh app được ủy quyền — hệ thống tự tải và kiểm tra file này lúc cài đặt (`autoVerify="true"`). Đối lại, một link `https://example.com/note/42` bấm từ bất kỳ đâu (tin nhắn, email, trình duyệt) sẽ **mở thẳng app**, không hiện hộp thoại "Open with" như custom scheme có thể gặp khi nhiều app cùng khai báo trùng scheme. Code mẫu khai báo cấu hình này chỉ để bạn thấy đúng cú pháp — không hoạt động thật trên máy bạn vì `example.com` không phải domain bạn sở hữu.

33.5 `PendingIntent` nhìn lại từ góc độ liên ứng dụng

Chương 20 đã dùng `PendingIntent` để notification "biết cách" mở Activity dù app không đang chạy. Nhìn từ góc độ chương này: `PendingIntent` chính là cách an toàn để **trao quyền thực thi một Intent cho một thành phần bên ngoài** (ở đó là hệ thống Notification) — tương tự tinh thần Provider (Chương 32) trao quyền truy cập dữ liệu có kiểm soát, thay vì mở toang. `FLAG_IMMUTABLE` (đã dùng ở Chương 20) đảm bảo bên nhận `PendingIntent` không thể chỉnh sửa nội dung Intent bên trong trước khi thực thi — một biện pháp an toàn khi trao quyền cho bên ngoài.

Bài tập

1. Chạy code mẫu, thử luồng chia sẻ cả hai chiều (gửi đi và nhận từ app khác) như mô tả trong README.
2. Mở deep link bằng nút trong app, rồi thử lại bằng lệnh `adb` — xác nhận cả hai cách đều tới đúng `NoteDetailActivity` với đúng ID.
3. Thử đổi `android:pathPrefix="/note"` thành `/notes` (thêm "s") trong intent-filter App Link — không cần test thật (vì không sở hữu domain), chỉ giải thích bằng lời tại sao link `https://example.com/note/42` sẽ không còn khớp nữa nếu áp dụng thay đổi này cho app thật.

Lỗi thường gặp

- **Quên** `category.BROWSABLE`: trình duyệt/nguồn bên ngoài không mở được link, dù `category.DEFAULT` đã có — cả hai category đều cần thiết cho deep link mở từ nguồn ngoài app.
- **Dùng** `startActivity()` **trực tiếp thay vì** `Intent.createChooser()` cho hành động chia sẻ: có thể tự mở thẳng một app cụ thể mà không hỏi, gây trải nghiệm khó lường nếu máy chỉ cài đúng một app khớp.
- **Nhầm lẫn custom scheme với App Link thật**: tưởng chỉ cần khai báo `autoVerify="true"` là xong — thực chất phải sở hữu domain và host đúng file xác minh, nếu không hệ thống âm thầm không xác minh được và rơi về hành vi hỏi "Open with" như custom scheme thường.
- **Đọc** `Intent.EXTRA_TEXT` **mà không kiểm tra** `intent.getType()`: một app khác có thể gửi `ACTION_SEND` với kiểu dữ liệu khác (ảnh, file) — luôn kiểm tra đúng mimeType mong đợi trước khi xử lý.

Tóm tắt & tiếp theo

Bạn đã biết đầy đủ các cơ chế giao tiếp liên ứng dụng ở tầng Intent — gửi/nhận chia sẻ, mở màn hình qua deep link. Chương 34 đi sâu hơn nữa: giao tiếp liên tiến trình (IPC) thật sự bằng AIDL và Bound Service, cho phép hai app gọi thẳng phương thức của nhau, không chỉ trao đổi dữ liệu qua Intent.

Chương 34: AIDL & Bound Service — giao tiếp liên tiến trình (IPC) giữa các app

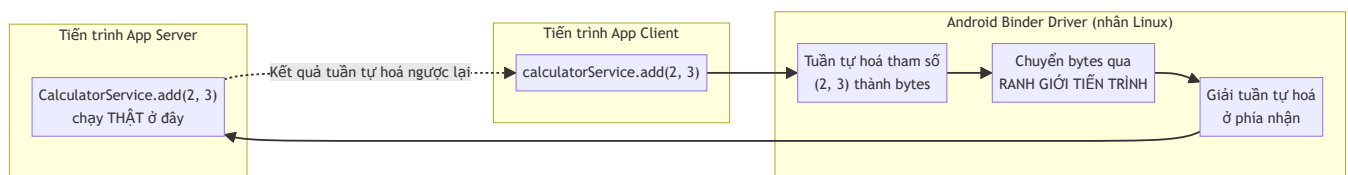
Mục tiêu học

- Hiểu IPC (Inter-Process Communication) là gì, và vì sao gọi hàm giữa hai app không đơn giản như gọi hàm bình thường.
- Viết một file `.aidl` định nghĩa interface, hiểu Android tự sinh code Binder từ nó.
- Bind một Service từ MỘT APP KHÁC (khác Chương 16, nơi bind cùng app), xử lý đúng vòng đời kết nối.
- Biết `RemoteException` là gì và vì sao nó chỉ xuất hiện với lời gọi liên tiến trình.

Code mẫu: `code/ch34-aidl-bound-service/` — hai app thật riêng biệt (`:server`, `:client`), nối tiếp đúng mô hình hai-app đã dùng ở Chương 32.

34.1 Vì sao gọi hàm giữa hai app không đơn giản như trong cùng app?

Chương 16 đã bind một Service **trong cùng app** — `LocalBinder` trả thẳng đối tượng Service, gọi hàm Java bình thường vì hai bên **chung một tiến trình, chung bộ nhớ**. Giữa hai app khác nhau, mỗi app chạy trong tiến trình Linux riêng (sandbox, Chương 1/26) — **không có bộ nhớ chung** để truyền thẳng một tham chiếu đối tượng.



Binder là cơ chế IPC riêng của Android (một phần của nhân Linux tùy biến) thực hiện toàn bộ việc tuần tự hoá/chuyển giao/giải tuần tự hoá này. **AIDL** (Android Interface Definition Language) là ngôn ngữ khai báo interface, để công cụ build **tự sinh code Java** làm việc với Binder — bạn không viết tay logic tuần tự hoá.

34.2 Viết file `.aidl`

```
// server/src/main/aidl/vn/example/ch34aidlserver/ICalculatorService.aidl
package vn.example.ch34aidlserver;

interface ICalculatorService {
    int add(int a, int b);
    int multiply(int a, int b);
    int getCallCount();
}
```

AIDL chỉ hỗ trợ một tập kiểu dữ liệu giới hạn: kiểu nguyên thủy, `String`, `List` / `Map` của chúng, và `Parcelable` tự khai báo — **không** truyền được object Java tùy ý như khi gọi hàm trong cùng tiến trình (khác hẳn `LocalBinder` ở Chương 16, vốn trả thẳng cả object Service).

Điểm dễ gây nhầm nhất: vì `:server` và `:client` là hai project Gradle hoàn toàn độc lập (không `implementation project(':server')`), file `.aidl` này phải được **chép sang cả hai module**, cùng khai báo `package vn.example.ch34aidlserver;` — công cụ build ở mỗi bên tự sinh code Java tương ứng (`ICalculatorService.Stub`, `ICalculatorService.Stub.Proxy`) khớp nhau nhờ cùng tên interface đầy đủ.

34.3 Phía Server: cài đặt `Stub`

```
public class CalculatorService extends Service {
    private final ICalculatorService.Stub binder = new ICalculatorService.Stub() {
        @Override
        public int add(int a, int b) throws RemoteException {
            callCount++;
            return a + b;
        }
    };

    @Override
    public IBinder onBind(Intent intent) {
        return binder;
    }
}
```

```
<service android:name=".CalculatorService" android:exported="true">
  <intent-filter>
    <action android:name="vn.example.ch34aidlserver.action.BIND_CALCULATOR" />
  </intent-filter>
</service>
```

`android:exported="true"` bắt buộc — cùng lý do đã gặp với `ContentProvider` (Chương 32): mặc định "false" chặn mọi app khác gọi tới.

34.4 Phía Client: bind bằng Intent tường minh + `setPackage`

```
Intent intent = new Intent("vn.example.ch34aidlserver.action.BIND_CALCULATOR");
intent.setPackage("vn.example.ch34aidlserver"); // BẮT BUỘC từ Android 5.0+
bindService(intent, connection, Context.BIND_AUTO_CREATE);
```

Khác Chương 16 (`new Intent(this, CounterService.class)` — chỉ định thẳng class vì cùng app), ở đây **không thể tham chiếu class `CalculatorService`** (nó nằm trong một APK khác, không có trên classpath của `:client`). Phải dùng implicit Intent (action) kèm `setPackage()` để giới hạn đúng app đích — thiếu `setPackage()`, hệ thống từ chối resolve intent tới bất kỳ Service nào của app khác vì lý do bảo mật.

```
private final ServiceConnection connection = new ServiceConnection() {
    @Override
    public void onServiceConnected(ComponentName name, IBinder binder) {
        calculatorService = ICalculatorService.Stub.asInterface(binder);
    }
    @Override
    public void onServiceDisconnected(ComponentName name) {
        // Tiến trình SERVER bị kill đột ngột – trường hợp gần như không xảy ra
        // khi bind cùng app (Chương 16), nhưng hoàn toàn có thể xảy ra ở đây.
    }
};
```

`ICalculatorService.Stub.asInterface(binder)` bọc `IBinder` thô nhận được thành interface Java quen thuộc — gọi `calculatorService.add(2, 3)` **trông như** một lời gọi hàm bình thường, nhưng thực chất kích hoạt toàn bộ chuỗi Binder ở mục 34.1.

34.5 RemoteException — dấu hiệu riêng của lời gọi liên tiến trình

```
try {
    int result = calculatorService.add(a, b);
} catch (RemoteException e) {
    // Tiến trình server đã chết giữa lúc gọi, hoặc lỗi truyền dữ liệu qua Binder
}
```

Mọi phương thức AIDL đều khai báo `throws RemoteException` — điều **không bao giờ xảy ra** với lời gọi hàm thông thường trong cùng tiến trình (như `LocalBinder` ở Chương 16). Đây là lời nhắc trực tiếp trong chữ ký hàm: "lời gọi này có thể thất bại vì lý do nằm ngoài tầm kiểm soát của bạn — tiến trình bên kia có thể đã không còn tồn tại".

Bài tập

1. Cài cả hai app theo README, thực hiện vài phép tính — xác nhận số lượt gọi (`getCallCount()`) tăng dần đúng, chứng minh cùng một `CalculatorService` instance phục vụ nhiều lời gọi liên tiếp.
2. Gỡ cài đặt app `:server`, mở lại app `:client` — xác nhận thấy thông báo "Không tìm thấy Calculator Server" thay vì crash.
3. Thử xóa dòng `intent.setPackage( ... )` ở `:client`, build và chạy lại — quan sát `bindService()` trả về `false` (không tìm được Service nào), minh chứng trực tiếp cho mục 34.4.

Lỗi thường gặp

- **Quên chép file `.aidl` sang module còn lại (hoặc chép nhưng sai package):** hai bên sinh ra interface không khớp, lỗi biên dịch hoặc lỗi runtime khó hiểu.
- **Quên khai báo `<queries>` trong manifest client (Android 11+):** `bindService()` âm thầm không tìm thấy Service, dù app server đã cài đúng — package visibility (đã gặp khái niệm tương tự cần cân nhắc ở Chương 32/33) chặn truy vấn package khác nếu không khai báo rõ.
- **Quên `setPackage()` khi bind implicit Intent sang app khác:** hệ thống từ chối resolve, không tìm được Service nào dù mọi khai báo khác đều đúng.
- **Không xử lý `RemoteException`:** app crash ngay khi tiến trình server bị hệ thống kill đúng lúc đang gọi — tình huống hiếm nhưng hoàn toàn có thể xảy ra trong thực tế.

Tóm tắt & tiếp theo

Bạn đã biết cơ chế giao tiếp liên tiến trình sâu nhất trong sách — nền tảng của mọi Binder-based API hệ thống (kể cả `ContentProvider` ở Chương 32 thực chất cũng dùng Binder bên dưới). Chương 35 chuyển hẳn sang phần cứng: kết nối Bluetooth Classic và BLE để giao tiếp với thiết bị bên ngoài, không chỉ giữa các app trên cùng máy.

Chương 35: Bluetooth — Classic & BLE, quét thiết bị, kết nối, truyền/nhận dữ liệu

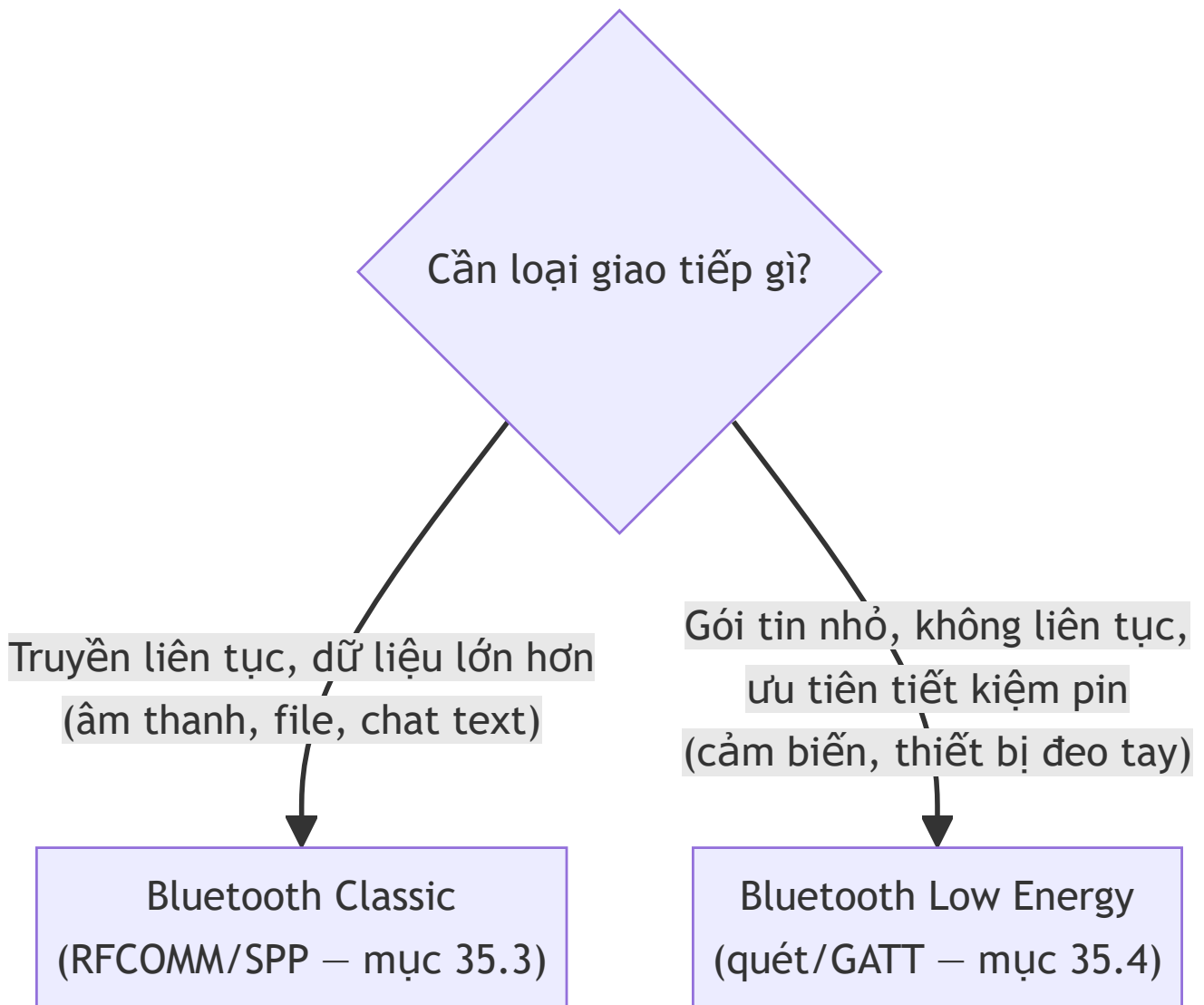
Lưu ý: chương này cần **thiết bị Android thật** để thực hành đầy đủ — máy ảo (Chương 3) không mô phỏng phần cứng radio Bluetooth thật. Cần ít nhất hai thiết bị thật cho phần Classic (chat hai chiều), một thiết bị cho phần quét BLE.

Mục tiêu học

- Phân biệt Bluetooth Classic (kết nối bền, truyền dữ liệu liên tục — âm thanh, file, chat) và BLE (Bluetooth Low Energy — tiết kiệm pin, truyền gói tin nhỏ, phổ biến ở thiết bị đeo tay/IoT).
- Xin đúng bộ quyền Bluetooth theo từng nhóm phiên bản Android (trước và từ Android 12).
- Cài đặt kết nối Bluetooth Classic hai chiều bằng mẫu Server/Client kinh điển (`AcceptThread` / `ConnectThread` / `ConnectedThread`).
- Quét thiết bị BLE xung quanh bằng `BluetoothLeScanner`.

Code mẫu: `code/ch35-bluetooth/`.

35.1 Bluetooth Classic và BLE — chọn loại nào?



Code mẫu chương này minh họa cả hai: **chat hai chiều** qua Bluetooth Classic (đúng chuẩn Serial Port Profile — SPP), và **quét thiết bị** qua BLE (không đi sâu vào việc đọc/ghi dữ liệu GATT của BLE, vốn phức tạp hơn và nằm ngoài phạm vi giới thiệu của sách).

35.2 Quyền Bluetooth — khác nhau rõ rệt trước/sau Android 12

Từ Android 12 (API 31+)

BLUETOOTH_SCAN — RUNTIME,
thay thế vai trò quét của ACCESS_FINE_LOCATION

BLUETOOTH_CONNECT — RUNTIME,
cần để kết nối/lấy tên thiết bị

Trước Android 12 (API ≤ 30)

BLUETOOTH, BLUETOOTH_ADMIN
— quyền 'normal', chỉ cần khai báo manifest

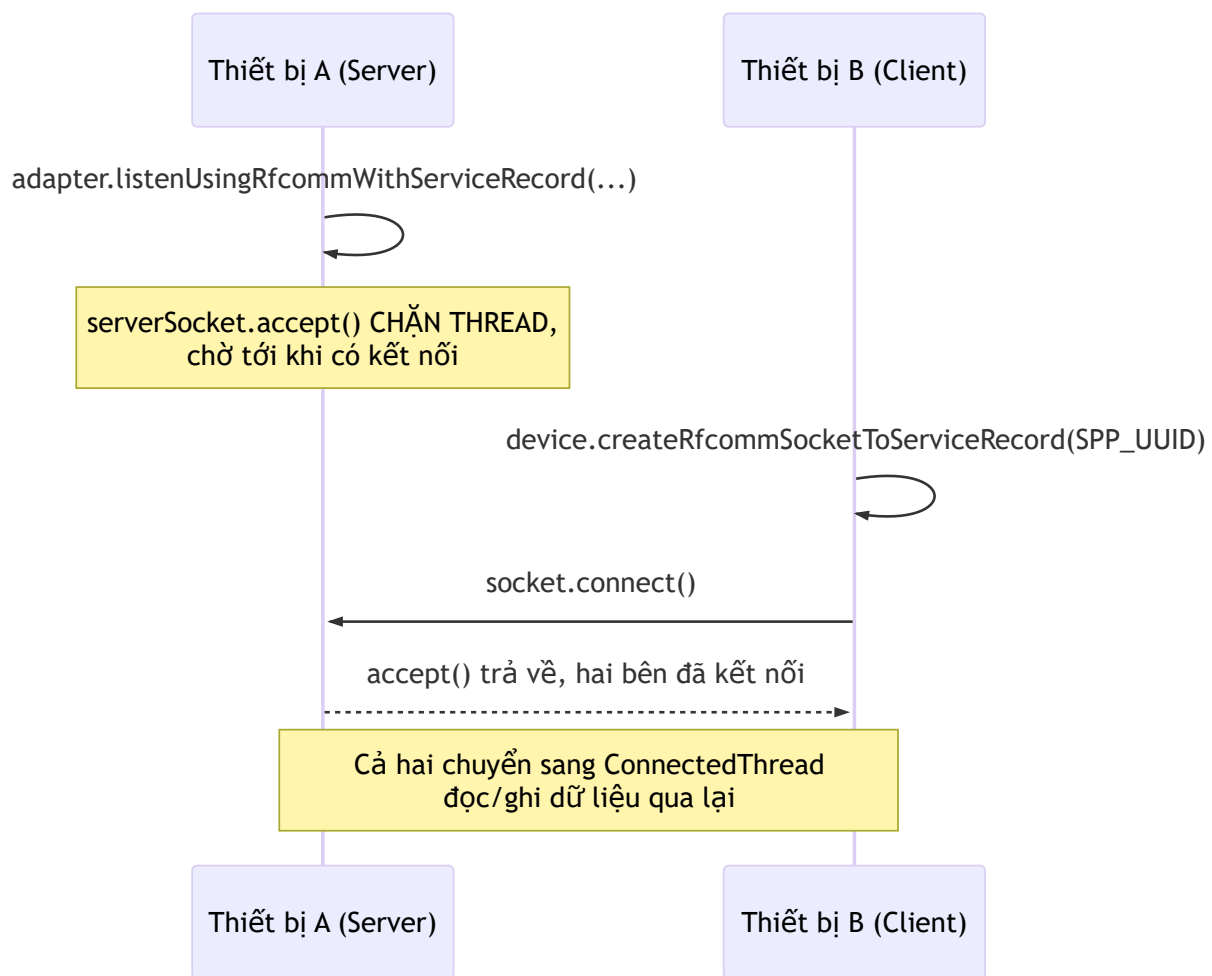
ACCESS_FINE_LOCATION
— RUNTIME, bắt buộc để QUÉT
(lý do lịch sử: suy ra vị trí qua BLE)

```
<uses-permission android:name="android.permission.BLUETOOTH" android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"
  android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
  android:maxSdkVersion="30" />

<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
  android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
```

`android:usesPermissionFlags="neverForLocation"` là khai báo quan trọng: nó nói với hệ thống "app này quét Bluetooth nhưng **không** dùng kết quả để suy ra vị trí người dùng" — nhờ đó xin được `BLUETOOTH_SCAN` **mà không cần kèm** `ACCESS_FINE_LOCATION` trên Android 12+. Code mẫu gom logic chọn đúng bộ quyền theo `Build.VERSION.SDK_INT` vào `BluetoothPermissions.java`, đúng tinh thần "gom logic vào một chỗ" đã áp dụng với `PrefsManager` ở Chương 12.

35.3 Bluetooth Classic — mẫu Server/Client/Connected kinh điển



Ba thread luôn xuất hiện trong mọi cài đặt Bluetooth Classic hai chiều:

```
// Server: chờ kết nối tới – accept() CHẶN THREAD
BluetoothServerSocket serverSocket = adapter.listenUsingRfcommWithServiceRecord(NAME,
SPP_UUID);
BluetoothSocket socket = serverSocket.accept();

// Client: chủ động kết nối – connect() cũng CHẶN THREAD
BluetoothSocket socket = device.createRfcommSocketToServiceRecord(SPP_UUID);
socket.connect();

// Cả hai, sau khi đã kết nối: đọc liên tục – read() CHẶN THREAD tới khi có dữ liệu
InputStream input = socket.getInputStream();
int bytes = input.read(buffer);
```

Cả ba lời gọi in đậm (`accept` , `connect` , `read`) đều **chặn (blocking)** — đúng nguyên tắc đã lặp lại xuyên suốt từ Chương 13/19: **không bao giờ** chạy chúng trên main thread. Code mẫu (`BluetoothChatManager.java`) đặt mỗi vai trò vào một `Thread` riêng (`AcceptThread` , `ConnectThread` ,

`ConnectedThread`), báo kết quả về main thread qua `Handler` — chính là mẫu hình đã quen thuộc từ Chương 16 (Service dùng Handler báo về Activity).

`SPP_UUID` (`00001101-0000-1000-8000-00805F9B34FB`) là UUID **chuẩn, cố định** cho Serial Port Profile — hầu như mọi thiết bị Bluetooth Classic hỗ trợ truyền dữ liệu nối tiếp đều dùng chung giá trị này, không phải do bạn tự đặt.

35.4 Quét BLE — không cần ghép đôi để "thấy" thiết bị

```
BluetoothLeScanner scanner = bluetoothAdapter.getBluetoothLeScanner();

ScanCallback callback = new ScanCallback() {
    @Override
    public void onScanResult(int callbackType, ScanResult result) {
        String name = result.getScanRecord().getDeviceName();
        int rssi = result.getRssi();    // cường độ tín hiệu – càng gần 0 càng mạnh
        // ...
    }
};

scanner.startScan(callback);
// ... sau một khoảng thời gian:
scanner.stopScan(callback);
```

Khác Bluetooth Classic (`getBondedDevices()` chỉ liệt kê thiết bị đã **ghép đôi** từ Settings), BLE scan phát hiện **mọi thiết bị đang quảng bá (advertising)** xung quanh — không cần ghép đôi trước. Đây là lý do tai nghe, vòng đeo tay thông minh thường "hiện ra ngay" khi mở app quét BLE, dù chưa từng kết nối với chúng.

Luôn giới hạn thời gian quét — code mẫu tự động `stopScan()` sau 12 giây bằng `Handler.postDelayed`, và dừng ngay trong `onStop()` nếu người dùng rời màn hình. Quét BLE liên tục vô thời hạn là nguyên nhân phổ biến khiến app bị coi là "hao pin bất thường".

Bài tập

- Với hai thiết bị thật đã ghép đôi Bluetooth, làm theo README: một bên Server, một bên Client, gửi tin nhắn qua lại.
- Mở màn hình quét BLE trên một thiết bị thật, xác nhận thấy danh sách thiết bị xung quanh kèm RSSI, không thiết bị nào trùng lặp (nhờ `Map` theo địa chỉ MAC trong `BleScanActivity`).
- Tắt Bluetooth trên thiết bị Client giữa lúc đang chat — quan sát thiết bị Server nhận được sự kiện "Đã ngắt kết nối" qua `ConnectedThread` bắt được `IOException` từ `read()`.

Lỗi thường gặp

- **Xin `ACCESS_FINE_LOCATION` trên Android 12+ mà quên `BLUETOOTH_SCAN`**: quét thất bại âm thầm hoặc ném lỗi quyền — hai bộ quyền phục vụ hai nhóm phiên bản khác nhau, không thể dùng chung.
- **Gọi `connect()` / `accept()` / `read()` trực tiếp trên main thread**: đứng UI ngay lập tức (các hàm này chặn vô thời hạn tới khi có sự kiện) — luôn bọc trong `Thread` riêng như code mẫu.
- **Quên `adapter.cancelDiscovery()` trước khi `connect()`**: quá trình quét (nếu đang chạy) làm chậm đáng kể, thậm chí khiến `connect()` thất bại — thói quen tốt luôn hủy discovery trước khi kết nối Classic.
- **Quét BLE không giới hạn thời gian, không dừng khi rời màn hình**: tốn pin nghiêm trọng — luôn có cơ chế tự dừng (timeout) và dừng trong `onStop()`.

Tóm tắt & tiếp theo

Bạn đã biết giao tiếp với thiết bị bên ngoài qua cả hai chuẩn Bluetooth phổ biến. Chương 36 tiếp tục với NFC — giao tiếp tầm cực gần, thường dùng cho thanh toán, đọc thẻ, và chia sẻ dữ liệu bằng cách chạm hai thiết bị vào nhau.

Chương 36: NFC — đọc/ghi tag, Host Card Emulation (HCE), giao tiếp giữa 2 thiết bị

Lưu ý: phần đọc/ghi thẻ NFC cần **thiết bị Android thật có NFC** — máy ảo không mô phỏng được sóng NFC thật. Phần Host Card Emulation (HCE) khó tự kiểm chứng hơn nữa, cần đầu đọc NFC chuyên dụng — xem giải thích chi tiết ở mục 36.5.

Mục tiêu học

- Hiểu NFC hoạt động ở khoảng cách cực gần (vài cm), khác hẳn tầm hoạt động của Bluetooth (Chương 35).
- Dùng Foreground Dispatch để giành quyền xử lý sự kiện chạm thẻ khi Activity đang mở.
- Đọc và ghi `NdefMessage` / `NdefRecord` — định dạng dữ liệu chuẩn cho tag NFC.
- Biết Host Card Emulation (HCE) là gì ở mức khái niệm — cơ chế đứng sau Google Wallet.

Code mẫu: `code/ch36-nfc/`.

36.1 NFC khác Bluetooth ở điểm nào?

NFC (chương này)		
Tầm hoạt động: VÀI CENTIMET	KHÔNG cần ghép đôi — chạm là nhận diện	Phù hợp: thanh toán, đọc thẻ, trao đổi lượng dữ liệu NHỎ tức thời
Bluetooth (Chương 35)		
Tầm hoạt động: vài mét tới vài chục mét	Cần ghép đôi (Classic) hoặc quét (BLE)	Phù hợp: truyền liên tục, dữ liệu lớn hơn

Chính khoảng cách cực gần này khiến NFC phù hợp cho các tình huống cần **chủ đích rõ ràng** của người dùng (chạm thẻ để thanh toán, chạm để mở khoá) — khó bị kích hoạt nhầm hay từ xa như Bluetooth.

36.2 Foreground Dispatch — giành quyền xử lý khi Activity đang mở

```
@Override
protected void onResume() {
    super.onResume();
    nfcAdapter.enableForegroundDispatch(this, pendingIntent, intentFilters, null);
}

@Override
protected void onPause() {
    super.onPause();
    nfcAdapter.disableForegroundDispatch(this);
}
```

Không có Foreground Dispatch, khi chạm thẻ, hệ thống tự chọn app xử lý dựa trên `<intent-filter>` tĩnh trong manifest — có thể hiện hộp thoại "Open with" nếu nhiều app cùng khai báo, hoặc mở nhầm app khác. `enableForegroundDispatch()` (gọi trong `onResume`) đảm bảo **Activity đang mở này luôn nhận sự kiện trước tiên**, không hỏi han gì — đúng cập bật/tắt theo vòng đời `onResume` / `onPause` đã quen thuộc từ nguyên tắc đăng ký/hủy đăng ký ở Chương 12/17.

```
private void setupForegroundDispatch() {
    Intent intent = new Intent(this,
        getClass()).addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP);
    pendingIntent = PendingIntent.getActivity(this, 0, intent,
        PendingIntent.FLAG_MUTABLE);
    intentFilters = new IntentFilter[]{
        new IntentFilter(NfcAdapter.ACTION_NDEF_DISCOVERED),
        new IntentFilter(NfcAdapter.ACTION_TAG_DISCOVERED)
    };
}
```

`PendingIntent` (đã học kỹ ở Chương 20, nhìn lại ở Chương 33) một lần nữa xuất hiện — đây là cách hệ thống "gọi ngược" vào đúng Activity đang mở khi phát hiện thẻ, dùng `FLAG_MUTABLE` (khác `FLAG_IMMUTABLE` ở Chương 20) vì hệ thống cần **chèn thêm dữ liệu thẻ** vào Intent trước khi gửi lại cho app.

36.3 Đọc `NdefMessage` từ thẻ

```
@Override
protected void onNewIntent(Intent intent) {
    super.onNewIntent(intent);
    Tag tag = intent.getParcelableExtra(NfcAdapter.EXTRA_TAG);
    Parcelable[] rawMessages =
intent.getParcelableArrayExtra(NfcAdapter.EXTRA_NDEF_MESSAGES);
    if (rawMessages != null) {
        NdefMessage message = (NdefMessage) rawMessages[0];
        for (NdefRecord record : message.getRecords()) {
            String text = decodeTextRecord(record);
            // hiển thị text
        }
    }
}
```

`NDEF` (NFC Data Exchange Format) là định dạng dữ liệu chuẩn cho tag NFC — một `NdefMessage` chứa một hoặc nhiều `NdefRecord`. Việc giải mã một record kiểu Text đòi hỏi đọc đúng cấu trúc byte: byte đầu tiên là độ dài mã ngôn ngữ, phần còn lại mới là văn bản thật:

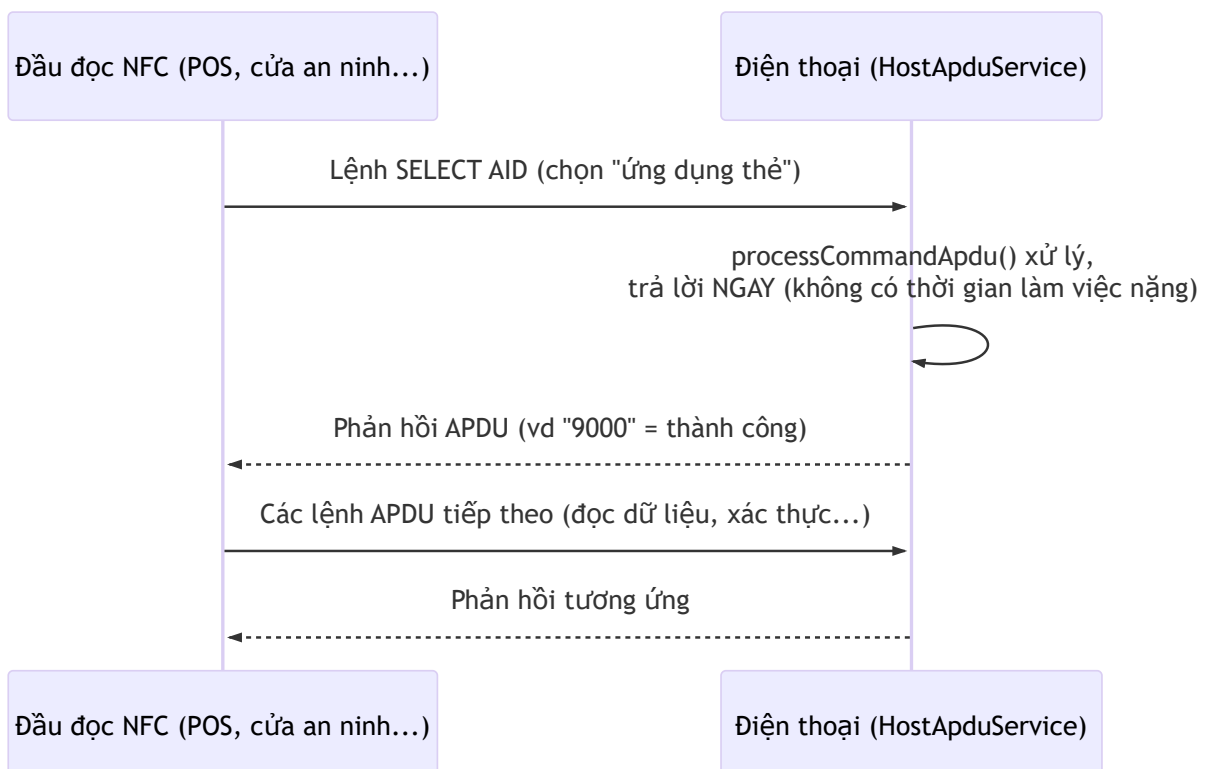
```
private String decodeTextRecord(NdefRecord record) {
    byte[] payload = record.getPayload();
    int langLength = payload[0] & 0x3F;
    return new String(payload, 1 + langLength, payload.length - 1 - langLength,
StandardCharsets.UTF_8);
}
```

36.4 Ghi `NdefMessage` vào thẻ

```
Ndef ndef = Ndef.get(tag);
if (ndef != null) {
    ndef.connect();
    if (ndef.isWritable()) {
        ndef.writeNdefMessage(message);
    }
    ndef.close();
} else {
    // Thẻ TRỐNG, chưa từng có định dạng NDEF – cần format trước khi ghi
    NdefFormatable formatable = NdefFormatable.get(tag);
    if (formatable != null) {
        formatable.connect();
        formatable.format(message);
        formatable.close();
    }
}
```

Hai đường khác nhau tùy trạng thái thẻ: `Ndef` (thẻ đã có định dạng NDEF, có thể ghi đè) và `NdefFormatable` (thẻ hoàn toàn trống, cần định dạng lần đầu trước khi ghi được). Luôn kiểm tra `isWritable()` — một số thẻ bị khoá ghi (read-only) sau lần format/ghi đầu tiên có chủ đích.

36.5 Host Card Emulation (HCE) — điện thoại "giả làm" thẻ



Khác mọi phần trước (điện thoại **chủ động đọc** thẻ), HCE khiến điện thoại đóng vai **bị động** — chờ một đầu đọc bên ngoài gửi lệnh APDU (theo chuẩn ISO 7816-4) tới. Đây chính là cơ chế đứng sau việc dùng điện thoại thay thẻ giao thông, thẻ ra vào văn phòng, hay Google Wallet.

```
public class HceService extends HostApuService {
    @Override
    public byte[] processCommandApu(byte[] commandApu, Bundle extras) {
        if (startsWith(commandApu, SELECT_APDU_HEADER)) {
            return SUCCESS_RESPONSE; // phải trả lời NGAY, không có độ trễ
        }
        return UNKNOWN_COMMAND;
    }
}
```

```
<aid-group android:category="other">
    <aid-filter android:name="F0010203040506" />
</aid-group>
```

AID (Application Identifier) đóng vai trò tương tự **AUTHORITY** của **ContentProvider** (Chương 32) — "địa chỉ" để đầu đọc chọn đúng "ứng dụng thẻ" cần giao tiếp trong số nhiều app HCE có thể cài trên máy. Dải AID bắt đầu bằng **F0** dành riêng cho mục đích proprietary/nội bộ, không trùng với AID thật của các hệ thống thanh toán.

Vì sao khó tự test tại nhà: HCE cần một **đầu đọc NFC thật** (không phải điện thoại khác) đã biết gửi đúng lệnh APDU khớp AID khai báo — khác phần đọc/ghi tag (mục 36.3-36.4) chỉ cần một thẻ NFC rẻ tiền là tự kiểm chứng được ngay.

Bài tập

1. Chạy code mẫu trên thiết bị thật, thử luồng ghi rồi đọc lại đúng nội dung trên cùng một thẻ NFC.
2. Thử chạm một thẻ NFC hoàn toàn mới (chưa từng format) — xác nhận app tự chuyển sang dùng **NdefFormatable** thay vì **Ndef**.
3. Đọc lại **HceService.java**, tự vẽ ra sơ đồ tương tự mục 36.5 nhưng cho tình huống app của bạn dùng để mở cửa văn phòng bằng điện thoại — xác định đâu là "đầu đọc", đâu là "lệnh APDU" trong ngữ cảnh đó.

Lỗi thường gặp

- Quên **disableForegroundDispatch()** trong **onPause()**: Activity tiếp tục giành sự kiện NFC dù không còn hiển thị, chặn cả Activity khác trong cùng app nhận đúng sự kiện của nó.

- **Không kiểm tra** `NfcAdapter.getDefaultAdapter(this)` **trả về** `null` : crash ngay trên thiết bị không có phần cứng NFC — luôn kiểm tra trước khi dùng, giống `BluetoothAdapter` ở Chương 35.
 - **Cố ghi vào thẻ mà không kiểm tra** `isWritable()` : một số thẻ bị khoá ghi có chủ đích (ví dụ thẻ NFC dùng cho mục đích công cộng, cố định nội dung) — `writeNdefMessage()` ném lỗi rõ ràng thay vì âm thầm thất bại.
 - **Xử lý chậm trong** `processCommandApdu()` : đầu đọc NFC thường có thời gian chờ phản hồi rất ngắn (vài trăm ms) — bất kỳ việc nặng nào (mạng, đĩa) đều khiến giao dịch thất bại; trả lời gần như tức thời là bắt buộc.
-

Tóm tắt & tiếp theo

Bạn đã hoàn thành ba chương phần cứng (Bluetooth, NFC) — đủ kiến thức cơ bản để giao tiếp với thế giới vật lý bên ngoài điện thoại. Chương 37 khép lại Phần 7 bằng việc hệ thống hoá toàn bộ các quyền runtime đã gặp rải rác (vị trí cho quét Bluetooth/BLE, quyền theo từng API level) thành một bức tranh đầy đủ.

Chương 37: Quyền & sandbox cho phần cứng — vị trí cho BLE/NFC, background location, runtime permission theo API level

Mục tiêu học

- Hệ thống hoá lại toàn bộ các quyền runtime đã gặp rải rác từ Chương 26, 35, 36 thành một bức tranh theo dòng thời gian API level.
- Xin đúng trình tự quyền vị trí foreground và background — hai bước bắt buộc tách biệt.
- Biết tự đặt câu hỏi "app có thật sự cần quyền này không?" trước khi xin — nguyên tắc xuyên suốt của cả Phần 7.

Code mẫu: `code/ch37-hardware-permission/`.

37.1 Toàn cảnh: quyền phần cứng đã đổi thế nào qua các phiên bản

API 21-22
Cấp quyền LÚC CÀI ĐẶT
(chưa có runtime permission)



API 23 (Android 6.0)
Runtime permission ra đời
(Chương 26)



API 29 (Android 10)
ACCESS_BACKGROUND_LOCATION
tách riêng khỏi foreground



API 30 (Android 11)

API 30 (Android 11)
Bắt buộc xin foreground TRƯỚC,
background SAU, không gộp chung
+ Package visibility (Chương 34)



API 31 (Android 12)
BLUETOOTH_SCAN/CONNECT thay thế
vai trò ACCESS_FINE_LOCATION
cho Bluetooth (Chương 35)



API 33 (Android 13)
POST_NOTIFICATIONS runtime
(Chương 20)

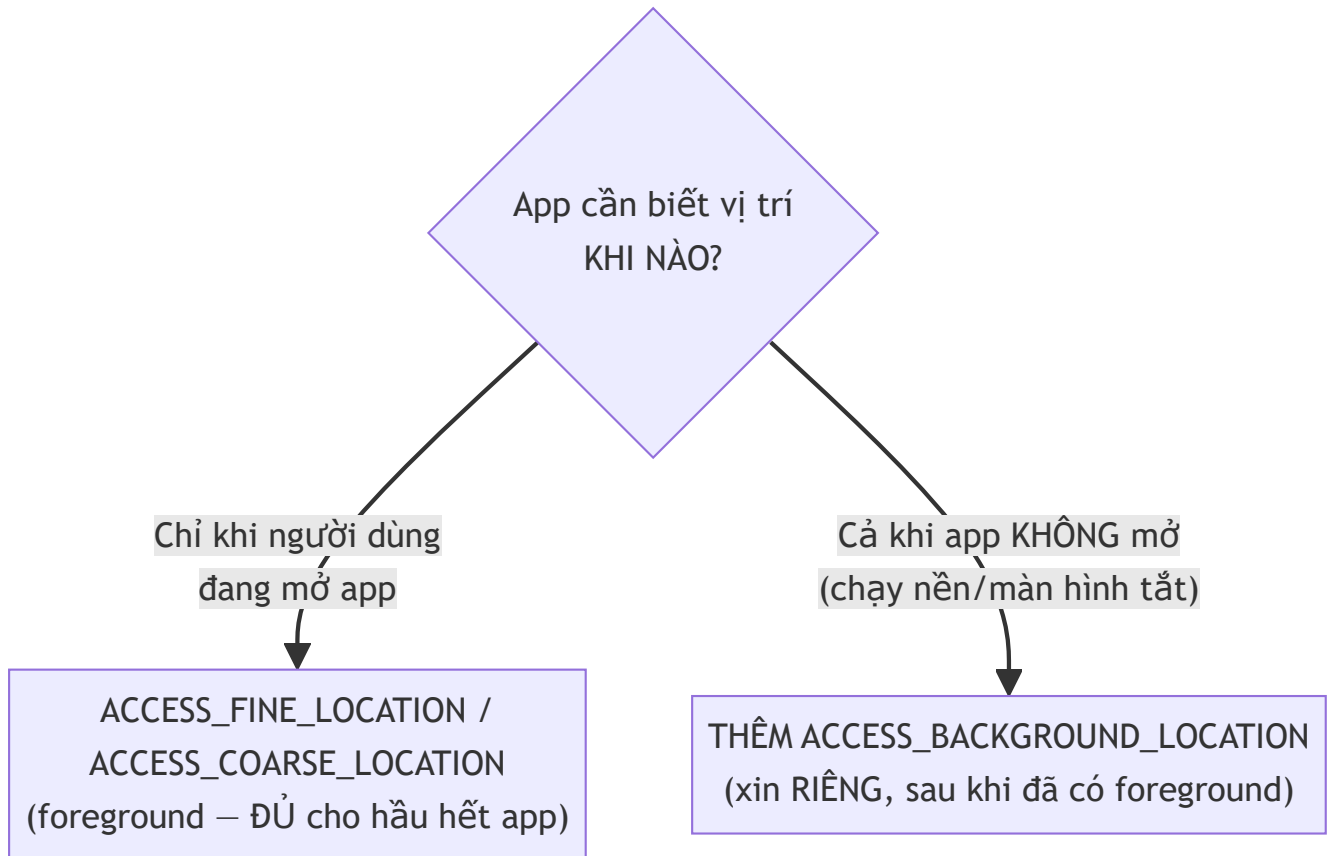
Mỗi mốc trong sơ đồ này đã xuất hiện rải rác: Chương 20 (thông báo), Chương 26 (mô hình chung + danh bạ), Chương 34 (package visibility), Chương 35 (Bluetooth). Chương này gom lại thành một bức tranh duy nhất, tập trung vào mảnh còn thiếu: **vị trí (location)**.

37.2 Vì sao quét Bluetooth/BLE từng cần quyền vị trí?

Đây là câu hỏi khiến nhiều người mới bối rối nhất: "Tại sao app Bluetooth của tôi lại đòi quyền vị trí?" Chương 35 đã nhắc: **trước Android 12**, hệ thống dùng chung `ACCESS_FINE_LOCATION` để bảo vệ cả việc truy cập vị trí GPS thật lẫn việc quét Bluetooth/Wi-Fi — vì địa chỉ MAC của các access point/thiết bị BLE xung quanh có thể dùng để **suy luận ra vị trí tương đối chính xác** của người dùng (kỹ thuật định vị không cần GPS, dựa vào cơ sở dữ liệu vị trí các access point đã biết).

Từ Android 12, hai quyền `BLUETOOTH_SCAN` / `BLUETOOTH_CONNECT` tách riêng, cho phép khai báo rõ ràng `neverForLocation` (mục 35.2) nếu app không dùng kết quả quét để suy ra vị trí — tách bạch hai mục đích trước đây bị gộp chung.

37.3 Vị trí Foreground và Background — hai quyền, hai câu hỏi khác nhau



```
// Bước 1 – foreground, xin trước, dùng được ngay khi app đang mở
foregroundLauncher.launch(new String[]{
    Manifest.permission.ACCESS_FINE_LOCATION,
    Manifest.permission.ACCESS_COARSE_LOCATION
});

// Bước 2 – CHỈ xin nếu thật sự cần, và CHỈ SAU KHI đã có bước 1
backgroundLauncher.launch(Manifest.permission.ACCESS_BACKGROUND_LOCATION);
```

Từ Android 11 (API 30), hệ thống **không cho phép** xin cả hai quyền trong cùng một hộp thoại — phải tách thành hai bước tuần tự như trên. Nhiều phiên bản Android còn tự động **chuyển hướng người dùng sang màn hình Settings** để chọn "Allow all the time" cho quyền nền, thay vì hiện hộp thoại Allow/Deny quen thuộc — hành vi giao diện cụ thể do hệ thống quyết định, code gọi API vẫn giữ nguyên cách làm.

Câu hỏi quan trọng nhất trước khi viết dòng code xin quyền nền: "Tính năng này có thật sự cần vị trí khi app không mở không?" Một app bản đồ tra cứu đường đi, hay tìm cửa hàng gần đây, **không cần** `ACCESS_BACKGROUND_LOCATION` — chỉ ứng dụng như theo dõi hành trình chạy bộ liên tục, hay chia sẻ vị trí thời gian thực khi đã tắt màn hình, mới thật sự cần. Google Play cũng yêu cầu giải trình rõ ràng mục đích sử dụng quyền này khi phát hành (liên quan tới Chương 39) — xin quá tay không chỉ ảnh hưởng lòng tin người dùng mà còn có thể bị từ chối duyệt.

37.4 Bảng tổng hợp — tra cứu nhanh

Tính năng	Quyền cần thiết	Từ API level	Xin runtime?
Mạng (Chương 21)	<code>INTERNET</code>	Mọi phiên bản	Không (normal)
Foreground service (Chương 16)	<code>FOREGROUND_SERVICE</code>	28+	Không (normal)
Thông báo (Chương 20)	<code>POST_NOTIFICATIONS</code>	33+	Có
Đọc danh bạ (Chương 26)	<code>READ_CONTACTS</code>	Mọi phiên bản	Có (dangerous)
Quét Bluetooth/BLE (Chương 35), trước 12	<code>ACCESS_FINE_LOCATION</code>	≤ 30	Có
Quét/kết nối Bluetooth (Chương 35), từ 12	<code>BLUETOOTH_SCAN</code> , <code>BLUETOOTH_CONNECT</code>	31+	Có
NFC (Chương 36)	<code>NFC</code>	Mọi phiên bản	Không (normal)
Vị trí khi dùng app (chương này)	<code>ACCESS_FINE_LOCATION</code> / <code>ACCESS_COARSE_LOCATION</code>	Mọi phiên bản (runtime từ 23)	Có
Vị trí khi chạy nền (chương này)	<code>ACCESS_BACKGROUND_LOCATION</code>	29+	Có , xin RIÊNG sau foreground

Bài tập

1. Chạy code mẫu, xin quyền foreground trước, xác nhận vị trí cập nhật đúng.
2. Thử xin quyền background — quan sát hành vi hệ thống trên máy ảo/thiết bị bạn đang dùng (hộp thoại thường, hay chuyển sang Settings).
3. Tự tra bảng ở mục 37.4, chọn ra 3 quyền bạn nghĩ dễ bị lạm dụng/xin quá mức nhất trong các app thường gặp trên điện thoại của mình, giải thích lý do.

Lỗi thường gặp

- **Xin `ACCESS_BACKGROUND_LOCATION` cùng lúc với `ACCESS_FINE_LOCATION` trong cùng một lời gọi:** bị hệ thống từ chối/bỏ qua trên Android 11+ — phải tách thành hai bước tuần tự đúng như mục 37.3.
- **Xin quyền vị trí nền "cho chắc" dù tính năng không thật sự cần:** tăng nguy cơ bị từ chối khi đăng Google Play, và giảm lòng tin người dùng khi thấy yêu cầu quyền không tương xứng với chức năng hiển thị.
- **Quên rằng hành vi UI của hộp thoại xin quyền vị trí nền thay đổi giữa các phiên bản Android:** đừng giả định luôn có nút "Allow"/"Deny" đơn giản — một số phiên bản đưa thẳng người dùng vào Settings.
- **Nhầm lẫn quyền cho việc quét Bluetooth (Chương 35) với quyền vị trí thật của chương này:** dù cùng nhóm "location-related" trong lịch sử Android, `BLUETOOTH_SCAN` (từ Android 12) và `ACCESS_FINE_LOCATION` (dùng cho GPS thật) là hai quyền độc lập, phục vụ hai mục đích khác nhau.

Tóm tắt & tiếp theo

Bạn đã khép lại **Phần 7 — Giao tiếp liên ứng dụng & phần cứng nâng cao**: chia sẻ dữ liệu (ContentProvider), deep link/share, IPC qua AIDL, Bluetooth, NFC, và toàn cảnh hệ thống quyền chi phối tất cả. Phần 8 — phần cuối cùng của sách — chuyển sang việc đưa ứng dụng ra thế giới thật: chuẩn bị bản release, ký ứng dụng, và phát hành lên Google Play.

Phần 8 — Xuất bản ứng dụng

Chương 38: Chuẩn bị release build, ký ứng dụng, tối ưu kích thước (App Bundle)

Mục tiêu học

- Cấu hình `signingConfig` để Gradle tự động ký bản release, đọc thông tin nhạy cảm từ file KHÔNG commit.
- Hiểu `versionCode` / `versionName` khác nhau ra sao và quy tắc tăng chúng qua mỗi lần phát hành.
- Phân biệt APK và Android App Bundle (AAB) — vì sao Google Play khuyến nghị AAB.

Code mẫu: `code/ch38-release-build/` — hoàn thiện nốt phần build release mà Chương 27 mới dừng ở bước ProGuard/R8.

38.1 Không bao giờ để mật khẩu keystore trong file commit

Chương 27 đã cảnh báo: mất keystore đồng nghĩa không update được app cũ. Cảnh báo thứ hai cũng quan trọng không kém: **để lộ mật khẩu keystore** (commit nhầm lên Git, kể cả repo riêng tư) cũng nguy hiểm tương đương — ai có mật khẩu đều ký được bản cập nhật giả mạo dưới danh nghĩa app của bạn.

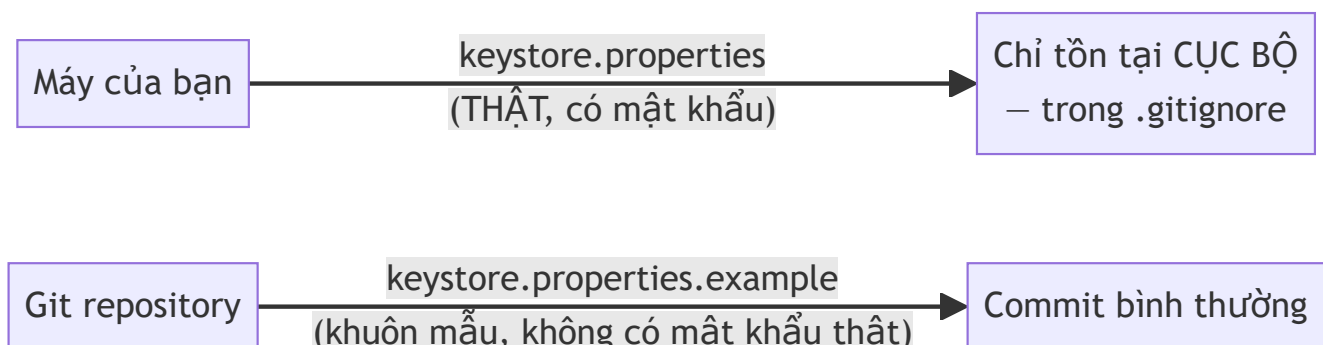
```

def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file("keystore.properties")
def hasKeystoreConfig = keystorePropertiesFile.exists()
if (hasKeystoreConfig) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            if (hasKeystoreConfig) {
                storeFile rootProject.file(keystoreProperties['storeFile'])
                storePassword keystoreProperties['storePassword']
                keyAlias keystoreProperties['keyAlias']
                keyPassword keystoreProperties['keyPassword']
            }
        }
    }
    buildTypes {
        release {
            if (hasKeystoreConfig) {
                signingConfig signingConfigs.release
            }
        }
    }
}

```

`keystore.properties` nằm trong `.gitignore` — thông tin thật (mật khẩu, đường dẫn file `.jks`) chỉ tồn tại **cục bộ trên máy** của người thực hiện build release, không bao giờ đi qua hệ thống quản lý phiên bản. `keystore.properties.example` (có commit) chỉ là **khuôn mẫu** để người khác biết cần điền gì.

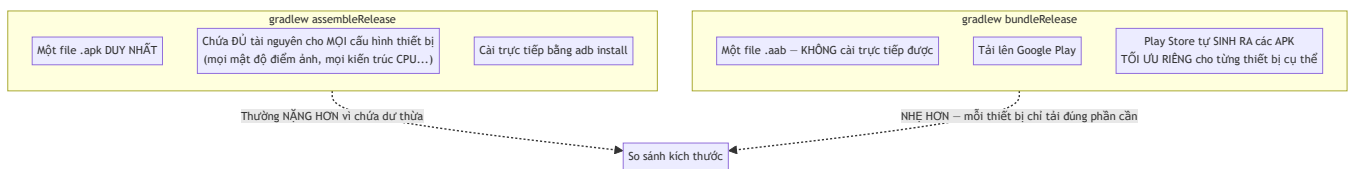


38.2 `versionCode` và `versionName` — hai con số, hai vai trò khác nhau

```
defaultConfig {  
    versionCode 3          // SỐ NGUYÊN – Play Console dùng để biết bản nào MỚI HƠN  
    versionName "1.2.0"    // CHUỖI hiển thị cho người dùng – tự do đặt quy ước  
}
```

`versionCode` **bắt buộc tăng dần** qua mỗi lần tải bản mới lên Play Console — Google Play từ chối thẳng một bản tải lên có `versionCode` bằng hoặc nhỏ hơn bản đã có. `versionName` chỉ mang tính hiển thị (người dùng thấy trong Play Store, trong Settings app), không ảnh hưởng logic cập nhật — quy ước phổ biến là semantic versioning (`MAJOR.MINOR.PATCH`), nhưng hoàn toàn tự do đặt theo cách bạn muốn.

38.3 APK và Android App Bundle (AAB) — vì sao Google khuyến nghị AAB?



APK truyền thống đóng gói **mọi biến thể tài nguyên** (mọi mật độ màn hình, mọi kiến trúc CPU) vào một file duy nhất — thiết bị nào cũng tải về đủ cả, dù chỉ dùng một phần nhỏ. AAB giao cho Google Play việc "cắt" ra đúng APK phù hợp cho từng thiết bị tải xuống, giảm dung lượng tải về đáng kể. Từ 2021, Google Play **yêu cầu bắt buộc** định dạng AAB cho app mới — sẽ dùng trực tiếp ở Chương 39.

```
.\gradlew.bat assembleRelease    # ra .apk  
.\gradlew.bat bundleRelease      # ra .aab – dùng cho Chương 39
```

Bài tập

- Build thử `assembleRelease` khi chưa có `keystore.properties` — xác nhận vẫn ra được file APK (chưa ký).
- Tự tạo keystore thử nghiệm theo README, tạo `keystore.properties` trỏ tới nó, build lại — dùng `apksigner verify --print-certs` xác nhận APK đã được ký.
- Tăng `versionCode` lên 4, build lại — so sánh với bước 2, xác nhận Gradle không phàn nàn gì (không có ràng buộc gì ở tầng build cục bộ, ràng buộc `versionCode` chỉ được Play Console kiểm tra khi tải lên thật, xem Chương 39).

Lỗi thường gặp

- **Commit nhầm file** `keystore.properties` **thật (không phải `.example`) lên Git**: rò rỉ mật khẩu keystore — nếu đã lỡ commit, phải coi như mật khẩu đã bị lộ, đổi mật khẩu/tạo lại toàn bộ keystore nếu có thể (dù việc đổi keystore cho app đã public gặp đúng vấn đề Chương 27 đã cảnh báo).
- **Quên tăng** `versionCode` **trước khi build bản mới**: Google Play từ chối bản tải lên với thông báo lỗi rõ ràng — dễ khắc phục nhưng dễ quên nếu không có quy trình nhắc nhở.
- **Tương file** `.aab` **cài trực tiếp được như** `.apk`: `adb install app-release.aab` báo lỗi — AAB chỉ dùng để tải lên Play Console, muốn cài thử cục bộ cần dùng công cụ `bundletool` để tự sinh APK từ AAB (nằm ngoài phạm vi chi tiết của sách).
- **Đặt cùng một mật khẩu cho** `storePassword` **và** `keyPassword`: hoạt động được nhưng làm giảm một lớp bảo vệ — dù không bắt buộc phải khác nhau, cân nhắc dùng hai mật khẩu riêng biệt cho dữ liệu thật.

Tóm tắt & tiếp theo

Bạn đã có một file AAB đã ký, sẵn sàng để phát hành. Chương 39 hướng dẫn đưa file này lên Google Play Console — bước cuối cùng để ứng dụng của bạn tới được tay người dùng thật.

Chương 39: Đưa ứng dụng lên Google Play Console

Mục tiêu học

- Nắm được quy trình tổng quát để đưa một app từ file `.aab` (Chương 38) tới tay người dùng thật qua Google Play.
- Chuẩn bị đúng nội dung mô tả cửa hàng (store listing), đúng giới hạn ký tự của Play Console.
- Hiểu khái niệm "release track" (Internal/Closed/Open testing, Production) và vì sao nên đi qua từng bước thay vì phát hành thẳng.

Code mẫu: `code/ch39-google-play-console/` — chương này không có logic Android chạy trên thiết bị, thay vào đó là cấu trúc quản lý nội dung cửa hàng kèm script tự kiểm tra giới hạn ký tự trước khi đăng.

39.1 Toàn cảnh quy trình phát hành

File .aab đã ký
(Chương 38)

```
graph TD; A[File .aab đã ký (Chương 38)] --> B[Tạo tài khoản Google Play Console Developer (trả phí một lần)]; B --> C[Tạo app mới trong Console: tên, package name, danh mục]; C --> D[Điền Store Listing: mô tả, ảnh chụp màn hình, icon]; D --> E[Khai báo nội dung:];
```

Tạo tài khoản Google Play
Console Developer (trả phí một lần)

Tạo app mới trong Console:
tên, package name, danh mục

Điền Store Listing:
mô tả, ảnh chụp màn hình, icon

Khai báo nội dung:

Content rating, Data safety, quyền riêng tư



Chọn Release Track:
Internal → Closed → Open → Production



Tải file .aab lên track đã chọn



Google xét duyệt
(vài giờ tới vài ngày)



App xuất hiện trên Play Store

Chương này tập trung vào hai bước dễ chuẩn bị trước và dễ mắc lỗi nhất: **Store Listing** (mục 39.2) và **Release Track** (mục 39.3) — các bước còn lại (tạo tài khoản, khai báo Content rating/Data safety) là các form khai báo tuần tự trên giao diện Console, không có nhiều logic kỹ thuật để trình bày sâu hơn ở đây.

39.2 Store Listing — chuẩn bị trước, tránh sửa đi sửa lại trên form

Google Play giới hạn nghiêm ngặt độ dài từng trường:

Trường	Giới hạn
Tiêu đề (Title)	30 ký tự
Mô tả ngắn (Short description)	80 ký tự
Mô tả đầy đủ (Full description)	4000 ký tự
Thông tin phiên bản mới (Release notes)	500 ký tự mỗi ngôn ngữ

Thay vì soạn trực tiếp trên form Console (dễ mất nội dung nếu mất kết nối, khó version-control), code mẫu tổ chức nội dung thành file text riêng theo ngôn ngữ, kiểm tra bằng script trước:

```
store-listing/vi-VN/title.txt
store-listing/vi-VN/short_description.txt
store-listing/vi-VN/full_description.txt
release-notes/vi-VN/default.txt
```

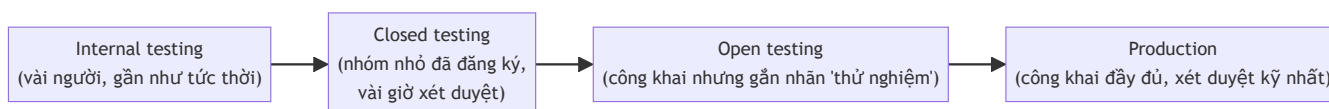
```
node tools/validate-store-listing.js
```

```
OK      store-listing/vi-VN/title.txt (24/30 ký tự)
OK      store-listing/vi-VN/short_description.txt (77/80 ký tự)
OK      store-listing/vi-VN/full_description.txt (486/4000 ký tự)
OK      release-notes/vi-VN/default.txt (147/500 ký tự)
```

Tất cả mục đều hợp lệ.

Cách làm này còn có lợi ích phụ: nội dung mô tả app được **quản lý phiên bản cùng mã nguồn** (commit vào Git), dễ đối chiếu lịch sử thay đổi, và **tải sử dụng được cho CI/CD** — nhiều đội phát triển dùng công cụ tương tự (phổ biến nhất là Fastlane) để tự động hoá toàn bộ quy trình tải bản build + nội dung mô tả lên Play Console mà không cần thao tác tay trên giao diện web mỗi lần phát hành.

39.3 Release Track — vì sao không phát hành thẳng lên Production?



Mỗi track có mức độ xét duyệt và phạm vi tiếp cận khác nhau. Quy trình khuyến nghị: **luôn đi qua Internal testing trước** (phát hiện lỗi hiển nhiên trong vài phút, không cần chờ xét duyệt lâu), rồi Closed/Open testing với nhóm nhỏ người dùng thật trước khi đẩy lên Production. Bỏ qua các bước này

và phát hành thẳng Production tăng rủi ro một lỗi nghiêm trọng (crash ngay khi mở app) tới tay hàng loạt người dùng thật trước khi kịp phát hiện.

Điểm quan trọng liên quan trực tiếp tới Chương 38: mọi track đều yêu cầu `versionCode` tăng dần — một bản đã tải lên Internal testing với `versionCode 3` thì bản tiếp theo (dù ở track nào) phải có `versionCode` lớn hơn 3, không thể quay lại dùng số cũ.

Bài tập

1. Chạy `node tools/validate-store-listing.js` trong code mẫu, xác nhận toàn bộ hợp lệ.
 2. Cố tình sửa `full_description.txt` dài vượt 4000 ký tự (copy lặp lại nội dung nhiều lần), chạy lại script — xác nhận bắt đúng lỗi trước khi bạn kịp dán nhầm vào Play Console thật.
 3. Tự phác thảo (chỉ viết ra giấy/text, không cần Console thật) nội dung Store Listing tiếng Việt cho MỘT trong các app mẫu đã xây trong sách (ví dụ app Room ở Chương 13) — thực hành đúng giới hạn ký tự.
-

Lỗi thường gặp

- **Soạn mô tả app trực tiếp trên form Console, không lưu bản nháp:** dễ mất công soạn lại nếu phiên làm việc bị gián đoạn — nên soạn trong file text trước, dán vào sau cùng.
 - **Bỏ qua Internal/Closed testing, phát hành thẳng Production:** rủi ro lỗi nghiêm trọng tới tay số đông người dùng ngay từ lần đầu, thay vì phát hiện sớm trong nhóm nhỏ.
 - **Quên rằng `versionCode` phải tăng dần XUYỀN SUỐT mọi track**, không riêng Production: tải lên track testing với số cũ hơn bản production hiện tại cũng bị từ chối.
 - **Không kiểm tra kỹ Content rating/Data safety trước khi nộp:** khai sai (dù vô ý) thông tin về việc app có thu thập dữ liệu gì có thể khiến app bị gỡ hoặc tài khoản bị cảnh cáo sau khi phát hành — cần khai chính xác dựa trên những gì app THẬT SỰ làm (quyền đã xin ở Chương 26, 35, 37 là manh mối để tự rà soát).
-

Tóm tắt & tiếp theo

Bạn đã có bức tranh đầy đủ để đưa một app từ file build tới tay người dùng thật trên Google Play. Chương 40 khép lại sách với việc theo dõi ứng dụng sau khi phát hành — thu thập crash report và số liệu sử dụng cơ bản.

Chương 40: Theo dõi sau phát hành — Crashlytics, Analytics cơ bản

Mục tiêu học

- Hiểu cơ chế đứng sau crash reporting: `Thread.UncaughtExceptionHandler`, và vì sao không bao giờ được "nuốt" crash âm thầm.
- Hiểu vì sao SDK analytics luôn **gom sự kiện (batching)** thay vì gửi ngay từng cái một.
- Biết cách đọc lại crash log kèm `mapping.txt` (Chương 27) khi crash xảy ra trên bản release đã làm rồi tên.
- Biết Firebase Crashlytics/Analytics là gì, và tại sao thường là lựa chọn thực dụng hơn tự xây khi làm dự án thật.

Code mẫu: `code/ch40-crash-analytics/` — tự cài đặt cơ chế crash reporting và analytics batching, không phụ thuộc dịch vụ ngoài, để hiểu rõ "hộp đen" trước khi dùng SDK thật.

40.1 Vì sao cần theo dõi sau phát hành?

Chương 24-25 đã dạy kiểm thử **trước khi** phát hành (Unit test, Espresso) — nhưng không có bộ test nào bắt được **mọi** tình huống thực tế: thiết bị lạ, phiên bản Android hiếm gặp, dữ liệu người dùng nhập bất thường. Theo dõi sau phát hành là tấm lưới an toàn cuối cùng — biết được app đang crash ở đâu, bao nhiêu người bị ảnh hưởng, ngay cả khi bạn không tự tái hiện được lỗi đó trên máy mình.

40.2 Cơ chế crash reporting: `UncaughtExceptionHandler`

```
Thread.UncaughtExceptionHandler defaultHandler =
Thread.getDefaultUncaughtExceptionHandler();

Thread.setDefaultUncaughtExceptionHandler((thread, throwable) → {
    writeCrashLog(appContext, thread, throwable); // 1. Ghi lại đầy đủ ngữ cảnh
    defaultHandler.uncaughtException(thread, throwable); // 2. BẮT BUỘC gọi lại
});
```

Exception không được xử lý
(uncaught) xảy ra



Handler tùy chỉnh của bạn
chạy **TRƯỚC TIÊN**



Ghi log: stack trace,
thiết bị, phiên bản app



Gọi lại handler **MẶC ĐỊNH**

của hệ thống



Hệ thống tiếp tục quy trình
chuẩn: đóng app, hiện dialog lỗi

Điểm quan trọng nhất, dễ làm sai nhất: **luôn giữ và gọi lại handler mặc định** sau khi ghi log xong. Bỏ qua bước này để "tự xử lý" exception không đúng cách khiến app rơi vào trạng thái treo bất thường, tệ hơn hẳn một crash bình thường — mục tiêu của crash reporting là **quan sát**, không phải **ngăn chặn** crash.

Cài đặt càng sớm càng tốt — trong `Application.onCreate()` (Chương 6), trước cả `MainActivity`:

```
public class MonitoringApplication extends Application {  
    @Override  
    public void onCreate() {  
        super.onCreate();  
        CrashReporter.install(this);  
    }  
}
```

40.3 Đọc lại crash log ở lần chạy tiếp theo

Vì tiến trình đã chết ngay sau khi crash, không thể "gửi lên server ngay lúc đó" một cách đáng tin cậy — cách thực dụng là **ghi xuống đĩa trước** (giống nguyên tắc offline-first ở Chương 23), rồi thử tải lên ở lần khởi động tiếp theo:

```
List<String> logs = CrashReporter.getPendingCrashLogs(this);
if (!logs.isEmpty()) {
    // Ở app thật: gọi OkHttp/Retrofit (Chương 21-22) tải log lên server,
    // dùng WorkManager (Chương 18) để đảm bảo tải được kể cả khi mất mạng lúc
    // mở lại app – rồi mới CrashReporter.clearCrashLogs() sau khi tải thành công.
}
```

Code mẫu dừng ở bước hiển thị log ngay trong app để bạn tận mắt thấy nó đã được ghi lại đầy đủ — một dịch vụ thật (mục 40.5) tự động hoá toàn bộ phần "tải lên server" này.

40.4 Analytics — vì sao luôn gom theo đợt (batching)?

```
public static void logEvent(String name, String... params) {
    pendingEvents.add(...); // chỉ THÊM VÀO HÀNG ĐỢI, KHÔNG gửi ngay
}

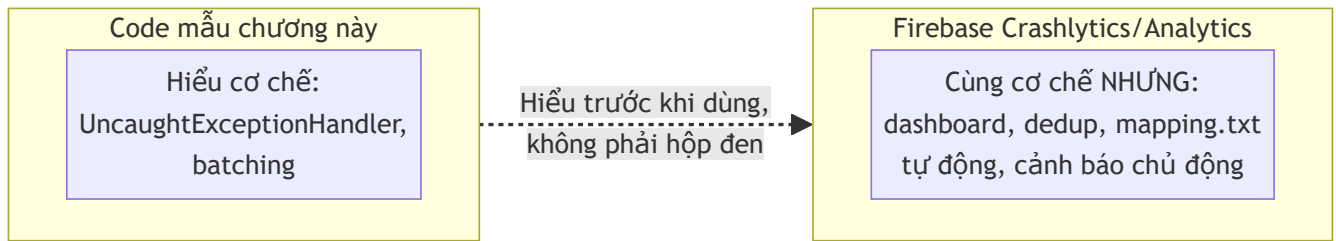
// Chạy định kỳ, độc lập với logEvent()
scheduler.scheduleWithFixedDelay(AnalyticsLogger::flush, 10, 10, TimeUnit.SECONDS);
```

Gửi một request mạng cho **mỗi** sự kiện (mỗi lần bấm nút, mỗi lần cuộn màn hình) sẽ tốn pin và dữ liệu di động đáng kể nếu người dùng thao tác nhiều — đúng vấn đề đã cảnh báo từ Chương 21 về chi phí một request HTTP. Mọi SDK analytics thật (Firebase Analytics, Amplitude, Mixpanel...) đều **gom nhiều sự kiện lại, gửi định kỳ một lần** — đây là lý do đôi khi bạn thấy sự kiện "xuất hiện trễ vài phút" trên dashboard thật, không phải lỗi của SDK mà là hành vi thiết kế có chủ đích.

40.5 Firebase Crashlytics/Analytics — khi nào nên dùng bản thật thay vì tự xây?

Code mẫu chương này giúp hiểu **cơ chế**, nhưng cho dự án thật, hầu như luôn nên dùng dịch vụ đã có sẵn (Firebase Crashlytics/Analytics là lựa chọn phổ biến nhất, miễn phí cho quy mô vừa) thay vì tự xây toàn bộ, vì những lý do tự xây khó tái tạo đầy đủ:

- **Dashboard trực quan:** biểu đồ, lọc theo phiên bản/thiết bị, không phải tự đọc log thô.
- **Tự động dedup:** hàng nghìn người dùng gặp CÙNG một lỗi được gộp thành một "issue" duy nhất, không phải hàng nghìn dòng log riêng lẻ.
- **Tích hợp sẵn `mapping.txt`:** tự động dịch ngược tên đã bị R8 làm rối (Chương 27) khi hiển thị crash từ bản release, không cần tự quản lý thủ công.
- **Cảnh báo chủ động:** gửi thông báo khi tỷ lệ crash tăng đột biến, không cần tự mở app kiểm tra.



Xem `README.md` của code mẫu để biết các bước cụ thể tích hợp Firebase thật khi cần.

Bài tập

1. Chạy code mẫu, gây crash thử, xác nhận log được ghi lại đầy đủ ở lần mở app tiếp theo.
2. Bấm nút ghi sự kiện nhiều lần liên tiếp trong vài giây, quan sát Logcat xác nhận chúng được gộp thành một đợt gửi duy nhất.
3. Đọc kỹ `CrashReporter.install()`, giải thích bằng lời tại sao dòng gọi lại `defaultHandler.uncaughtException( ... )` là bắt buộc, dựa trên mục 40.2.

Lỗi thường gặp

- **Không gọi lại handler mặc định sau khi ghi log:** app rơi vào trạng thái treo/lỗi khó hiểu thay vì đóng gọn gàng như một crash bình thường.
- **Gửi một request mạng cho mỗi sự kiện analytics:** tốn pin/dữ liệu di động không cần thiết — luôn gom theo đợt như mục 40.4.
- **Cài đặt crash reporting trễ (ví dụ trong `MainActivity.onCreate()` thay vì `Application.onCreate()`):** bỏ lỡ những crash xảy ra sớm hơn trong quá trình khởi động app.
- **Quên lưu giữ `mapping.txt` (Chương 27) của mỗi bản release đã phát hành:** dù dùng Crashlytics thật, không có mapping đúng thì vẫn không dịch ngược được tên đã bị làm rối trong crash log.

Lời kết

Bạn đã đi trọn hành trình từ dòng lệnh cài đặt Android Studio đầu tiên (Chương 2) tới việc theo dõi một ứng dụng đã phát hành thật (chương này) — 40 chương, mỗi chương kèm một project chạy được, cùng xây dựng dần một mô hình phát triển Android hoàn chỉnh: từ nền tảng, lưu trữ, xử lý nền, mạng, kiểm thử, kiến trúc, tới giao tiếp phần cứng và xuất bản. Phụ lục cuối sách tổng hợp lại các lỗi thường gặp xuyên suốt các chương, cùng danh sách tài liệu tham khảo để tiếp tục học sâu hơn từng chủ đề riêng lẻ.

Phụ lục

Phụ lục A: Bảng tổng hợp lỗi thường gặp & cách xử lý

Bảng tra cứu nhanh — mỗi dòng tóm tắt một lỗi đã giải thích chi tiết trong chương tương ứng. Dùng Ctrl+F tìm theo thông báo lỗi hoặc từ khoá khi gặp sự cố, rồi quay lại đúng chương để đọc phần giải thích đầy đủ.

Cài đặt & build

Triệu chứng	Nguyên nhân thường gặp	Xem lại
'adb' is not recognized	Path chưa trỏ tới platform-tools, hoặc terminal mở trước khi cập nhật biến môi trường	Chương 2
Máy ảo chạy cực chậm/treo lúc khởi động	Chưa bật ảo hoá phần cứng (VT-x/AMD-V), hoặc xung đột Hyper-V/WSL2	Chương 3
INSTALL_FAILED_UPDATE_INCOMPATIBLE	App cũ trên máy ký bằng key khác bản đang cài — gỡ cài đặt cũ trước	Chương 5
Thiếu android:exported trên Activity/Service/Receiver có intent-filter	Bắt buộc khai báo tường minh từ Android 12 (API 31)	Chương 4, 8, 32, 34
Build release crash/null dữ liệu dù debug chạy tốt	R8 đổi tên field mà Gson/reflection cần đúng tên gốc — thiếu rule -keep	Chương 27

Vòng đời & UI

Triệu chứng	Nguyên nhân thường gặp	Xem lại
Dữ liệu mất khi xoay màn hình	Không lưu trong onSaveInstanceState, hoặc tưởng nhầm nó là lưu trữ lâu dài	Chương 6
Giữ Activity / Context trong biến static hoặc trong ViewModel	Rò rỉ bộ nhớ — Activity không bao giờ được giải phóng	Chương 6, 28
View "biến mất"/lệch vị trí dù XML trông đúng	Thiếu ràng buộc ConstraintLayout theo một chiều (ngang hoặc dọc)	Chương 7
NoMatchingViewException khi test Espresso	Sai ID, hoặc bàn phím ảo che View cần thao tác (quên closeSoftKeyboard())	Chương 25

Dữ liệu & mạng

Triệu chứng	Nguyên nhân thường gặp	Xem lại
<code>IllegalStateException: Cannot access database on the main thread</code>	Gọi Room trực tiếp trong <code>onClickListener</code> , quên bọc qua <code>Executor</code>	Chương 13
<code>NetworkOnMainThreadException</code>	Gọi <code>OkHttpClient.execute()</code> (đồng bộ) trên main thread thay vì <code>enqueue()</code>	Chương 21
Field JSON map ra <code>null</code> dù tên đúng chính tả trông giống	Gõ sai <code>@SerializedName</code> , hoặc lệch kiểu dữ liệu	Chương 22
Mất mạng khiến màn hình trống trơn	Thiết kế "network-first" thay vì offline-first (luôn đọc cache trước)	Chương 23

Chạy nền & thông báo

Triệu chứng	Nguyên nhân thường gặp	Xem lại
<code>ForegroundServiceDidNotStartInTimeException</code>	Gọi <code>startForeground()</code> quá trễ sau <code>startForegroundService()</code>	Chương 16
Broadcast tự định nghĩa không tới nơi	Thiếu <code>setPackage()</code> , hoặc kỳ vọng receiver khai báo tĩnh vẫn nhận được (không còn đúng từ Android 8+)	Chương 17
<code>PeriodicWorkRequest</code> "chạy chậm hơn khai báo"	Chu kỳ tối thiểu 15 phút — WorkManager tự nâng lên, không phải lỗi	Chương 18
Bấm Huỷ tác vụ nhưng vẫn chạy tới hết	<code>Future.cancel(true)</code> chỉ đặt cờ interrupted — code phải tự kiểm tra <code>isInterrupted()</code>	Chương 19
Notification không hiện ra, không báo lỗi	Thiếu <code>NotificationChannel</code> (Android 8+), hoặc thiếu quyền <code>POST_NOTIFICATIONS</code> (Android 13+)	Chương 20

Quyền & bảo mật

Triệu chứng	Nguyên nhân thường gặp	Xem lại
<code>SecurityException</code> dù đã khai báo <code><uses-permission></code>	Quyền thuộc nhóm "dangerous" — phải xin thêm ở runtime, khai báo manifest chỉ là điều kiện cần	Chương 26
Xin lại quyền không có tác dụng, không hiện hộp thoại	Người dùng đã chọn "Không hỏi lại" — chỉ còn cách hướng dẫn vào Settings	Chương 26
Quét Bluetooth thất bại âm thầm trên Android 12+	Xin <code>ACCESS_FINE_LOCATION</code> thay vì <code>BLUETOOTH_SCAN</code> — hai quyền phục vụ hai nhóm phiên bản khác nhau	Chương 35, 37
Xin quyền vị trí nền bị từ chối ngay lập tức	Cố xin <code>ACCESS_BACKGROUND_LOCATION</code> cùng lúc với quyền foreground — phải tách hai bước	Chương 37

Liên ứng dụng & phần cứng

Triệu chứng	Nguyên nhân thường gặp	Xem lại
App khác không đọc được <code>ContentProvider</code> / <code>Service</code> của bạn	Thiếu <code>android:exported="true"</code>	Chương 32, 34
<code>bindService()</code> sang app khác không tìm thấy gì (API 30+)	Thiếu khai báo <code><queries></code> (package visibility) trong manifest	Chương 34
Deep link không mở được từ trình duyệt/nguồn ngoài	Thiếu <code>category.BROWSABLE</code> trong intent-filter	Chương 33
Kết nối Bluetooth/đọc NFC làm đung UI	Gọi <code>connect()</code> / <code>accept()</code> / <code>read()</code> (đều CHẶN thread) trực tiếp trên main thread	Chương 35, 36

Xuất bản

Triệu chứng	Nguyên nhân thường gặp	Xem lại
Google Play từ chối bản tải lên	<code>versionCode</code> không tăng so với bản trước đó	Chương 38, 39
Không đọc được crash log từ bản release đã phát hành	Thiếu <code>mapping.txt</code> đúng phiên bản để dịch ngược tên đã bị R8 làm rối	Chương 27, 40
Mất khả năng phát hành bản cập nhật cho app cũ	Làm mất file keystore release và không dùng Google Play App Signing	Chương 27, 38

Phụ lục B: Tài liệu tham khảo & nguồn học thêm

Sách này ưu tiên giải thích khái niệm và cơ chế bằng tiếng Việt kèm code chạy được thật — tài liệu chính thức bên dưới luôn là nguồn cập nhật nhất khi Android ra phiên bản mới, hoặc khi cần tra cứu chi tiết API vượt ngoài phạm vi một chương giới thiệu.

Tài liệu chính thức Android

- **developer.android.com/guide** — tài liệu hướng dẫn chính thức, theo từng chủ đề (Activity, Fragment, Intent, Service...) — nguồn tham khảo hàng đầu cho Phần 0, 1, 3.
 - **developer.android.com/reference** — tài liệu tham khảo API đầy đủ (Javadoc), tra cứu chính xác tham số/hành vi từng phương thức.
 - **developer.android.com/training/permissions** — tài liệu chi tiết về mô hình quyền runtime, cập nhật theo từng phiên bản Android mới (bổ sung cho Chương 26, 37).
 - **developer.android.com/topic/architecture** — hướng dẫn kiến trúc chính thức của Google (MVVM, Repository, UDF) — mở rộng cho Chương 28, 31.
 - **developer.android.com/jetpack** — tổng quan các thư viện Jetpack (Room, Lifecycle, Navigation, Hilt, Compose...) đã dùng xuyên suốt sách.
 - **developer.android.com/jetpack/compose** — tài liệu đầy đủ về Jetpack Compose, mở rộng cho Chương 30.
 - **developer.android.com/guide/topics/connectivity/bluetooth** và **[.../nfc](https://developer.android.com/guide/topics/connectivity/nfc)** — tài liệu chi tiết Bluetooth/NFC, mở rộng cho Chương 35, 36.
 - **developer.android.com/studio/publish** — hướng dẫn chính thức về ký ứng dụng, App Bundle, phát hành — mở rộng cho Chương 38, 39.
 - **source.android.com** — tài liệu về nội bộ hệ điều hành Android (AOSP), cho ai muốn tìm hiểu sâu hơn tầng dưới Chương 1.
-

Thư viện chính đã dùng trong sách

- **square.github.io/okhttp** và **square.github.io/retrofit** — tài liệu chính thức của OkHttp/Retrofit (Chương 21-22).
 - **developer.android.com/jetpack/androidx/releases/room** — ghi chú phát hành và tài liệu Room (Chương 13, 23).
 - **dagger.dev/hilt** — tài liệu chính thức Dagger Hilt (Chương 29, 31).
 - **github.com/google/gson** — tài liệu Gson (Chương 22-23).
-

Kotlin (cho ai muốn mở rộng sau Chương 30)

- **kotlinlang.org/docs** — tài liệu ngôn ngữ Kotlin chính thức.

- developer.android.com/kotlin — tài liệu Android dành riêng cho Kotlin, bao gồm Coroutines (nhắc tới ở Chương 19) và Compose đầy đủ (Chương 30).
-

Dịch vụ theo dõi sau phát hành (mở rộng Chương 40)

- firebase.google.com/docs/crashlytics — tài liệu Firebase Crashlytics.
 - firebase.google.com/docs/analytics — tài liệu Firebase Analytics.
-

Công cụ build tài liệu (dùng để tạo sách này)

- mermaid.js.org — cú pháp vẽ sơ đồ Mermaid, dùng trong toàn bộ hình minh họa của sách.
 - [markdown-it](#) (thư viện npm) — bộ chuyển đổi Markdown sang HTML dùng trong `tools/build.js` của repo này.
-

Đã đọc hết 40 chương và hai phụ lục — cảm ơn bạn đã theo sách tới đây. Toàn bộ mã nguồn ví dụ nằm trong thư mục `code/` của repo, độc lập với nội dung sách, có thể mở trực tiếp bằng Android Studio để tiếp tục thử nghiệm.